



- 能力导向的技术体系
- 问题驱动的技术要点
- 项目驱动的技术应用

科林 C 语言

内部培训教程

科林海哥

科林明伦 | 清华大街 163 号 4 层

编写说明

设计理念

本教程编写目的是让 C 语言从零到一的学习更具有实用性。C 语言作为语言的语言，它更为底层，掌握 C 语言及其底层逻辑的人在驾驭新的语言时会更从容的上手。本教程从编程运用能力角度出发，把必要的重点知识以问题驱动的方式将知识点的来龙去脉及应用技巧进行解答；力求让学习者对于 C 语言的必要知识点有较深度的认知。另外本教程从实用角度出发收集遴选了经典的题目及项目，对常规笔试题、学科考试、课程设计、小白练手都具有极大的借鉴意义与参考性。并为未来继续学习其它编程语言提供一套高可复用的学习方法与实操流程。

编程能力

想到即做到是所有初学者的愿望。从有一个想法到完成一个可以运行的软件项目是需要知识体系的认知、需求逻辑的设计，代码编写并调试等非常综合的能力。但是无论多么复杂的问题也是能够被分解的，编程能力也是如此。本教程从运用编程语言能够做项目为目标定义了六项能力对初学者进行讲授。它们分别是：

- 1、过程控制能力：用流程结构描述算法逻辑的能力。
- 2、数据存储能力：结合数据类型的存储原理，合理实用内存空间的能力。
- 3、模块化管理代码能力：利用函数及各种严谨性关键词构建项目模块化的能力。
- 4、文字处理能力：利用字符串原理及标准库函数解决文字处理算法的能力。
- 5、自定义类型能力：能够用结构体、联合体、枚举等自创类型描述万物信息模型。
- 6、数据持久化能力：把内存数据平移到文件或数据库永久保存的处理能力。

编程思维

编程本质是思维转移并固化到电脑，之后让电脑代替人去执行的过程。所谓知己知彼，就是我们既要知道我们的逻辑意图，又要懂得计算机如何执行实现我们的意图。这个过程中语法、编程环境、各种执行原理是短期学习的重点。通过系统学习与刻意练习短期内就能够掌握。而从长远发展看，编程的过程就是建立解决问题的思维过程。编程思维伴随着学习深入会逐步建立，这里仅为初学者打开一扇意识之窗。短期学习中逐步理解语言机制的思维如模块化、结构化、面向过程。随着解决问题的增多，比如研究算法数据结构时、做不同种类项目时就会形成经验沉淀，这些都是编程思维。总之编程思维是学习、内化、实践后自然而然总结抽象出来的经验体系，唯有持续实践、学习、分享、交流才能逐渐完善，逐步形成自己的编程思维体系。

问题驱动

学编程、用编程是一个发现问题甚至是创造问题并解决问题的过程。做的越多才能错的越多，错的越多才越能够成长更快。所以学习编程不能简单大量输入（如：看书、停课）就能掌握的。比较有效的方式之一就是列出几百个经典问题与解决。经典问题可能是算法的，可能是基于原理分析结果的。总之以问题为导向，能够系统的，知其然知其所以然的解决最具典型性问题是快速熟悉一门语言的捷径。

项目实战

如说问题驱动是了解熟悉一门语言的基本面，那么用其做不同种类的项目才是学习编程真正的目的。项目更接近工作本身。项目不仅仅是写代码，其还囊括需求分析与设计、代码规范与管理、团队协作。很多编程思想（如重构、设计模式等）都是基于项目而提出的。而且在项目中更能体会技术的价值与重要程度。

目 录

编写说明	1
编程能力一：过程控制	11
Part 1 C 语言基本认知	12
1、 什么是 hello world?	12
2、 为什么建议 C 语言学习采用 VS2019~2022 ?	12
3、 C 语言的模块化、结构化、面向过程与编程过程有关系吗?	12
4、 main 函数的作用是什么? 以及注意事项。	13
5、 C 语言的语句是什么, 分类有哪些?	15
6、 中英文符号混淆后会产生什么错误?	16
7、 VS2019 主要的快捷键有哪些?	16
8、 C 语言源代码中注释符号有哪些及使用技巧?	17
Part 2 变量	17
9、 变量的基本概念及用法?	17
10、 为什么说 C 语言是强类型语言?	18
11、 变量命名语法规则有哪些? 并列出典型的错误变量名	18
12、 C89/C90/C99/C11 是什么?	19
13、 变量命名规范与命名规则有什么区别?	19
14、 能够改变变量存储值的运算符有哪些? 举例说明作用	19
15、 变量的典型 bug 有哪些? 并请说明原因。	20
Part 3 printf 与 scanf	22
16、 printf 的 f 指什么? 怎么理解其蕴含的编程思维?	22
17、 printf 的设置参数常有有有哪些, 到哪里找, 如何查看?	22
18、 使用 scanf 时, 变量前不加&会怎样?	24
19、 scanf 与 scanf_s 的区别是什么?	24
20、 scanf 在输入 float 与 double 类型时的区别是什么?	24
21、 scanf 输入时默认的数据间隔有哪些?	25
22、 scanf 的返回值是干什么的?	25
23、 为什么输入的数据无法存入到变量? scanf 的工作原理又是什么?	26
Part 4 源代码执行原理	27
24、 高级编程语言、汇编语言、机器语言的关系是什么?	27

25、	从源代码到 exe 需要经过哪几个时期？	27
26、	不同时期里的各个环节分工作用是什么？	28
27、	为什么会有语法错误的提示？	30
28、	LNK... 提示的错误产生的原因有哪些？	30
29、	编程入门时期的 bug 是什么原因产生的？	30
Part 5	流程结构	31
30、	流程结构分几种？它们的思维逻辑是什么？	31
31、	多分支选择结构可以用什么语法表达？	31
32、	C 语言有 bool（布尔）类型吗？其逻辑真与假用什么表示？	33
33、	switch 结构中，如果没有 break 语句会怎样？	33
34、	while、do while、for 三种循环的思维逻辑有何差异？	33
35、	死循环是什么？用 while 和 for 举例	34
36、	为什么说 for 循环是万能型循环？	35
37、	什么是流程结构的嵌套结构？	35
38、	break 语句与 continue 语句的区别？	36
39、	调试器的作用及使用步骤？	37
Part 6	运算符与表达式	38
40、	C 语言的表达式与语句的区别是什么？	38
41、	%运算符的作用？-9%-2 的计算结果如何分析？	38
42、	7/2 与 7.0/2 的计算区别是什么？	38
43、	复合赋值运算符有哪些？赋值表达式的计算结果如何分析？	38
44、	i++表达式与++i 表达式的计算结果有何不同？	39
45、	int a = 5, b = 5; a+= a++ + ++b; 变量 a 和 b 的值分别是多少？	39
46、	关系运算符与逻辑运算符都有哪些？其表达式计算结果是什么？	39
47、	&&与 的区别是什么？什么是短路问题？	40
48、	位运算中& ^ 的优先级从高到低的顺序是什么？它们有哪些特性。	40
49、	位运算的<<和>>是什么？另外>>时正负数如何处理？	41
50、	条件（或称三目）运算符是什么？举例说明作用。	42
51、	sizeof 是函数还是运算符？其作用是什么？	42
52、	逗号运算符的作用及计算结果是什么？	42
53、	~, !, ~ 举例说明三种具有“反”功能的作用？	42
	笔试真题	44

编程题	44
编程能力二：数据存储	45
Part 1 基本数据类型	46
1、基本数据类型的分类有哪些？整体介绍一下。	46
2、计算机中的字节是什么？	47
3、十进制转二进制的数学计算方法是什么？	47
4、为什么计算机还要有八进制与十六进制的表示呢？	47
5、整型数的存储原理是什么？	48
6、-1 的二进制存储过程是什么？	48
7、16-9 这个算术运算用二进制计算处理的过程是什么？	49
8、整型存储的溢出现象指什么？	50
9、什么是字面值？整型字面值常量前置写法有哪些？	50
10、整型数据字面值后缀有哪些，分别是什么？	51
11、有符号与无符号的区别是什么？	51
12、如何得到不同整型的数据范围？	51
13、printf 输出不同的整型数据时，其格式都是%d 吗？	52
14、字符型的原理是什么？为什么其存储原理是定点存储？	53
15、字符型字面值设置方式有哪些？	54
16、转义字符有哪些及其作用是什么？	55
17、ASCII 码与 Unicode 码具体是什么？	55
18、char 与 wchar_t 的区别是什么？	55
19、汉字如何输入与输出？	55
20、字符类型的常见算法有哪些？	56
21、什么是浮点型数据类型？	56
22、浮点型数据的字面值与 printf 格式是什么？	57
23、浮点型存储原理是什么？	57
24、float 与 double 有效数字的分别是多少？	58
25、基本数据类型的类型转换方式有哪些？请举例说明。	59
Part 2 指针基础	61
26、什么是内存地址，它有何特性？	61
27、x86 与 x64 的区别是什么？	61
28、指针与地址的关系？	61

29、	指针与指针变量的区别？	61
30、	&与*在指针运算中的作用。	62
31、	指针变量自身的内存空间大小和指向的变量类型有关吗？	62
32、	指针变量增加或减少 1 个整数会得到什么？	63
33、	void* 是什么类型的指针，有何作用？	64
34、	NULL 是什么？	64
35、	大端模式与小端模式指什么？	64
36、	如果一个 char*指针变量指向了一个 int 型变量会报出编译错误吗？	65
37、	写一段代码判断是否是小端模式？	65
38、	如果指针指向了不属于本程序的内存空间会产生什么样的错误？	66
39、	int* x, y, z; 三个变量都是指针变量吗？	66
40、	多级指针是什么？	67
Part 3	数组	67
41、	什么是数组？它有什么特性	67
42、	数组下标越界会产生编译错误吗？	68
43、	什么是数组的遍历？	68
44、	一维数组初始化的方式有哪些？	69
45、	如何计算得出数组的成员数量？	69
46、	一维数组和二维数组的思维逻辑有何不同？	69
47、	二维数组的定义与初始化的写法是怎样的？	69
48、	二维数组为什么又称为：一维数组的一维数组？	69
49、	如何计算二维数组的行数与列数？	70
Part 4	指针与数组	70
50、	一维数组名有何特性？	70
51、	[] 运算符的计算逻辑是什么？	71
52、	数组名是指针变量还是指针常量？	71
53、	利用基本类型的指针变量遍历数组成员的方式有哪些？	71
54、	一维数组名+1 与&一维数组名+1 的区别？	72
55、	指针数组与数组指针分别是什么？	73
56、	二维数组遍历的底层过程是什么？	75
57、	二维数组名+1 与一维数组名+1 的区别？	76
58、	如何利用一维数组指针遍历二维数组所有成员？	76

59、 如何利用二维数组指针遍历二维数组所有成员？	76
笔试真题	77
编程题	77
编程能力三：模块化管理代码	78
Part 1 自定义函数	79
1、 重构的基本思想是什么？	79
2、 函数与模块化的关系是什么？	79
3、 函数的三种状态及关系是什么？	79
4、 自定义函数的构成有哪些？举例说明	80
5、 函数可以嵌套定义吗？	81
6、 函数的执行原理是什么？	81
7、 函数之间的数据传递方向与值传递的方式有哪些？	82
8、 利用函数把数据“带回”到调用位置的手段有哪些？	83
9、 实参的传递顺序是什么？	83
10、 为什么数组做形参最好能够传入数组的有效长度（字符数组除外）？	85
11、 return 的作用与注意事项有哪些？	86
内存区域	86
12、 内存区域有哪些及其作用？	86
13、 函数名及函数执行的本质？	87
14、 函数指针变量是什么？举例说明	88
15、 全局变量与局部变量的区别？	89
16、 形参是局部变量吗？	89
17、 堆内存使用方法与注意事项？	89
18、 栈内存与堆内存的区别？	92
19、 extern 的作用？	92
20、 static 修饰变量的作用？	93
21、 const 修饰全局变量与局部变量的差异是什么？	94
22、 const 修饰指针变量的方式有哪些？	96
typedef	97
23、 typedef 的作用及用法？	97
24、 如何用 typedef 简化指针相关类型的定义？	97
25、 typedef 可以嵌套定义吗？	98

enum	99
26、 什么是枚举类型？	99
27、 枚举的基本用法有哪些？	99
28、 枚举类型创建的变量能保存整数吗？	100
预处理	100
29、 预处理是什么？	100
30、 头文件在项目里有什么作用？	101
31、 #include 的本质是什么？其后的<>与“”的区别是什么？	101
32、 #define 宏替换的作用及基本用法？	101
33、 typedef 与#define 的区别？	102
34、 宏替换能嵌套定义吗？	103
35、 如何用 typedef 与宏创建一个布尔类型？	103
36、 什么是有参数的宏？举例说明	103
笔试真题	104
编程题	104
编程能力四：文字处理	105
相关技术构成	105
字符串相关原理	106
1、 字符数组与字符串的区别？	106
2、 字符串常量的原理是什么？	106
字符串处理函数	108
3、 如何找到字符串处理函数？	108
4、 如何得到字符数组的长度和字符串长度？	109
5、 strcpy() 与 memcpy() 的区别？	109
6、 strcat() 的用法及仿真？	109
7、 strstr() 字符串匹配函数的用法及仿真？	109
8、 strcmp() 字符串比较函数的用法及仿真？	110
9、 字符串与数字类型的互相转换类型有哪些？	111
10、 字符的输入输出函数有哪些？	112
11、 字符串的输入输出函数有哪些？	112
12、 如何实现密码输入时的掩码效果？	113
笔试真题	114

编程题	114
编程能力五：自定义类型	115
结构体与共用体	116
1、 结构体的作用及定义语法？	116
2、 如何创建结构体变量并为其赋值？	116
3、 结构体变量的输入与输出？	117
4、 用结构体类型创建数组并对内容进行排序？	118
5、 结构体指针变量的定义方式？	119
6、 结构体成员访问运算符 . 与->的用法是什么？	119
7、 结构体的对齐、补齐是什么？	119
8、 结构体位域是什么？	121
9、 结构体的嵌套定义方法？	122
10、 结构体与共用体的区别？	123
11、 使用共用体的注意事项是什么？	123
笔试真题	124
编程题	124
编程能力六：数据持久化	125
文件处理	126
1、 文本文件与二进制文件的区别？	126
2、 文件打开函数 fopen 的处理模型是什么？	126
3、 文件的打开方式有哪些？	127
4、 删除文件与修改文件名的函数是什么？	127
5、 创建文件的方法有哪些？	127
6、 向文本文件写入数据的函数有哪些？	127
7、 如何读取一个文本文件中不确定长度的所有整数？	128
8、 文件光标指什么？如何得到和设置文件光标位置？	128
9、 如何对一个文本文件的内容进行改写操作？	129
10、 二进制文件的读写操作函数 fread 与 fwrite 的用法？	129
11、 如何实现文件的复制功能？	130
数据库连接	131
1. 数据库的准备工作有哪些？	131
2. C 语言连接 MySQL 有哪些设置及流程？	131

3.	在 C 语言中如何对数据库实现增加、修改、删除的数据操作?	133
4.	在 C 语言中如何实现查询结果的显示?	133
easyX 图形化编程		135
easyX 技术简介		135
一、	环境准备	135
二、	窗口相关	137
1.	窗口设置	137
2.	关闭窗口	139
3.	对话框	139
4.	改变“标题栏”	140
5.	坐标原点	141
6.	缩放绘图	142
三、	颜色模式	143
1.	颜色原理	143
2.	颜色设置	143
3.	获取颜色值	144
4.	红蓝对调	145
四、	绘制图形	145
1.	获取窗口大小	错误!未定义书签。
2.	直线	145
3.	圆形	146
4.	椭圆形	148
5.	矩形	149
6.	圆角矩形	151
7.	圆弧形	152
8.	扇形	153
9.	多边形	154
10.	区域填充	156
11.	镂空图形	157
五、	图像处理	157
1.	加载图像	159
2.	绘制图像	160

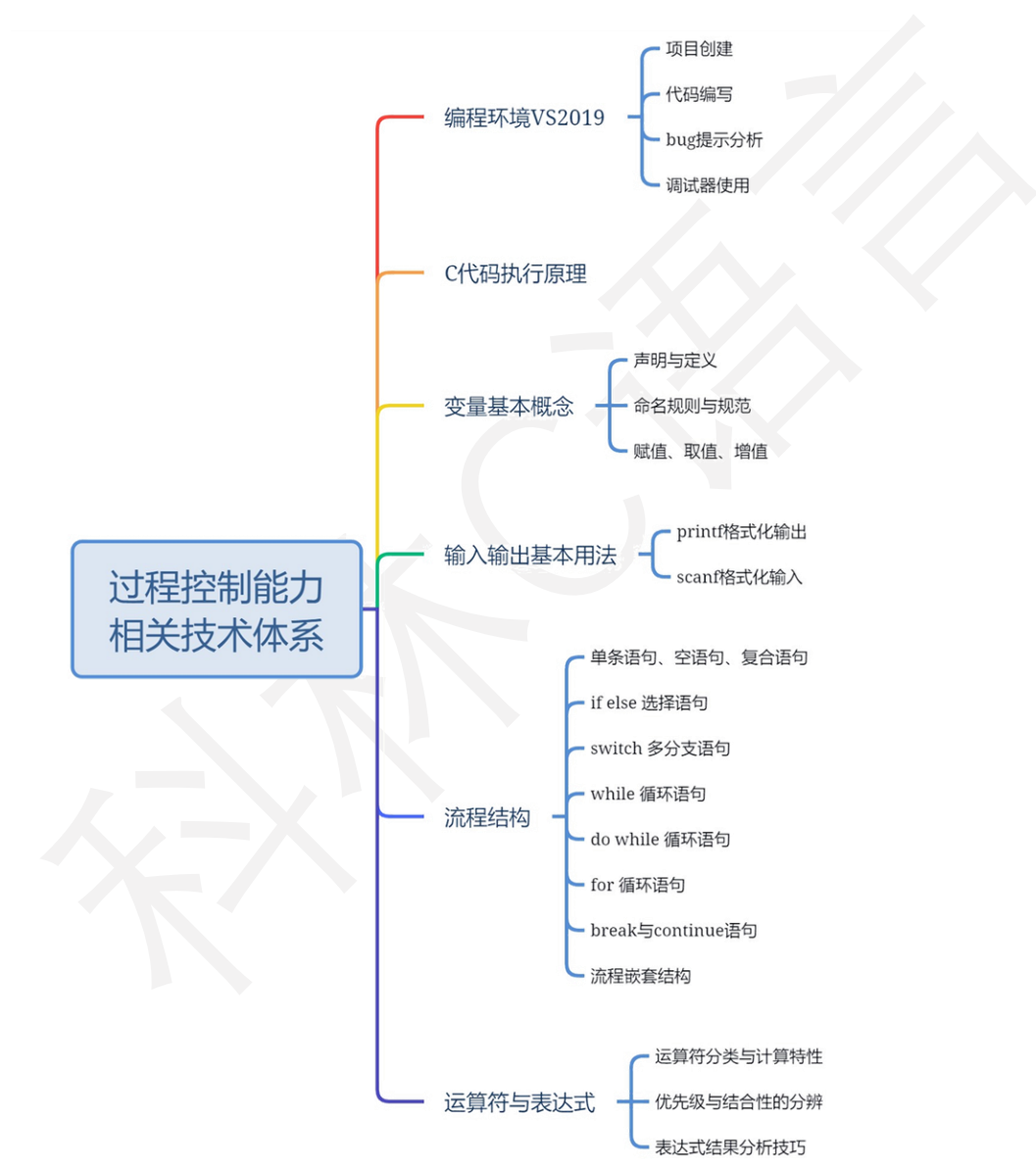
3.	旋转绘制	160
4.	窗口截图	161
5.	截图保存	162
六、	动画框架	163
1.	绘图动画原理	163
2.	图像帧动画原理	163
3.	动画框架原理	164
4.	动画简单例子	166
七、	像素处理	167
八、	文字输出	168
1.	坐标处输出	168
2.	区域内输出	169
3.	动态文字内容	170
九、	鼠标处理	171
十、	键盘处理	173
十一、	音乐播放	176
十二、	游戏框架	177
1.	初始值基本游戏信息	177
2.	鼠标键盘动作的监听	177
3.	界面绘制的处理结构	178
4.	机器人控制处理结构	179
5.	退出游戏框架	180
6.	游戏框架的最佳用法	180
C 语言项目实战		186
游戏项目开发		186
1.	经典贪吃蛇	186
2.	俄罗斯方块	193
组件化项目开发		206
链表组件化项目		207
MIS 管理信息系统开发		227
成绩单管理系统		227

编程能力一：过程控制

能力简介

过程控制能力就是能够把一个处理逻辑的细节与步骤设计清楚，在指定的集成开发环境（如：VS2019）里运用 C 语言的流程结构控制的代码表达出来，并能够调试通过的编程能力。

相关技术构成：



学习技巧建议：

- 1、熟练使用编程环境 VS2019 编写代码并执行程序
- 2、熟悉语法及基本概念。
- 3、多用调试器观察执行细节，用自己的话总结技术点的执行原理。

4、主动犯错误，去观察不同错误在 VS2019 中的提示说明，并结合原理理解错误的原因。

Part 1 C 语言基本认知

1、什么是 hello world?

Hello world 是开发者接触一个新的编程语言或搭建一个新的编程环境时，为了验证编程语言的语法或编程环境调试的“通用手段”。如果能够运行出 hello world 字样的内容，说明此环境是可以继续写程序的。反之就需要调试跑通环境。才能继续后续的学习。

2、为什么建议 C 语言学习采用 VS2019~2022 ?

“工欲善其事必先利其器”。科林推荐的编程环境是因为我们的服务对象是未来进入到 IT 行业的软件研发工程师。所以选择编程环境尽量选择行业普遍使用的，不断版本更新的编程环境。学习编程过程中，熟练使用编程环境也是学习的一部分。

编程环境又称为 IDE (Integrated Development Environment) 集成开发环境，所谓“集成”包括了代码编写过程中各个环节的支持：源代码文件管理、代码提示、语法检查、错误警告提醒、调试器、执行结果查看...。尤其是调试工具，编程在学习与开发过程中，我们都需要利用调试器提供的各类窗口观察（如变量、监视表达式、函数调用、内存情况、反汇编等）代码背后的执行原理，从而更深刻理解计算机的执行细节。知其然知其所以然。进而编写出更准确、更高性能的代码。



VS2019 编程工具的获得、安装、工具注册、环境测试等每个步骤的操作视频可以通过百度网盘链接获得。

3、C 语言的模块化、结构化、面向过程与编程过程有关系吗？

当然有。编程的过程是逻辑思维的转移与传递的过程。“知己知彼”我们既要知道自己的逻辑意图，也要理解计算机这个工具如何能够接纳、理解、执行我们的逻辑意图。所以更好的理解计算机编程语言本身的设计逻辑（或称编程思维），更有助于我们使用它表达出自己的逻辑。C 语言编程思维的底层思维就是：模块化思维、结构化思维、面向过程思维。理解了这三种思维，就能更有“感觉”的写出代码，而不是单纯的“抄写”代码。



如果把代码开发比作为生产一辆车，那么：

模块化思维：已存在有大量的、标准化的、可重复使用的各种零部件。

结构化思维：安装车辆的各个部位把对应的零部件放置其中（如发动机不能放在后备箱）。不同的结构代表不同的功能框架。

面向过程思维：按照设计的逻辑步骤化的把零部件组装在一起的过程。

再比如：小孩子喜欢玩的乐高玩具：按照图纸，寻找对应的零件、一步步的把一个成品的各个部位搭建好，最终再组装在一起的过程就是这样的思维方式。因此在初期写代码时，就把这种思维形成意识融入到写代码的过程中来。

建议按如下方式与步骤写代码：

Step 1 模块化思维	C 语言已经提供了很多《标准库函数》（函数也可以称为功能模块）因此写代码时可以使用它们。如何用这些模块呢？ <code>#include</code>（译为：包含）是一种预处理指令相当于“打开”某个“库”，在本代码文件中准备使用其中的某些模块的功能。 <code>#include <stdio.h></code> 目的：打开标准输入输出模块的“功能清单”（体验 C 语言的传统命名风格——半个单词组合模式） <code>std</code>:standard 的简写译为——标准的 <code>io</code>:input/output 的简写译为——输入/输出 <code>.h</code> 即 head 头。<code>.h</code> 扩展名的文件也常称为“头文件”。
Step 2 结构化思维	你真正的功能代码 <code>printf("hello world.\n");</code>；必须要写在一个称之为 <code>main</code> 函数（或称主函数）的 <code>{ }</code> 内部。 Why? 因为 <code>main</code> 是一种结构（详情参见函数相关内容）是一种语法约束下的格式结构。只有符合此格式，我们的代码才会被 C 语言编译器识别并执行。 请记住：结构化思维=固定格式+可变内容 <pre>int main() { return 0; }</pre> 上面的代码就是：固定格式。务必要正确且完整的写好结构后再去写里面的代码。【注意】 不要有中文的标点符号。
Step 3 面向过程思维	简单理解就是控制思维。把你的意图逻辑转移给计算机的过程。 “告诉”计算机干什么、怎么干（详情参见流程结构相关内容） <code>printf("hello world.\n");</code>；就是告诉计算机：在控制台打印一段文字 <code>hello world.</code> 并换行。这个代码必须要写在 <code>main</code> 结构的 <code>{ }</code> 内部。如同在一个格子里写字，不要写在结构外面。

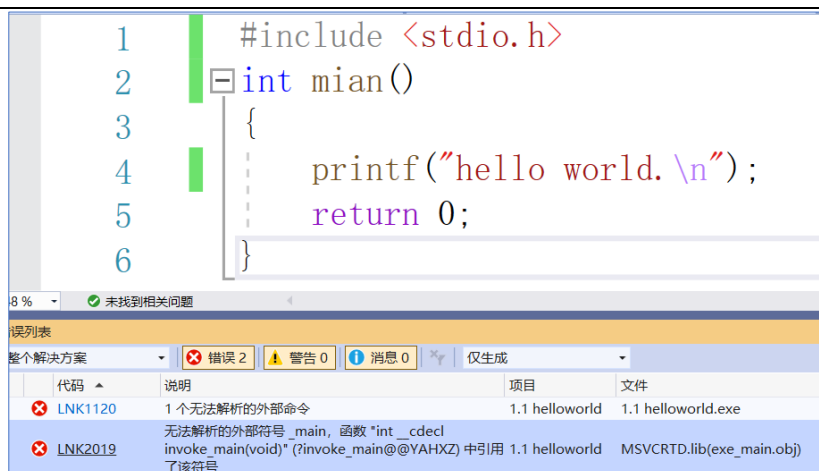
“不要重新发明轮子”是软件行业的“口头禅”。其从某角度告诉我们代码的复用很重要。C 语言的库函数就是可被重复使用的、已经开发好的功能。对于编程的人就是“拿来就用”的工具。不需要重复开发会用即可。

4、main 函数的作用是什么？以及注意事项。

`main` 函数或称主函数是 C 语言开发的软件项目必须有的一个入口模块。也就是说 C 的项目由很多模块（或称函数）构成，但是只能拥有一个 `main` 函数。当一个项目中缺少 `main` 函数或者存在一个以上的 `main` 函数都会引发项目的编译失败。

初学者首次接触 C 语言，就一定会学习使用 `main` 函数。如果不小心写出不符合 `main` 要求的代码就会产生不易于初学者理解的错误（或称 bug）。下面就列举几个写 `main` 函数的常见 bug。

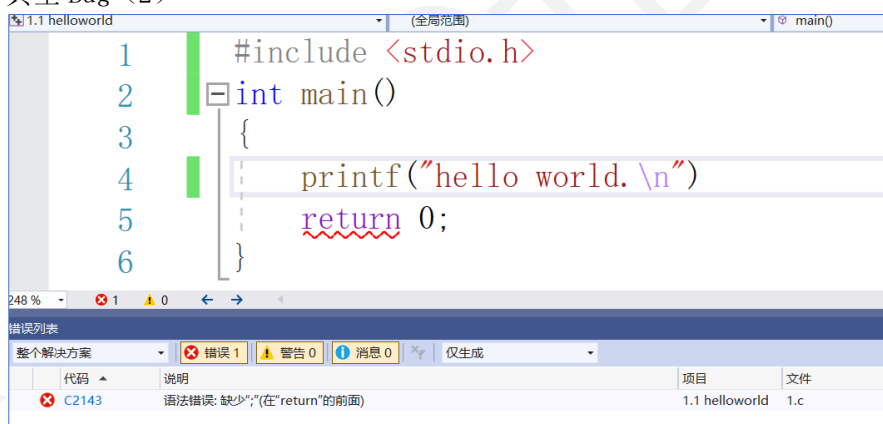
典型 Bug（1）：



Bug 提示解读:

- 1、**无法解析**: 通常代表不能识别的函数名或变量名。说人话: “没引荐不认识”
- 2、**外部符号**: int、return 这类 C 语言关键字属于内部符号, 是可被编译器识别的。main 并不是内部符号, 但确是约定好的一个名字; 用于本项目首先被执行的函数。谁来执行呢? **MSVCRTD**
- 3、**MSVCRTD**: MS 即微, VC 即 (VisualC++: 微软的 C++产品) RT 即 (runtime 运行时, 能启动运行 C/C++的执行程序) D (debug 调试版本)。也就是说 MSVCRTD 用来执行你的项目代码, 但它需要找到 main 函数并执行它; 然而并没有找到。

典型 Bug (2)



Bug 提示解读:

语法错误: 图中 “C2143” 字样中 C 代表 compile 即编译, 编译器在对源代码解析翻译过程中需要对语句进行识别, 当识别到 return 这条新的语句时, 其上一条的 printf 调用函数的语句还没有结束, 编译器就此 “卡 bug” 编译不下去了。从而产生了语法错误的提示字样。

典型 Bug (3)

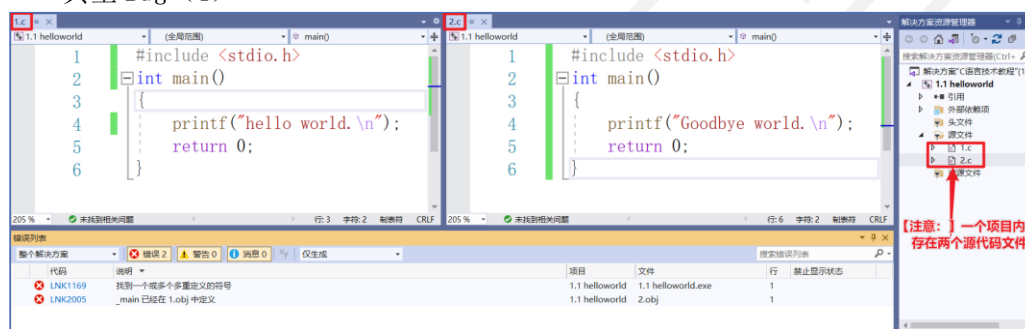


Bug 提示解读:

⚠ 仅仅是一种警告，并会影响代码编译执行。而❌是不可“绕开”的错误。

printf 并不是 C 语法的关键字（或称保留字；即内部符号）。如果能被编译器识别需要提前“告知”有这样一个 printf 函数。然后在 main 中才可以调用执行它。当项目中缺少#include<stdio.h>时就会有此 bug 提示。

典型 Bug (4)



Bug 提示解读:

找到一个或多个多重定义的符号: 即相同作用域内（如本项目内），存在相同的函数名或变量名，使得“连接器”无法区分它们。

main 已经在...中定义: 即 main 函数在本项目中已经存在于 1.c 源代码文件中，而在 2.c 中又被定义了，从而产生了重定义的错误。

提示：一个 C 语言的项目内部是由很多函数构成；每个函数负责实现一个功能模块。函数名不允许重复，否则无法识别调用。但有且只有一个 main 函数作为“执行入口”；即第一个被执行的模块。因此哪段函数代码要先执行，那么它才能称为 main；不被执行的函数可以换个名字，只要没有重名冲突即可。

5、C 语言的语句是什么，分类有哪些？

`printf("hello world.\n");` 就是一条语句，或称为指令。其作用就是命令计算机完成一个功能的执行。语句可以分为：单条语句、空语句和复合语句。

1、单条语句:

它以英文的分号作为语句的结束标记。C 语言编译器在编译时以 ; 号为标记来识别：一条语句的结束和另一条语句的开始。

2、空语句:

仅仅写上英文分号；而没有实质的功能执行，此类语句为空语句。主要和 if、while、for 等流程控制指令语句配合，起到结构占位作用。【注意:】如果有意用空语句占位，务必让其单独占用一行。提醒代码阅读者这是有意为之的。

<pre>#include <stdio.h> int main() { if (3 > 5); else printf("正确"); return 0; }</pre>	<pre>#include <stdio.h> int main() { if (3 > 5) else printf("正确"); return 0; }</pre>
---	--

3、复合语句：

复合一词在编程语言中经常使用。复合（即多个操作合并在一处的简称）语句就是如果某逻辑可能需要多条语句才能表达出来，那么就用一对英文的 { } 把每条语句放入其中。复合语句从语法上可以理解为“一句话”。例如：

```
#include <stdio.h>
int main()
{
    复合语句
    {
        printf("我是谁? \n");
        printf("从哪里来? \n");
        printf("到哪里去? \n");
    }
    return 0;
}
```

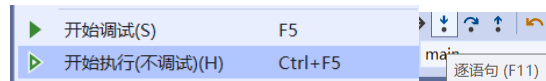
6、中英文符号混淆后会产生什么错误？



如图所示会产生语法错误。中文的符号如 ; 或 “ ” 或 () 这些都不是预设在编译器识别范围内的符号。所以 VS2019 会提示：“未声明的标识符”这样的错误提示。

7、VS2019 主要的快捷键有哪些？

基本的文本操作：ctrl+a 全选、ctrl+c 复制、ctrl+v 粘贴、ctrl+h 查找替换、ctrl+s 保存。其实很多菜单或工具栏按钮都有快捷键的提示如：



还有一些需要按两次组合成的快捷键如变量重命名快捷键：



需要按一次 ctrl+r，然后再按一次 ctrl+r 即可完成操作。快捷键体现了程序员熟练程度与效率的体现。日常练习时注意积累即可。

8、C 语言源代码中注释符号有哪些及使用技巧？

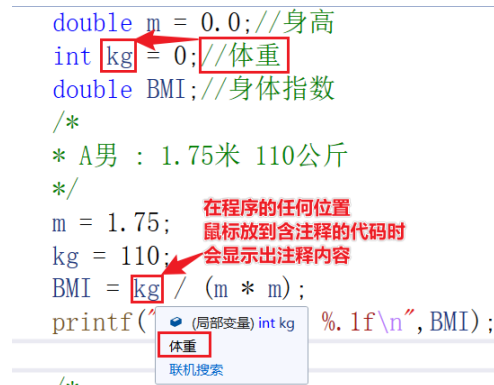
源代码中的 `//` 和 `/* */` 称之为注释符号。其作用有两个。

作用 1： 对程序中某块代码的逻辑或设计用意进行描述说明；以便于代码被人读懂。

【注意】代码的可阅读性在开发中非常重要。

`//` 称之为“行注释”，适合对一行代码进行解释说明，通常放在行上面或代码行尾部。

`/* */` 称之为“块注释”，适合对一大段描述性文字的“包围”。在包围圈内容任意书写内容。某变量或函数的注释在 VS2019 中任意位置悬停时可查看到注释。



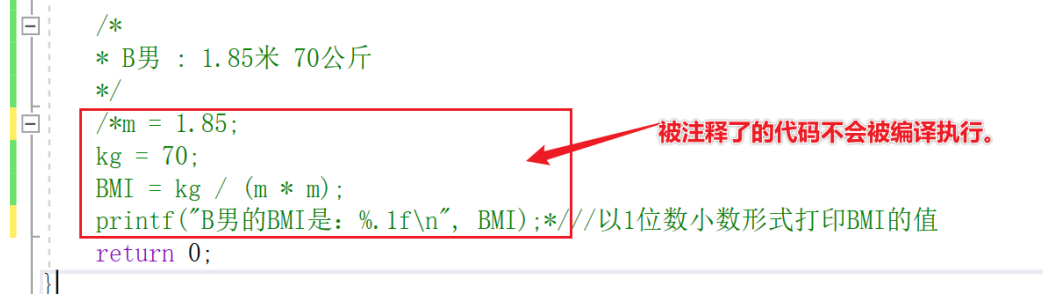
```
double m = 0.0; //身高
int kg = 0; //体重
double BMI; //身体指数
/*
 * A男 : 1.75米 110公斤
 */
m = 1.75;
kg = 110;
BMI = kg / (m * m);
printf("(局部变量) int kg %.1f\n", BMI);
/*
```

在程序的任何位置
鼠标放到含注释的代码时
会显示出注释内容

体重
联机搜索

【注意】程序代码被注释的内容是不会被编译的。

作用 2： 辅助测试调试。对不想执行的代码用注释符号进行注释。



```
/*
 * B男 : 1.85米 70公斤
 */
/*m = 1.85;
kg = 70;
BMI = kg / (m * m);
printf("B男的BMI是: %.1f\n", BMI);*/ //以1位数小数形式打印BMI的值
return 0;
```

被注释了的代码不会被编译执行。

这样就起到辅助调试的效果了。

操作技巧： VS2019 中把不想要执行的代码先选中。然后找到工具栏的按钮



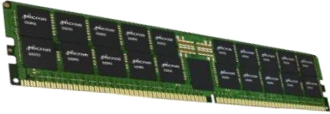
即可把选中的代码进行注释掉。如果想恢复代码的执行，就点



Part 2 变量

9、变量的基本概念及用法？

变量： 即其代表的值是可以改变的量，类似于数学方程中的未知数 x, y 的作用。代表一个不确定的数值。在代码中变量就是程序员向计算机内存中申请的一个存储空间，用于存放程序在执行中的计算结果。

变量的使用可分为：变量创建、变量赋值、变量值读取等操作。例如：	
<pre>#include <stdio.h> int main() { double m = 0.0; //身高 int kg = 0; //体重 double BMI; //身体指数 .. }</pre> 定义变量或称创建变量	定义变量（define variable）就是在计算机内存中  申请一个属于本段代码专属的存储空间，并用不同的名字标识这些空间。
<pre>#include <stdio.h> int main() { double m = 0.0; //身高 int kg = 0; //体重 double BMI; //身体指数 .. }</pre> 数据类型 变量名 赋值运算符 初始值	变量定义的格式： 【格式 1】数据类型 变量名 ； 【格式 2】数据类型 变量名 = 初始值； 【格式 3】数据类型 变量 1, 变量 2····； 建议：尽量采用【格式 2】 每个变量的定义单独占一行并加入“//行注释”进行说明。
<pre>m = 1.85; kg = 70; BMI = kg / (m * m);</pre>	一个“=”为赋值号 目的：把右侧的计算数值存储进左侧的变量所在的内存空间中。
<pre>printf("B男的BMI是: %.1f\n", BMI);</pre>	读取变量的值用于打印呈现结果。

10、 为什么说 C 语言是强类型语言？

计算机编程语言种类繁多，这些编程语言根据不同的特性角度可以分为不同的种类。根据“变量在定义时，是否需要显式地指明数据类型”，可以分为强类型语言与弱类型语言。C 语言, C++, JAVA 这些语言对于变量的类型“极其敏感”。不同数据类型定义的变量会拥有不同的存储原理与空间大小，如果变量在被赋值或读取值时，类型与值不匹配会产生编译错误。因此要想数量掌握 C/C++这种强类型语言，不同数据类型的底层原理就要很了解。与之相反的称为弱类型语言，如 JavaScript（JS）、Python、PHP 等。弱类型语言在定义变量时不需要显式的指明类型。编译器（或解释器）会根据最新赋值给变量的数据值自动推导类型。一个变量的类型值可自由切换。
--

11、 变量命名语法规则有哪些？并列出典型的错误变量名

变量名的命名规则主要是编译器在解析编译时不产生语法错误的规则。这个规则包括
1、变量名可由中英文字符、数字、下划线_、\$符号组合构成 2、变量名不可以有空格等标点符号。 3、变量名不可以用数字开头：8hello 是错误的、_hello 是可以的 4、变量名不可以和编程语言的保留字（或称关键字）重名。

C保留字

C89/C90

char	struct	short	do	if	break	const	typedef
int	union	long	while	else	continue	static	sizeof
float	enum	unsigned	for	switch	return	register	extern
double	void	signed		case	goto	volatile	
				default		auto	

C99

_Bool	restrict
_Complex	inline
_Imaginary	

C11

_Atomic	_Alignas	_Generic
_Thread_local	_Alignof	_Noreturn
		_Static_assert

【建议】实际开发项目时尽量不用汉字和\$符号命名变量（练习测试无所谓）。

12、 C89/C90/C99/C11 是什么？

C89、C90、ANSI C 通常指的是同一个 C 语言标准。1989 年，美国国家标准协会 (ANSI) 推出 C 语言和 C 标准库的标准。该标准通常被称为 ANSI C。由于该标准是 1989 年推出的，因此也被称为 C89。时隔一年，1990 国际标准协会 ISO 参照 ANSI 标准，推出一模一样的 C 语言和 C 标准库标准，由于该标准是 1990 年提出的，因此被称为 C90 标准。因此，C89，C90，ANSI C 是同一个标准。1994 年，ANSI/ISO 联合的组织想要为 ANSI C 标准加入有限的改动，使 C 标准在国际字符、一些明显的缺陷、数值计算上更上一层楼，推出了 C99 标准。2011 年，标准委员会推出了 C11 标准。

13、 变量命名规范与命名规则有什么区别？

变量的**命名规则**是应对编译器的语法检查。如果不满足（问题 11）的规则会产生编译不通过的语法错误。而**命名规范**通常是指行业约定速成的命名形式。一般变量名尽量采用有含义的英文单词的名词或形容词（不要用汉字命名变量）来增强代码可读性。如果变量名由多个单词构成，例如：苹果的价格，如用 apple+price 两个单词命名时，比较主流的方式有：

- 1、下划线隔开每个单词：apple_price
- 2、小驼峰式：applePrice 即首单词首字母小写其后每个单词都需要首字母大写。
- 3、大驼峰（或称帕斯卡）式：ApplePrice。每个单词首字母都要大写。
- 4、匈牙利命名法（不推荐）：属性+类型+大驼峰。例如：g_iApplePrice 其中 g_代表全局整型变量 i 代表 int 整型。属性与数据类型缩写都需要额外记忆，增加了额外的学习成本。

14、 能够改变变量存储值的运算符有哪些？举例说明作用

变量本质就是一个存储空间，我们可以把它形象的理解为一个“储钱罐”

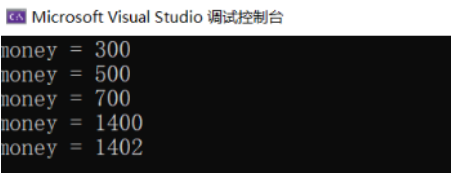
“存钱” ——赋值	赋值行为就类似存钱，区别是赋值是一种 覆盖行为 ：即第二次存入新值，前一次的就被覆盖，也就是仅仅保存最后一次的值。 【格式】 变量名=值；
“取钱” ——取值	读取变量的值。 例如：printf(“%d”，变量名)；

“增值” ——自我改变	就是让变量的值在原有值基础上发生增长性的改变。例如： 变量名 = 变量名 + 值 其计算过程是：首先赋值号右侧“变量名 + 值”的先计算得出结果。再将结果赋值存储进左侧的变量空间里。 【建议】上面的自增长逻辑表达式按下面表达式实现。 变量名 += 值 其中+=属于复合运算符（即先在原有变量值基础上先增加后赋值进变量的运算过程） 类似的还有：-- 自减、*= 自乘、/= 自除、% 自求余等 此外：变量名++ 或 ++变量名这样的写法都代表变量值自增1。 变量名-- 或 --变量名则代表变量自减1 【注意】上述组合型运算符中间不允许有空格隔开。 例如：“+空格=”、“+空格+”都是错误的用法。
------------------------	---

例如：

```
#include <stdio.h>
int main()
{
    int money = 0;
    //赋值（覆盖原有值）
    money = 100;
    money = 300;
    printf("money = %d\n", money); //打印结果：money=300 即覆盖了上一次的赋值
    //增值（自我改变）：
    //方式一：
    money = money + 200;
    printf("money = %d\n", money); //打印结果：money=500
    //方式二：复合赋值
    money += 200;
    printf("money = %d\n", money); //打印结果：money=700
    money *= 2;
    printf("money = %d\n", money); //打印结果：money=1400
    //方式三：++ 自增1 - 自减1
    money++;
    ++money;
    printf("money = %d\n", money); //打印结果：money=1402
    return 0;
}
```

运行结果如下（建议使用调试器的局部变量窗口仔细观察变量值的变化）。



15、 变量的典型 bug 有哪些？并请说明原因。

典型 Bug (1):

```

1  #include <stdio.h>
2  int main()
3  {
4      pi = 3.141596;
5      s = PI * 6 * 6;
6      printf("面积是: %.2f\n", s);
7      return 0;
8  }

```

错误列表

整个解决方案 错误 4 警告 2 消息 0 仅生成

代码	说明
C2065	"s": 未声明的标识符
C2065	"s": 未声明的标识符
C4477	"printf": 格式字符串 "%.2f" 需要类型 "double" 的参数, 但可变参数 1 拥有了类型 "int"
C2065	"pi": 未声明的标识符
C2065	"PI": 未声明的标识符
C4244	"=": 从 "double" 转换到 "int", 可能丢失数据

Bug 提示解读：“未声明的标识符”说的是 `s` 和 `pi` 变量没有声明定义，即不存在这个变量对应的存储空间，而使用不存在变量存取数据是不可能的。

【记住】变量要：“先声明定义、后使用”，也就是说：先创建出存储空间；再存取数据值。

典型 Bug (2):

```

#include <stdio.h>
int main()
{
    /* ... */
    int salary; //月薪
    salary += 1000;
    printf("%d\n", salary);
    return 0;
}

```

代码 说明

C4700 使用了未初始化的局部变量 "salary"

Bug 提示解读：“未初始化的局部变量”，但凡在 `{ }` 内定义的变量都称为“局部变量”即 `{ }` 被执行时，变量被创建。`{ }` 内都可访问此变量的存储空间。一旦 `{ }` 内所有代码执行完，其里面定义的局部变量就会“被回收”即不可被外部访问了。这类的局部变量如果没有明确的赋初始值，却在其上做类似：`++`、`--`、`+=`、`-=`、`*=`...等自增类赋值运算时就会产生此错误提示。

典型 Bug (3):

```

int main()
{
    int sex = 1;
    char sex = 'M';
    printf("%d\n", sex);
    return 0;
}

```

代码 说明

参见 "sex" 的声明

C2371 "sex": 重定义: 不同的基类型

Bug 提示解读：“XX 变量重定义”。当变量在一个作用域范围内（比如 `{ }`），一个变量名被定义了多次就会产生此提示。因为一个变量名在一个作用域范围内只能代表一个存

存储空间。如果按如下方式修改代码：

```

1  #include <stdio.h>
2  int main()
3  {
4      int sex = 1;
5      {
6          char sex = 'M';
7          printf("%c\n", sex);
8      }
9      printf("%d\n", sex);
10     return 0;
11 }

```

Microsoft Visual Studio 调试控制台

M
1

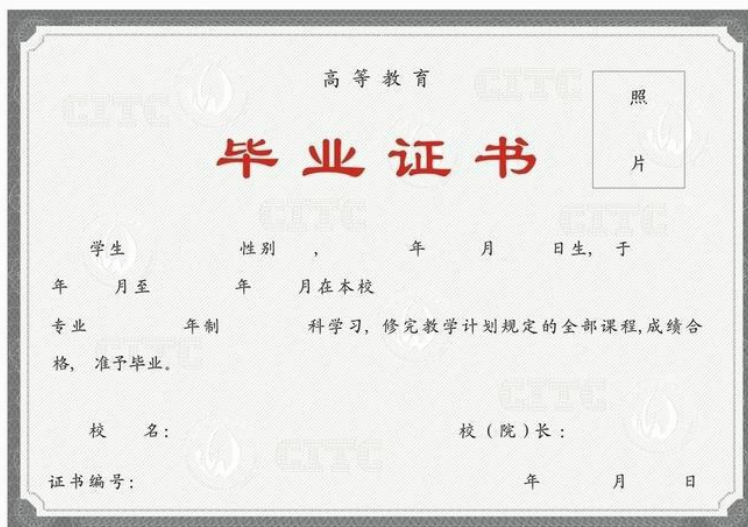
这样，里面的{ }就形成了一个新的作用域。由于两个 sex 变量名分属于不同的作用域，因此就不会有同名问题。而按照“临近原则”，在最里面的{ }内会访问最近的 sex 变量空间，其外部同名的 sex 变量会被“屏蔽”。因此里面的打印结果是 M，里面{ }结束后，其内部的 sex 存储空间就会被“撤销并回收”。

Part 3 printf 与 scanf

16、 printf 的 f 指什么？怎么理解其蕴含的编程思维？

printf 由 print（译为打印）+format（译为格式）两个单词组合而成。其功能是按照设定的输出格式打印出便于阅读的数据。此函数声明来自 stdio.h 头文件。

printf 函数功能属于**模板化思维**的编程技术。什么是模板化思维呢？打个比方：



一张毕业证书就是模板化思维：模板化=固定格式+可变内容。

例如：9*9=81 其中数字部分的 9、9、81 就是可变内容。而 “*” 及 “=” 两个符号便是固定格式。因此用 printf 的编程思维描述就是：

```
printf("%d*%d=%d\n", 9, 9, 9*9);
```

其中 “%d*%d=%d\n”，就是一个模板格式。变化的数字位置用 % 开头，再匹配一个预定义好的字母 d（代表十进制整数）即可，所以 %d 就代表此处是一个整数。其后与 %d 对应的数据就要做到类型匹配、一一对应，不能多也不能少。

17、 printf 的设置参数常有哪些，到哪里找，如何查看？

printf 函数的详细信息可以参考如下网址：

<https://cplusplus.com/reference/cstdio/printf/>

进入网址后可以看到 printf 第一个参数 format 的格式说明符模板：

function

printf

```
int printf ( const char * format, ... );
```

Print formatted data to stdout

Writes the C string pointed by **format** to the standard output (**stdout**). If **format** includes additional arguments following **format** are formatted and inserted in the resulting string

Parameters

format

C string that contains the text to be written to **stdout**. It can optionally contain embedded **format specifiers** that are replaced by the value formatted as requested.

格式说明符模板

A **format specifier** follows this prototype: [\[see compatibility note below\]](#)
`%[flags][width][.precision][length]specifier`

即：%[标记][整体宽度][.其中小数位数][类型长度]类型说明符
 其中[]圈起的部位为可选项，可以不使用。

下面把主要的 **specifiers**（说明符）的预定义设置如下。

类型说明符	输出说明	输出样例
%d 或 %i	Signed d ecimal i nteger 有符号 十进制 整型数	18 或 -18
%u	Unsigned decimal integer 无符号 十进制 整型数	128
%o	Unsigned o ctal integer 无符号 八进制 整型数	610
%x 或 %X	Unsigned h exadecimal integer 无符号 十六进制 整型数	5af 或 5AF
%f 或 %F	Decimal f loating point 十进制 浮点数（实数）	3.14
%e 或 %E	Scientific notation 科学计数法 mantissa/ e xponent 尾数/指数	3.14e5
%g 或 %G	Use the shortest 去掉多余的 0，最短小的方式显示浮点数	
%c	c haracter 字符型（字符影像）	A
%s	s tring of characters 字符串	Hello
%p	p ointer address 指针地址	0017FF10
%%	让%不起到可变数据占位作用，仅仅是个普通文字	%

【注意：】每个 **specifiers** 说明符需要结合数据类型才可真正理解并使用。

案例 1：

```
printf("北京时间：%d年%d月%d日 %02d:%02d\\n", 2023, 5, 23, 8, 9);
```

北京时间：2023年5月23日 08:09

说明：%02d：2 代表数据的展示宽度为 2 个文字位。如果对应的数值（比如 8）不足 2 个文字位则用 0 补充。如果大于等于 2 个文字位，则以实际数据为准；宽度限制无效。

案例 2：

```
printf("图书价格：%f\\n", 18.256);
printf("图书价格：%.2f\\n", 18.256);
printf("图书价格：%8.2f\\n", 18.256);
printf("图书价格：%08.2f\\n", 18.256);
printf("图书价格：%-.2f\\n", 18.256);
printf("图书价格：%+.2f\\n", 18.256);
```

图书价格：18.256000
 图书价格：18.26
 图书价格：18.26
 图书价格：00018.26
 图书价格：18.26
 图书价格：+18.26

说明：%f 代表小数形式，小数的整数部分默认实际的数据宽度，小数部分默认 6 个小数

位，不足补 0。

%2f：格式规定小数部分只显示 2 位，其会对第 3 小数位进行四舍五入。

%8.2f：规定整体宽度 8 位，其中小数位（不含小数点）占 2 位，如实际数据总位数不足时，数据右对齐左补空格。

%08.2f：在%8.2f 基础上，实际数据位数不足时则左补 0。

%-8.2f：在%8.2f 基础上，数据左对齐 右侧补空格。

%+8.2f：让数据显示其正负数的符号。

18、 使用 scanf 时，变量前不加&会怎样？

scanf 由 scan（译为扫描）format（译为格式）两个单词组合而成。其功能是按照设定的格式扫描我们输入的一串文字。如果我们输入的文字满足格式的要求，就会把对应的数据存入到指定变量的内存地址对应的空间中。它和 printf 不同之处就是对变量的使用。printf 在打印输出时用变量名即可，但是 scanf 在输入时，必须用“&取地址”运算符计算变量的实际内存地址，然后再根据%d 等类型说明符把数据按对应数据类型的存储原理存入到变量所在地址的一段内存空间内。所以必须要理解不同数据类型的字节数及存储原理。例如：

```
int age = 0;
scanf( "%d" , &age);
```

这段代码中首先会先计算 age 变量在内存中的实际地址，然后根据%d 的设置会把用户输入的数据以有符号整型的存储原理存放到 age 变量开始地址的 4 个字节空间内。学习过指针后会对这块的理解能更透彻一些。

19、 scanf 与 scanf_s 的区别是什么？

在使用 VS2019 时，如果使用 scanf 作为数据的输入手段可能会报出如下的错误提示。

代码	说明
 C4996	'scanf': This function or variable may be unsafe. Consider using scanf_s instead. To disable deprecation, use _CRT_SECURE_NO_WARNINGS. See online help for details.

上面的提示信息说明：scanf 作为标准的库函数，存在**安全隐患**。因为此函数在被执行时，需要把读取到的信息存入到内存地址对应的空间。只要函数涉及到内存空间访问就会存在这样的安全隐患。因此微软的 VS2019 会有这样的提醒。根据它的提示，我们可以有两种解决的办法：

方式一：采用微软独有的安全函数 scanf_s。但是它并不是标准的库函数，使用它会使 C 代码移植性受到影响。所以它还会提供第二种方式解决。

方式二：使用_CRT_SECURE_NO_WARNINGS 这个“宏”来关闭“C 运行时 CRT_保护 SECURE 无 NO_警告 WARNINGS”的警告提示。但务必保证将

```
#define _CRT_SECURE_NO_WARNINGS
```

这句代码放置在第 1 行处，即#include<stdio.h>的前面。

如采用方式一，则只需要将scanf改成scanf_s即可。

另外：scanf_s在字符或字符串输入方面必须要指定所在内存的存储空间大小。

例如：

```
char ch;
char str[10];
scanf("%c",&ch,1);//此处的 1 代表一个字节的空间大小
scanf("%s",str,10);//此处的str是数组名，本身就是内存地址无需&计算内存地址，10代表可用的内存空间为10个字节。
```

20、 scanf 在输入 float 与 double 类型时的区别是什么？

先看如下代码：

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include<stdio.h>
int main()
{
    float m;
    double n;
    scanf("%f%lf", &m, &n);
    printf("%f %f ", m, n);
    return 0;
}
```

上例代码中 float 类型变量 m 和 double 类型变量 n 在 scanf 输入时需要严格区分类型格式的。但它们在 printf 打印时只需要用%f 输出小数，无需单独区分。这些细节是可以用手册查询到的。

<https://cplusplus.com/reference/cstdio/scanf/>

	specifiers				
	length	d i	u o x	f e g a	c s [] [^] p n
	(none)	←	←	float*	char* void** int*
hh		signed char*	unsigned char*		signed char*
h		short int*	unsigned short int*		short int*
l	←	long int*	unsigned long int*	double*	wchar_t* long int*
ll		long long int*	unsigned long long int*		long long int*
j		intmax_t*	uintmax_t*		intmax_t*
z		size_t*	size_t*		size_t*
t		ptrdiff_t*	ptrdiff_t*		ptrdiff_t*
L				long double*	

上图是 scanf 在输入时，不同类型的对应。其中 f 代表小数形式输入，(none) 和 l（注意：l 是英文字母 L 的小写）代表长度前缀。所以%f 对应 float 类型进行输入，它会把用户输入的数据以浮点型 4 个字节的长度存入到变量 m 的内存空间内。而%lf 则以 8 个字节的内存空间长度接收数据。

21、scanf 输入时默认的数据间隔有哪些？

当我们使用 scanf 连续输入若干个数据时，建议采用连续的数据格式设置。即格式符中间不要掺杂不必要的其它字符。例如：%d%d%f 这种格式。我们在从键盘输入时可以用空格或换行都可以间隔前后的数据。但是如果采用了诸如逗号为数据间的间隔符号，我们会看到只有第一个数据被正确读取了。其余的变量都得不到数据。

22、scanf 的返回值是干什么的？

在 VS2019 中使用 scanf 会经常被提示：**C6031: 返回值被忽略: "scanf"**。那么 scanf 的返回值是什么呢？看如下例子：

```
#define _CRT_SECURE_NO_WARNINGS
#include<stdio.h>
int main()
{
    int a=0, b=0, c=0, d=0;
    int count = scanf("%d%d%d%d", &a, &b, &c, &d);
    printf("a=%d b=%d c=%d d=%d\n", a, b, c, d);
    printf("共成功存入了%d 个变量\n", count);
    return 0;
}
```

上面代码要求输入 4 个整数到 4 个变量中。scanf 返回的是被正确无误的存入到变量的数量。如果返回 4，则说明数据全部正确存入到了所有设置的变量里。

23、为什么输入的数据无法存入到变量？scanf 的工作原理又是什么？

scanf 中的 f 即 format。即按预先设定的格式扫描数据。如果我们输入的数据与设定的格式不匹配，则从不匹配开始，后面的所有数据都不会被处理了。

接《问题 22》的代码，如果输入 1,2,3,4 换行，那么只有 a 变量能够得到数据。而 b, c, d 三个变量都不会得到值。scanf 的工作原理可以参考下图：



1：当我们输入 1,2,3,4 换行时，这些连续的文字会被存入到一个“键盘输入缓冲区”的地方排队等待读取并验证。读取验证的规则就是“%d%d%d%d”。当第一个%d 读取时，由于 d 代表“有符号十进制整数”，所以它认为只有“0 1 2 3 4 5 6 7 8 9 + -”这些字符才是合法的整数字符。其它都是非法的文字符号，因此 1 被正确读取；当读取到 逗号时，它认为是非法的整数字符，所以第一个%d 就结束了读取任务并把满足格式的整数 1 存入进了变量 a 的内存空间；并记录了成功读取的个数为 1。

2：继续读取。由于%d 后面还是%d，这种连续的数据格式之间默认用空格或换行作为间隔。所以它需要读取到一个空格或者换行后，才能继续处理第 2 个%d。但此时读取的却是一个逗号字符，并不是期待的空格或换行。因此 scanf 会认为是“非法的输入”。所以就停止了 scanf 的后续执行。因此后面的变量 b, c, d 都没有读取到需要的数据而保持了原有的初始值。

【总结】scanf 工作时，我们输入的数据必须按约定的格式输入。即固定的内容必须“写完整”，直到到%d 时才能录入对应的变化数据。

观察如下代码的输入：

```
#include<stdio.h>
int main()
{
    int a = 0, b = 0, c = 0, d = 0;
    int count = scanf_s("a=%d,b=%d,c=%d,d=%d",&a,&b,&c,&d);
    printf("a=%d b=%d c=%d d=%d\n",a,b,c,d);
    printf("共成功存入了%d个变量\n",count);
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
a=10, b=20, c=30, d=40
a=10 b=20 c=30 d=40
共成功存入了4个变量
```

其中：“a=”是格式里的固定内容，必须照原样输入。等到%d 位置时需要十进制的整数。因此 10 被录入，再然后逗号是固定的内容必须照样输入……。一旦某个位置不满足规则的规则 scanf 就会立刻停止。后面的变量都不会收到数据。

其实上面的设计本意是：先提示输入“a=”然后用户再录入数据。所以上面的代码改造如下：

```
int main()
{
    int a = 0, b = 0, c = 0, d = 0;
    printf("a=");
    scanf_s("%d", &a);
    printf("b=");
    scanf_s("%d", &b);
    printf("c=");
    scanf_s("%d", &c);
    printf("d=");
    scanf_s("%d", &d);
    printf("a=%d b=%d c=%d d=%d\n", a, b, c, d);
    return 0;
}
```



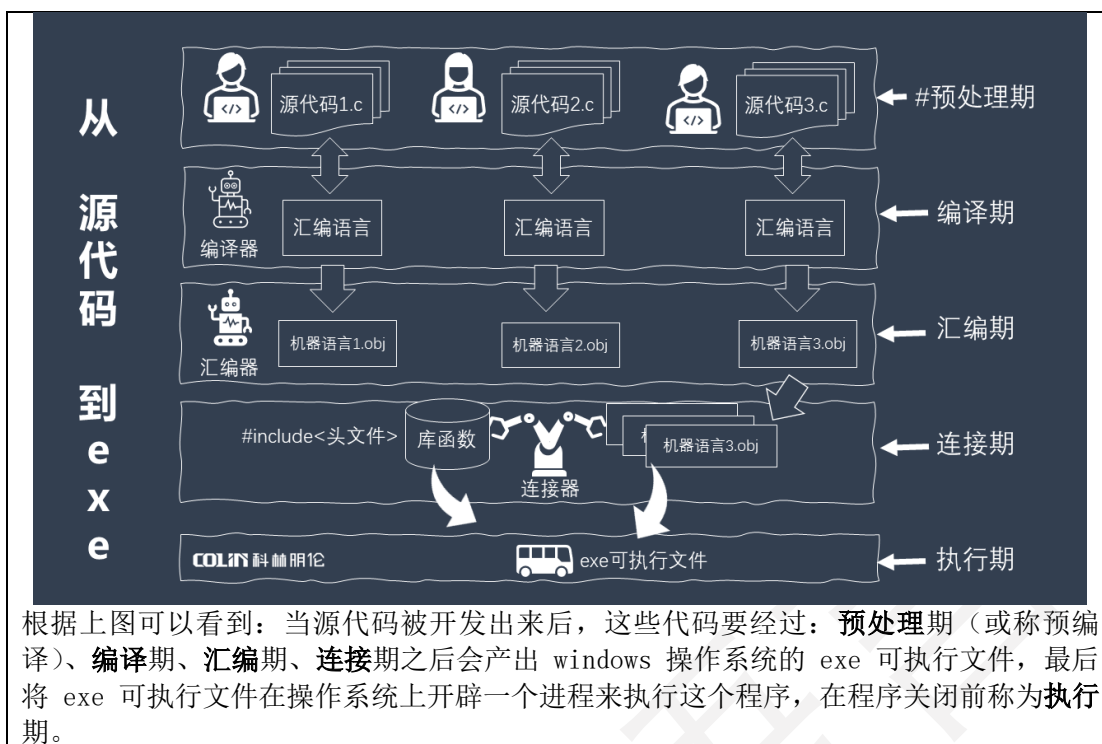
```
Microsoft Visual Studio 调试控制台
a=1
b=2
c=3
d=4
a=1 b=2 c=3 d=4
```

Part 4 源代码执行原理

24、 高级编程语言、汇编语言、机器语言的关系是什么？

所谓高级编程语言就是用英文单词+计算符号+数字构成的指令代码，经过学习和重复实践就能够掌握。我们把接近人类自然语言的计算机指令称为高级编程语言。我们目前常用的编程语言 C/C++、java、python 等都属于高级编程语言，甚至把数据库中的 SQL 语句称为“第四代语言”。因为它更接近人类的自然语言表达。高级编程语言的“高级”代表的是接近了人类，但却远离了机器。最终高级语言的每一条指令还是最终要变为 0 或 1 的机器语言的指令才能被机器执行。但是 0 或 1 组成的机器指令是很难被大多数人理解并学习的。所以人们研究开发出了汇编语言，汇编语言也称为机器语言的助记符号，也就是把 01 的指令用一个单词来代表，这样相对于机器语言更利于学习和使用。所以我们用 VS2019 编写的 C 语言源代码，计算机不会立刻能够执行它们，而是需要把它们最终变为 01 的机器指令后，再由计算机去执行机器指令。

25、 从源代码到 exe 需要经过哪几个时期？



26、 不同时期里的各个环节分工作用是什么？

第一步：源代码编写。



一个项目可以是一个人或多个人共同开发多个源代码文件。每个源代码文件可以包含多个不同的自定义函数，每个函数就是一个功能模块。我们把人编写源代码使用的编程语言成为：“高级语言”像 C++、java、C#、python 等。

所谓“高级语言”就是相对贴近人类的自然语言，但计算机能够识别并执行的是机器语言（即二进制指令）。更直白的讲我们的 C 语言源代码计算机是不能够直接执行的。它需要第三步**编译**的环节。

第二步：预处理（或称预编译）

预处理是在编译（第三步）之前对源代码继续“变形”的过程。也就是对所有#开头的指令进行处理。比如#include 会把被包含的文件复制粘贴进我们的源代码，#define 会根据宏名对源代码中宏名替换成对应内容……。

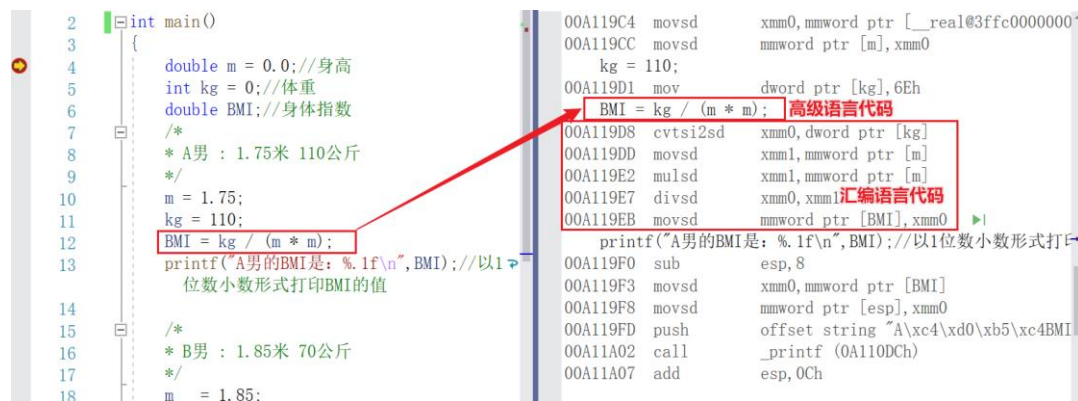
第三步：编译



我们编写的源代码需要一个编译器 Compiler（能够识别源代码语法的程序），对源代码进行词法分析、语法分析、优化等操作，最后生成汇编语言代码。当然如果在转换成汇编语言代码过程中由于语法不符合编译就会报出编译错误的提示；比如之前的语法

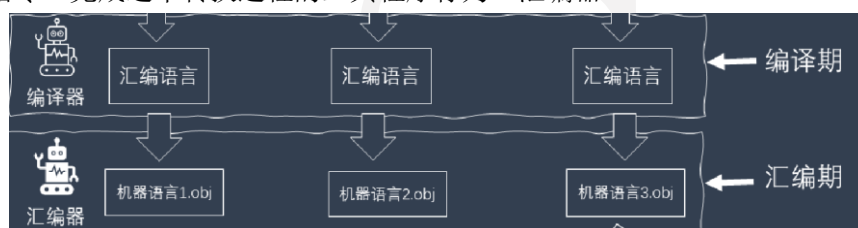
错误等。

什么又是汇编语言呢？为什么是汇编语言？所谓汇编语言：就是针对本台计算机 CPU 的二进制机器指令提供的一一对应的助记符号。因为机器指令是由二进制 0 或 1 构成的序列，不利于记忆和使用，因此人们为机器语言设计出的一一对应的辅助记忆的符号。也就是说：我们人类写的源代码如果能被编译器“解析翻译”成汇编语言代码，就一定能变成计算机能够识别的机器语言。利用 VS2019 调试器中的“反汇编”窗口可以浏览到每行源代码都会对应一条或多条汇编语言代码。

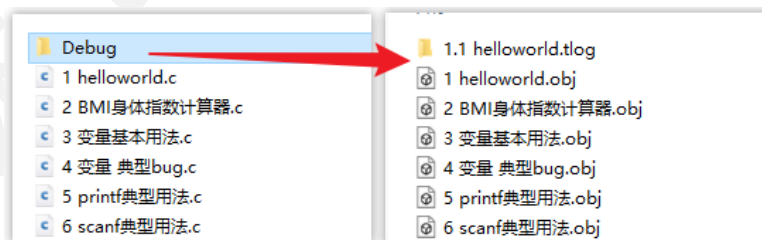


第四步：汇编

把编译器产生的汇编语言继续转换成机器语言的过程称为汇编。这个转换期间称为汇编期。也就是根据每个源代码文件都会自动生成一对一的一个文件名相同但不同扩展名的 .obj 或 .o 文件；我们称之为“目标文件”。即程序源代码最终目标是变为机器能够识别的指令。完成这个转换过程的工具程序称为“汇编器”。



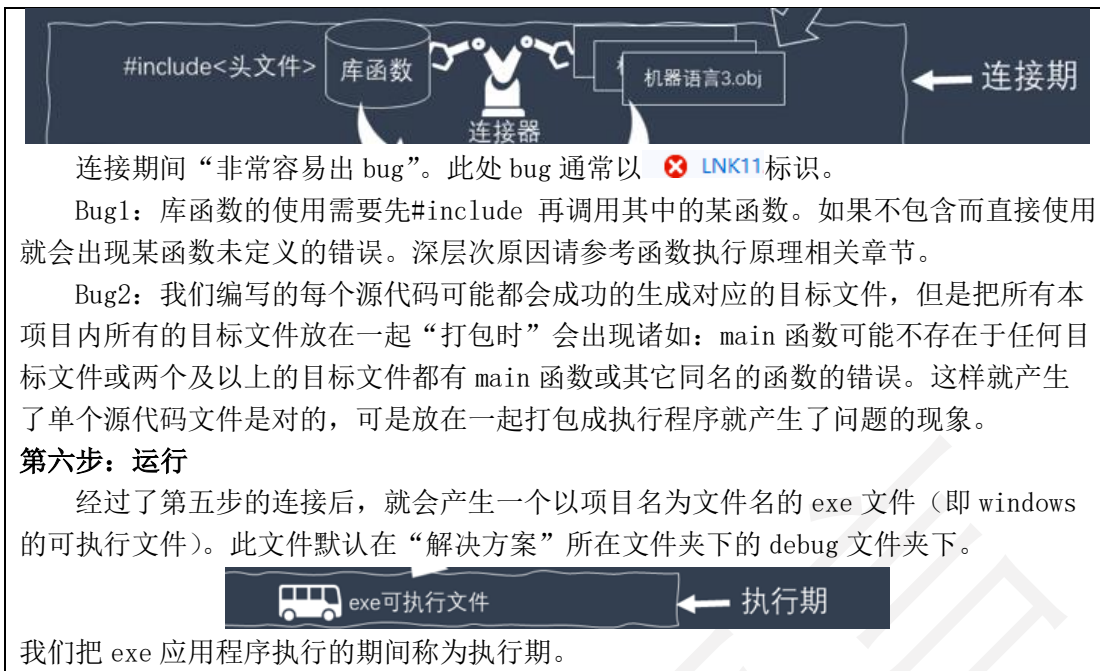
VS2019 中我们可以打开项目所在文件夹，并能够看到源代码文件。如果源代码被生成或执行后会产生 debug 文件夹；继续进入 debug 文件夹就能够看到与源代码文件匹配的 .obj 目标文件。



当然：如果某个源代码文件有语法错误没能经过编译期，则其没有对应的 .obj 目标文件。

第五步：连接

所谓的连接就是把第四步的目标文件及用到的某些库函数一同连接或理解为“打包”进一个 exe 文件的过程。整个期间称为连接期。完成这个任务的程序称为“连接器”——Linker。



27、 为什么会有语法错误的提示？

人们编写好源代码后，编译器会对源代码进行词法分析、语法检查及优化。如果是正确语法下的源代码是会被“翻译”成对应的汇编语言，反之错误的语法将无法翻译；这个过程就停止了。所以要求开发者进行代码的修正。一般报出语法错误尽量看错误列表中上面的提示，因为代码前面的语法错误会连锁反应到后面的一系列错误，所以前面改正了后面也许就全部正确了。所以“出错了，向前看”

28、 LNK... 提示的错误产生的原因有哪些？

LNK 提示的错误是在连接期内，连接器会根据头文件及代码中对某函数的调用去把对应的函数代码指令“打包”到我们项目的 exe 内。如果此时找到不到对应的代码就会报出某函数未定义的错误。此时要么没有包含对应的头文件（因为头文件中通常包含被调用的函数声明），或者通过声明找不到对应的函数定义。总之，找不到你调用的功能函数模块。

还有一种情况是我们所有的源代码经过了编译期排除了语法错误。汇编期生成了机器指令文件。但是当把它们汇总时可能发现 main 函数或其它函数有重名的情况。

29、 编程入门时期的 bug 是什么原因产生的？

刚刚接触编程时是很容易触犯一些“基础性的低级错误”的。但这是必须经历的一个过程。那么如何减少甚至避免一些语法类错误呢，从而让我们的重心放在逻辑算法上，想到既能做到呢？“原理不通、习惯不好”是多年接触初学者的一个总结。

所谓“原理不通”指的是每行代码背后的执行过程是什么，代码写完了计算机是如何处理的，能产生什么样的计算结果。很多人会用数学思维去理解套用，这样不完全对。很多的数学学习习惯在计算机执行时不完全一致。所以多站在计算机的执行原理角度理解代码在干什么，多试错，多总结。知己知彼，慢慢就可形成“人机合一”的思维方式及感觉了。

还有一个原因就是“习惯不好”。很多人写代码类似“抄写”。代码写的过程最好能够遵循计算机编程的底层思维和执行过程去写。比如：理解了模块思维就应该先把需要的库函数 `#include` 包含打开。理解了结构化思维就需要先把结构写好，比如成对的 `()`, `{}` 等；再回到结构的不同功能位置补充功能。这样逐层的写代码，站在执行原理角度写代码就会更准确。

Part 5 流程结构

30、 流程结构分几种？它们的思维逻辑是什么？

所谓流程就是完成一件事情的逻辑步骤及实现过程。比如刷牙、洗脸、待人接物、考公、考研。只要有行为或具体的动作，就是有一个过程来具体表现逻辑步骤。编程的过程就是把人们对一个具体行为的实现逻辑转移到计算机的过程，也就是把控制思维通过编程语言（计算机能够理解的指令）以软件的形式固化到计算机的过程。简单概括：**流程就是描述控制的具体操作过程。**

问题：**如何表达人类的控制思维？**

编程语言提供了一套控制的语法——流程结构。用以实现人们控制思维的表达。之所以称之为“结构”，就是语法提供了固定的“格式”；我们按照格式填写需要的条件或内容就可以实现控制目的——让机器完成人的思维意图。只要是控制逻辑都会落实成一个“动作/动词”。无论具体动作是什么多复杂，都可归为三个大的动作类型。

动作类型 1：步骤化的行为。比如刷牙。我们称为**顺序结构**。表达步骤先后的逻辑。

动作类型 2：根据条件判断结果后，再决定的行为。比如点餐的行为。我们称为**选择结构**。表达先判断再行动的逻辑。

动作类型 3：规律性重复的行为。比如跳绳比赛中“跳”的行为。我们称为**循环结构**。表达在满足条件范围内，不间断的重复一个行动。这三种动作类型（或称流程结构）可以任意的交叉组合（流程的嵌套结构），就可以描述出任意的复杂动作。

所以理解了流程的逻辑思维，再掌握与控制思维对应的语法结构，就可以编写出“听你话”的流程结构的程序了。

31、 多分支选择结构可以用什么语法表达？

方式一：**if else if else if...**结构

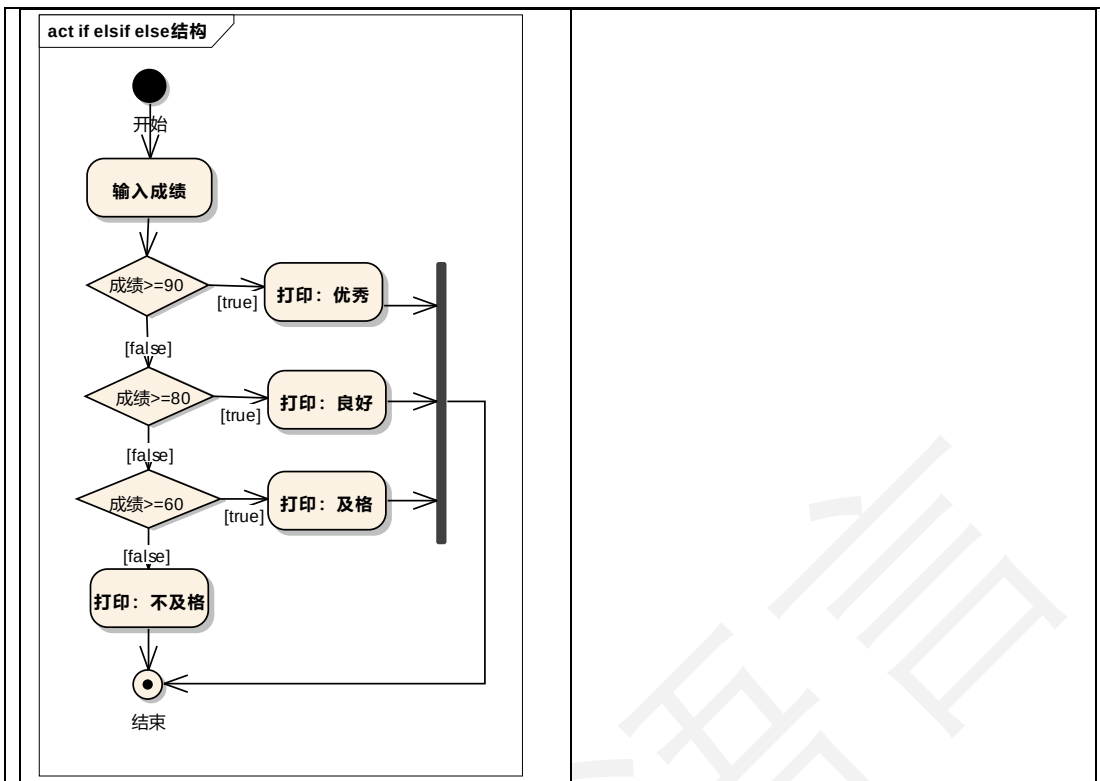
一种**多选一**的逻辑结构。也就是把多个不同的判断条件组织起来，数据只要满足其中一种判断即结束这个逻辑结构。

例如：根据得分判断对应的评价。成绩在 90 分以上优秀、90~80 分为良好 79~60 分为及格、低于 60 分为不及格。

逻辑流程图

代码实现

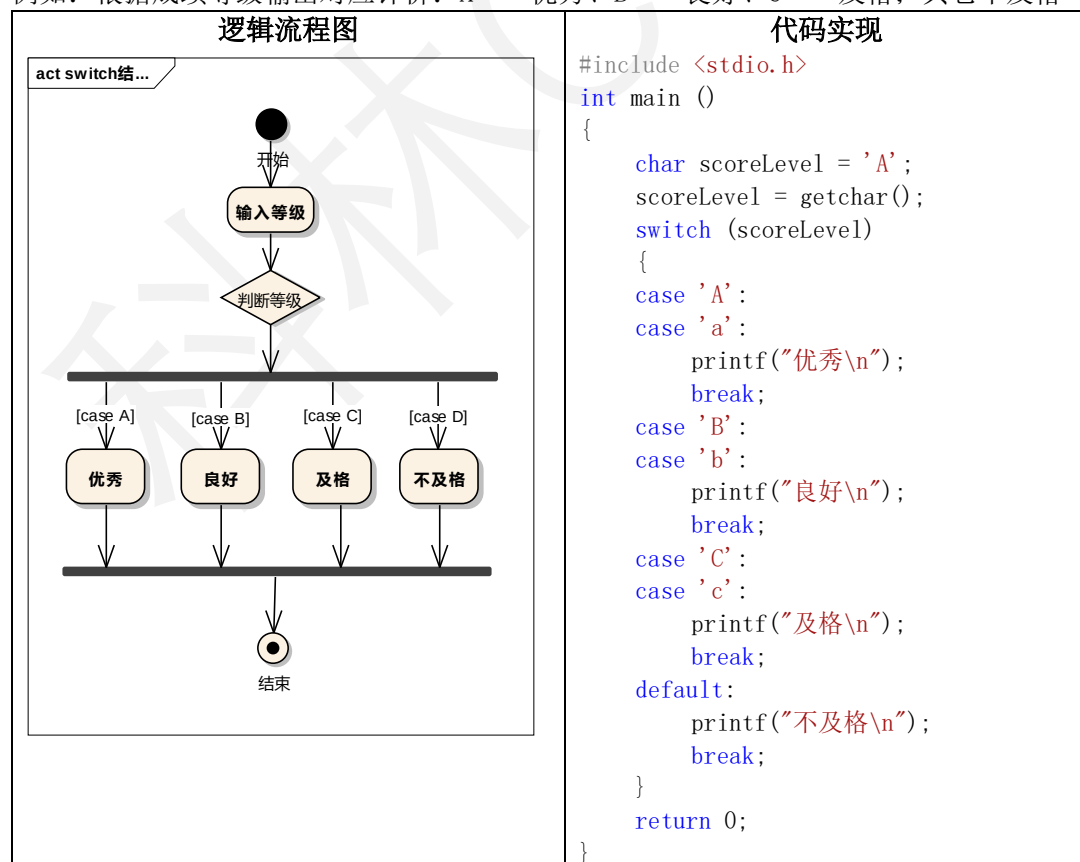
```
#include <stdio.h>
int main()
{
    int score = 0 ;
    scanf_s("%d",&score);
    if (score>=90){
        printf("优秀\n");
    }
    else if (score>=80) {
        printf("良好\n");
    }
    else if (score >= 60) {
        printf("及格\n");
    }
    else {
        printf("不及格\n");
    }
    return 0;
}
```



方式二: **switch** 结构

另一种**多选一**的逻辑结构。它根据 `switch` 后面的判断结果去匹配对应的处理逻辑。

例如: 根据成绩等级输出对应评价: A——优秀、B——良好、C——及格; 其它不及格



上面的例子中: `getchar()` 函数用于从键盘输入一个字符存入到 `scoreLevel` 变量中。

switch 关键字引导一个多分支的结构，先判断小括号中表达式的值，再利用 case 关键字匹配 switch 的计算结果。由于用户从键盘输入的可能是小写字母也可能是大写字母。因此代码中 `case 'A':` 表示的是无论 A 或 a 都做同样的处理，即打印“优秀”。break 关键字用于结束退出 switch 的多选一结构。

32、 C 语言有 bool（布尔）类型吗？其逻辑真与假用什么表示？

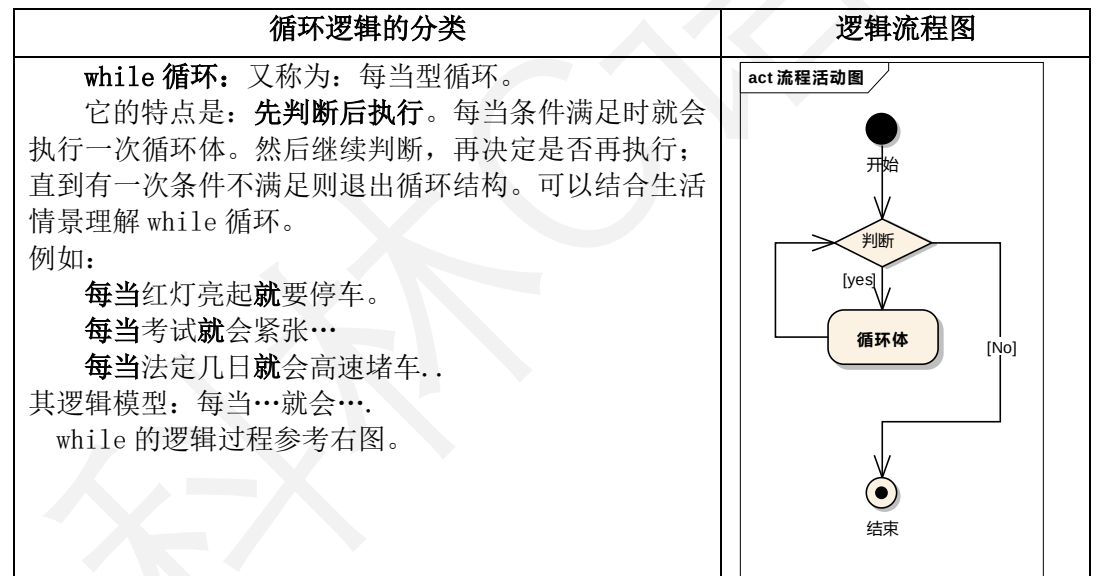
C 语言不具有很多编程语言都具有的 boolean 类型（即布尔类型或称逻辑类型）。boolean 类型通常有两个值：true 代表逻辑真，false 代表逻辑假。C 语言中的“替代”方法是用 int 代表 boolean 类型。非 0 的整数代表逻辑真，0 代表逻辑假。

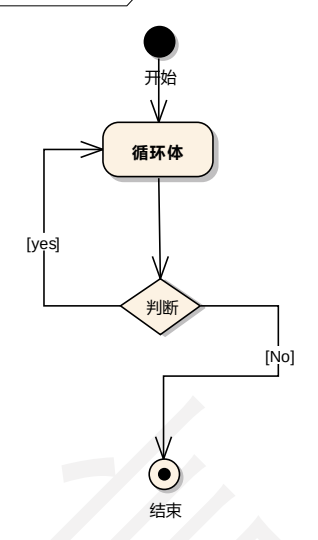
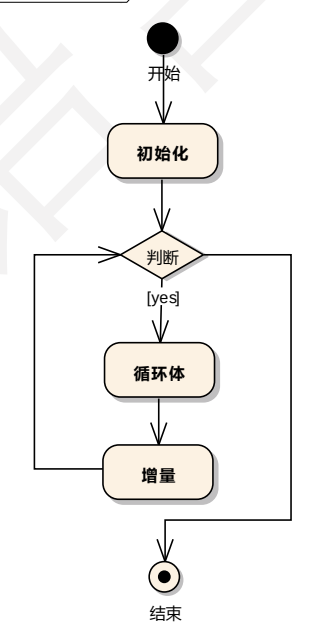
33、 switch 结构中，如果没有 break 语句会怎样？

break 作为 switch 结构的“出口”语句，在没有特殊情况下不要省略。如果只有 case 的入口，而没有 break 的出口，那么代码的执行会进入下一个 case 入口继续执行，直到遇到能退出 switch 结构的语句。

34、 while、do while、for 三种循环的思维逻辑有何差异？

什么是循环的逻辑？就是有规律的重复一个处理行为。C 语言中根据循环结构的逻辑特征提供了三种循环结构的语法，它们分别是：while 循环、do-while 循环和 for 循环。在某些需求方面它们也可以互相转换。



do while 循环：又称为直到型循环。 它的特点时：先执行一次，如果条件满足就继续执行。直到条件不满足就会退出此循环结构。结合生活场景理解就是各种尝试类的情形。 例如： 先试吃，味道可以再继续吃。 先试用期，能力与待遇双向匹配就继续工作。 其逻辑模型： 先行动一次…再判断…直到不满足 do- while 的逻辑过程参考右图。	act 流程活动图  <pre> graph TD Start((开始)) --> LoopBody([循环体]) LoopBody --> Decision{判断} Decision -- [Yes] --> LoopBody Decision -- [No] --> End(((结束))) </pre>
for 循环：更适用于明确重复次数的场景。 例如： 剪 10 个指甲 写 5 遍自己的名字 本科上 4 年 每周的周一 ~ 周五为工作日… 其逻辑模型是： 设置： 从哪里开始（初始值）； 到哪里结束（判断条件）； 每一次的变化规律（增量） 执行：每次重复做什么（循环体）。 其执行过程请参考右图。	act 流程活动图  <pre> graph TD Start((开始)) --> Init([初始化]) Init --> Decision{判断} Decision -- [Yes] --> LoopBody([循环体]) LoopBody --> Increment([增量]) Increment --> Decision Decision -- [No] --> End(((结束))) </pre>

35、 死循环是什么？用 while 和 for 举例

所谓死循环就是让循序永远重复的执行不退出。有时候死循环是不小心写的一个 bug 比如：

```

#include <stdio.h>
int main()
{
    int a = 10;
    while (a >= 0); //此处的 ; 是个 bug
    {
        a--;
    }
    return 0;
}
  
```

while 后面不小心写的分号其实是个“空语句”做了 while 的循环体。代码是不会

执行到 a—的。

在某些应用当中死循环是很有用的比如游戏框架中利用死循环不间断检测玩家在游戏中的是否有鼠标或键盘的操作，如若有的立即处理。如果是有意为之的死循环，那么死循环的代码通常如下：

```
while (1)
{
    ...
}
```

由于整数 1 是一个非 0 的数，在 C 语言中代表逻辑真，所以上面代码就是个死循环。另外 for 的如下写法

```
for ( ; ; )
{
    ...
}
```

for 中循序判断的部位如果为空置状态，则此时的 for 就是一个死循环。

36、 为什么说 for 循环是万能型循环？

for 循环的逻辑主要是面向明确循环起止及次数的场景。但是 for 循环的写法非常灵活。比如下面是标准 for 循环的用法：

```
for ( int i=0; i<5; i++)
{
    ...
}
```

我们可以把初始化的部位提出到外面：

```
int i = 0;
for ( ; i<5; i++)
{
    ...
}
```

也可以把增量部位放在循环体的里面：

```
int i = 0;
for ( ; i<5; )
{
    i++;
    ...
}
```

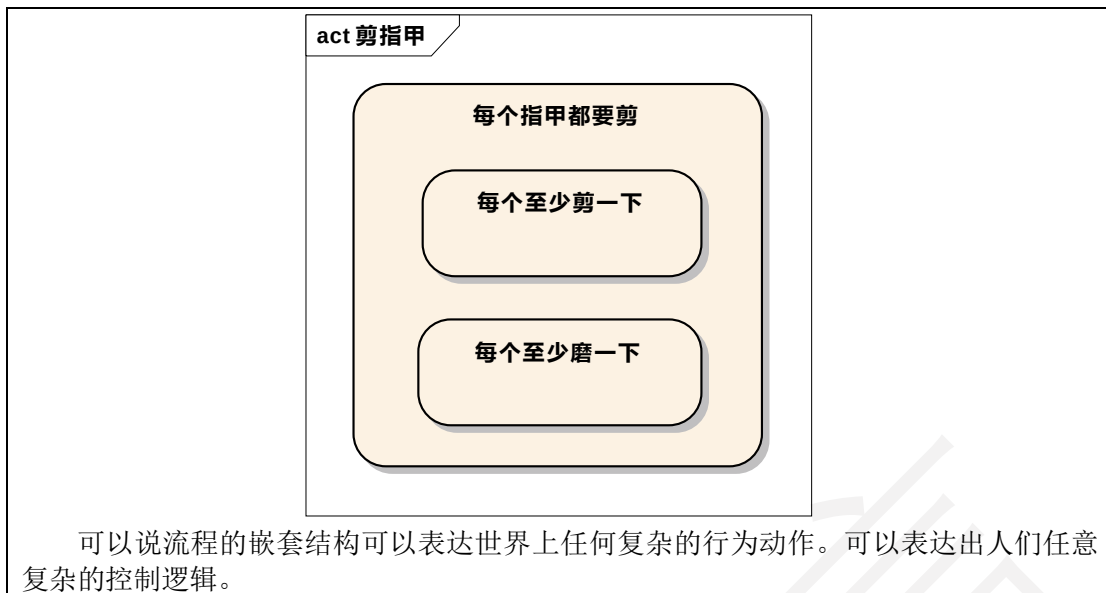
这样的逻辑功能就和 while 循环的功能一致了。

如果有多个循环变量还可以用逗号连接不同变量的变化表达式。例如：

```
for (int i=0, j=5 ; i<j; i++, j--)
{
    printf("i=%d j=%d\n", i, j);
}
```

37、 什么是流程结构的嵌套结构？

流程的嵌套结构就是流程内部可以还有子流程。也就是选择结构里面有选择或有循环；循环体里面还有子循环。比如：剪指甲的逻辑。



38、 break 语句与 continue 语句的区别？

break 除了在 switch 结构中应用外，还可以在循环中使用。其作用是提前结束所在循环的执行。例如：

```
for (int i=1 ;i<=10 ;i++ )
{
    if (i%5==0)
    {
        break;
    }
    printf("%d\n", i);
}
```

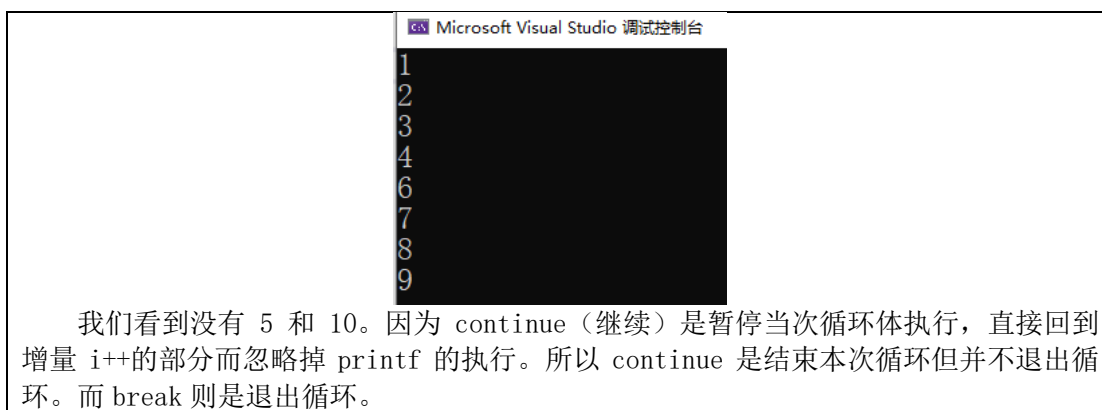
Microsoft Visual Studio 调试控制台

```
1
2
3
4
```

此时 break 和 if 条件语句配合, 当 i 是 5 时提前退出了 for 循环。所以结束循环的方式就变成了两种。一种是正常结束循环, 如 $i \leq 10$ 的相反逻辑。另一种是中途结束, 即 if 与 break 的配合。

如果把上面例子中的 break 替换成 continue 会怎样呢？

```
for (int i=1 ;i<=10 ;i++ )
{
    if (i%5==0)
    {
        continue;
    }
    printf("%d\n", i);
}
```



39、 调试器的作用及使用步骤？

调试器（debugger）是开发软件项目必不可少的工具。可以说善于用调试器找 bug 的程序员是悲哀的。在入门阶段调试器可以帮助我们理解代码的执行原理。在做项目时它可以方便快速的帮助我们定位 bug，修正 bug。

VS2019 中使用调试器的步骤如下：

1、设置断点 ：在行号前面的灰色处点击可设置或取消断点。学习起步阶段断点可以设置在代码的第一行，实际开发时断点设置在可能出错的代码位置。

2、启动调试器执行程序： 本地 Windows 调试器 此时代码的执行位置就停留在了断点上  6

3、点击  或按 F11 快捷键进行单步逐语句执行：可以观察出流程结构的执行顺序。

另外：在调试状态下点击“调试”->”窗口”可以打开不同的窗口以多个角度查看代码在执行过程中的信息情况。



4、点击  可以停止调试。点击  可以重新调试。点击  继续(C) 代码会继续向下执行后续的代码，如果后面没有其它断点了，本项目代码会被执行完，如果有其它断点，则执行指向就会、停留在下一个断点上。

【注意】在调试状态下不要做任何代码编写或项目文件的操作。当代码没有问题

时，一定要停止调试后再做其它 VS2019 的操作。

Part 6 运算符与表达式

40、 C 语言的表达式与语句的区别是什么？

学习好 C 语言一定要区分表达式与语句的区别。

C 语言的语句可以用“;”号构成单条语句或一对 { } 构成复合语句。

例如：168; 是一条字面值常量语句。

3+4; 是一条加法语句。

a++; 或 ++a; 是一条 ++ 语句。

a = 3+4; 是一条赋值语句。但是在赋值操作之前，需要把 3+4 这个加法表达式优先计算出结果 7，然后才能执行“=”这个赋值操作。此时的 3+4 就是一个表达式。

只要被确定是一条语句内的表达式，就一定要会分析这个表达式的计算结果。而 a++; 这是一条 ++ 语句，就不需要计算分析 a++ 表达式的结果。

例如：b=a++; 这里 a++ 就是一个 ++ 的表达式，其计算结果要存入 b 变量。此时就必须能分析出 a++ 这个表达式的值是什么。

【注意】 a++ 表达式与 ++a 表达式的计算结果是不同的。

41、 % 运算符的作用？ -9%-2 的计算结果如何分析？

“%”是求余数运算符又称“取模”运算。如果分析负整数求余数计算结果时，建议可以先进行整除计算，然后用“被除数-商*除数”的方法就可计算出余数。所以分析 -9%-2 的余数时首先计算 -9/-2，整除后得到 4，然后计算 -9-(4*-2) 得到余数 -1。

可以用此方法计算分析如下代码的执行结果。

```
printf("%d.....%d\n", 9 / 2, 9 % 2);
printf("%d.....%d\n", -9 / 2, -9 % 2);
printf("%d.....%d\n", 9 / -2, 9 % -2);
printf("%d.....%d\n", -9 / -2, -9 % -2);
```

42、 7/2 与 7.0/2 的计算区别是什么？

7/2 的计算结果是 3，而 7.0/2 的计算结果是 3.5。主要原因是 7 和 2 是整数，在计算机里以整型存储原理对其保存，而 7.0 虽然在数学角度看和 7 没差别但是在计算机中它将以浮点型存储原理来保存。所以当计算 7/2 时，由于两个数都是整型存储计算结果就是整除（没有小数）。而 7.0/2 由于 7.0 是浮点型存储 2 是整型存储，计算机按自动类型转换规律将整型的存储形态向浮点型存储形态进行转换，所得最终结果就是浮点型数据，所以 7.0/2 会得到具有小数的结果 3.5。

43、 复合赋值运算符有哪些？赋值表达式的计算结果如何分析？

复合赋值运算符就是把一个计算符号和赋值号组合成一个新的运算符号。

和算术运算符结合的有：+=、-=、*=、/=、%=。

和位运算符结合的有：<<=、>>=、&=、^=、|=。

它们的共同特点都是：先把算术运算或位运算的结果首先计算好后再赋值回变量保存结果。赋值表达式的计算结果是左侧变量的最终值。例如：

```
int a, b, c, d;
a = b = c = d = 1;
printf("%d %d %d %d\n", a, b, c, d);
```

如何正确分析上例中 a = b = c = d = 1; 的计算过程与结果呢。在分析混合计算结果时，按照“先看优先级、优先级相同再看结合性”的原则，因为三个“=”是相同优先

级的，所以需要查看结合性；其结合性是从右向左，因此d=1这个赋值表达式先执行。既然是个表达式，那么这个表达式的值是什么呢？赋值表达式的值就是左值（即赋值号“=”左侧变量空间最后存储的值）本例中就是变量d的值——1。再下一步计算c=1这个表达式，其表达式值是变量c的最终值——1。依次类推。所以我们看到的效果就是变量a、b、c、d都被赋予了相同的值——1。再例如：

```
int m = 5;
m += m * 2;
printf("%d\n", m);
```

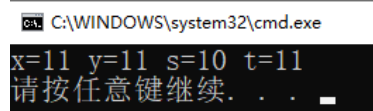
上面代码中m的最终计算结果是20。由于+=和*=是相同优先级，因此按照结合性的要求m*=2这个表达式先被计算执行；即m自乘2后m变量的值是10，m*=2作为赋值表达式，变量m的当前值10就是赋值表达式的值。然后再计算的表达式m+=10。此刻变量m的值是10，再自增加10最终结果就是20。

44、 i++表达式与++i 表达式的计算结果有何不同？

先看如下代码的计算结果：

```
int x = 10, y = 10;
int s = x++;
int t = ++y;
printf("x=%d y=%d s=%d t=%d\n", x, y, s, t);
```

执行结果如下：



我们可以看到 x++与++y 这两个表达式的写法不同，根据打印结果可知：无论++运算符在前或在后，参与计算的变量 x 和 y 都自增了 1。但作为++表达式的值则完全不同。

区别在于当++运算符在后而变量名在前时(如 x++)，其规则是把变量 x 的当前值作为++表达式的值被返回，然后变量 x 再自增 1。所以 s 变量保存的是自增前的 x 的值。而++y 这个表达式计算时，++在前就先进行自增 1，然后把增加 1 后的变量 y 的值作为表达式的值返回；因此 t 得到了 11。--运算符也是如此。

45、 int a = 5, b = 5; a+= a++ + ++b; 变量 a 和 b 的值分别是多少？

如何分析此计算类结果？建议熟悉不同运算符的优先级，优先级相同再看结合性。用这个顺序划分出表达式，再继续分析不同表达式的计算结果最终分析出每个变量的值。

以本题为例：变量 a,b 的初始值都是 10。a+= a++ + ++b;这条语句中包含了+=、+、++三个运算符，按照优先级查看，它们是不同的优先级。所以构建表达式的计算顺序：

- 1、a++：a 在++之前，变量 a 的当前值作为表达式的值然后再自增加。结果 a++ 表达式的值是 5，但变量 a 的值是 6。
- 2、++b：由于++在前，所以 b 先自增 1 后其值为 6，6 也作为++b 表达式的值被返回。
- 3、+运算：即左侧 a++的表达式值加上右侧++b 表达式的值，最终得到 5+6 为 11。
- 4、a+=11：a 变量的当前值是 6，+=是在 a 的基础上增加 11，最终变量 a 的值是 17，变量 b 的值是 6。

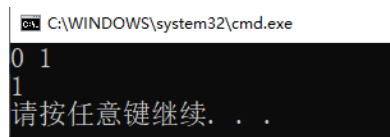
46、 关系运算符与逻辑运算符都有哪些？其表达式计算结果是什么？

关系运算符包括：>大于、>=大于等于、<小于、<=小于等于，和==等于、!=不等于

两大部分。前一部分的优先级高于后一部分。在混合运算时注意区分。例如：

```
printf("%d %d\n", 3 > 5, 3 < 5);
printf("%d\n", 3 > 8 == 5 < 4);
```

代码打印结果如下：



根据执行结果可以看到：关系表达式的计算结果只有 1 和 0。其中 1 代表逻辑真而 0 代表逻辑假。我们再看第二个关系表达式，由于 == 与 > 和 < 是不同的优先级，所以 3>8 先被执行得到 0（逻辑假）然后 5<4 被执行得到的结果还是 0；最后计算 0==0 得到的结果是 1（逻辑真）。

47、 && 与 || 的区别是什么？什么是短路问题？

&& 是逻辑与运算符。代表是并且的逻辑意思。&& 两边都是真（非 0）结果才是真；有一侧结果是假（0），&& 逻辑与的计算结果就是假。类似我们生活中买一张火车票：身份证与人民币缺一不可，它们之间的关系就是 && 逻辑与的概念。

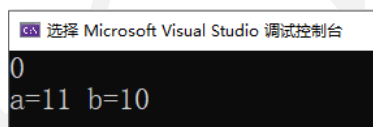
|| 是逻辑或运算符。代表“或者”的意思。|| 如有一侧为真则逻辑或表达式的结果就为真。只有当两侧都为假是，|| 表达式的结果才是假。

【注意】&& 是比 || 高一个优先级的运算。混合运算时注意优先级。

短路问题通常指逻辑运算符 && 和 || 的计算特点。观察如下代码的计算结果。

```
int a = 10, b = 10;
printf("%d\n", a++ > 10 && b-- <= 10);
printf("a=%d b=%d\n", a, b);
```

执行结果为：



我们看到虽然 ++ 和 -- 的优先级高于 &&，但是变量 b 并没有变化还是初始值 10。说明 b-- 这个表达式没有被执行。&& 的逻辑是有假则假，全真则真；其在计算时如发现左侧为假那么右侧的表达式就不会被执行。通常把这种选择性执行某部分表达式的行为称为“短路问题”。其逻辑特点可以用如下的电路关系表示：



|| 逻辑或也是存在短路问题的。特点是如果左侧为逻辑真，那么右侧的表达式则不被执行。因为无论执行结果是什么最终逻辑与表达式的结果都是真。只有左侧计算为逻辑假时才会去执行右侧的表达式。

48、 位运算中 & | ^ 的优先级从高到低的顺序是什么？它们有哪些特性。

位运算就是二进制 01 进行处理的运算。其中 &（位与运算符）的优先级高于 ^（异

或运算符)，而 $\wedge$ 异或的优先级高于 $|$ （位或运算符）。下面举例说明它们的计算特点。

& 按位与运算

例如：

```
printf("%d\n", 10&5);
printf("%d\n", 0b1010
    &0b0101);
```

十进制的 10 经过原码、反码、补码后在计算机内部的存储就是 00001010（以一个字节 8 个位为例）。5 的存储是 00000101。 $\&$ （按位与）的计算特点是二进制位对齐后，上下位（按全 1 则 1，有 0 就 0 的规则）进行计算，最终得到的结果就是 00000000。所以打印结果就是 0。理解了位运算的计算过程就可以直接用二进制方式设置字面值常量了，这样就更容易分析计算结果。

| 按位或运算

根据对 $\&$ 按位与的理解，按位或的特点是“全 0 则 0 有 1 则 1”。所以上面案例换成 $|$ 按位或后：

```
printf("%d\n", 10|5);
printf("%d\n", 0b1010
    |0b0101);
```

其二进制的计算结果是 00001111，用十进制打印就是 15。

$\wedge$ 按位异或运算

按位异或的计算特点是“相同为 0 不同为 1”。所以上面案例换成按位或后

```
printf("%d\n", 10^5);
printf("%d\n", 0b1010
    ^0b0101);
```

二进制的计算结果是 00001111，用十进制打印也是 15。

请注意： $\&$ $\wedge$ $|$ 是不同的优先级的运算符，所以它们混合运算时要注意优先级。

例如：

```
printf("%d\n", 10|9&8^-1);
```

分析此类位运算表达式的计算结果的过程如下：

把操作数变为二进制形态	(10 的二进制) 00001010 (9 的二进制) 00001001 (8 的二进制) 00001000 (-1 的二进制) 11111111
$\&$ 的计算	(9) 00001001 (8) 00001000 $\&$ 后: 00001000
$\wedge$ 的计算	(-1) 11111111 $\wedge$ 后: 11110111
$ $ 的计算	(10) 00001010 $ $ 后: 11111111

最终以 $\%d$ 有符号十进制打印的数据为-1。

49、 位运算的 $\ll$ 和 $\gg$ 是什么？另外 $\gg$ 时正负数如何处理？

$\ll$ 左移运算符

即二进制数向左移动若干位，前面被移出的数据被舍弃而后面补充 0。例如：

```
printf("%d\n", 10<<1);
printf("%d\n", 10<<2);
```

其计算过程是：10 的二进制为 00001010，左移 1 位后为 00010100。换算成十进制打印就是 20。所以对于整数左移一位的运算结果相当于乘以 2。

$\gg$ 右移运算符

对于非负数右移补 0。但对于负数右移会补 1。例如：

```
printf("%d\n", 10>>1);
```

```
printf("%d\n", -1>>5);
```

其计算过程是：10 的二进制为 00001010，右移 1 位后为 00000101。换算成十进制打印就是 5。所以正整数右移 1 位相当于整除 2。

而计算机处理 -1 时，经过原码、反码、补码处理后其结果是 11111111（即全 1）。其中左侧开头的 1 代表符号位即负数（如果是 0 则代表正数）。此时无论向右移动多少的二进制位都会补充 1。最终结果还是十进制的 -1。

50、 条件（或称三目）运算符是什么？举例说明作用。

条件运算符是唯一的三目运算符，即其表达式需要三个操作数。其格式为：

判断条件 ? 非 0 的处理 : 0 的处理

其逻辑就是是一个简化版的 if else 选择结构。举个例子：

//取得两个整数的最大值

```
int a = 3, b = 5;
```

```
int max = a > b ? a : b;
```

//取得一个整数的绝对值

```
int x = -9;
```

```
int abs = x >= 0 ? x : -x;
```

由上面代码可以看到条件运算符的执行过程是：先判断“?”号前的表达式的值，如果判断结果为非 0（即真）就把“:”左侧的表达式计算结果返回，如果判断结果为 0（即假）时，就把“:”右侧表达式计算结果返回。

51、 sizeof 是函数还是运算符？其作用是什么？

sizeof 是 C 语言保留字（或称关键字），并不是函数；只是使用时的写法和函数调用很像。它的作用是得到一个数据类型或一个表达式计算结果的字节数。由于不同的编译器及版本对数据类型支持的字节数可能会有差异，所以用 sizeof 运算符来获取当前编译器下字节数的大小更准确，代码的移植性会更好些。

52、 逗号运算符的作用及计算结果是什么？

“,” 逗号运算就是用英文逗号链接起来多个表达式从而形成的一串表达式。逗号表达式也是有计算结果的，那就是最后一个（或最右侧）表达式的计算结果将代表整个逗号表达式的结果。例如：

```
int a;
```

```
a = (1, 2, 3, 4, 5);
```

```
int x, y, z;
```

```
z = (x = 5, ++x, y = x++);
```

```
printf("%d %d %d\n", x, y, z);
```

上例中由于逗号运算符的优先级最低，所以要想保存逗号表达式的值必须要用（）提高其优先级。变量 a 的最终值是 5，即最后一个字面值常量 5 的值代表了整个逗号表达式的值被返回。下一个逗号表达式的执行过程是：x 赋值为 5，++ x 使得 x 变为 6，x++表达式的值是 6 但 x 的值是 7；因此 y 变量的值是 6，最后 y 变量的值作为逗号表达式的值返回，再存入到 z 变量中。

53、 -、 !、 ~ 举例说明三种具有“反”功能的作用？

“-”号如果仅有一个操作数时它的作用就是取得这个操作数的相反数。相反数的计算规则是从右向左计算。例如：printf("%d\n", - - - 8);先计算8的相反数。注意：-与8中间有一个空格。这个是计算相反数运算。得到-8，再取-8的相反又得到8……。

相反数的二进制实现原理是先反码，再补码。例如：

8 的二进制：0000 1000

8 的相反数过程：先反码 1111 0111 再补码 1111 1000 得到的是 -8。

-8 的相反数过程：先反码 0000 0111 再补码 0000 1000 得到 8

! 的作用就是取一个整数的相反逻辑值。printf("%d %d %d\n", !9, !0, !- 9);
!9 表达式的值就是 0。即 9 代表的是逻辑真!9 就是逻辑真的相反逻辑即逻辑假(0)。

~是按位取反运算符。即每个二进制位（无论是否是符号位）都取相反的二进制数：即 0 变 1、1 变 0。例如：printf("%d %d\n", ~0, ~-1); 由于 0 的二进制为全 0，-1 的二进制（经过原码、反码、补码后）为全 1。~0 的结果就是把所有 0 取反后就变成了全 1。

~-1 就是把所有二进制的 1 全部取反变为 0。所以输出结果为：-1 0

笔试真题

编程题

科林 C 语言

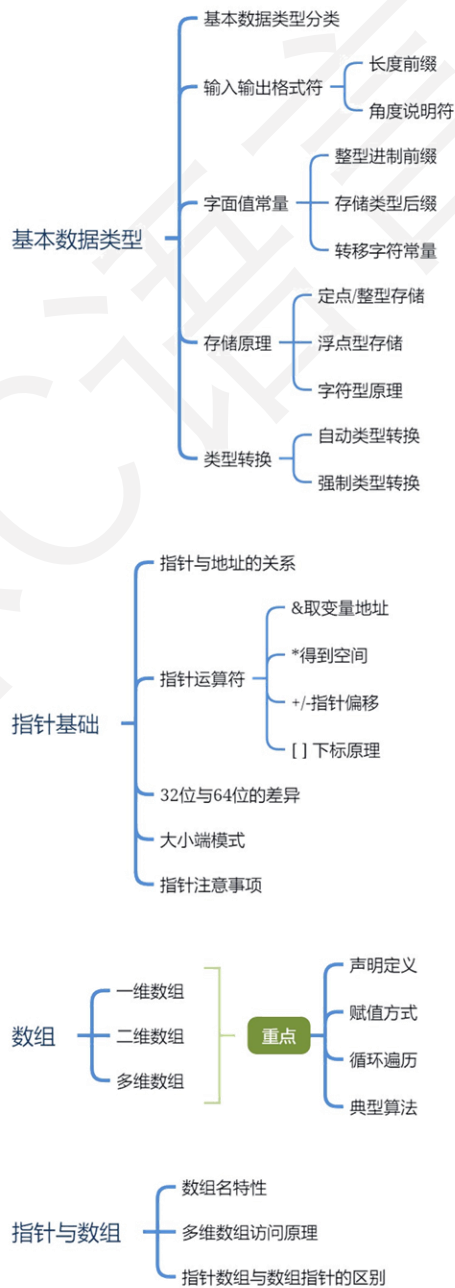
编程能力二：数据存储

能力介绍

数据存储能力就是掌握 C 语言基本数据类型、指针及指针扩展、复合类型或称扩展类型（数组、结构体、联合体、枚举等）的存储原理、字面值表示及使用思维。能够利用不同的数据类型准确描述现实世界的信息构成。充分理解强类型语言的类型特性。

相关技术构成

数据存储能力 相关技术体系



学习技巧建议：

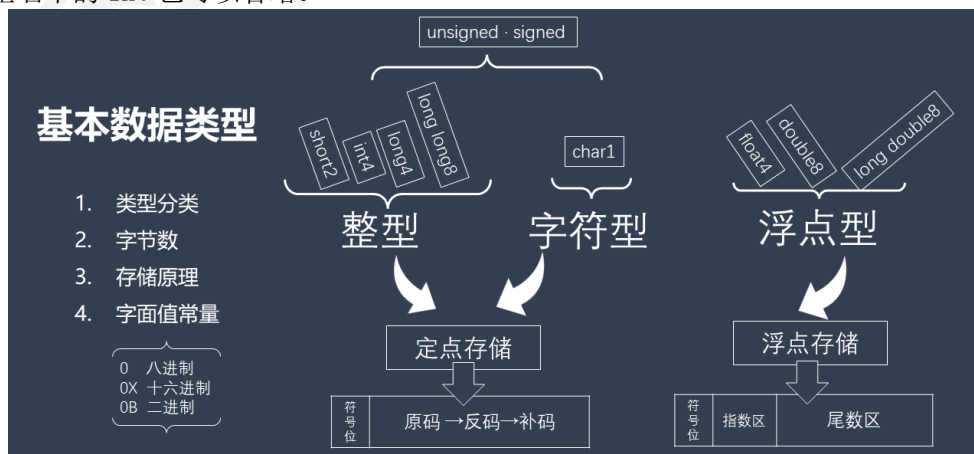
- 1、扎实记住基本数据类型的分类与其存储原理的差异。
- 2、理解指针变量使用的基本模式——找到地址、得到空间。
- 3、多用调试器的内存窗口查看数据的实际存储与指针偏移。
- 4、典型的数组算法要在理解的基础上多重复默写。
- 5、指针的理解要在熟悉概念的基础上多“刷题”

Part 1 基本数据类型

- 1、基本数据类型的分类有哪些？整体介绍一下。

COLIN 科林 C 语言基本数据类型						
	类型关键字	类型名称	字节数	数据范围	字面值后缀	打印格式
1	signed char	有符号字符型	1	-128~127	-	%c
2	unsigned char	无符号字符型	1	0~255	-	%hu
3	signed short int	有符号短整型	2	-32768~32767	-	%hd
4	unsigned short int	无符号短整型	2	0~65535	-	%hu
5	signed int	有符号整型	4	-2147483648~2147483647	-	%d或%i
6	unsigned int	无符号整型	4	0~4294967295	U或u	%u
7	signed long int	有符号长整型	4	-2147483648~2147483647	L或l	%ld
8	unsigned long int	无符号长整型	4	0~4294967295	UL或ul	%lu
9	signed long long int	有符号长长整型	8	-9223372036854775808 ~9223372036854775807	LL或ll	%lld
10	unsigned long long int	无符号长长整型	8	0~18446744073709551615	ULL或ull	%llu
11	float	单精度浮点型	4	3.4e-38~3.4e38	F或f	%f或%e
12	double	双精度浮点型	8	1.7e-308~1.7e308	-	%f或%e
13	long double	长双精度类型	8	1.7e-308~1.7e308	L或l	%f或%e

基本数据类型从大的应用上分为：整数类型（整型）、浮点数类型（浮点型）和字符型。它们分别应对生活中的整数、实数和文字。根据上表可以看到不同类型都有对应的类型关键字，其中暗色是可省略的。比如 signed long long int 是该类型的全名，编程时用 long long 这个简写即可。有符号关键字 signed 可以省略，需要 int 配合的类型名中的 int 也可以省略。



所有基本数据类型从存储的原理角度分为定点存储（或称整型存储）和浮点存储。由于字符的存储本质是固定的整数编码，编码也是以整型方式存储；所以从存储原理角

度字符型与整型的存储原理是相同的。另外不同类型的字节数不同，它们表达的数据范围也不同，字节数越大可表示的数据范围也越大。所以在实际使用时要根据程序运行的软硬件环境和实际问题选择更适合的数据类型创建变量。

2、计算机中的字节是什么？

我们目前使用的计算机为二进制计算机。任何信息(文字、数字、图片、视频、程序等)最终采用 0 或 1 进行处理。为了更好的管理 0 或 1 的数据量，人们会把 8 个二进制位编组为一个字节 (byte, 简称 B)。字节也被称为计算机最小的存储单位。又因为 2 的十次方等于 1024，它接近我们日常生活中的 1 千，所以把 1024 个字节就称为 1KB (称为：千字节)。为了标识更大的存储空间数量，又把 1024KB 称为 1MB, 1024MB 称为 1GB。依次类推还有更大的存储单位：1TB、 1PB ...

3、十进制转二进制的数学计算方法是什么？

计算机的底层是二进制的，而我们生活中一般是十进制的。所以开发者要掌握不同进制转换的计算方法。其中：二进制与十进制的转换方法如下：

十进制转二进制：整数部分除二取余，小数部分乘二取整

例如：56.125 转为二进制：

整数部分	小数部分
56 ÷ 2 = 28.....0	0.125×2 = 0.25 取整 0
28 ÷ 2 = 14.....0	0.25×2 = 0.5 取整 0
14 ÷ 2 = 7.....0	0.5×2 = 1.0 取整 1
7 ÷ 2 = 3.....1	
3 ÷ 2 = 1.....1	
1 ÷ 2 = 0.....1	
最终整数结果：111000	最终小数部分结果：001

即：56.125 转二进制的结果为：111000.001

反之：二进制转十进制的方法如下：

整数部分： $1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 32 + 16 + 8 + 0 + 0 + 0 = 56$

小数部分： $0 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 0 + 0 + 0.125 = 0.125$

4、为什么计算机还要有八进制与十六进制的表示呢？

八进制的数字范围：0 1 2 3 4 5 6 7。

十六进制的数据范围：0 1 2 3 4 5 6 7 8 9 A B C D E F。

因为 3 个二进制位的数据范围：000~111 按十进制换算：0~7 正好是 1 位八进制的数据范围。所以八进制的字面值设置可以更方便口算把八进制转为二进制：即 1 八进制位相当于 3 进制位。

例如：八进制 123 转为二进制：001 010 011

按上面的方式推导：4 个二进制位的数据范围：0000~1111 按十进制换算是 0~15。其正好是 1 位十六进制数的数据范围。所以 1 个十六进制位就是相当于 4 个二进制位。

例如：十六进制 5A 转为二进制就是：0101 1010。

另外：一个字节是 8 个二进制位，所以可以用 2 个十六进制数表示 1 个字节的数据情况。通常在表示内存中每个字节的存储数据时默认用十六进制表示。

内存 1	
地址:	0x006FF8E0
0x006FF8E0	11 00 00 00 cc cc cc cc 08 f9 6f 00 c3 1e 22 00 01 00 00 00 d0
0x006FF8F5	54 95 00 a0 97 95 00 01 00 00 00 d0 54 95 00 a0 97 95 00 64 f9
0x006FF90A	6f 00 17 1d 22 00 d1 f1 cb d2 23 10 22 00 23 10 22 00 00 b0 55
0x006FF91F	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x006FF934	00 00 00 00 00 00 00 00 00 00 00 00 7c a5 22 00 88 a5 22 00 00
0x006FF949	00 00 00 10 f9 6f 00 00 00 00 00 d0 f9 6f 00 40 37 22 00 65 98

5、整型数的存储原理是什么？

1、符号位：

首先把整数数据变成二进制。但是如何表达一个数字是正数或负数呢？

由于二进制计算机只有 0 或 1 两种物理表示，因此规定：用一个二进制位单独代表符号——用二进制 0 代表正数，二进制 1 代表负数。如果让计算机表达正整数 5 就会有如下的表示：5 的二进制为 101；5 默认为 int 类型需要 4 个字节的存储空间。最左侧为符号位，0 代表正。所以如下表格就是 5 在计算机中的表示形式。

00000000	00000000	00000000	00000101
----------	----------	----------	----------

我们把上述的表示手段称为：原码。

2 原码：

就是符号位加上整数的绝对值的组合。比如要想表示-5。则是

10000000	00000000	00000000	00000101
----------	----------	----------	----------

其实计算机中最终保存的并不是原码，原码仅仅是个处理过程的一步。下一步是反码操作。

3 反码：

所谓反码（按位取反的简称）就是每个二进制位变为相反的值：0 变 1 或 1 变 0。

但是：正整数与负整数的反码处理是不同的。正整数其实是不做反码处理的，也可以说正整数的反码是原码本身。但负整数是要处理的，其中符号位的 1 不参加反码处理，符号位以外的各个位都要按位取反。所以上面-5 的原码转变成反码如下：

11111111	11111111	11111111	11110101
----------	----------	----------	----------

此时计算机还要再做最后一步的处理——补码。

4 补码：

如果是正整数，那么补码还是原码本身。可如果是负整数则在上一个（反码）的基础上再补加一个二进制的 1。

11111111	11111111	11111111	11110101
			+1
11111111	11111111	11111111	11110111

最终计算机存储起来的是补码后的结果。

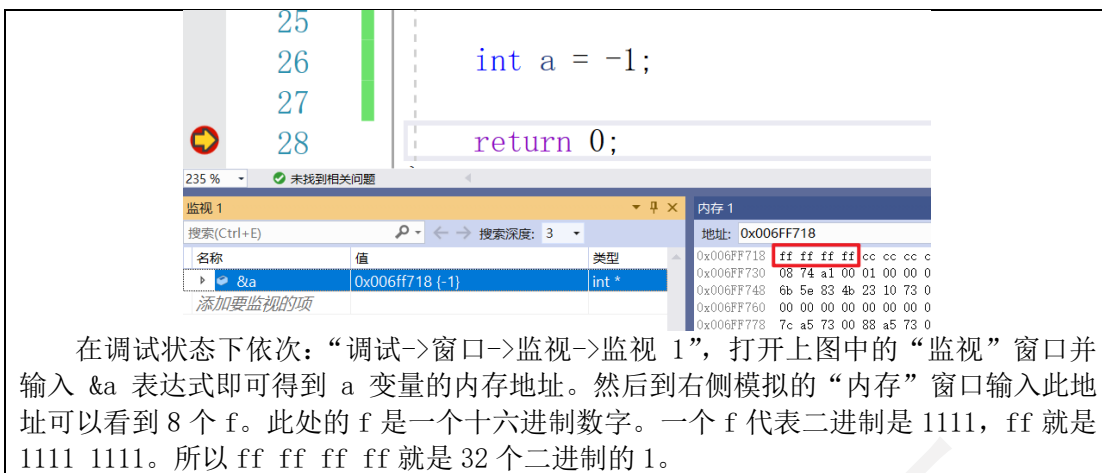
6、-1 的二进制存储过程是什么？

-1 的内存存储处理过程如下：

原码： 1 代表负、0 代表正	10000000	00000000	00000000	00000001
反码： 符号位不动，其余按位取反	11111111	11111111	11111111	11111110
补码： 在反码基础上补加 1	11111111	11111111	11111111	11111111

我们可以看到生活中“小小的”-1 在计算机二进制世界里“竟然”是个全 1 的“最大二进制数”。

在 VS2019 中我们也可以用“内存 1”模拟窗口查看-1 的存储情况。



在调试状态下依次：“调试->窗口->监视->监视 1”，打开上图中的“监视”窗口并输入 &a 表达式即可得到 a 变量的内存地址。然后到右侧模拟的“内存”窗口输入此地址可以看到 8 个 f。此处的 f 是一个十六进制数字。一个 f 代表二进制是 1111，ff 就是 1111 1111。所以 ff ff ff ff 就是 32 个二进制的 1。

7、16-9 这个算术运算用二进制计算处理的过程是什么？

在计算机看来 16-9 其实是 $16 + -9$ 。也就是说把减法变成了加法。其细节步骤如下（为了简便，此处用 1 个字节的空间举例）：

步骤	处理过程	二进制结果
1	16 的二进制存储： 由于 16 是正整数，原码、反码、补码是相同的	0001 0000
2	-9 的二进制存储： 原码：1000 1001 反码：1111 0110（符号位不变，其余按位取反） 补码：1111 0111（补加 1）	1111 0111
3	二进制的加法操作。 $\begin{array}{r} 0001\ 0000 \\ +\ 1111\ 0111 \\ \hline \cancel{1}0000\ 0111 \end{array}$ 但一个字节的存储空间只有 8 位上面的结果多了 1 位，那么多出的位就被舍弃掉。	0000 0111

如何把如上的二进制数 0000 0111 反推出日常的十进制数呢？

第一步：看符号位。0 代表正说明是正整数。

第二步：正整数原码、反码、补码是相同的。所以符号后面的 000 0111 直接转为十进制；最终对应的有符号十进制就是 7。

用上述同样的方法再计算另一个算术式： $-16-9$ 。

按计算机处理方式会变成： $-16 + -9$ 。其细节步骤如下：

步骤	处理过程	二进制结果
1	-16 的二进制存储： 原码：1001 0000 反码：1110 1111（符号位不变，其余按位取反） 补码：1111 0000（补加 1）	1111 0000
2	-9 的二进制存储： 原码：1000 1001	1111 0111

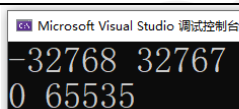
	反码: 1111 0110 (符号位不变, 其余按位取反) 补码: 1111 0111 (补加 1)	
3	二进制的加法操作。 $\begin{array}{r} 1111\ 0000 \\ +\ 1111\ 0111 \\ \hline \cancel{1}1110\ 0111 \end{array}$ 但一个字节的存储空间只有 8 位上面的结果多了 1 位, 那么多出的位就被舍弃掉。	1110 0111

8、整型存储的溢出现象指什么?

由于不同的整型占据的内存空间大小是固定的, 也就是说不同的整数存在数据范围的边界问题。例如: 1 个字节情况下有符号的数字的范围是 -128~127, 无符号的数字范围则是 0~255。在没有负数情况下正整数范围翻倍了。

两个字节情况下, 有符号的数据范围是: -32768~32767, 无符号的数字范围则是: 0~65535。那么如果一个数据已经是该数据类型范围内的最大值或是最小值, 如果再继续增加或继续减小会怎么样呢? 观察如下代码及打印结果:

```
short a = 32767; //有符号下的最大值
short b = -32768; //有符号下的最小值
unsigned short x = 65535; //无符号下的最大值
unsigned short y = 0; //无符号下的最大值
printf("%hd %hd\n", a+1, b-1);
printf("%hu %hu\n", x+1, y-1);
```



```
Microsoft Visual Studio 调试控制台
-32768 32767
0 65535
```

从打印结果可用看出: 有符号下的最大值 32767, 再继续增加 1 会得到有符号数 -32768 即从最大值反而变成了最小值。反之最小值的变量 b 存储的 -32768 让其继续减 1, 本打算让其继续变小反而变成了最大值。无符号下的情况亦然。我们这种只有把整型存储才有的现象称为“溢出现象”。我们以无符号的二进制变化过程也是可以理解溢出的。x 变量的内存情况是: 11111111 11111111。此时再 +1 会得到 1 00000000 00000000 但存储空间只有 16 个位。多出的舍弃掉, 所以结果就是 0。

9、什么是字面值? 整型字面值常量前置写法有哪些?

所谓字面值即通过文字表面就能知道其代表的数据是什么, 反之就是变量。比如: 8 就是数值 8。相对于变量而言 8 不能被修改成别的数值, 因此也称为字面值常量。所谓常量就是不可被修改的数据。

整型数据字面值的格式为: **进制前缀+字面值+类型后缀**

C 语言中整型字面值常量设置时可以用二进制、十进制、八进制、十六进制进行设置。

```
int a;
a = 0B1010; //二进制字面值, 可用数字: 0, 1
a = 10; //十进制字面值, 可用数字: 0~9
a = 0123; //八进制字面值, 可用数字: 0~7
a = 0x123A; //十六进制字面值, 可用数字: 0~9 A~F
```

以 0b 或 0B 开头的为二进制整型的字面值常量。其数据表示只能是 0 或 1, 如果用其它数字就会产生编译错误。

以 0 开头由 0~7 共 8 种数字组成的为八进制整型字面值常量。

以 0x 或 0X 开头, 0~9 及 Aa~Ff 共十六种数字组成的为十六进制字面值常量。没有任何特殊前缀的由 0~9 共十种数字组成的为十进制字面值常量。

10、 整型数据字面值后缀有哪些，分别是什么？

为了更好的在代码中标识不同类型匹配的数值，我们可以在字面值的后面增加数据类型后缀（可以不区分大小）参考如下代码：

```
int a1 = 9;
unsigned a2 = 9U;

long b1 = 9L;
unsigned long b2 = 9UL;

long long c1 = 9LL;
unsigned long long c2 = 9ULL;
```

上例中 9、9U、9L、9LL、9ULL 其实都是标识 9 这个值。但是增加后缀标识后其处理时的存储大小就不同了。在 VS2019 中把鼠标悬停到不同 9 的上面会看到其对应的存储

类型。例如： 没有后缀的 9 就是 int 类型。其它的类型后缀如下：

   
。

其中：9i64 的 i 代表 integer 整型，64 代表 64 位即 8 个字节。注意：低于 int 字节数的 short 没有自己专属的字面值常量后缀。

11、 有符号与无符号的区别是什么？

只有 C/C++ 语言的整型是区分有符号与无符号的，其它编程语言如 java、python 等语言基本不予区分。所谓有符号指数据被二进制化（原码、反码、补码）存储后，第 1 位的符号位是否起到符号的作用。如果是，那么这个有限的字节范围内既能表示正整数又能表示负整数。如果不是符号，那就是数字；那么这个字节范围仅仅用于表示正整数，则相比有符号正整数的数据范围就翻倍了。以 short 类型两个字节为例：有符号情况下其数据范围为 -32768~32768。无符号情况下，其表示的数据范围为 0~65535。

12、 如何得到不同整型的数据范围？

如果我们去单独记忆不同类型整数准确的数据范围是没有意义的，但有时是需要知道数据的大致范围进而选择更合适的数据类型避免溢出现象的发生。我们可以有两种方式得到。

方式一：利用 limits.h 头文件中定义好的最大值与最小值。

```
#include <stdio.h>
#include <limits.h>
int main()
{
    printf("char 最大值: %hhd 最小值: %hhd\n", CHAR_MAX, CHAR_MIN);
    printf("unsigned char 最大值: %hhu 最小值: %hhu\n", 0, UCHAR_MAX);

    printf("short 最大值: %hd 最小值: %hd\n", SHRT_MAX, SHRT_MIN);
    printf("unsigned short 最大值: %hu 最小值: %hu\n", 0, USHRT_MAX);

    printf("int 最大值: %d 最小值: %d\n", INT_MAX, INT_MIN);
    printf("unsigned int 最大值: %u 最小值: %u\n", 0, UINT_MAX);

    printf("long long 最大值: %lld 最小值: %lld\n", LLONG_MAX, LLONG_MIN);
    printf("unsigned long long 最大值: %llu 最小值: %llu\n", 0ULL, ULLONG_MAX);
    return 0;
}
```

方式二：利用溢出现象计算得到。

```
unsigned char a = -1; //或者 0xFF
unsigned short b = -1; //或者 0xFFFF
unsigned int c = -1; //或者 0xFFFFFFFF
unsigned long long d = -1LL; //或者 0xFFFFFFFFFFFFFFFF

printf("char 最大值: %hhd 最小值: %hhd\n", a/2, a / 2+1);
printf("unsigned char 最大值: %hhu 最小值: %hhu\n", 0, a);

printf("short 最大值: %hd 最小值: %hd\n", b/2, b/2+1);
printf("unsigned short 最大值: %hu 最小值: %hu\n", 0, b );

printf("int 最大值: %d 最小值: %d\n", c/2, c/2+1);
printf("unsigned int 最大值: %u 最小值: %u\n", 0, c );

printf("long long 最大值: %lld 最小值: %lld\n", d/2, d/2+1);
printf("unsigned long long 最大值: %llu 最小值: %llu\n", 0, d);
```

选择 Microsoft Visual Studio 调试控制台

```
char 最大值: 127 最小值: -128
unsigned char 最大值: 0 最小值: 255
short 最大值: 32767 最小值: -32768
unsigned short 最大值: 0 最小值: 65535
int 最大值: 2147483647 最小值: -2147483648
unsigned int 最大值: 0 最小值: 4294967295
long long 最大值: 9223372036854775807 最小值: -9223372036854775808
unsigned long long 最大值: 0 最小值: 18446744073709551615
```

在方式二中由于-1 经过原码、反码、补码后在二进制计算机里就是全 1。所以-1 可以作为无符号变量初始的最大值。当然也可以用十六进制的 F 进行设置。一个 f 为 4 个 1111，ff 就是一个字节全 1。这样就实现了无符号的最大值设置。

无符号变量如果是最大值，再整除 2 就得到有符号角度的最大值。在这个最大值基础上再加 1；由于“溢出现象”在有符号角度下就能得到其最小值。

13、 printf 输出不同的整型数据时，其格式都是%d 吗？

观察如下代码的打印结果：

```
printf("%d %u\n", -25, -25);
printf("%hhd %hhu\n", -25, -25);
```

选择 Microsoft Visual Studio 调试控制台

```
-25 4294967271
-25 231
```

printf 的功能仅仅是根据打印的格式设置来把同样的二进制 0 或 1 序列呈现出不同的数据样式。其格式如下表格所示。

specifiers							
length	d i	u o x X	f F e E g G a A c	s	p	n	
(none)	int	unsigned int	double	int	char*	void*	int*
hh	signed char	unsigned char					signed char*
h	short int	unsigned short int					short int*
l	long int	unsigned long int			wint_t	wchar_t*	long int*
ll	long long int	unsigned long long int					long long int*
j	intmax_t	uintmax_t					intmax_t*
z	size_t	size_t					size_t*
t	ptrdiff_t	ptrdiff_t					ptrdiff_t*
L			long double				

%d 对应的是 int，即以有符号十进制 4 个字节的长度展示对应的数据。

%u 对应的是 unsigned int，即以无符号十进制 4 个字节的长度展示数据。
 %hhd 对应的是 signed char，即以有符号十进制 1 个字节长度展示数据。
 %hhu 对应的是 unsigned char，即以无符号十进制 1 个字节长度展示数据。
 -25 作为十进制 int 类型，经过原码、反码、补码后的二进制存储形态为：
 11111111 11111111 11111111 11100111

如果用 %d 展示，其打印结果是：-25

如果用 %u 展示结果就是是：4294967271

而 %hhd 用 1 个字节只能展示数据的低位字节 11100111 结果也是 -25。

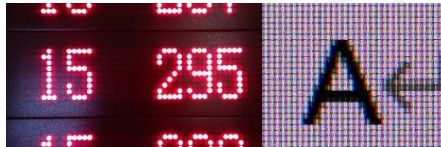
%hhu 用 1 个字节展示数据的低位字节 11100111，以无符号形式展示就是 231。

【注意】printf 就是用不同的字节**长度**结合数据展示的**角度**（d 有符号十进制角度，u 无符号十进制角度，o 无符号八进制角度，x 无符号十六进制的角度）展示指定内存空间内的数据，printf 并不能转变内存空间内数据类型的存储形态。

14、 字符型的原理是什么？为什么其存储原理是定点存储？

所谓字符就是文字符号，即我们日常的字母、数字、各种标点及符号等。由于二进制计算机能够存储的任何信息都得以 0 或 1 方式，所以字符所代表的文字类信息也将“想办法”变成 0 或 1。我们先理解**字符影像与字符编码**的关系：

我们在屏幕上看到的任何文字类型信息都可以称为文字影像。比如：



上图左侧是路边广告屏上的文字，右侧是电脑显示屏照片放大的效果。所谓“字符影像”就是不同像素点的点亮及颜色构成的图案。我们日常的文字都被编制好了不同的影像图案。这个过程是显示适配器（或称显卡）预制好了字符影像的图案及对应的图案编号（即字符编码）。当我们编程告诉显卡字符编码号后，显卡去“告知”显示器哪个像素去点亮即形成我们看到的文字。

综上，我们对字符型数据的编程其实就是对字符编码号进行编程。所有字符影像编码号的集合称为“字符集”。目前统一的字符集编码有 ASCII 码（1 个/单字节）如下图，和 Unicode 码（2 个/双字节）等。本教程纯 C 语言只采用单字节 ASCII 码的字符举例。

十进制	二进制	符号	十进制	二进制	符号	十进制	二进制	符号	十进制	二进制	符号
0	0000 0000	NUL	32	0010 0000	[空格]	64	0100 0000	@	96	0110 0000	`
1	0000 0001	SOH	33	0010 0001	!	65	0100 0001	A	97	0110 0001	a
2	0000 0010	STX	34	0010 0010	"	66	0100 0010	B	98	0110 0010	b
3	0000 0011	ETX	35	0010 0011	#	67	0100 0011	C	99	0110 0011	c
4	0000 0100	EOT	36	0010 0100	\$	68	0100 0100	D	100	0110 0100	d
5	0000 0101	ENQ	37	0010 0101	%	69	0100 0101	E	101	0110 0101	e
6	0000 0110	ACK	38	0010 0110	&	70	0100 0110	F	102	0110 0110	f
7	0000 0111	BEL	39	0010 0111	\	71	0100 0111	G	103	0110 0111	g
8	0000 1000	BS	40	0010 1000	(	72	0100 1000	H	104	0110 1000	h
9	0000 1001	HT	41	0010 1001	)	73	0100 1001	I	105	0110 1001	i
10	0000 1010	LF	42	0010 1010	*	74	0100 1010	J	106	0110 1010	j
11	0000 1011	VT	43	0010 1011	+	75	0100 1011	K	107	0110 1011	k
12	0000 1100	FF	44	0010 1100	,	76	0100 1100	L	108	0110 1100	l
13	0000 1101	CR	45	0010 1101	-	77	0100 1101	M	109	0110 1101	m
14	0000 1110	SO	46	0010 1110	.	78	0100 1110	N	110	0110 1110	n
15	0000 1111	SI	47	0010 1111	/	79	0100 1111	O	111	0110 1111	o
16	0001 0000	DLE	48	0011 0000	0	80	0101 0000	P	112	0111 0000	p
17	0001 0001	DC1	49	0011 0001	1	81	0101 0001	Q	113	0111 0001	q
18	0001 0010	DC2	50	0011 0010	2	82	0101 0010	R	114	0111 0010	r
19	0001 0011	DC3	51	0011 0011	3	83	0101 0011	S	115	0111 0011	s
20	0001 0100	DC4	52	0011 0100	4	84	0101 0100	T	116	0111 0100	t
21	0001 0101	NAK	53	0011 0101	5	85	0101 0101	U	117	0111 0101	u
22	0001 0110	SYN	54	0011 0110	6	86	0101 0110	V	118	0111 0110	v
23	0001 0111	ETB	55	0011 0111	7	87	0101 0111	W	119	0111 0111	w
24	0001 1000	CAN	56	0011 1000	8	88	0101 1000	X	120	0111 1000	x
25	0001 1001	EM	57	0011 1001	9	89	0101 1001	Y	121	0111 1001	y
26	0001 1010	SUB	58	0011 1010	:	90	0101 1010	Z	122	0111 1010	z
27	0001 1011	ESC	59	0011 1011	;	91	0101 1011	[	123	0111 1011	{
28	0001 1100	FS	60	0011 1100	<	92	0101 1100	\	124	0111 1100	
29	0001 1101	GS	61	0011 1101	=	93	0101 1101	]	125	0111 1101	}
30	0001 1110	RS	62	0011 1110	>	94	0101 1110	^	126	0111 1110	~
31	0001 1111	US	63	0011 1111	?	95	0101 1111	_	127	0111 1111	DEL

总结：字符的本质就是一个整数编码号对应一个“显示影像”。我们对字符编程就是对字符编码号的使用。由于字符影像是有限的（如上图的 ASCII 码表），所以一个字节的数据范围就可以表示英文的相关字符。因此把存储一个字节的数据类型定义为 char（character）即字符型。字符型变量的本质就是一个整数（编码号）。因此字符型和整型从存储上原理是相同的。

所以根据上表：字符 A 就是 65。用 65 也可以给字符变量设置值。

15、 字符型字面值设置方式有哪些？

```
char a = 'A'; //所见即所得——影像方式
char b = 65; //十进制表示的编码号
char c1 = 0101; //八进制表示的编码号
char c2 = '\101'; //八进制表示的编码号
char d1 = 0x41; //十六进制表示的编码号
char d2 = '\x41'; //十六进制表示的编码号
printf("%c %c %c %c %c %c ", a, b, c1, c2, d1, d2);
```

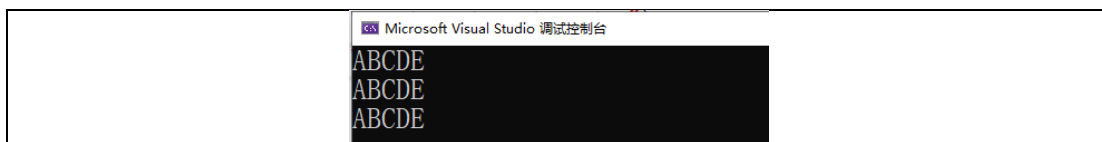
Microsoft Visual Studio 调试控制台

A A A A A A

可用%c 格式即字符影像方式展示，所以看到的结果都是 A。

但是我们看到八进制的编码表示可以用 0101 或 '\101'。这两个写法都是八进制的字符编码号设置变量的值。但是字符型变量赋值时建议用单引号类的字面值写法。因为单引号可以连续放到双引号内形成“字符串”，例如：

```
printf("ABCDE\n");
printf("\101\102\103\104\105\n");
printf("\x41\x42\x43\x44\x45\n");
```



16、 转义字符有哪些及其作用是什么？

有些字符比较特殊；例如和代码内置符号相同的字符如单引号’、双引号”，反斜杠\等；它们在代码中有特殊的作用，如果设置它的字面值就不可以像字母字符或数字字符那样。另外像回车（ASCII—13）、换行（ASCII—10）、缩格 tab（或称制表符）（ASCII—9）、退格 backspace（ASCII—8）等也都是字符型数据；但它们都无法直接输入。ASCII 码表中它们都有对应的字符编号。这类的特殊字符通常用预定好的转义形式进行设置。常见的转义字符如下：

\r	回车：让光标退回到本行的头部
\n	换行：让光标进入到下一行的头部 相当于按了一下 enter 键
\b	退格：让光标向左侧移动一个字符 相当于 backspace 退格键
\t	缩格：产生一个类似 8 个空格的空间 相当于 tab 键
\'	就是一个单引号
\"	就是一个双引号
\\	就是一个反斜杠

17、 ASCII 码与 Unicode 码具体是什么？

ASCII 字符集编码——全称 ASCII (American Standard Code for Information Interchange, 美国信息交换标准代码) 是基于拉丁字母的一套电脑编码系统。它主要用于显示现代英语，而其扩展版本 EASCII 则可以部分支持其他西欧语言，并等同于国际标准 ISO/IEC 646。ASCII 一共定义了 128 个字符，包括 33 个控制字符。

Unicode 字符集编码——中文：万国码、国际码、统一码、单一码。是计算机科学领域里的一项业界标准。它对世界上大部分的文字系统进行了整理、编码，使得电脑可以用更为简单的方式来呈现和处理文字。Unicode 伴随着通用字符集的标准而发展，同时也以书本的形式对外发表。Unicode 至今仍在不断增修，每个新版本都加入更多新的字符。目前最新的版本为 2016 年 6 月 21 日公布的 9.0.0，已经收入超过十万个字符（第十万个字符在 2005 年获采纳）。

Unicode 涵盖的数据除了视觉上的字形、编码方法、标准的字符编码外，还包含了字符特性，如大小写字母。Unicode 发展由非营利机构统一码联盟负责，该机构致力于让 Unicode 方案替换既有的字符编码方案。因为既有的方案往往空间非常有限，亦不适用于多语环境。

18、 char 与 wchar_t 的区别是什么？

char 类型也称单字节字符，就是用一个字节来保留字符的编码。所以 char 类型只能保留 ASCII 码中的字符。wchar_t 称为双字节字符，即用两个字节的空间保留字符。再进一步查看可以看到 wchar_t 的类型定义代码为：typedef unsigned short wchar_t; 其类型来源是 unsigned short，所以 wchar_t 可以表达的字符数量是两个字节共 65536 个字符。

19、 汉字如何输入与输出？

方式一：用 wchar_t 处理字符汉字

```
#include <stdio.h>
#include <locale.h>
```

```
int main()
{
    setlocale(LC_ALL, "CHS");
    wchar_t word = L'科';
    wprintf(L"%lc", word);

    wscanf_s(L"%lc", &word, sizeof(wchar_t));
    wprintf(L"%lc",word);
    return 0;
}
```

上例中locale.h是本地化头文件，setlocale(LC_ALL, "CHS");是设置本地语言为简体中文。L前缀的L'科'则代表双字节字符常量，L"%lc"代表双字节的字符串常量。其中%lc是双字节的输入输出格式。wprintf与wscanf_s是以双字节方式处理字符类数据。

方式二：用char型数组处理汉字。

因为汉字编码用两个字节空间，所以在C语言中也可以用char型数组的连续空间保留。具体代码如下：

```
char str[20];
scanf_s("%s",str);
printf("%s\n",str);
```

20、 字符类型的常见算法有哪些？

仔细观察字符集 ASCII 码表会发现有部分字符的编码号具有规律性：如数字字符、字母字符等它们的编号都是连续的。因此可以用这些规律实现如下算法：

是否是数字字符	<pre>char ch = '3'; if (ch>='0' && ch<='9') { printf("数字字符"); }</pre>
是否是大写字母字符	<pre>if (ch>='A' && ch<='Z') { printf("大写字母字符"); }</pre>
是否是小写字母字符	<pre>if (ch>='a' && ch<='z') { printf("小写字母字符"); }</pre>
是否是字母字符	<pre>if (ch>='A' && ch<='Z' ch >= 'a' && ch <= 'z') { printf("字母字符"); }</pre>
字母的大小写互转 【注】大小写字母的 字符编码对应差值为 32	<pre>char letter = 'A'; letter += 32; printf("%c\n",letter);</pre>
数字与数字字符互转	<pre>char num = '9'; printf("字符影像: %c \n", num); printf("字符编码: %d \n",num); printf("数字字符转数字%d\n",num-'0');</pre>

上例中：&&称为“逻辑与”运算符，代表并且；||为逻辑或运算符，代表或者。

21、 什么是浮点型数据类型？

浮点数据类型主要用于存储小数。之所以称为浮点数，是因为其变为二进制后，最终被存储的小数部分要进行指数化；也就是小数点会“移动”，所以称为浮点数。相对于浮点数的整型数据存储也可称为“定点数”。

浮点数的分类与内存大小：

11	float	单精度浮点型	4	3.4e-38~3.4e38	F或f	%f或%e
12	double	双精度浮点型	8	1.7e-308~1.7e308	-	%f或%e
13	long double	长双精度类型	8	1.7e-308~1.7e308	L或l	%lf或%le

```
#include<stdio.h>
int main()
{
    float pi = 3.14F;
    double pi2 = 3.14;
    long double pi3 = 3.14L;
    printf("float类型字节数: %d\n", sizeof(float));
    printf("double类型字节数: %d\n", sizeof(double));
    printf("long double类型字节数: %d\n", sizeof(long double));
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
float类型字节数: 4
double类型字节数: 8
long double类型字节数: 8
```

我们发现: long double 与 double 都是一样的内存大小。long double (多精度浮点类型或长精度浮点类型) long double 是 1999 修订后的 C 语言标准 “关键字”, C 语言的库函数里用了 long double 这个词,但对 long double 的处理,取决于编译器。ANSI C 标准规定了 double 变量存储为 64 位 (8 个字节) 浮点数值,但并未规定 long double 的确切精度。所以对于不同平台可能有不同的实现。通常是 10 个字节,有的是 12 字节或 16 字节。IEEE 754 规定 long double 的精度不少于 double 的精度,就像 int 和 long int 一样。关于具体的编译器的情况,可以用 sizeof(long double) 计算后得知具体的字节数。

22、 浮点型数据的字面值与 printf 格式是什么?

浮点数字面值可以用小数形式与指数形式 (即科学计数法) 来表达。

//地球距离太阳的距离 : 149597870千米

```
double distanceA;
```

```
double distanceB;
```

//小数形式:

```
distanceA = 149597870.0; //后面的0 可以省略
```

//指数形式:

```
distanceB = 1.49597870e8; //8代表10的8次方
```

```
printf("指数形式打印: %e,%e\n", distanceA, distanceB); //默认6位尾数
```

```
printf("小数形式打印: %f,%f\n", distanceA, distanceB);
```

printf("相对短形式打印: %g , %g \n", distanceA, distanceB); //小数与指数哪个短,显示哪个。

Microsoft Visual Studio 调试控制台

```
指数形式打印: 1.495979e+08, 1.495979e+08
小数形式打印: 149597870.000000, 149597870.000000
相对短形式打印: 1.49598e+08 , 1.49598e+08
```

上面代码中: distanceB = 1.49597870e8; 的 e8 代表 10 的 8 次方。是一种指数表示方式。我们在设置时不一定严格科学计数法要求 (整数部分必须是个位数)。也可以写成: 149.597870e6 。

23、 浮点型存储原理是什么?

浮点数与整型数的存储原理是不同的。两者相比浮点数存储更为复杂。float 和 double 虽然占据的内存空间大小不同,但是存储原理是相同的。它们将自己所在的内存空间按——符号位+指数区+尾数区的形式进行存储分配。其中:
float 的内存空间 (4 个字节 32 位): 1 个符号位+8 个指数区+23 个尾数区。

double 的内存空间（8 个字节 64 位）：1 个符号位+11 个指数区+52 个尾数区。

下面以一个 float 类型实例简要说明一下浮点型数据的存储过程：



例如：float a = 10.127F;

步骤	说明
1、二进制格式化：	整数部分除 2 取余，小数部分乘 2 取整 10 的二进制：1010 0.127 的二进制： 0010000010000011000100100110111010010111100011010101 注意：0.127 连续乘了 52 个 2 才被乘尽。
2、指数化	1010. 0010000010000011000100100110111010010111100011010101 01 小数点浮动 3 位（二进制表示的科学计数法，保留一个整数位） 0100010000010000011000100100110111010010111100011010101 浮动的 3 即二进制的 11 将被保存进指数区。剩余的 0100010000010000011000100100110111010010111100011010101 将被保存到尾数区。很明显尾数区只有 23 个二进制，如今有 55 个 二进制要被保存，那么剩余的就只能舍弃。最终保留了 01000100000100000110001。这就说明计算机保存的小数并不是 足够精确的。这 23 个尾数二进制位就是有效数字了，对应十进制 就是 6~7 位。如果是双精度类型，52 位尾数就可以表达 15~16 位 十进制有效数字。
3、指数偏移	指数化时二进制浮动了 3 个二进制位，这个 3（二进制的 11）在 存储到指数区时需要增加 0111111 进行偏移处理。 即 127+3 得到的二进制为 10000010。
4. 最终二进制形式	符号位：0 【注意】0 代表正 1 代表负 指数区：10000010 尾数区：01000100000100000110001 组合在一起即： 01000001001000100000100000110001

这个过程的结果我们也可以到 IEEE-754 浮点转换器 (<https://www.h-schmidt.net/FloatConverter/IEEE754.html>) 查看任意小数的二进制存储形态。

24、 float 与 double 有效数字的分别是多少？

float 类型的存储中小数尾数有 23 个二进制位，可以表示 6~7 位十进制有效数字。
double 类型的存储中小数尾数是 52 个二进制位，可以表示 15~16 位十进制有效数字。

25、基本数据类型的类型转换方式有哪些？请举例说明。

整型与字符型在存储时采用的是定点存储（原码→反码→补码）。特点是准确、数据范围小、存在溢出现象。浮点型存储（符号+指数+尾数）的特点是能够存储小数，存在有效数字位数问题。当不同的存储原理的数据进行混合算术运算时，由于其自身存储原理的不同，会把数据类型转换成统一的存储形态才能运算出结果。这种转换分为自动类型转换（或称隐式方式）和强制类型转换（或称显示方式）。

看下面的案例：

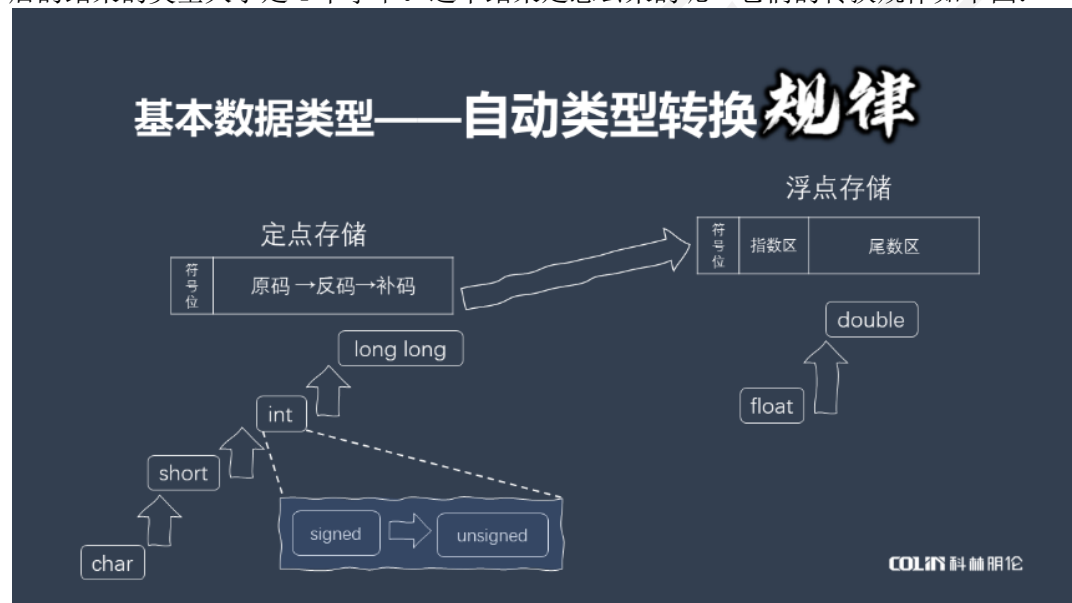
```
printf("%d\n", sizeof( 3+6LL+0.1f));
```

输出结果为：

Microsoft Visual Studio 调试控制台

4

上例中可以看到：字面值 3 没有后缀，所以为 int 类型占 4 个字节存储空间，6LL 的后缀说明是 long long 类型 8 个字节，0.1f 的后缀是 float 类型 4 个字节。它们相加后的结果的类型大小是 4 个字节。这个结果是怎么来的呢？它们的转换规律如下图：



规律 1：当有整型与浮点型同时运算时：整型向浮点型转换（无关字节数的大小）。

规律 2：当存储原理相同时：字节少的向字节多的转换。

规律 3：char 或 short 变量参加的算术表达式，计算结果最小为 int 类型。

例如：如下的计算结果为 4 个字节

```
char ch = 'A';
short s = 123;
printf("%d\n", sizeof( ch+s));
```

什么是强制类型转换呢？就是我们需要它（表达式的值）是什么样的类型，它就是什么类型。强制类型转换的语法格式：（现有数据类型）表达式。例如：

```
printf("%d\n", sizeof( 3+6LL+(int)0.1f));
```

上例中：(int)0.1f 这个表达式就是把 0.1f 这个 float 类型存储变为 int 类型，其计算结果是只保留整数部分其余舍弃，即结果为 0。这样，3+6LL+(int)0.1f 的类型就是：int+long long+int，此表达式中的三个操作数是同一种存储原理，按字节少向字节多转换，最终结果是 long long 类型 8 个字节的大小。

综上就能理解如下代码的执行结果了。

```
printf("%d %g\n", 7/2, 7./2 );
```

```
printf("%d %g\n", 7/2, (double)7/2);
```

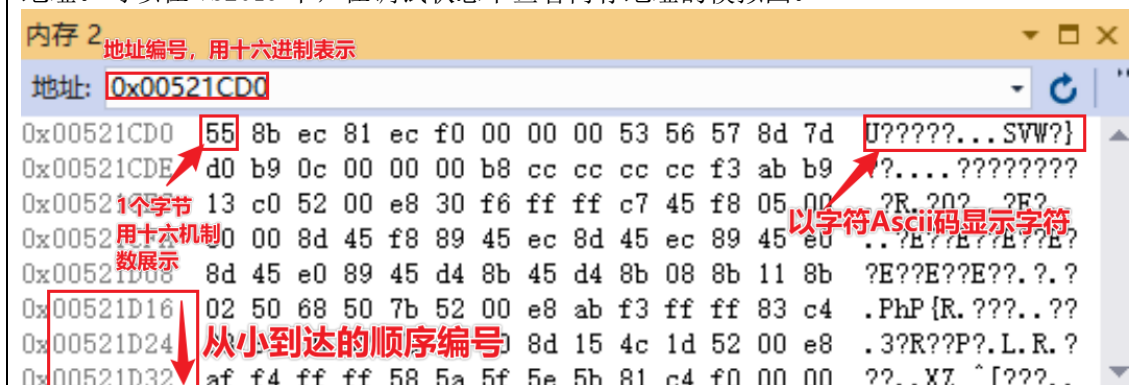
3 3.5

3 3.5

Part 2 指针基础

26、 什么是内存地址，它有何特性？

二进制计算机以 8 个二进制位构成一个字节，并以字节作为最小存储单位。内存就像一个网格状的仓库，字节就是其中的一个“小格子”。操作系统会为每个字节“编制一个地址”便于找到它并向它读取或存入数据。内存地址的分配管理与生活中一个小区或一个仓库的管理模式类似，具有唯一性即不会重复地址编号，连续性并从小编号地址到大编号号地址。可以在 VS2019 中，在调试状态下查看内存地址的模拟图。



27、 x86 与 x64 的区别是什么？

1: 支持的内存不同

32 位的操作系统，最多支持 4G 的内存，实际内存为 3.25G；64 位系统支持 4G 8G 16G 32G 64G 128G、256G 内存，理论上可以无限支持，只要你主板上有足够的内存条。

2: 支持的处理器不同

64 位的操作系统支持基于 64 位的处理器，而 32 位的系统却不能完全支持 64 位的处理器。

3: 支持的软件不同

32 位的操作系统，支持基于 32 位的软件，不能运行 64 位的软件；而 64 位的系统一般这两种类型的都支持，基本上与各种软件都兼容，特别是 adobe 公司的软件，现在的新版本的都只支持 64 位。

4: 处理数据的能力

32 和 64 表示 CPU 可以处理最大位数，一次性的运算量不一样，理论上 64 位的会比 32 位快 1 倍，内存寻址也不一样。

5: 系统体积大小

64 位系统都比 32 位系统大的多，比如 win7 64 位比 win7 32 位系统大 700M 左右。

28、 指针与地址的关系？

在 C 语言中指针 (pointer) 就是地址，此刻它们是同义词。地址更容易理解。但是指针更形象。因为我知道了你的地址，所以可以说我指向了你。因为我指向了你就可以找到你所在空间并进行数据的存储或读取操作。尤其在数据结构、界面编程、网络编程中用指针表达逻辑的画面感会更形象、直观。

29、 指针与指针变量的区别？

指针就是地址。C 语言中二者是同义词。**指针变量就是存储地址的变量**。类似于数字 3 与 int 类型变量的关系。初学者最好严格区分；因为在 C 语言中能够代表地址的知识点还有很多，如：数组名、函数名、字符串常量等；它们本质就是一个指针常量（即固定不

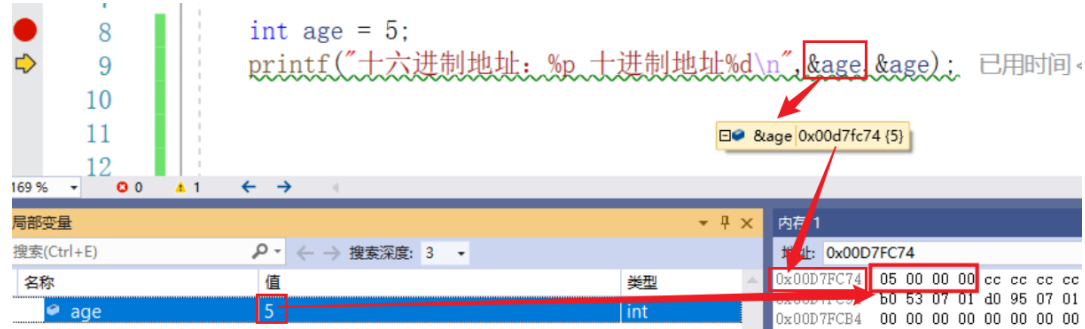
可改变的地址)而非是指针变量。

30、&与*在指针运算中的作用。

&在单目状态下代表取地址运算符，即得到一个变量在内存中的实际地址。例如：

```
int age = 5;
printf("十六进制地址: %p 十进制地址%d\n", &age, &age);
```

代码中: &age 可以计算得到 age 变量在本次执行时被分配到的内存空间的实际地址 (注意: 变量每次被分配的空间可能不一样, 但一定会为其分配一个存储空间)。printf 中%p 代表指针类型的打印, 即以十六进制形式打印 age 变量的实际内存地址。当然也可用%d 以十进制的方式打印。利用 VS2019 中调试器的内存窗口可以直观的看到变量 age 的地址以及空间内的值。



通过上图可以看到: 由于 age 变量是 int 类型, 其占据 4 个字节的存储空间。又由于每个字节都有地址编号, 所以&age 计算得到的是 age 变量所占 4 个字节空间中最小的地址也称为首地址。

【注意】“*”运算符在这里的计算过程是: 找到指针指向的地址, 再结合指针的类型得到指向空间的值 (这种利用指针获得空间使用权的方式也称为: “间接访问”内存空间或称“解指针”)。

```
int* p = &age;
*p = 8;
```

上例中: p 指针变量保留的 age 变量所占 4 个字节空间的首地址 (或称最小号地址)。那么*p 就是一个间接访问 age 变量空间的表达式; 即*p 和 age 能达到一个目的——把一个 8 存入到这个内存空间里。

另外如果有如下写法: *&age = 9; 此时*&是一对相同优先级的互逆运算, &负责得到一个地址, *负责根据这个地址得到一个空间。所以它们互逆抵消后就变成 age=9。

31、指针变量自身的内存空间大小和指向的变量类型有关吗?

答案是无关。指针变量在定义时要按: “数据类型* 指针变量名;” 的格式创建指针变量。其中*号是指针类型的标记, 但是前面必须指明是什么类型的指针。指针变量的本质就是:

1、“记住”一个地址。

2、利用这个地址和*结合计算得到一段内存空间。

所以无论什么类型的指针都只能是记录一个字节的地址编号。

那么指针变量占据的内存空间大小和什么有关呢? 答案是: 和当前选择的运行平台位数有关。如果选择的是 x86 即 32 位平台下, 指针 (内存地址编号) 将以 4 个字节空间存储地址编号。如果选择的是 x64 即 64 位平台下, 则以 8 个字节长度对地址编号。

```
int age = 5;
int* p;
p = &age;
char* pc = &age;
double* pd = &age;
unsigned long long* pull = &age;
printf("%d %d\n", sizeof(p), sizeof(int*));
```

```
printf("%d %d %d\n", sizeof(p), sizeof(int*), sizeof(*p));
printf("%d %d %d\n", sizeof(pc), sizeof(char*), sizeof(*pc));
printf("%d %d %d\n", sizeof(pd), sizeof(double*), sizeof(*pd));
printf("%d %d %d\n", sizeof(pull), sizeof(unsigned long long*), sizeof(*pull));
```

Microsoft Visual Studio 调试控制台

```
4 4 4
4 4 1
4 4 8
4 4 8
```

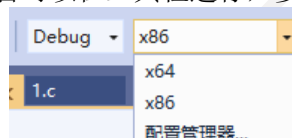
根据上面代码的执行结果可以看到：无论指针变量是何种类型打印结果都是 4。因为当前的运行平台选择的是 x86 即 32 位系统。如果切换到 x64 则结果如下：

Microsoft Visual Studio 调试控制台

```
8 8 4
8 8 1
8 8 8
8 8 8
```

可以看到指针变量的大小、指针类型的大小都和运行平台的位数有关。但最后一个值即利用指针得到空间大小时才和指针指向的类型有关。

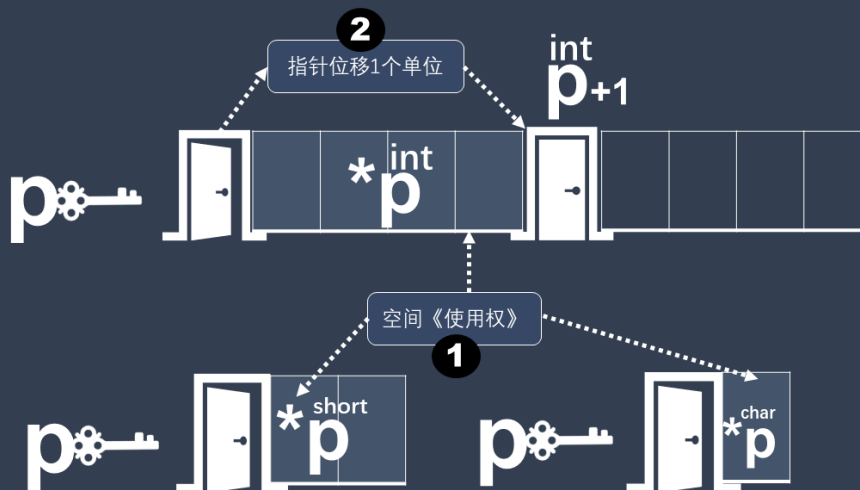
在 VS2019 中选择切换运行平台可以在工具栏进行，参考下图：



32、 指针变量增加或减少 1 个整数会得到什么？

《指针变量类型的作用》

COLIN 科林明伦



上图是指针变量应用的最基本模型。我们在定义指针变量时，务必指明指针所指向的类型。这是因为一旦一个指针变量保留了一个字节的地址编号，那么利用这个指针就可以完成两个操作。一个是“*”操作，即根据指针变量指向的地址再结合指针变量指向的数据类型就能获得一段内存空间。例如：`int*`类型指针变量就可以获得 4 个字节的内存空间，`short*`类型指针变量能够获得 2 个字节的内存空间。我们对这段内存空间可以有读取的权力，也可以有修改的权力，我们就统一把 `*p` 的操作称为获得空间使用权。

另一个操作就是指针的偏移。一个指针变量如果增加或减少一个整数（比如 1），那么会得到什么呢？这里的 1 不要理解为一个字节而是一个类型的字节数；是 1 个单位的概念。例如一个 `int*` 指针变量 `p` 指向了一个字节所在地址，那么指针变量 `p+1` 会得到由 `p` 开始的 4 个字节以外的另一个地址。也就是偏移了一个 `int` 的空间。所以指针变量增加或减

少一个整数其实是在当前地址为基础增加或减少一个指向类型的字节数，从而得到另一个地址。看如下测试案例：

```
int age = 5;
int* p;
p = &age;
char* pc = &age;
double* pd = &age;
unsigned long long* pull = &age;
printf("%d %d %d %d\n", p-1, p, p+1, p+2);
printf("%d %d %d %d\n", pc - 1, pc, pc + 1, pc + 2);
printf("%d %d %d %d\n", pd - 1, pd, pd + 1, pd + 2);
printf("%d %d %d %d\n", pull - 1, pull, pull + 1, pull + 2);
```

由于指针变量 p、pc、pd、p11 指向了同一个地址。但是由于指向类型的不同，所以加减同样的整数得到的地址也不同，这个地址的间隔差与指针定义的指向类型有关。

选择 Microsoft Visual Studio 调试控制台

14219724	14219728	14219732	14219736
14219727	14219728	14219729	14219730
14219720	14219728	14219736	14219744
14219720	14219728	14219736	14219744

33、 void* 是什么类型的指针，有何作用？

void 通常翻译为“无”，代表没有类型，是**无类型**的占位关键字。void*代表一个没有具体指向类型的指针，用此类型创建的指针变量仅仅用于保留一个地址。但是用这个指针变量无法完成上一问题中的指针模型两大操作：即获得空间使用权的操作和指针偏移的操作。请看如下代码：

```
int age = 5;
void* p = &age;
*p = 8;
p + 1;
```

上述代码中的*p=8 在 VS2019 中会提示：“表达式必须是可修改的左值”这样的错误提示，意思是说*p 这个表达式无法完成一个空间使用权的获得。因为 p 指向的类型是 void，没有具体类型就无法得到一个空间的大小。而 p+1 这句代码都会报出“表达式必须是指向完整对象类型的指针”这样的错误提示；意思也是说因为 p 指向的类型不具体，也就是不知道类型的字节数，最终无法偏移到另一个地址。

34、 NULL 是什么？

NULL 在 C 语言中被称为“空指针”。NULL 并不是保留字而是#define NULL 0。是一个宏替换的名字，它代表为 0。也就是 0 号地址。因为即使在现实生活中也不会为地址设置为 0 号，所以就约定 0 号地址就是空地址，代表指针变量不指向任何的内存空间。

```
int* p = NULL;
if (p==NULL)
{
    printf("p为空指针\n");
}
else
{
    printf("p指向的值是: %d\n", *p);
}
```

35、 大端模式与小端模式指什么？

首先，观察如下代码，看一个内存的存储现象：

```
int num = 0x12345678;
```

```
int* p = &num;
```

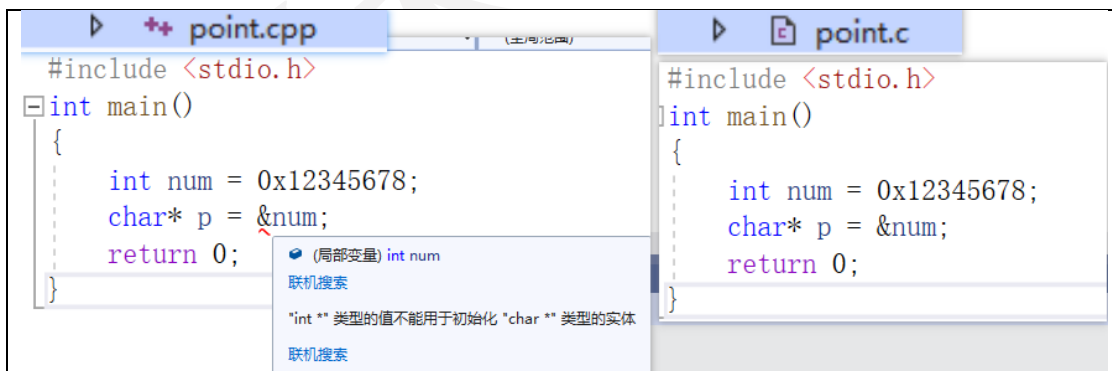
内存 1									
地址: 0x00F9FABC									
0x00F9FABC	78	56	34	12	cc	cc	cc	cc	cc
0x00F9FADC	c0	53	52	01	48	72	52	01	48
0x00F9FAFC	00	00	00	00	00	00	00	00	00

我们发现内存中所看到的数据和我们习惯的数据展示不一样。我们习惯的数据展示方式是——数据的高位数在左侧而低位数在右侧（即 12345678）。但是在内存中它们却是以字节为单位的翻转着（78563412）。我们把这种数据的存储现象称为小端模式存储（简称小端 little-endian）。即内存的地址小号的编号存储了数据的低位数，大的地址编号存储的是高位数。反之如果小地址编号存储了高位数而大地址编号存储了低位数就称之为大端模式(big-endian)。



根据上图可以看到:一般在网络传输中的数据通常采用大端模式。而内存中保留数据时则采用小端模式。

36、 如果一个 char* 指针变量指向了一个 int 型变量会报出编译错误吗？



如上图所示，如果我们的源代码文件是 .cpp 时，就会产生编译器错误。而变成 .c 时就不会报编译错误。这是原因 .cpp 时，VS2019 会采用 C++ 的编译器对源代码进行编译，而 .c 则采用纯 C 语言编译器。而 C++ 编译器在语法检查方面则更为严格。这种情况下如果想通过编译器的语法检查可以对 `&num` 的计算结果进行强制类型转换，使其类型与左侧指针变量的指向类型保持一致。即 `char* p = (char*) #` 这样无论 C++ 编译器还是 C 编译器都不会报错。

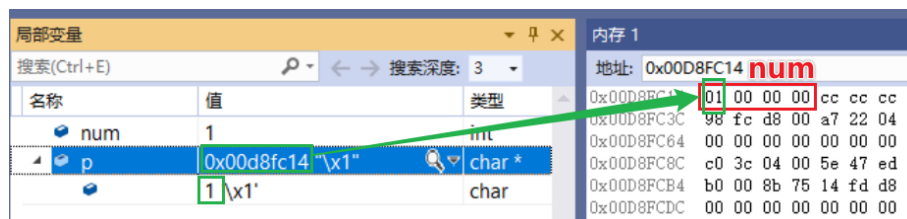
37、 写一段代码判断是否是小端模式？

```
#include <stdio.h>
int main()
{
    int num = 1;
```

```

char* p =(char*) &num;
if (*p==1)
{
    printf("小端模式\n");
}
else
{
    printf("大端模式\n");
}
return 0;
}

```



通过调试器的内存窗口可以观察到：变量 num 的值是 1，在内存中的 4 个字节存储形态为 01 00 00 00。&num 得到的是 4 个字节中的最小地址（即首地址）交给 p 保存。由于 p 指针变量的类型是 char*，即*p 代表 p 指向的是 1 个字节的空间使用权。如果是小端存储模式，1 这个低位数会在小地址号的空间存储，如果是大端模式则高位数 00 会存储在小地址号的空间里。

38、 如果指针指向了不属于本程序的内存空间会产生什么样的错误？

指针最好要指向我们申请的内存空间。什么是我们申请的合法空间呢？例如 main 函数内定义的局部变量或申请的堆内存空间，如果指向了不属于我们本程序的内存空间，此时操作系统会阻止我们对这个空间的访问。

例如：

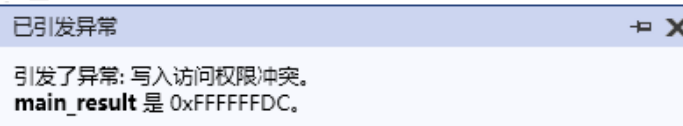
```

int a = 5;
int* p = &a;
*p = 10;
p += 10;
*p = 8;
printf("%d\n", *p);

```

上面代码中，a 的内存空间是我们申请的合法空间，而指针变量偏移了 10 个 int 空间后，那里的空间就不属于我们的程序了。而此时用*p 来强行使用此空间会导致我们的程序会被中止执行。有如下提示时，通常是我们的程序没有执行到 return 0 就被停止了。这个 -1073741819 都是内存访问冲突的原因造成的中途被关闭的错误。

(进程 25388) 已退出，代码为 -1073741819。如果用调试器查看，就会看到“访问权限冲突”字样的提示。也都是指针指向了非法空间所致。



39、 int* x, y, z; 三个变量都是指针变量吗？

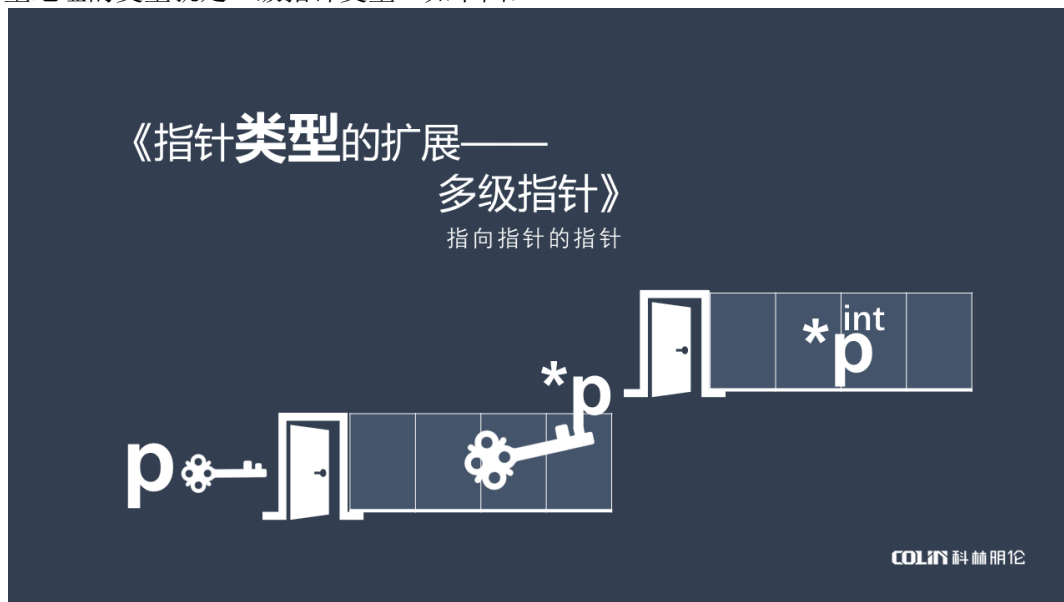
这里只有 x 是指针变量，其它的 y, z 都是 int 类型变量。如果 y, z 也要变为指针类型变量则需要按如下方式定义：

```
int* x, *y, *z;
```

即每个变量名字前面都需要*号，这样 x、y、z 就都是 int 型指针变量了。

40、 多级指针是什么？

多级指针也可称为指向指针变量的指针变量。因为指针变量也是变量，只要是变量就要占用一段内存空间。如果一个指针变量的内存地址被保存，那么存储这个指针变量地址的变量就是二级指针变量。如下图：



p 是一个指针变量，但其指向的空间里面存储的是一个 int 型数据的内存地址，我们拿着这个地址又去访问了一个真正的 int 型整数空间。这里的 p 指针变量就是一个多级指针。观察如下代码：

```
int a = 5;
int* p = &a; //一级指针
int** pp = &p; //二级指针
int*** ppp = &pp; //三级指针
printf("%d\n", ***ppp);
```

根据上面的代码我们把一个“*”的指针称为一级指针，它能够指向一个 int 类型的空间。“**”称为二级指针，它能够指向一个“*”的指针变量。三级及更多层的规律也是如此。

***ppp 的计算过程是：按从右到左的结合性看，*ppp 先结合得到一个地址，此地址再和前面的*结合得到另一个地址，再和前面的*结合才能得到真正的整数。用调试器查看的效果如下：

局部变量		
名称	值	类型
a	5	int
p	0x00effb08 {5}	int *
pp	0x00effafc {0x00effb08 {5}}	int **
ppp	0x00effaf0 {0x00effafc {0x00effb08 {5}}}	int ***
	0x00effafc {0x00effb08 {5}}	int **
	0x00effb08 {5}	int *
	5	int

Part 3 数组

41、 什么是数组？它有什么特性

数组（英文 array）就是一组同类型变量的集合。它具有三个特性：长度固定、连续存储，有序编号每个成员，编号最小为 0，最大为长度减 1。C 语言的数据类型包括基

本数据类型、指针类型还有扩展出的复合类型。数组就是这种复合类型的一种，它是由现有类型为基础，做了数量的延伸。建议：数组在定义时，把 `int [长度]` 理解为一个数据类型：其代表一串连续的整数。只不过数组的名字要写在类型与[长度]之间。

数组被定义时，其长度务必要确定好。VS2019 的编译器不支持数组长度的动态创建。如下例子打算用变量作为数组的长度，VS2019 是不允许的：

```
int len = 8;
int arr[len];

return 0;
```



变量是不能作为数组长度的。只能用整型常量才可以。如果想要实现动态长度的数组，除了用堆内存的方式以外，还可以先申请一个足够长的数组，再用一个变量来记录数组当前的有效长度。

```
int len = 8;
int arr[1000];
```

上面代码定义一个 1000 个长度的数组，但实际使用的长度为 `len` 个。

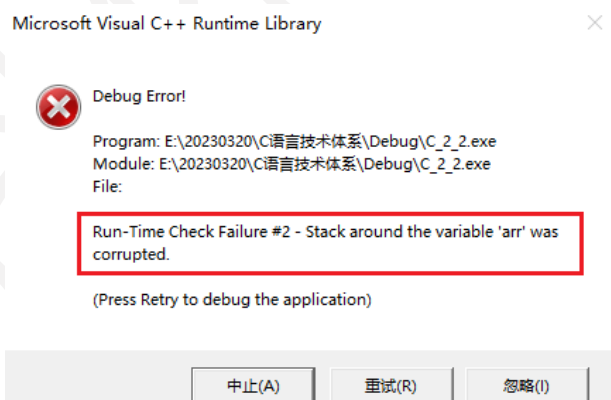
42、 数组下标越界会产生编译错误吗？

```
int arr[5] ;
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;
arr[3] = 40;
arr[4] = 50;
```

根据上述代码，数组长度为5，下标范围是0、1、2、3、4。所以如下两行代码属于是越界访问。

```
arr[5] = 9;
arr[-1] = 6;
```

但当我们编译生成时是不会报错误的。因为[]在代码中是一种运算符，其作用是计算出一个地址并获得一个空间的使用权。只有在程序运行时，如果[]计算得到的空间属于非法访问的话，我们的程序才有可能被关闭。比如运行时出现下图的提示，通常代表内存访问的问题，即非法访问不属于我们的内存空间。



43、 什么是数组的遍历？

遍历就是对管理的数据进行“清点”的过程。由于数组的访问用下标号的方式，而且数组的存储空间是连续的，所以用有规律的下标配合循环就能清点所有的数组成员。

```
for (int i = 0; i < 5 ; i++)
{
    printf("%d ", arr[i]);
}
```

44、 一维数组初始化的方式有哪些？

方式一：完全初始化

```
int arr1[5] = {1, 2, 3, 5, 2};
```

如果初始长度超过预定的长度就会产生编译错误提示：初始值设定项值太多

方式二：不完全初始化。即仅给出部分成员的值，其它未赋值的默认为 0

```
int arr2[5] = { 1, 2 };
```

```
int arr3[5] = { 0 }; // 初始全0
```

方式三：省略长度，由初始化成员的实际个数决定数组的长度。

```
int arr4[] = {8, 9, 8, 5, 6, 5, 2, 4};
```

45、 如何计算得出数组的成员数量？

用数组名做 sizeof 运算符的参数可以计算整个数组所占据的内存字节总数。

又由于数组至少有一个成员，所以再用 sizeof 计算一个成员的字节数。这样数组总的字节数除以一个成员的字节数的结果就是数组的成员数量。代码如下：

```
printf("数组的字节数 : %d\n", sizeof(arr));
printf("数组的成员字节 : %d\n", sizeof(arr[0]));
int len = sizeof(arr) / sizeof(arr[0]);
```

46、 一维数组和二维数组的思维逻辑有何不同？

数组的维度是一个逻辑概念。其实数组无论是几维，其实际在内存中都是连续存储的。一维数组的思维逻辑属于线性思维，适合表达一串、一排、一序列等的应用场景。而二维数组的思维类似一个表格或矩阵，适合如棋盘、地图等应用场景。

47、 二维数组的定义与初始化的写法是怎样的？

二维数组定义的格式：数据类型 数组名 [行数][每行列数];

二维数组初始化的方式：

方式一： 完全初始化，即完全满足定义的总的变量个数。

```
int arr1[2][3] = {1, 2, 3, 4, 5, 6};
```

这种形式的初始值看似一维数组，但是能够正确的把数据存入到各个位置。究其原因二维数组虽然从思维逻辑上是个表格形状，但在内存中却是成线性连续存储的。如下图内存模拟图所示：

内存 1

地址: 0x0075F77C

0x0075F77C 01 00 00 00 02 00 00 00 03 00 00 00 04 00 00 00 05 00 00 00 06 00 00 00 cc

0x0075F797 cc 07 23 f3 40 bc f7 75 00 23 30 d9 00 01 00 00 00 30 0b a7 00 b0 50 a7 00

但为了方便直观，也可以用 {} 二层区分一下行的逻辑：

```
int arr2[2][3] = { {1, 2, 3}, {4, 5, 6} };
```

这样可以更直观形象的观察到二维数组的行列的构成。

方式二：不完全初始化。即只给出部分成员的值，其余置为0。

```
int arr3[2][3] = { 1, 2 }; 其余的4个数为0。
```

```
int arr4[2][3] = { {1}, {4, 5} } 相当于{{1, 0, 0}, {4, 5, 0}}
```

方式三：根据初始化值决定行数，但必须指明具体列数。

```
int arr5[][2] = { 1, 2, 3, 4, 5, 6, 2 }; 其中列数2不能省略。
```

48、 二维数组为什么又称为：一维数组的一维数组？

以 `int arr2[2][3] = { {1, 2, 3}, {4, 5, 6} };` 为例用调试器观察“局部变量”窗口如下：

名称	值	类型
从行的角度看：是一个一维数组		
[0]	0x010ffc64 {0x010ffc64 {1, 2, 3}, 0x010ffc70 {4, 5, 6}}	int[2][3]
[0]	0x010ffc64 {1, 2, 3}	int[3]
[0]	1	int
[1]	2	int
[2]	3	int
[1]	0x010ffc70 {4, 5, 6}	int[3]
[1]	4	int
[2]	5	int
[2]	6	int

但每一行又是一个一维数组

所谓二维数组（或多维数组）其实从管理上看每一个维度都是一个“一维数组”，只不过每个维度的大小不同。行维度是一个一维数组，但是每行的大小是 3 个 int 变量。进而到了列的维度，每列大小就是一个 int 变量了。以此类推三维数组、四维数组也是这样的管理逻辑。因此在操作二维数组具体成员时一定以“先行后列”的原则，先确定行维度的空间，然后在行内部再寻找列维度的空间。

49、 如何计算二维数组的行数与列数？

```
以 int arr5[][2] = { 1, 2, 3, 4, 5, 6, 2 }; 为例:
printf("二维数组整体大小: %d\n", sizeof(arr5));
printf("二维数组一行的大小: %d\n", sizeof(arr5[0]));
printf("二维数组一列的大小: %d\n", sizeof(arr5[0][0]));

printf("二维数组总行数: %d\n", sizeof(arr5) / sizeof(arr5[0]));
printf("二维数组每行的列数: %d\n", sizeof(arr5[0]) / sizeof(arr5[0][0]));
```

Microsoft Visual Studio 调试控制台

```
二维数组整体大小: 32
二维数组一行的大小: 8
二维数组一列的大小: 4
二维数组总行数: 4
二维数组每行的列数: 2
```

由于二维数组的成员数量至少有一行一列。又由于 C 语言的二维数组每行都是相同大小的。所以都用下标 0 作为 sizeof 的参数。

arr5[0]代表的是第 0 行的空间。

arr5[0][0]代表第 0 行第 0 列的空间。

Part 4 指针与数组

50、 一维数组名有何特性？

参看上图，在调试器中可以看到数组名 as 本质就是一个地址，按照这个地址在内存模拟窗口中可以查看到这个地址开始的数组成员在内存中的排列情况。

接下来用 `printf("%d %d %d\n", as, as+1, as+2);` 这行代码查看数组名增加 1 个整数会得到什么？

Microsoft Visual Studio 调试控制台

9436640 9436644 9436648

根据打印结果可以看到：都是地址。数组名增加一个整数，地址编号就增加一个 4（即 `int` 的类型大小）。所以结论是：数组名+整数得到的是数组内某成员的地址。数组名 `as` 也可以理解为 `as+0`，那么数组名就代表第 0 个成员的地址。如果计算结果是个地址，那么这个地址能做什么呢？当然是利用地址可以间接访问一个内存空间。所以：
`printf("%d %d %d\n", *as, *(as + 1), *(as + 2));` 的打印结果是

Microsoft Visual Studio 调试控制台

1 2 3

即数组某成员的值。和我们下标访问成员的值是同样的结果。

我们把刚才的行为（`as+1` 和 `*(as + 1)`）用两个动词概括就是“找到”和“得到”。所谓“找到”就是找到了一个地址（指针），“得到”就是利用这个地址（指针）结合指向的类型获得一个空间的使用权。

总结：一维数组的数组名是一个代表数组首元素的地址，一维数组名加整数是数组成员间的偏移，即指向某数组成员。

51、 `[]` 运算符的计算逻辑是什么？

`[]` 不仅仅是数组的标志，其本身是一个运算符；优先级和 `()` 一样，都是最高的优先级。其使用格式为：**参照地址[偏移量]**。我们熟悉的写法是 `arr[1]` 即访问数组的下标 1 成员空间。但我们要理解其背后的执行过程是这样的：

参照地址[偏移量] 等价于 `*(参照地址+偏移量)`。也就是 `arr[1]` 背后的计算过程是 `*(arr+1)`。也就是 `arr` 这个数组名代表的是数组首元素的地址，`arr+1` 代表从第 0 个成员地址开始偏移指向下标 1 成员的地址。`*(arr+1)` 就代表下标 1 成员的空间。我们也可以理解成 `arr[N]` 相当于 `*(arr+N)` 的快捷方式。日常开发建议用 `[]` 访问数组成员的空间，简单直观。但是也要知道其背后的计算逻辑。

52、 数组名是指针变量还是指针常量？

数组名是指针常量。当我们试图对数组名有赋值类行为时，VS2019 会提示我们“表达式必须是可修改的左值”。意思是说数组名是个指针常量，不可被修改。究其原因数组在被创建时，计算机就会为其开辟连续的存储空间。数组名就是这个空间的起始位置。而如果允许数组名被修改，那么原来的空间也许就找不到了。所以数组名是个指针常量。

```
int main()
{
    int as[]={1, 2, 3, 4, 5};
    as += 3;
    as++;
}
```

(局部变量) int as[5]
 联机搜索
 表达式必须是可修改的左值
 联机搜索

只有指针变量才可以被赋值操作即让其指向另一个新的地址。

53、 利用基本类型的指针变量遍历数组成员的方式有哪些？

有如下代码，让一个基本类型 `int` 的指针指向一个一维数组的第 0 个成员：

```
int as[]={1, 2, 3, 4, 5};
```

```
int* p = as;
```

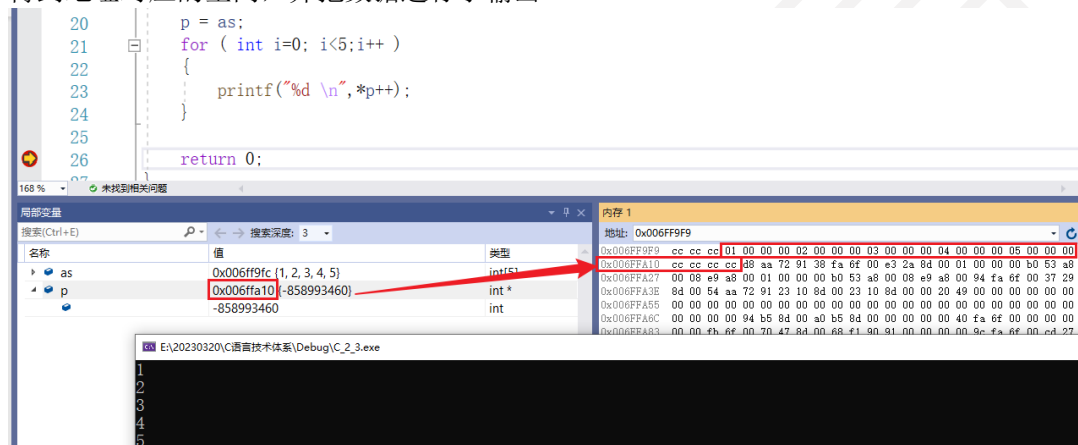
我们可以用“固定地址法”即 p 作为参照地址不动来访问数组成员。

```
for (int i=0 ; i<5; i++)
{
    printf("%d %d %d %d\n",as[i], *(as +i) ,p[i], *(p+i ));
}
```

也可以用“移动地址法”。让指针变量 p 从数组首地址开始不断的向下偏移一个成员。

```
p = as;
for ( int i=0; i<5;i++ )
{
    printf("%d \n",*p++);
}
```

请注意上例中 *p++ 这个表达式；由于 * 与 ++ 为相同优先级，它们的结合性是从右至左，所以 ++ 先执行；按照 ++ 表达式的特点，因为 p 在前，++ 在后，所以 p 当前指向的地址作为表达式的值去与 * 结合，但是 p 会指向下一个成员。最后 * 和 ++ 表达式的值结合得到地址对应的空间，并把数据进行了输出。



从上图的调试结果可以看到：p 遍历完每个数组成员后已经指向了数组最后一个成员的后面。所以我们在用指针遍历数组成员时务必让指针变量指向数组的头部。

54、 一维数组名+1 与&一维数组名+1 的区别？

先看一段代码：

```
int as[] = { 1 , 2 , 3 , 4 , 5 };
printf("%d %d\n", as, as+1);
printf("%d %d\n", &as, &as + 1);
```

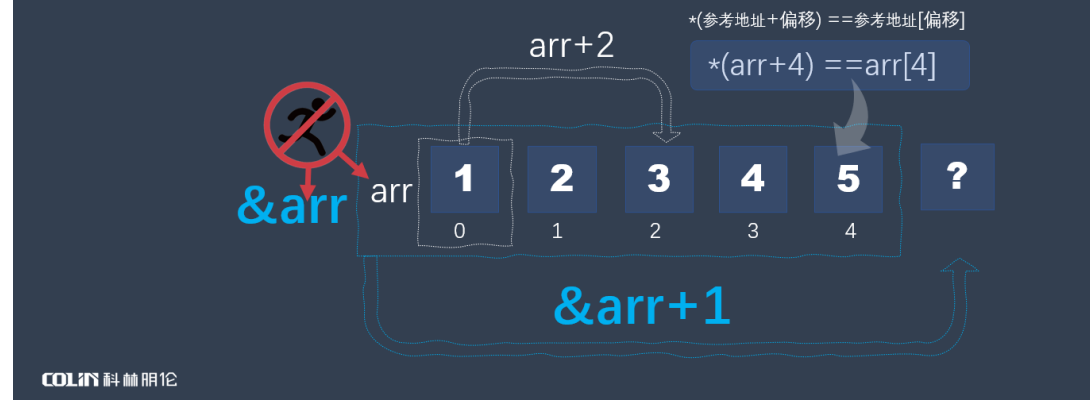
Microsoft Visual Studio 调试控制台

```
1703532 1703536
1703532 1703552
```

根据打印的结果可以看到：数组名 as+1 得到的地址差是 4 即一个 int 类型的空间大小，而 &as+1 地址差是 20；也就是 5 个 int，即整个数组的空间大小。

《一维数组·名·特性》

```
int arr[5]={1,2,3,4,5}
```



根据上图可以得到如下结论：

- 1、数组名是首个成员的地址，所以**数组名+整数**是数组成员间的移动。
- 2、数组名用&取地址运算后，得到的是整个数组类型（即 5 个 int 类型变量空间）的指针，所以再加 1 是整个数组类型的空间偏移。也就是找到的是数组 as 后面的地址。

```
printf("%d %d\n", sizeof(as), sizeof(*as));
```

感受一下上面的代码：`sizeof(as)` 得到的是整个数组空间的大小，其背后的过程是：`&as` 的计算结果是一个 `int [5]` 这个数组类型的指针。再用“*”操作得到的是 `int [5]` 这个数组类型的空间使用权。由于*和&为互逆运算，所以抵消后就是数组名。请试着解释如下代码执行结果。

```
int* p = as;
```

```
printf("%d %d\n", sizeof(p), sizeof(as));
```

p 是指针变量，所以指针变量的大小和 x86 与 x64 的选择有关，如果 x86 结果是 4。而 as 是指针常量，`sizeof(as)` 计算得到的是其代表的整个数组空间的大小。

55、 指针数组与数组指针分别是什么？

C 语言中的指针就是内存地址。数组是某类型变量的数量扩展，也是一种数据类型。所谓指针数组说的是指针类型的数组，数组的每个成员都是一个指针变量。逻辑如下图所示：



```

int a=1,b=2,c=3,d=4;
int arr[3] = { 7,8,9 };
int* p[4] = { &a, &arr[1],arr,arr+2 };
int len = sizeof(p) / sizeof(p[0]);
for (int i=0 ;i<len ; i++)
{
    printf("%d\n",*p[i] );
}

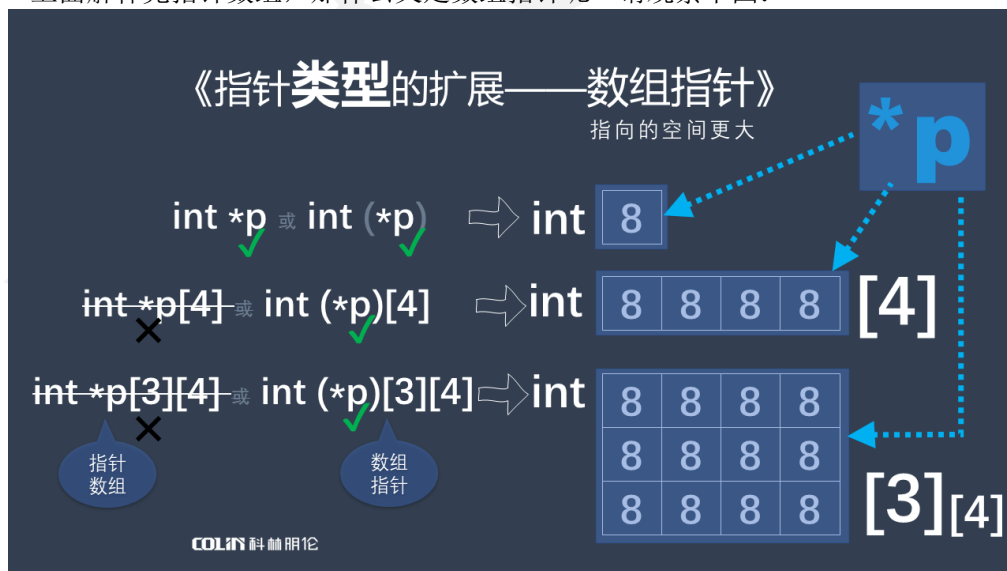
```

观察上面的代码，`int*p[4]`就是一个指针数组。【注意】对于指针相关类型的辨识一定要会观察其代表的类型，`int*`代表的是个 `int` 型指针，`[4]`是该类型变量数量的扩展，所以 `int* [4]` 整体代表的是一个数组类型，即拥有 4 个成员的指针类型的数组（简称：指针数组）。另外用调试器局部变量窗口查看指针数组 `p` 的初始值发现：每个成员都是 `int` 类型的指针。再来观察其初始值：`&a` 代表一个变量的地址，`&arr[1]`代表下标 1 成员的地址，`arr` 数数组名代表首元素地址，`arr+2` 代表地址偏移俩个类型变量也就是数组下标 2 成员的地址。

在遍历时，`*p[i]`是如何计算的呢？首先`[]`的优先级别高，所以 `p[i]`先执行得到指针数组成员的值（即一个指针），然后再执行“*”计算，得到地址对应空间的值。我们用调试器查看内存情况如下：

p	0x00effbe4 {0x00effc40 {1}, 0x00effc00 {8}, 0x0...	int *[4]
[0]	0x00effc40 {1}	int *
	1	int
[1]	0x00effc00 {8}	int *
	8	int
[2]	0x00effbfc {7}	int *
	7	int
[3]	0x00effc04 {9}	int *
	9	int

上面解释完指针数组，那什么又是数组指针呢？请观察下图：



数组指针是对指针可以指向更大内存空间的扩展类型。比如 `int*` 是可以指向一个 `int` 类型空间的指针，利用这个类型的指针变量就可以得到 4 个字节的空间使用权。如果比 `int` 大出几倍的空间如何处理呢？这个时候数组就出现了。比如：`int [4]`代表的是 4 个 `int` 的空间，可以理解为一种扩展的数据类型。此时如何定义这个空间的指针呢？方法是把数组名的位置用“*”代替，即 `int(*)[4]`，其就可以是一个扩展空间了的指针类型，代表的是指向 4 个 `int` 的指针类型，又因为 4 个 `int` 就是一个 `int` 类型的一维数组所以简称为：一维数组指针。依次类推。如果一个指针能够指向一个二维数

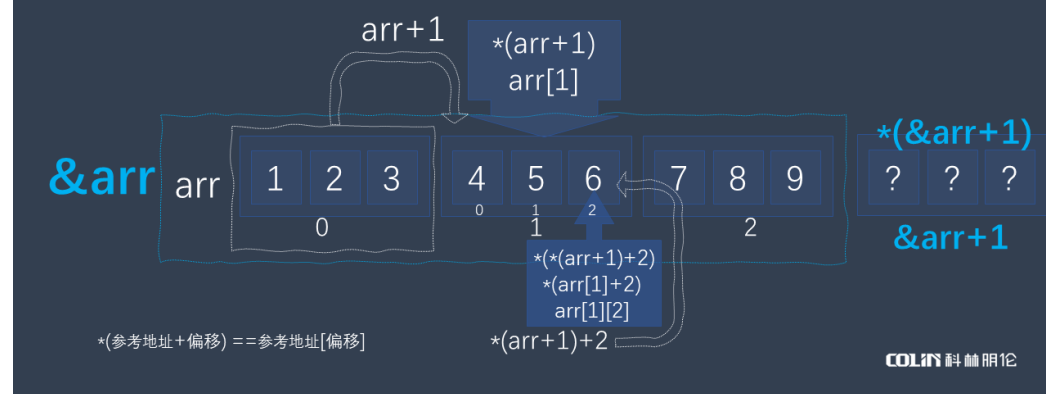
组，那么它的定义方式为：

- 1、int [3][4] 先确定数组的整体空间大小。
- 2、在数组名的位置上用(*)代替。最终得到 int (*) [3][4]；即二维数组指针类型。
- 3、在*的后面加上指针变量名例如 parr, 就得到一个二维数组指针的变量。
完整写法：int (*parr) [3][4]。

56、 二维数组遍历的底层过程是什么？

《二维数组·名·特性》

```
int arr[3][3]={1,2,3,4,5,6,7,8,9}
```



```
int arr[3][4]={1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 };
```

如下代码用于检测二维数组名增加整数后，其偏移字节数的规律。

```
printf("%d %d %d\n", arr, arr+1, arr+2);
```

Microsoft Visual Studio 调试控制台

```
3864776 3864792 3864808
```

根据打印结果可以看到二维数组名每增加 1 都是 16 个字节的偏移。由于数组的基本类型是 int，所以 16 个字节就是 4 个 int 的空间，也就是二维数组一行的空间大小。所以二维数组名增加整数得到的是某行的行首地址。我们把这个过程称为“找到”行的地址。那么在这个地址基础上，再利用*运算符将“得到”整行的行空间。如下代码：

```
printf("%d %d %d\n", sizeof(*(arr + 0)), sizeof(*(arr + 1)), sizeof(*(arr + 2)));
```

Microsoft Visual Studio 调试控制台

```
16 16 16
```

用 sizeof 运算符计算“行空间”的大小得到结果是 16 个字节。即每行有 4 个 int 类型的变量总和。

由于[]运算符的执行过程是：地址[偏移]等价于*(地址+偏移)。所以上面代码也可用[]的方式来表示“行空间”。代码如下：

```
printf("%d %d %d\n", arr[0], arr[1], arr[2]);
```

但是行空间并不是具体的一个 int 类型变量，所以需要在行空间的“内部”继续访问列的空间。下面代码就以下标为 1（即第一行）的行空间为例，演示如何找到行内的某个列空间的地址。

```
printf("%d %d %d\n", *(arr + 1)+0, *(arr + 1) + 1, *(arr + 1) + 2);
```

Microsoft Visual Studio 调试控制台

```
4060992 4060996 4061000
```

根据打印结果我们看到输出全是地址，它们之间的差为 4 个字节。又由于*(arr + 1)等价于 arr[1]所以上面代码也可写成：

```
printf("%d %d %d\n", arr[1] + 0, arr[1] + 1, arr[1] + 2);
```

上面的代码的作用就是“找到”行内的某列地址，那么接下来就利用“找到”的地址，进而“得到”对应的列空间。代码如下：

```
printf("%d %d %d\n", *((arr + 1) + 0), *((arr + 1) + 1), *((arr + 1) + 2));
```

由于 $*(arr + 1)$ 等价于 $arr[1]$ ，所以代码可以修改如下：

```
printf("%d %d %d\n", *(arr[1] + 0), *(arr[1] + 1), *(arr[1] + 2));
```

利用**地址[偏移]**等价于 **$*(地址+偏移)$** 的关系再简化代码的写法：

```
printf("%d %d %d\n", arr[1][0], arr[1][1], arr[1][2]);
```

就形成了我们比较熟悉并常用的“先行后列”下标的访问方式了。理解了这个过程也就能理解更多多维数组的实现原理了。

57、 二维数组名+1 与一维数组名+1 的区别？

不同维度的数组名+1 时，这个 1 代表的是什么？是当前数组最高维度的空间大小。比如 `int arr[4]` 这个一维数组的最高维度就是列，因此数组名+1 就是列的偏移。二维数组 `int arr[3][4]` 的最高维度是行，所以 `arr+1` 就是行的偏移。

以此类推 `int arr[3][4][5]` 这样一个三维数组，其数组名 `arr+1` 就是“一个平面”的偏移即 4 行×5 列×4 个字节（`int` 类型大小为 4），最终是 80 个字节的偏移。

58、 如何利用一维数组指针遍历二维数组所有成员？

```
int arr[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};
int(*ph)[4] = arr;
for (int h=0;h<3 ;h++)
{
    printf("%d %d %d %d %d\n", *(ph+0), **ph, *(ph+1), (*ph)[2], *ph[3], );
    ph++;
}
```

上面代码中定义了一个一维数组指针变量 `ph`，并且让其指向二维数组的第 0 行。在循环里让 `ph++`，使其不断移动到下一行实现二维数组内的行偏移。

在打印不同列的值时，代码中列举了可能的写法；其中最后的写法是错误的。下面我们分析每个写法的计算逻辑。

$*(ph+0)$ 的执行过程：由于 `ph` 指向了一行，`*ph` 就会得到行空间，`*ph+0` 相当于在行空间内移动到第 0 列，最后 $*(ph+0)$ 得到的是第 0 列的空间值。由于 $*ph+0$ 的计算结果还是`*ph` 本身，所以 $*(ph+0)$ 就变为了`**ph`，给人的感类似一个二级指针，但其表达的目的不是二级指针的思维模式。这种迷惑性写法比较容易以笔试题形式出现。

$*(ph+1)$ 的执行过程遵循“找到地址得到空间”的过程分析，使 `ph` 能够“找到”二维数组某行的地址，`*ph` 代表“得到”指向行的行空间，`*ph+1` 代表“找到”行空间内的下标 1 列的地址， $*(ph+1)$ 代表的是“得到”本行空间内，下标为 1 的成员的空間。

$(ph)[2]$ 的执行过程： (ph) 先执行，目的是“得到”行空间， $[2]$ 代表的是在行空间内再“得到”列空间。

`ph[3]` 这个写法是错误的。因为`[]`的级别高于`*`所以 `p[3]` 先执行，但 `ph[3]` 代表了什么呢？当我们要理解`[]`的执行结果时，需要把其拆解为“找到”和“得到”的计算过程：即 `ph[3]` 变为 $*(ph+3)$ ，由于 `ph` 的类型是 `int(*)[4]`，即 4 个 `int` 的一维数组指针类型，所以 `ph+3` 就是在 `ph` 当前指向地址的基础上偏移 3 行（每行：4 个成员×4 给字节），最终偏移了 48 个字节。分析到这里时，`ph` 已经指向了不属于我们的内存空间了。

所以使用数组指针访问指向成员时务必理解指针变量的类型以及运算符的优先级。

59、 如何利用二维数组指针遍历二维数组所有成员？

```
int arr[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};
int(*pm)[3][4] = &arr;
for (int h=0 ;h<3 ;h++)
{
    for (int k=0 ;k<4 ;k++)
    {
        printf("%d %d %d\n",arr[h][k], (*pm)[h][k],    *((*pm+h)+k)    );
    }
}
```

```
}  
}
```

上面代码中 `int(*pm)[3][4]` 是一个二维数组指针变量，其初始值 `&arr` 代表的是整个二维数组类型空间的首地址，虽然去掉 `&` 也不会影响执行结果，但意义不同；`arr` 代表第 0 行的行空间首地址。另外请观察在利用二维数组指针变量 `pm` 访问数组成员的写法与数组名访问有哪些不同。由于 `pm` 指向了整个二维数组，`*pm` 就代表“得到”整个二维数组的空间。`(*pm)[h][k]` 就代表 `pm` 指向的二维数组空间内的某行某列的值。将其展开后的写法为 `*(*pm+h)+k`。其过程为：

- `*pm+h` 代表了二维空间内“找到”某行的地址，
- `*(*pm+h)` 代表了“得到”某行的行空间使用权，
- `*(*pm+h)+k` 代表了在行空间内“找到”某列的地址
- `*(*(*pm+h)+k)` 代表了“得到”某列的空间使用权。

我们一定要对[]背后的展开过程要非常熟悉，这样更利于理解指针的本质：找到地址 • 得到空间 • 偏移新地址。对笔试题中分析代码执行结果非常有帮助。

笔试真题

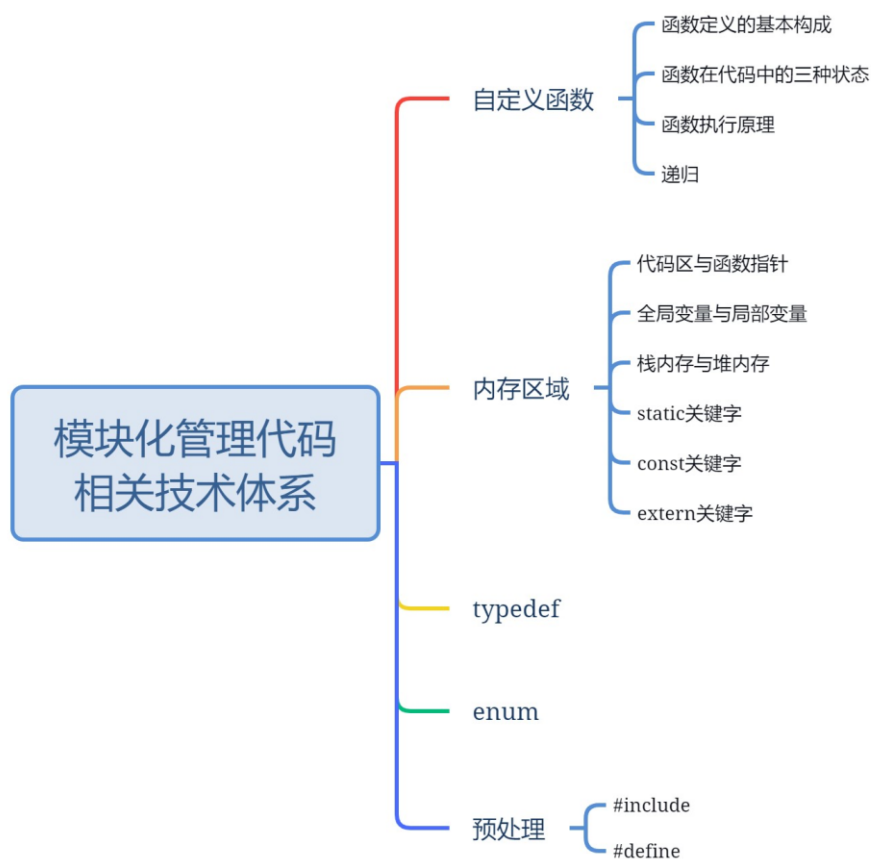
编程题

编程能力三：模块化管理代码

能力介绍

当创建一个真正的软件项目而非一个算法或 demo 时，代码量会增加许多，甚至需要多人协作完成。这时就需要管理的代码了。如何让代码便于阅读、易于维护扩展，增强复用性？这就需要采用模块化思想，按照高内聚低耦合的原则将项目划分为多个子程序，每个子程序内部有多个功能模块。分别用头文件与对应的源代码文件做函数功能规格设计与函数功能实现的分离。C 语言还有专门的语法能够让项目内的模块更具严谨性。

相关技术构成



学习技巧建议

- 1、用重构思维、测试驱动的手段使用函数的三种状态。
- 2、用调试器的局部变量、调用堆栈等窗口理解函数执行原理细节。
- 3、多用封装的思维理解函数封装流程、typedef 封装类型等代码技巧。
- 4、多做项目，在项目中体会高内聚低耦合原则的运用，以及不同关键字在项目中的作用。

Part 1 自定义函数

1、重构的基本思想是什么？

重构 (refactor) 其字面理解是重新改变代码结构。重构的前提是既有的代码已经完成了项目的功能，但内部的结构可能是杂乱的，这就需要对这个结构进行优化，让代码变得更易于人的阅读、方便维护、增强复用、以更灵活的应对可能的变化。但是无论怎样调整代码的内部结构都不要改变用户对软件功能的使用。重构是软件开发过程中非常日常的行为。并不是说代码完成功能就万事大吉了。优秀的开发者要经常性的 review (复审) 代码，优化代码。当我们对陌生领域项目缺乏经验时，无法用瀑布模型自上而下的做详细周密设计，就可以用原型迭代的思路像滚雪球一样开发一部分，重构优化，再继续下一部分；最终完成项目。

编程语言提供了专门的语法来实现结构化思维的编程。比如函数就是这样的语法。我们可以把流程代码封装进函数。函数就形成了一个功能模块，谁需要谁调用。这样重构思想就有了实现手段。所以函数的好处就是让项目变得可阅读、可维护、可复用。

2、函数与模块化的关系是什么？

模块化编程是编程的思维。软件也被称为“软件工程”。能够称之为工程说明它是复杂的、系统化的；是需要分工的，按工序分步实现组装的。根据生活中其它行业的工程化经验引入到软件开发中就是模块化的思维。函数 (英文 function 也可被称为方法 method) 是模块化思维的具体实现的技术手段之一。函数一词源于数学的概念，比如一个计算公式，参数的变化导致结果的变化。什么时候想计算，找到这个公式即可。再后来的面向对象语言中通常把函数称为方法，想表达的是一个功能用什么样的方法 (即具体流程步骤) 可以实现。一旦功能被实现了只需要找到它并执行它。

3、函数的三种状态及关系是什么？

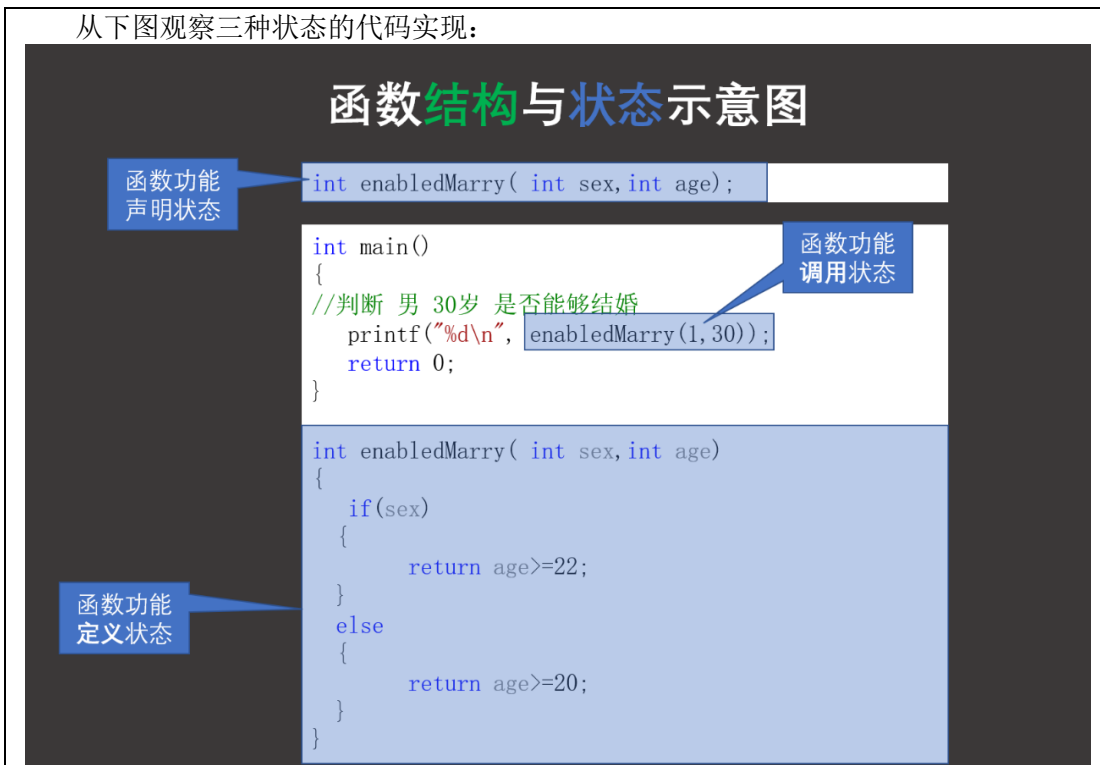


函数即是技术又是思维。从技术的角度看函数需要三种状态才能运行起来。

1、前置声明状态：即在调用函数功能前需要提前“告知”有此功能。
2、定义实现状态：即函数的功能具体如何实现？用流程结构描述具体实现的过程。我们把这个过程也称为“封装”——函数封装流程结构，流程结构必须写在函数里封装起来才不会触犯语法错误。

3、调用执行状态：函数的功能实现后是“死的”，只有被调用才能“激活”执行。

从下图观察三种状态的代码实现：

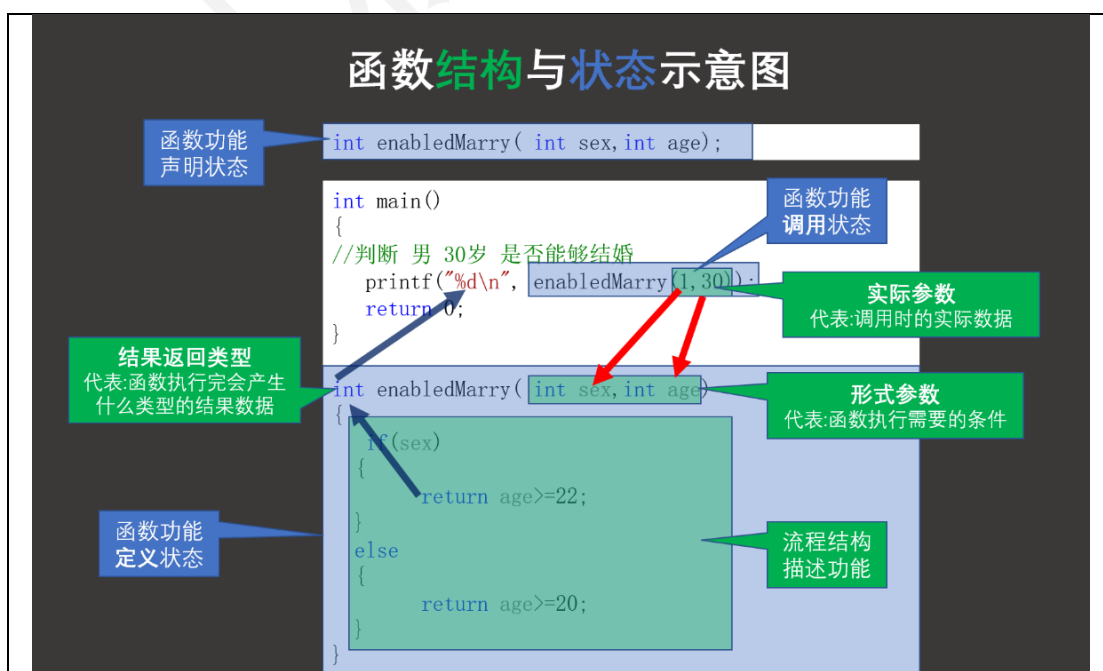


下面再从开发思维角度说明一下三种状态的关系。

从开发流程角度看，函数的开发过程建议：

- 1、先写函数声明：声明的语法是只有函数的头部而没有代码的实现过程，所以这个特点比较适合功能的特征（参数与返回类型）描述与设计思维描述的环节。很多语言也把这个语法称为接口。
- 2、再写函数调用：这样的目的是从应用场景角度验证函数规划的参数条件于返回结果是否满足应用的需要。如参数个数，顺序，类型等是否合理。
- 3、最后写函数定义：经过应用场景调用下的验证，我们基本对这个函数的设计目的及应用已经非常清楚了，这时开发就会更加的有的放矢。

4、自定义函数的结构组成有哪些部位？举例说明



根据上图，一个函数就是一个输入与输出的功能模型。输入就是此函数功能需要外部支持的条件，是以参数的方式来实现。参数分为形式参数（简称：形参）和实际参数（简称：实参）。所谓形参就是表达一种“需要”的列表，比如说：完成一道菜需要哪些食材与调料。但实际调用执行时，具体传入的就是实参。实参不同最终的结果也就不同。

函数功能的实现过程就是在一对 { } 里面的流程结构的执行。最终这个控制流程处理完成后会产生一个结果（注意不是多个）。这个结果需要说明类型即返回值类型。

return 关键字后面的表达式就是配合返回类型的语句。使用时尽量做到返回的实际值与返回类型要匹配。如果不需要一个具体的数据值被返回，那么返回值类型位置用 void 关键字进行占位。

5、函数可以嵌套定义吗？

所谓的函数嵌套定义是指函数定义体内，又定义了另一个函数。比如：

```
int fun()
{
    int b()
}
```

上例中 b 函数的定义写在了 fun 函数的定义里就是嵌套定义，此时会报出语法错误的。在 C 语言中函数不可以嵌套定义。C 语言中函数之间是彼此独立的，是互相被调用的关系。

6、函数的执行原理是什么？



参见上图，函数的执行原理（或称函数的执行过程）可以分次第性的理解。

表面上看，函数类似阿拉丁神灯中的神仙，它具备完成愿望的功能。但是只有“擦灯”的时候才能调用激发这个功能。一旦其完成了人发出的愿望（实际参数）后，此功能就结束“消失”了。等再一次调用时才会被重新执行。这个过程简单看来就是：调用执行、执行完消失。

再进一步看函数调用执行的过程中；操作系统会为本项目内所有函数的调用次序维护一个称为 stack（栈）的数据结构，我们可以称其为“函数调用栈”。VS2019 在调试状态下会有一个“调用堆栈”窗口来展示函数当前的调用状态，如下图：



栈结构的特点是 Last in first out (LIFO) 即最后入栈的先出栈，最先入栈的最后出栈。这个过程简称（后进先出）。被调用的函数先入栈，一个程序至少会有 main 函数，因此 main 先入栈，main 函数再去调用其它函数，其它函数还会调用别的函数，谁被调用谁入栈。等在栈最上面的（栈顶）函数执行完，它就会被操作系统从这个调用栈中被 pop（即弹出）出栈。哪个函数执行完，哪个函数就出栈。当然最后一个出栈的就是 main 函数了。再深入了解细节就需要了解内存区域及函数指针的相关概念了。

7、函数之间的数据传递方向与值传递的方式有哪些？

函数之间的关系为调用关系。就如同两个人，一个人买可乐，另一个人卖可乐。一个人负责发起调用（购买），并传入钱数和瓶数；另一个人负责执行（售卖配送）并返回所需要的结果——一手交钱一手交货。根据上面的场景描述，函数的数据传递方向可以分为**传入**→与**返回**。那么数据值在传入与返回过程中是如何被传递的呢？——数据值传递的过程是其“复制方式”的过程。下面用两个例子说明：

```
void exchange(int a, int b);
void swap(int* a, int* b);
int main ( void )
{
    int a = 10, b = 20;
    printf("a=%d b=%d\n", a, b);

    exchange(a, b);
    printf("a=%d b=%d\n", a, b);

    swap(&a, &b);
    printf("a=%d b=%d\n", a, b);

    return 0;
}

void exchange(int a, int b)
{
    int t;
    t = a; a = b; b = t;
}

void swap(int*a, int *b)
{
    int t;
    t = *a; *a = *b; *b = t;
}
```

Microsoft Visual Studio 调试控制台

```
a=10 b=20
a=10 b=20
a=20 b=10
```

根据上面的结果可以看到，第一次调用的 exchange 函数虽然其逻辑是交换两个变量的值，但是并没有改变 main 函数中变量 a, b 的值。那是因为变量 a, b 作为实参时，是把这两个变量的当前值以复制的方式传递给了 exchange 函数的形参 a, b。这里注意两点：

第一点：形参也是一种局部变量，其是伴随函数调用而被创建出来的临时存储空间，一旦函数执行结束完毕后，exchange 函数内相关局部变量就会被注销回收。

第二点：函数之间数据传递（无论是传入或是返回）都是以数据值的复制方式赋值到对应的形参变量。所以 exchange 函数被执行时，交换的其实是形参 a,b 两个局部变量的值，可是当 exchange 函数执行完毕后两个形参就被注销回收，其交换结果不会反应到 main 函数的局部变量 a,b 中的。如何能够改变 main 函数内部的 a、b 变量的值呢。唯有利用 a、b 变量在内存中的唯一地址，通过指针间接访问唯一空间实现其交互。

所以第二个 swap 函数的形参是指针类型，对应的实参是 main 函数中 a、b 两个变量的地址。地址在内存中是唯一的，所以 swap 函数内利用收到的地址交换了 main 函数变量 a、b 两个空间的值，所以 swap 函数执行结束后，main 函数也能看到交换的结果。我们通常把变量地址的实参传递称为**传址**，把变量自身当前值的传递称为**传值**。传址如果形象的理解相当于把变量自身传递出去的感觉。但无论传值还是传地址都是以复制的方式把表达式的计算结果复制到形式参数这个局部变量的存储空间内。

8、利用函数把数据“带回”到调用位置的手段有哪些？

按照自定义函数的模型，函数可以有多个参数数据的传入，但只能有一个数据的返回。但是有的时候我们需要函数执行完毕后能够把多个数据“带回”。

此时就可以用指针作为参数来解决。方法就是调用函数之前，先准备好存储空间，调用时把存储空间的地址作为实参传递，这样函数执行时可以把数据存储到实参指向的内存空间，用这样的方式就可以实现多个数据的“带回”效果。这个过程类似于到超市购物，自备了几个购物袋；把不同的物品放到对应的购物袋中。在很多 C 语言的库函数的设计手法都采用了这样的方式：即提供返回值方式的“带回”，又提供形参指针方式的“带回”。例如得到当前系统的时间函数：

```
#include <stdio.h>
#include <time.h>
int main()
{
    time_t t;
    //方式1: 返回值方式
    t=time(NULL);
    printf("%lld\n",t);
    //方式1: 形参指针方式
    time(&t);
    printf("%lld\n", t);
    return 0;
}
```

time 函数的函数结构：time_t time (time_t* timer); 。time_t 是被 typedef 后的类型别名，其本质类型是 long long int 类型。通过 time 函数的声明可以看到，我们既可以用其返回值类型的设置得到一个当前时间，也可以把一个存储时间变量的地址传递给它，让其把当前时间存入到我们指定的内存空间。

9、实参的传递顺序是什么？

观察如下代码是否会产生运行的 bug？

```
#include <stdio.h>
#include <time.h>
int fun(int a, int b, int c);
int main()
{
    int x;
    fun(x=3, x + 5, x ++);
    fun(x++, x + 5, x = 3);
    return 0;
}

int fun(int a, int b, int c)
```

```
{
    printf("a=%d b=%d c=%d\n", a, b, c);
}
```

当我们在 VS2019 中执行时会产生如下的 bug 提示：



它的提示意思是：x 变量被使用了，但是其并没有初始化。

这个提示主要是对：fun(x=3, x + 5, x ++);这句调用而提示的。为什么是这样呢？我们可能会认为参数的执行顺序应该从左侧开始，依次把实参传入到对应的形参。但从这个提示来看，这个过程恰恰是相反的。它会从最右侧开始计算第一个实参 x++。因为局部变量 x 没有初始值，x++就无法自增 1，因此产生了错误提示。所以用 fun(x++, x + 5, x = 3);这句调用就不会有这种运行时的 bug 提示了。其运行结果如下。

是否和你想的一样呢？也许你认为应该是：a=3 b=8 c=4。其实结果并不是！

分析一下：首先 x=3 这个表达式先执行，x 的值 3 将代表 x=3 这个赋值表达式的值，从道理上 3 应该传递给形参 c。可是打印结果却是 4。这是为什么呢。利用反汇编窗口，查看高级语言的源代码，编译器对应的汇编指令如下：

```
int x;
int y;
fun(x++, x + 5, x = 3);
007C63B5 mov     dword ptr [x], 3
007C63BC mov     eax, dword ptr [x]
007C63BF mov     dword ptr [ebp-0DCh], eax
007C63C5 mov     ecx, dword ptr [x]
007C63C8 add     ecx, 1
007C63CB mov     dword ptr [x], ecx
007C63CE mov     edx, dword ptr [x]
007C63D1 push    edx
007C63D2 mov     eax, dword ptr [x]
007C63D5 add     eax, 5
007C63D8 push    eax
007C63D9 mov     ecx, dword ptr [ebp-0DCh]
007C63DF push    ecx
007C63E0 call    _fun (07C143Dh)
007C63E5 add     esp, 0Ch
return 0;
```

如果感兴趣，观察研究汇编语言可以大致了解到：

- 1、3 存入到了 x 变量
- 2、x++表达式的值 3 被存入到寄存器
- 3、x 自增 1 后存入计数器
- 4、把 x 值 4 加入寄存器，并 push 到实参栈
- 5、把 x+5 的值存入寄存器，并 push 到实参栈

6、把第 2 步的结果 3 也 push 到实参栈

当实参栈准备好后，最终调用 fun 函数，fun 函数的形参 a,b,c 分别收到了 3, 9, 4。

结论：实参的传递顺序从右到左，建议尽量不要写出本例中不易理解的执行顺序代码。

10、为什么数组做形参需要传入实参数组的有效长度（字符数组除外）？

观察如下代码的三个自定义函数，其目的是把一个整型数组的和计算出来并返回。

```
#include <stdio.h>
int getArraySum_m1(int arr[]);
int getArraySum_m2(int arr[], int length);
int getArraySum_m3(int (*arr)[6]);
int main()
{
    int arr[] = {1, 5, 7, 9, 8, 2};
    printf("%d\n", getArraySum_m1(arr));
    printf("%d\n", getArraySum_m2(arr, 6));
    printf("%d\n", getArraySum_m3(arr));
    return 0;
}

int getArraySum_m1(int arr[])
{
    int sum = 0;
    int length = sizeof(arr) / sizeof(arr[0]);
    for (int i=0; i<length;i++ )
    {
        sum += arr[i];
    }
    return sum;
}

int getArraySum_m2(int arr[], int length)
{
    int sum = 0;
    for (int i = 0; i < length; i++)
    {
        sum += arr[i];
    }
    return sum;
}

int getArraySum_m3(int (*arr)[6])
{
    int sum = 0;
    int length = 6;
    for (int i = 0; i < length; i++)
    {
        sum += (*arr)[i];
    }
    return sum;
}
```

Microsoft Visual Studio 调试控制台

```
1
32
32
```

从打印结果来看，第一个是错的，其余两个正确。

第一个错误的根本原因是形参的语法是一个数组类型，但是其本质是一个指针类型变量。在VS2019中，把鼠标放置在函数名字上可以看到其函数声明。

```
int getArraySum_m1(int arr[])
{
    int
```

即使给出数组形参的长度也是一样的。

```
int getArraySum_m1(int arr[6])
{
    int SUM
```

从截图可以看到形参数组arr无论是否给予长度，都会被识别成一个指针变量。因此getArraySum_m1函数中的计算数组长度代码：

```
int length = sizeof(arr) / sizeof(arr[0]);
```

其sizeof(arr)计算的是指针变量arr的空间大小，如果是x86环境下，计算结果是4。由于arr[0]相当于*(arr+0)，即arr指向的空间大小。

由于arr的指针指向类型是int所以sizeof(arr[0])的计算结果也是4。所以sizeof(arr) / sizeof(arr[0])的计算结果是1。很显然这个getArraySum_m1函数是不会正确计算出数组的总和的。

基于这种情况，比较常规的方法是把数组的首地址和实际的有效长度都作为参数传递给函数，例如getArraySum_m2这个函数的做法。

我们再观察getArraySum_m3这个函数的做法，它的形参为int(*arr)[6]，即数组指针。arr此时的类型是指向了6个int空间的指针变量，利用arr这个数组指针可以访问整个数组空间。它的优点是函数参数只有一个即可。这样做的缺点是这个函数的灵活度就比较差，因此数组指针arr定义时明确说明了指向的数组空间的大小必须是6个成员，如果实际传入的数组空间长度变大或变小就不灵活了。另外：建议此方式的实参还是写成&arr（整个数组类型的首地址）更合理，这样类型就保持一致了。。

综合来看建议采用int getArraySum_m2(int arr[], int length)这种方式，可以最大灵活的应对不同长度的数组处理。

11、 return 语句的作用与注意事项有哪些？

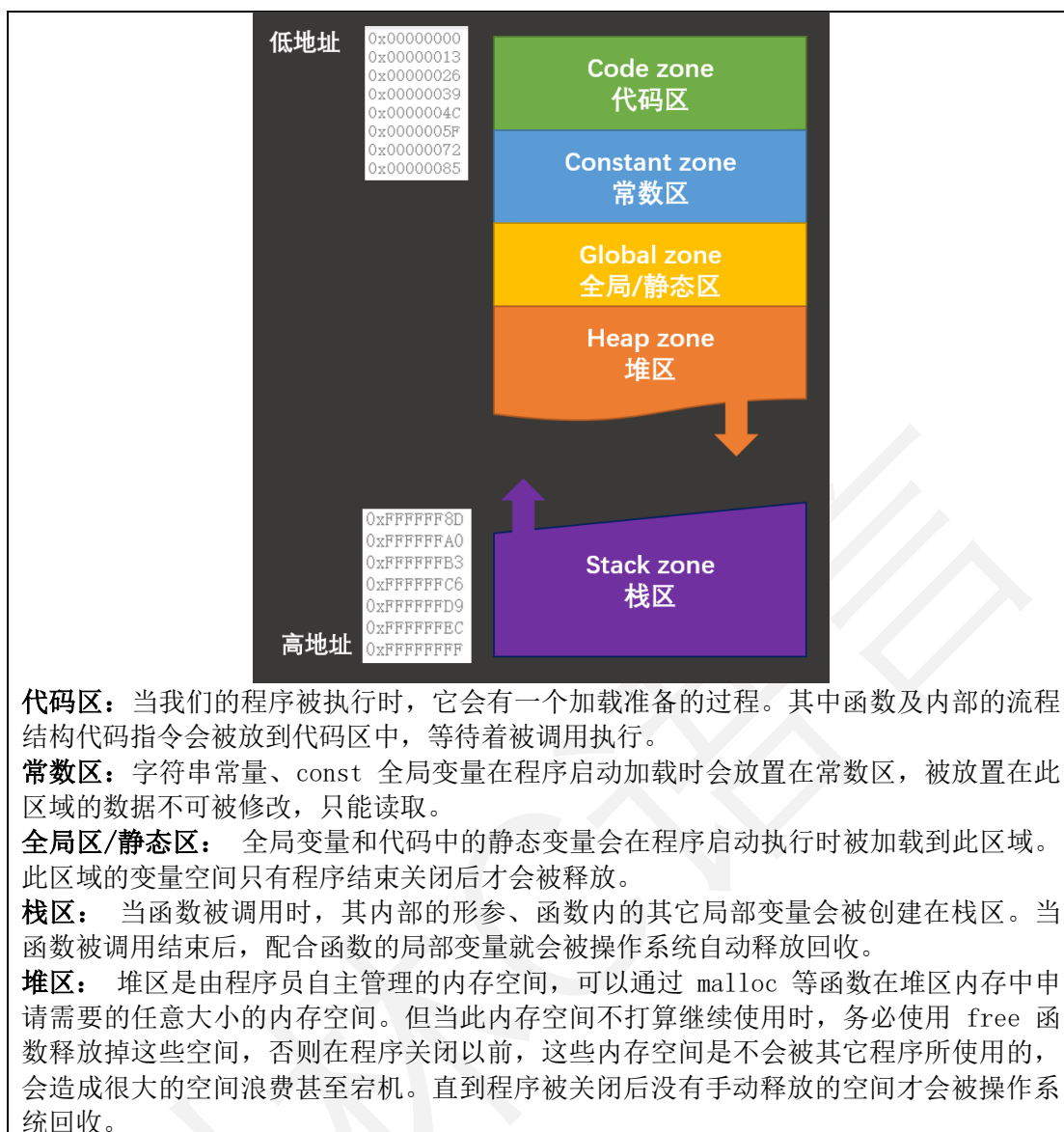
return 的作用是结束函数调用并返回符合返回类型设定的一个结果值。在使用return时有如下几点需要注意：

- 1、如果返回类型是 void 即无类型，也就是不需要 return 返回一个结果，那么函数最后一句的 return; 可以省略。
- 2、如果返回类型不是 void，那么函数中必须有一个 return 返回一个匹配类型的值。如果是 C++ 编译器会报编译错误。C 编译器由于语法检测不算非常严格可能不报错误。建议明确写明。
- 3、如果函数内的流程结构打算中途结束函数执行，那么 return 前务必以 if 条件语句进行控制。

Part 2 内存区域

12、 内存区域有哪些及其作用？

内存空间是由操作系统来管理的，这是个比较复杂的过程。我们在编程过程中创建了变量，如果仅仅知道是在内存中创建的还是不足够的。内存的管理可理解为有很多的功能片区。它类似一所大房子，虽然都是空间，有的是卧室、有的是书房。不同的物品放在不同的房间。我们需要知道内存存在支撑我们程序的运行过程中，它被为了哪些不同的区域空间，如下图所示：

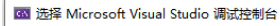
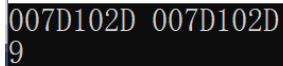


13、 函数名及函数执行的本质？

请观察如下代码：

```
#include <stdio.h>
int add(int a, int b);
int main()
{
    printf("%p %p\n", add, &add);
    printf("%d\n", add(4, 5));

    return 0;
}
int add(int a, int b)
{
    int sum = 0;
    sum = a + b;
    return sum;
}
```

上例代码中 `printf("%p %p\n", add, &add);` 的打印内容是函数名和函数名取地址后的计算结果，从打印结果看没有不同，总之都是一个地址。又由于项目被启动时程序中的所有函数都会被加载到代码区中，每个函数的名字就是其所在代码区的内存地址，所以通过函数名调用函数的本质是：使用函数名所代表的代码区地址找到函数代码，加上 () 传入实参就能启动执行这段代码。

14、 函数指针变量是什么？举例说明

函数指针变量就是能够存储函数所在代码区地址的变量。因为函数名的本质就是地址，是项目被执行时项目内的函数代码被加载放置的代码区的内存空间地址。所以如果能够找到这个地址，就可以用 () 并传入实参的方式执行这个地址空间里对应的代码指令。所以函数指针变量一旦保留了某函数在代码区的地址，就可以在项目的任意位置通过这个地址去执行其所在代码区里面的函数代码。例如：

```
#include <stdio.h>
int add(int a, int b);
int main()
{
    int (*p_add)(int , int );
    p_add = add;
    printf("%d\n", add(4,5));
    printf("%d\n", p_add(40, 50));
    printf("%d\n", (*p_add)(400 ,500));
    return 0;
}
int add(int a, int b)
{
    int sum = 0;
    sum = a + b;
    return sum;
}
```

上例代码中 `int (*p_add)(int , int );` 就是一个函数指针变量。在定义函数指针变量时先把其要指向的“函数声明”准备好，比如例中的 `int add(int a, int b);`；然后把函数名字用 (*) 代替，`int (*)(int , int );`；此时的写法就是描述了一个函数指针类型，此时只需要把函数的特征表达清楚即可。这个特征包括函数的参数类型及个数，还有返回值类型。此时参数的名字是可以没有的，使用时根据参数类型与个数等这样的函数特征去执行对应的函数代码。最后像所有指针变量的创建一样，在 * 的后面加上指针变量名即可。

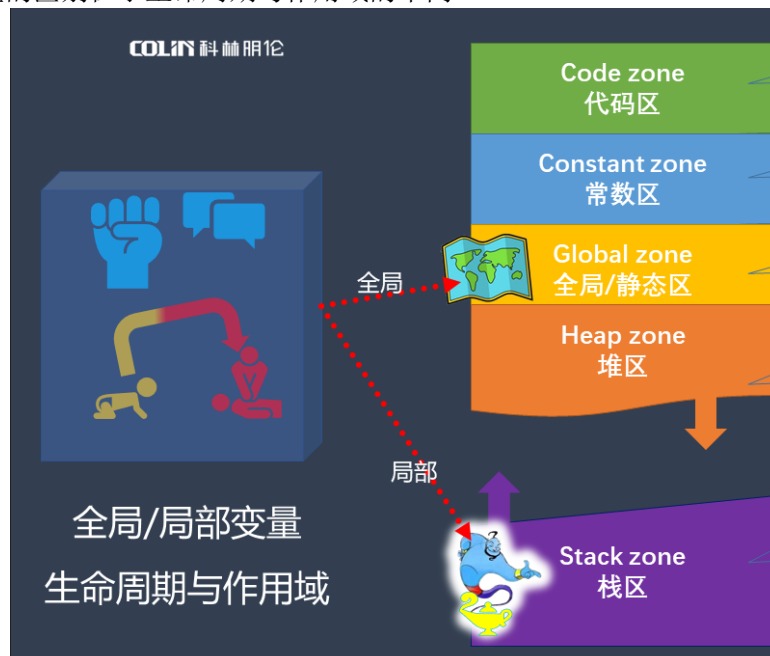
`p_add = add;` 此句就是让函数指针变量保留了符合它指向特征的函数——add 的代码区地址。

`p_add(40, 50)` 此写法可以称为隐式调用函数，也就是从字面上看不出是函数名还是函数指针变量，总之调用过程就是通过函数地址找到函数代码，以 () + 实参的方式执行代码指令。`(*p_add)(400 , 500);` 此写法也能执行 `p_add` 指向的函数代码。这个过程很符合一般指针“找到·得到”的使用习惯，这种方式可以称为显示调用。一般比较推荐隐式方式调用，代码比较简洁。其执行结果如下：




15、 全局变量与局部变量的区别？

全局变量通常指在函数体外部创建的变量。程序被启动执行时，它们会被创建在全局区。而局部变量指函数体内的定义的变量，包括形式参数在内都是局部变量。它们是伴随函数调用而被创建于栈区，函数调用结束后从栈区被回收的临时性变量。全局变量与局部变量的区别在于生命周期与作用域的不同。



从生命周期角度看：全局变量的生命周期在程序启动时就被创建在全局区，并且默认初始值置为 0。只有程序被关闭后，其占用的内存空间才被释放掉。而局部变量是其在函数被执行时才被创建，函数执行结束后被回收。当函数再被执行时局部变量再重新创建。

从作用域的角度看：全局变量可以被项目内任何的函数访问；而局部变量的访问只能在函数体内部，函数体外部是不能通过局部变量名字访问函数内的变量。另外，在一个函数被执行时，恰巧其函数内定义的局部变量名与全局变量名重名时，按照临近原则，全局变量被屏蔽，只有最近的局部变量才能被访问。所以局部变量与全局变量重名是不影响编译运行的。但是在同一个作用域内是不允许变量重复定义的。

16、 形参是局部变量吗？

形式参数是局部变量。形参的作用是在函数调用时接收实参的赋值。所以当函数被调用时，形式参数及函数体 {} 内的变量就会被全部创建在栈区内存空间，当函数执行完毕后，这些放置在栈区内配合此函数执行的局部变量就会被操作系统自动回收。

17、 堆内存使用方法与注意事项？

堆 (heap) 内存是由程序员根据程序的需要而自行申请的内存空间。堆内存给了程序员极大的自主性使用一片内存空间。但是当程序不需要这个空间时需要及时自行释放申请的堆内存空间。我们在申请内存空间时有三种方式。

方式一：

```
#include <stdio.h>
#include <malloc.h>
int main()
{
    int* p = malloc(sizeof(int));
    *p = 10;
```

```

    printf("%d\n", *p);
    free(p);
    return 0;
}

```

其中 malloc 函数用于向堆内存空间申请一片指定字节数量的内存空间，其参数是需要的字节总数量。建议用 sizeof 运算符计算出当前的编译器对 int 数据类型支持的字节数，不建议直接写 4。malloc 的返回值是一个 void* 类型的指针。因为 malloc 不确定你申请一片内存空间是干什么用的，它只能返回给你准备好的内存片空间的首地址。如果是 c++ 编译器，其语法检测比较严格，会认为 malloc 返回的类型是 void* 而左侧保留其地址的指针变量是 int*，两个类型不一致。此时如果报出这种编译错误可以用强转类型转换的语法强行匹配，从而消除编译错误。

例如：`int* p = (int*) malloc(sizeof(int));`

free 函数用于释放掉不使用的堆内存空间。我们只需要把申请的内存片区空间的首地址交给 free 函数即可。

如果像数组那样申请若干个 int 空间，那么代码如下：

```

#include <stdio.h>
#include <malloc.h>
int main()
{
    int size = 5;
    int* p = malloc(sizeof(int)*size);
    p[0] = 1;
    p[1] = 3;
    p[2] = 5;
    p[3] = 7;
    p[4] = 9;
    for (int i=0 ;i<size ;i++ )
    {
        printf("%d\n", p[i]);
    }
    free(p);
    p=NULL;
    return 0;
}

```

`sizeof(int)*size` 是向 malloc 函数提供的总共需要的空间大小。

方式二：

创建一个类似数组的一大片内存空间除了上述方式以外，还可以用 calloc 函数来实现。其声明语法为：`void*calloc(size_t count, size_t, size);`

其中第一个参数代表数据成员总数量，第二个参数代表其中一个数据成员所需的空间大小。具体代码如下：

```

#include <stdio.h>
#include <malloc.h>
int main()
{
    int size = 5;
    int* p = calloc(size, sizeof(int));
    p[0] = 1;
    p[1] = 3;
    p[2] = 5;
    p[3] = 7;
    p[4] = 9;
    for (int i=0 ;i<size ;i++ )
    {

```

```

        printf("%d\n", p[i]);

    }
    free(p);
    p=NULL;
    return 0;
}

```

那么 malloc 与 calloc 在创建类似数组空间有什么区别呢？表面上是参数不同，malloc 需要计算总的空间大小，calloc 则分开设置数量与单个的大小。

还有个区别是 calloc 具有空间初始化清零的功能：

内存 1	
地址: 0x00C50590	
0x00C50590	00 fc
0x00C505A8	00 00 00 00 00 00 00 00 24 7f 9b db 0f bb 00 08 48 4d c5 00 00
0x00C505C0	40 09 c5 00 5c ab 95 35 2a 04 8e 4c b9 42 2d 05 9b fe b1 b1 00

而 malloc 的空间初始化如下：

内存 1	
地址: 0x00C20590	
0x00C20590	cd cd cd cd cd cd cd cd cd cd cd cd cd cd cd cd cd cd cd fd fd fd fd
0x00C205A8	00 00 00 00 00 00 00 00 6c 86 d4 f7 60 59 00 08 48 4d c2 00 00 00 00
0x00C205C0	40 09 c2 00 5c ab 95 35 2a 04 8e 4c b9 42 2d 05 9b fe b1 b1 00 00 00 00

方式三：

如果当前的内存空间不够用，想要扩展一下大小该如何呢？可以使用 realloc 函数，例如：

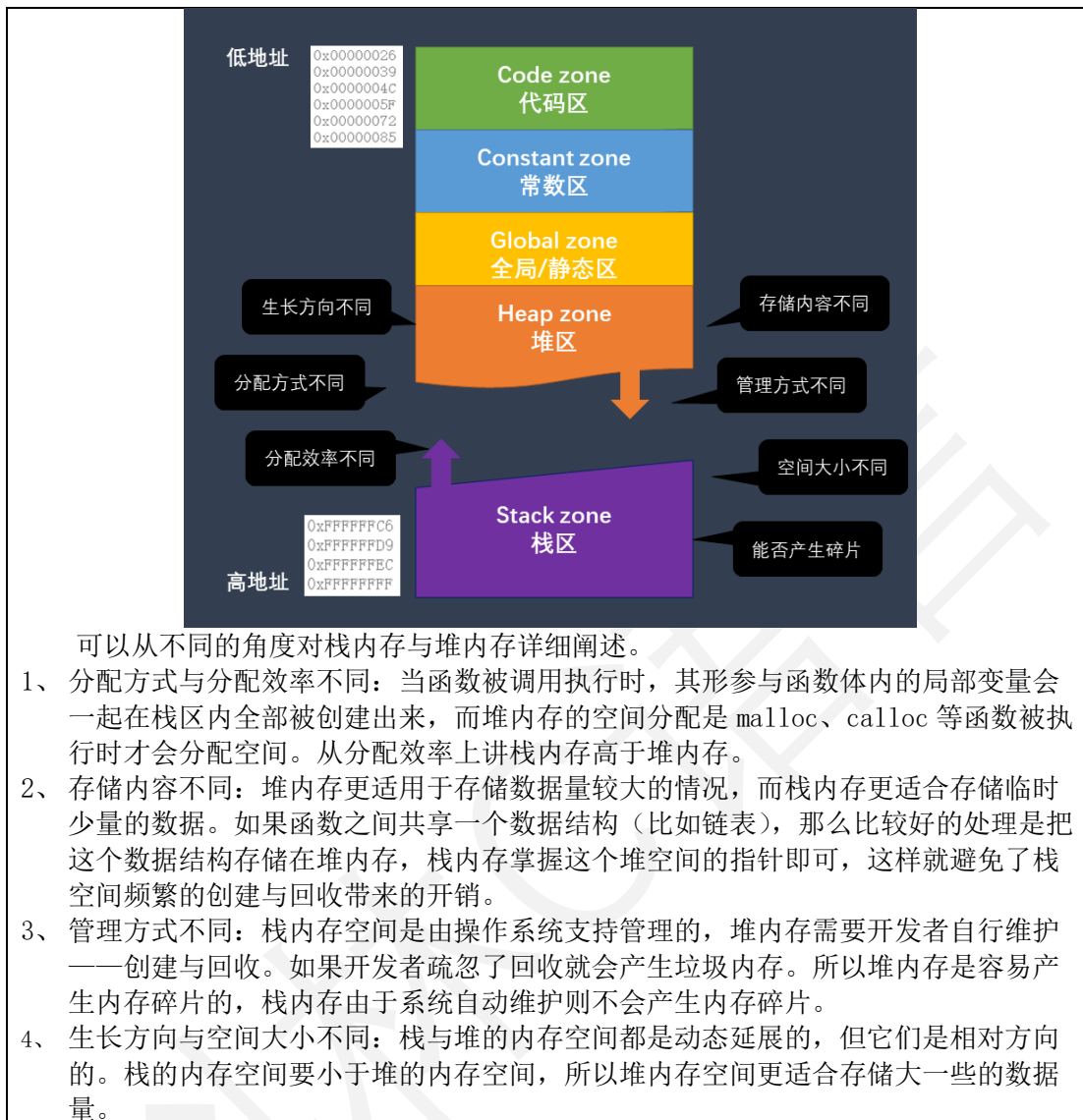
```

#include <stdio.h>
#include <malloc.h>
int main()
{
    int size = 5;
    int* p = calloc(size, sizeof(int));
    p[0] = 1;
    p[1] = 3;
    p[2] = 5;
    p[3] = 7;
    p[4] = 9;
    size += 3; //扩充后的总大小
    p = realloc(p, size * sizeof(int));
    p[5] = 10;
    p[6] = 20;
    p[7] = 30;
    for (int i=0 ;i<size ;i++ )
    {
        printf("%d\n", p[i]);
    }
    free(p);
    p=NULL;
    return 0;
}

```

从 `p = realloc(p, size * sizeof(int));` 这句代码可以看到 realloc 函数的第一个参数是现有的数据在堆的内存空间地址，第二个参数是扩容后最终空间的总大小。realloc 会根据扩容的情况进行空间的处理。如果现有空间的后续空间没有被使用且能够容纳所需的扩容，就在这个基础上扩充。如果不能，就会在堆内存中寻找一段能容纳我们要求大小的新的内存空间，再把现有的内容复制到新的空间，并返回新空间的地址。

18、 栈内存与堆内存的区别？



19、 extern 的作用？

extern 通常用于表达变量或函数来自外部（即其它源代码文件）。函数本身就是全局定义的，extern 是不需要在函数上写的。extern 用的比较多的地方还是对全局变量的声明。定义全局变量时一般是不需要写 extern 的，只需要给全局变量赋初始值就是明确定义了全局变量，而不写初始值则可能是声明。如果想在另一个源代码文件中使用此全局变量，则需要使用前用 extern 做一下这个变量的外部声明，用于告知此变量的创建来自其它源代码文件，这里仅仅是使用。注意声明是不能赋值的，如果赋值了就是定义行为，从而会产生全局变量重定义的错误。例如：

a. c 源代码文件的代码

```
int age = 5;
```

b. c 源代码文件中的文件

```
#include <stdio.h>
int main()
{
    extern int age;
    printf("%d\n", age);
}
```

```
return 0;
}
```

上例中 a.c 中定义了一个全局变量 age 并初始值为 5。我们要在 b.c 中准备使用这个 a.c 中 age 这个全局变量时，需要做一个外部变量的声明 `extern int age;` 这个声明相当于链接，就是说此变量的创建来自外部，我们这里只是使用它。如果代码改成如下写法：

```
#include <stdio.h>
extern int age = 8;
int main()
{
    printf("%d\n", age);
    return 0;
}
```

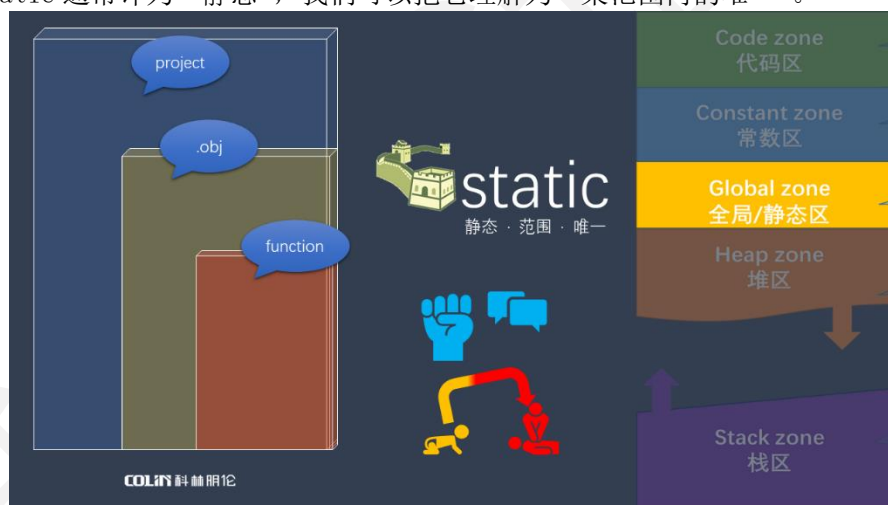
在 VS2019 中会提示如下的错误：

代码	说明
LNK1169	找到一个或多个多重定义的符号
LNK2005	_age 已经在 a.obj 中定义

因为有赋值行为就是定义变量，这样在全局变量区就存在一个名字被多重定义的错误。所以总结一个最佳实践就是全局变量定义时一定要赋初始值，不用其默认值 0。如果在项目的任意位置要访问这个变量时，用 `extern` 声明一下此变量来自外部，此时务必不要赋值。在项目中一个全局变量只能被定义一次但是可以声明无数次。

20、 static 修饰变量的作用？

`static` 通常译为“静态”，我们可以把它理解为“某范围内的唯一”。



这个范围指的是作用域，即变量可以访问的区域范围。访问范围可以分为三层。
 第一层访问范围：这个变量在整个项目（project）内的任意函数内都可被访问。
 第二层访问范围：这个变量只能被当前源代码文件内的所有函数访问，不可以被本源代码文件外的其它源代码文件的函数访问。
 第三层访问范围：这个变量仅能在函数内被访问即函数被调用执行过程中，变量能够被函数内访问，函数外用变量名不可访问到。

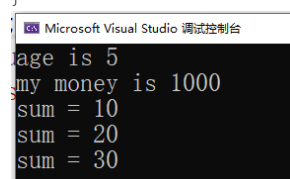
那么 `static` 在这三个层次的访问范围起到什么作用呢。先看如下代码：

```
文件 1: s1.c
int age = 5;
文件 2: s2.c
static int money = 1000;
int sayMoney()
{
    printf("my money is %d\n", money);
}
```

```

}
文件 3: s3.c
#include <stdio.h>
void test_static();
int sayMoney();
int main()
{
    extern int age;
    extern int money;
    printf("age is %d\n", age);
    //printf("money is %d\n", money);
    sayMoney();
    test_static();
    test_static();
    test_static();
    return 0;
}
void test_static()
{
    int num = 5;
    static int sum;
    num += 5;
    sum += num;
    printf("sum = %d\n", sum);
}

```



```

Microsoft Visual Studio 调试控制台
age is 5
my money is 1000
sum = 10
sum = 20
sum = 30

```

上面代码中的age变量是一个全局变量，在整个项目内都可以访问它。只需要用 `extern int age;` 做一下外部变量的声明即可。但是s2.c文件中的money变量，虽然写在了全局的位置上，但是其用static修饰后，它的作用域就被限定在了s2.c这个源代码文件内。所以s3.c要访问这个变量，即使做了 `extern int money;` 外部变量声明，但是当真正访问时：`printf("money is %d\n", money);` 就会报出：“无法解析外部符合money”这个编译错误。但是通过调用s2.c的 `int sayMoney();` 函数是可以访问money变量的。最后看 `void test_static()` 函数内的两个局部变量num和sum。其中num是非静态局部变量，当函数调用时它被创建在栈区，函数执行结束后，它的内存空间就被回收，函数第二次调用再重新创建…，而sum变量前用了static修饰符，它是在程序启动时，static修饰的变量就会创建在全局静态区并初始值为0，当函数被调用时，sum变量可以被函数内的代码访问到，当函数调用结束后，栈区的局部变量被回收，而static修饰的sum变量是不被回收的，放置全局静态区的变量是在程序关闭后才被回收。所以从打印结果可以看到sum变量的值在不断的累加并没有像num变量那样每一次都被重新创建。

总结一下：static修饰的变量其生命周期是程序启动时被创建在全局静态区，程序结束后才被回收。static修饰的变量可以将其作用域限定在某源代码文件内或某函数内。

21、 const 修饰全局变量与局部变量的差异是什么？

const 可以翻译成“常量”，即不可被改变的量。通常用于修饰变量让其内容在初始化后都不可以改变它，相当于 read-only（只读）。用它的目的是用语法的行为保护变量的值不被意外的修改。例如：

```
const double pi = 3.1415926;
```

```
int main()
{
    const double e = 2.71828;
    pi = 3.14;
    e = 2.71;
    return 0;
}
```

当我们试图修改变量 `pi` 和 `e` 的值时，在 vs2019 中就会报出“表达式必须是可修改的左值”这样的错误提示。说明两个变量的存储空间不可以通过变量名进行修改。

那么如果绕过变量名用指针的方式间接访问变量 `pi` 与 `e` 的内存空间是否可以实现修改呢？代码如下：

```
const double pi = 3.1415926;
int main()
{
    const double e = 2.71828;
    double* p_pi = &pi;
    double* p_e = &e;
    *p_pi = 3.14;
    *p_e = 2.71;
    printf("pi=%f e=%f\n", pi, e);
    return 0;
}
```

运行结果提示是：

已退出，代码为 -1073741819。当出现这个结果提示时，通常是指针间接访问空间的问题。我们用调试器观察如下：

The screenshot shows the Visual Studio 2019 IDE with a C++ project. The code is as follows:

```
7 double* p_pi = &pi;
8 double* p_e = &e;
9 *p_pi = 3.14;
10 *p_e = 2.71;
11 printf("pi=%f e=%f\n", pi, e);
12 return 0;
```

An error dialog box is open, titled "已引发异常" (Exception Raised). It shows the following details:

- Exception: 0xC0000005: 写入位置 0x00368BC8 时发生访问冲突。
- Location: 0x003622BA 处(位于 C_3_2.exe 中)引发的异常。
- Buttons: 复制详细信息 | 启动 Live Share 会话...
- Exception Settings:
 - ☒ 引发此异常类型时中断
 - 从以下位置引发时除外:
 - ☐ C_3_2.exe
- Buttons: 打开异常设置 | 编辑条件

The "局部变量" (Local Variables) window is also open, showing the following variables:

名称	值	类型
e	2.7182800000000000	const double
p_e	0x008ffe78 {2.7182800000000000}	double *
	2.7182800000000000	double
p_pi	0x00368bc8 {C_3_2.exe\const double pi} {3.1415926000000000}	double *
	3.1415926000000001	double

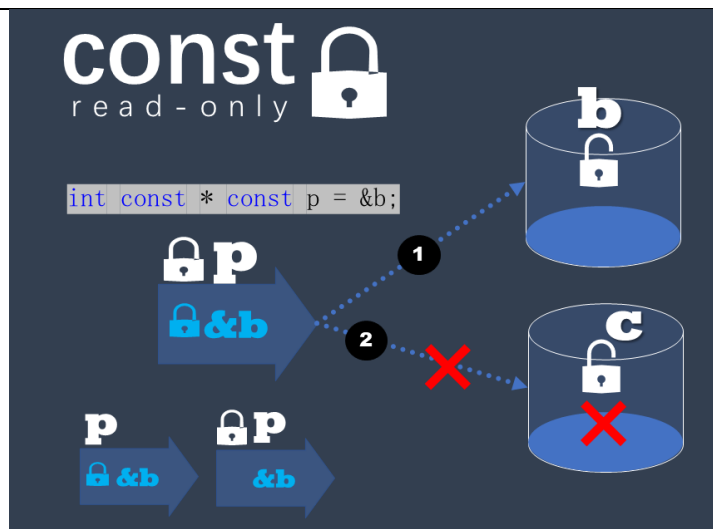
通过提示可以看到，利用指针变量 `p_pi` 去改变值时报出了：“发生访问冲突”的错误，仔细观察指向的地址，从局部变量窗口可以看到，这个地址是对应的一个全局的位置。其实 `pi` 这个变量加了 `const` 修饰符后，其创建的位置在常量区，常量区内的数据是不能被改变的，不论用直接访问还是间接访问的手段都不可以。把第 9 行去掉，代码可以通过编译并执行。

Microsoft Visual Studio 调试控制台

```
pi=3.141593 e=2.710000
```

从结果可以看到，利用 `p_e` 指针变量间接的修改了 `e` 变量空间的值。当然这种情况也和编译器的厂商及版本有关。这里只想说明全局变量加 `const`，其会被创建在常量区；而局部变量加 `const` 只能做语法级的初步检测，但是绕过变量名的访问就出现了矛盾。当然实际开发时不建议用这种手段。`const` 本质是提示开发者数据是只读的。

22、 const 修饰指针变量的方式有哪些？



观察上图回忆指针的基本使用模型是：首先有一个合法内存空间（全局静态区、栈区、堆区），然后用指针变量指向这个空间并实现间接访问。所以 `const` 在指针变量的修饰方面可以存在于两个位置的修饰：

1、避免指针不能指向其它的变量。

```
#include <stdio.h>
int main()
{
    int a = 3, b = 4;
    int* const p = &a;
    p = &b;
    return 0;
}
```

上面的代码中指针变量p在定义时，p前面有const关键字的限定使得p只能指向a变量，一旦写了p = &b;这条语句，p本打算指向b变量，但由于const的限定此时会报出编译错误，p是不可以被修改的。此时的指针变量p也可以称为“指针常量”。

2、避免指针不能间接修改指向的空间。

```
#include <stdio.h>
int main()
{
    int a = 3, b = 4;
    int const* p = &a;
    *p = 5;
    return 0;
}
```

上面代码中 `const` 放置在了 `*` 的前面（放在 `int` 前也可以），其意为让 `p` 不能用 `*p` 表达式修改指向的空间。所以 `*p=5` 会产生“表达式必须是可修改的左值”这样的编译错误。通常把这时的指针称为“常量指针”，即指针 `p` 指向了常量（不可修改其内容）。

3、让指针既不能改变指向又不能改变指向的空间内容。

```
#include <stdio.h>
int main()
{
    int a = 3, b = 4;
    int const* const p = &a;
    p = &b;
```

```

    *p = 5;
    return 0;
}

```

上面代码在定义 p 指针变量时用了两个 const，即让 p 不能指向其它的空间地址，也不允许 p 利用间接访问的 *p 表达式操作指向的空间内容。

Part 3 typedef

23、 typedef 的作用及用法？

typedef 的全称 type define 即类型定义，其目的就是对既有的类型另外再命名一个新的名字。新的名字与既有的名字都可使用。这样做的目的主要是让代码的可阅读性更强些。同一个数据类型在不同的应用场景里换个名称会让代码的阅读感更强。例如：

```

typedef char BYTE;
BYTE kb[1024];

```

char 通常理解为字符型。给人的概念就是保存一个字符。如果我们要表达 1K 的内存空间，可以用 char kb[1024] 表示，char 占一个字节，用它定义长度为 1024 的数组就是一个 KB 的内存空间。但是从阅读感看，感觉是让其存放 1024 个字符。此时把 char 另起一个别名 BYTE 即字节，用这个名字命名变量或数组，代码的阅读感会更强。

再比如：size_t、time_t、wchar_t 这样的类型，它们都是从基本数据类型重命名而来。

typedef unsigned int size_t 文件: vcruntime.h	typedef unsigned short wchar_t 文件: vcruntime.h	typedef long long time_t 文件: corecrt.h
--	---	---

size_t 的原始类型是 unsigned int。size_t 用于定义空间或数量大小的应用场景，这种情况下最小是 0，所以用 unsigned int 这个基本数据类型，但同 unsigned int 相比 size_t 的阅读感更强些。同理 time_t 作为时间的值，它要表达的是 1970 年 1 月 1 日到现在的秒数，这个数据比较大，它采用了 long long 类型。

24、 如何用 typedef 简化指针相关类型的定义？

首先要理解 C 语言指针的本质：是通过地址间接访问指向的空间。所以通过地址找到什么样的空间，如何使用空间的内容就是指针的指向类型所定义的。指针的类型包括基本类型指针、指向更大存储空间的数组指针以及指向函数所在代码区的函数指针。

如果是基本数据类型的指针用法比较简单：

```

typedef int* PINT;
PINT p1, p2;

```

上面代码中 p1, p2 都是 int* 类型的变量。对比之前的 int* q1, *q2; 此写法就不用再额外多写 * 号了。

数组指针与函数指针在定义相关指针变量时都需要描述性的写明指向类型的特征如：int (*p)[5]; 是一个数组指针变量，去掉 p 的 int (*)[5] 是数组指针类型。

double (*pfun)(double); 是函数指针变量，去掉指针变量名 pfun 后的

double (*) (double) 是函数指针类型。我们通常在用数据类型创建变量时更愿意用这样的格式：数据类型 变量名；即把数据类型放置在前面，变量名放置在后面。但上面两个指针变量需要在类型描述基础上把变量名放置在 * 后面。很多时候阅读起来比较麻烦，比较好的办法是把类型的描述过程用 typedef 封装起来。代码如下：

```

typedef double (*PFUN)(double);
typedef int (*P5INT)[5];

```

我们只需要用 typedef 关键字，然后把原指针变量名的位置用约定俗成的大写字母的类型名替换掉就完成了扩展指针类型的“封装”。之后就可以用 PFUN、P5INT 这样的类型名创建函数指针与数组指针了，例如：

```

P5INT pa1, pa2;
PFUN pf1, pf2;

```

局部变量

搜索(Ctrl+E) 搜索深度: 3

名称	值	类型
pa1	0xffffffff {?...	int[5] *
[0]	<无法读取内...	
[1]	<无法读取内...	
[2]	<无法读取内...	
[3]	<无法读取内...	
[4]	<无法读取内...	
pa2	0xffffffff {?...	int[5] *
[0]	<无法读取内...	
[1]	<无法读取内...	
[2]	<无法读取内...	
[3]	<无法读取内...	
[4]	<无法读取内...	
pf1	0xffffffff	double(*) (double)
pf2	0xffffffff	double(*) (double)

我们可以用调试器查看上面四个变量的类型。用 typedef 封装后可以增强代码的可读性，也会降低指针的使用难度。

25、 typedef 可以嵌套定义吗？

当然可以，所谓 typedef 的嵌套就是在新类型名的基础上继续创建新类型。例如：

```
typedef int ARR_INT_5[5];
typedef ARR_INT_5* P5INT;
typedef P5INT ARR_4_POINTER[4];
```

第一个是定义了一个 5 个整数的数组类型，第二个是在这个数组的基础上，为其指针创建了一个类型，这个类型就是一个数组指针，第三个为这个数组指针类型的 4 个数量的扩展（即数组）又创建了类型。最终 ARR_4_POINTER 所代表的类型是数组指针数组。我们用该类型创建一个变量并用调试器观察其类型：ARR_4_POINTER arr；

局部变量

搜索(Ctrl+E) 搜索深度: 3

名称	值	类型
arr	0x004ffe9c {0...}	int[5] *[4]
[0]	0xffffffff {?...	int[5] *
[1]	0xffffffff {?...	int[5] *
[2]	0xffffffff {?...	int[5] *
[3]	0xffffffff {?...	int[5] *

通过调试器可以看到这是个长度为 4 的数组，其每个成员都是一个指针，所以是指针数组。再展开其中一个数组成员：

局部变量

搜索(Ctrl+E) 搜索深度: 3

名称	值	类型
arr	0x004ffe9c {0...}	int[5] *[4]
[0]	0xffffffff {?...	int[5] *
[0]	<无法读取内...	
[1]	<无法读取内...	
[2]	<无法读取内...	
[3]	<无法读取内...	
[4]	<无法读取内...	
[2]	0xffffffff {?...	int[5] *
[3]	0xffffffff {?...	int[5] *

arr 是一个可以指向 5 个 int 类型的数组指针。通过上面的 typedef 嵌套的例子，我们可以把比较复杂的指针、数组、函数指针等混合类型用嵌套的方式进行分块定义。typedef int (*ARR_4_POINTER[4])[5]; 这一行实现的类型和上面代码是一样的，很明显一步到位的写法阅读感会差些。

Part 4 enum

26、 什么是枚举类型？

枚举 `enum` 是 C 语言的一种扩展数据类型。主要用于某数据类型的数据值是有限的且有意义的少量可能的值。例如血型可以是 A、B、O、AB。星期、星座、属相、方向等，这几个数据的值都是这样的特点。

27、 枚举的基本用法有哪些？

枚举的使用可分两步实现：

1、 创建枚举类型

其格式为 `enum 枚举名称 { 内容 1, 内容 2, 内容 3, ... };``enum 枚举名称 { 内容 1=整数, 内容 2=整数, 内容 3=整数, ... };`

例如

`enum blood_type {A, B, O, AB};`

此时 `blood_type` 为枚举类型名，A, B, O, AB 是可能的枚举值，每个值默认被赋予从 0 开始的整数，即 A=0, B=1, O=2, AB=3。当然也可以对枚举值指定整数。例如：

`enum blood_type { A=10, B=15, O, AB };` 此时 O 的值将为 16，即再上一个值的基础上进行累加 1。通过这里我们可以知道枚举的本质就是整型。

2、 使用枚举类型的变量

其格式为：`enum 枚举类型 = 预定的枚举值;` 例如：`enum blood_type xx = AB;``switch (xx)`

{

`case A:``break;``case B:``break;``case O:``break;``case AB:``break;``default:``break;`

}

通过上述代码可以看到创建枚举变量需要枚举关键字 `enum`。枚举变量的值一定是预制好的值 `A, B, O, AB` 的其中之一。枚举类型的作用就是增强代码的可阅读性。另外也可以用 `typedef` 简化枚举类型的使用。例如：

```
typedef enum blood_type {A, B, O, AB} BLOCK_TYPE;
BLOCK_TYPE xx = AB;
```

其中 `enum blood_type` 中的枚举名 `blood_type` 是可以省略的。

28、 枚举类型创建的变量能保存整数吗？

可以的，因为枚举值的本质就是整数，所以给枚举变量赋值可以用整型的字面值常量。但最好是定义枚举类型时的枚举值设定的整数。

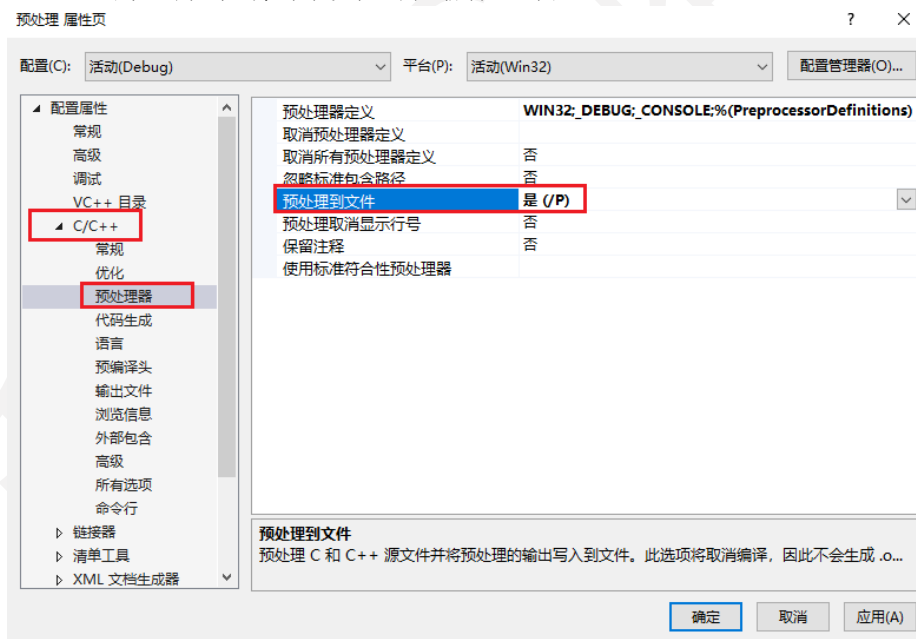
Part 5 预处理

29、 预处理是什么？

预处理也可称为预编译，就是在编译器工作前对源代码进行“变形”的过程。当我们写完如下代码后

```
#include<stdio.h>
int main()
{
    return 0;
}
```

在 VS2019 的“调试”菜单找到“调试属性”窗口



按图中的标注进行寻找设置，把“预处理到文件”选择为“是”。这样做的目的是可以查看到源代码被预处理后变成的“样子”。此时点击 VS2019 的“生成”菜单中的“生成 XX 项目”选项。然后再到项目所在的文件夹中 debug 文件夹来查看结果。进入后可以看到 .i 为扩展名的文件。用记事本等文本编辑器打开查看，可以看到：

```

10678 __pragma(pack(pop))
10679
10680 #pragma warning(pop)
10681 #line 2449 "C:\\Program Files
10682 #pragma external_header(pop)
10683 #line 2 "E:\\colinwork\\C语言老
10684 int main()
10685 {
10686
10687     return 0;
10688 }
10689

```

我们的 main 函数在最后，但是其前面有一万多行的代码。这个代码就是#include 的作用。即代码在编译器开始工作之前，被包含的头文件中的内容复制进我们的源代码内，如果这个过程没有任何问题时，才会把这个变形后的源代码进行编译。

总之所谓预处理就是在编译期前对源代码进行变形的过程。理解了之后别忘了把“预处理到文件”再设置为否。要不然代码将不会进入到编译期进行编译。这个步骤仅限于我们观察预处理后的源代码变形结果。

30、 头文件在项目里有什么作用？

在一个项目里头文件相当于一个项目的“目录大纲”或“功能清单”，我们可以把自定义的数据类型、全局变量的声明、自定义函数的声明等项目中的结构性信息存储在不同的头文件中。哪里需要这些信息哪个源代码文件就进行#include 即可。所以我们查看一个项目的头文件就可以大致了解一个项目的功能构成。

31、 #include 的本质是什么？其后的<>与“”的区别是什么？

#include 工作在预处理期，其本质就是把对应的头文件复制粘贴进源代码。#include 后面的<>代表在当前电脑系统（包括操作系统和 VS2019 编程环境系统）寻找头文件，相当于公共资源。””双引号则是在当前项目位置上按路径寻找头文件。主要对应本项目的自定义的头文件。

32、 #define 宏替换的作用及基本用法？

#define 称为宏替换简称宏，其作用相当于查找并替换，即按预定义好的宏名称进行查找，并把符合的源代码部位替换成对应的文字内容。

其基本格式为：#define 宏名 替换内容

注意：宏名与替换内容之间最好一个空格，另外替换内容后面不要随意加分号，因为分号也将作为替换内容的一部分去参与替换。这样替换成的源代码极有可能产生编译错误。

例如：

```
#define MAX 100; //注意此处的分号
```

```

int main()
{
    int a[MAX];
    return 0;
}

```

查看预处理的结果文件可以看到：

```

4 int main()
5 {
6     int a[100];
7     return 0;
8 }
9

```

a[100;]是明显的语法错误，就会产生编译错误。所以务必注意#开头的预处理指令不要任意加;号。

我们可以用#define 来定义值，用于增强代码的可读性，有点类似枚举。例如：

```
#define SUCCESS 0
```

```
#define FAIL 1
int main()
{
    return SUCCESS;
}
```

将 `SUCCESS` 定义为 0，所以 `return 0` 的可读性就不如 `return SUCCESS;` 的阅读感强。

33、 typedef 与 #define 的区别？

结合如下代码分析二者的区别：

```
#define PINT int*
typedef int* P_INT;
int main()
{
    PINT a, b, c;
    P_INT p, q, k;
    return 0;
}
```

上面代码预处理后的效果如下：

```
3 typedef int* P_INT;
4 int main()
5 {
6     int* a, b, c;
7     P_INT p, q, k;
8     return 0;
9 }
```

通过调试器查看每个变量的类型：

局部变量		
搜索(Ctrl+E)		
名称	值	类型
▶ a	0xffffffff {??}	int *
▶ b	-858993460	int
▶ c	-858993460	int
▶ k	0xffffffff {??}	int *
▶ p	0xffffffff {??}	int *
▶ q	0xffffffff {??}	int *

综合上面的几张图可以看到：b, c 变量的类型都是 `int`。而 p、q、k 的类型都是 `int*`。

表面上看很像的两行代码如下：

```
PINT a, b, c;
P_INT p, q, k;
```

区别在哪里呢。首先 `#define` 宏工作的时期是预处理期，即编译之前。它的作用是把源代码变形替换成另一个源代码。这个任务完成后 `#define` 也就不存在了。所以 `PINT` 变为 `int*` 后只有 a 变量是指针类型，其它的都是 `int` 类型变量。而 `typedef` 是要参加编译的，编译后其命名的数据类型 `P_INT` 一直可被项目所使用。所以 p, q, k 都是 `int*` 指针类型。

另外两者的使用侧重不同。`typedef` 的目的是简化数据类型，可以把复杂的类型进行封装。而 `#define` 目的是增强代码的维护性、可读性；从而辅助开发者对一些代码的查找替换。

34、宏替换能嵌套定义吗？

宏替换是可以嵌套定义的。其模型如下：

```
#define A B
      #define B C
            #define C D
```

A 被替换成 B，B 再替换成 C，C 再替换成 D…。 例如：

```
#define PR printf("hello world.");
#define A PR PR PR PR PR PR PR PR PR PR
#define B A A A A A A A A A A
```

上面的宏可以创建出 100 个 helloworld。

35、如何用 typedef 与宏创建一个布尔类型？

```
#define TRUE 1
#define FALSE 0
typedef int BOOLEAN;
int main()
{
    BOOLEAN isOK = TRUE;
    return 0;
}
```

C 语言中没有布尔 boolean 类型，通常用 int 代替，0 代表假 false、非 0 代表真 true。所以上例中让 typedef 与#define 配合实现这个效果。

36、什么是有参数的宏？举例说明

有参数的宏是指在替换时并不是整体替换，而是局部替换。模型如下：

```
#define 宏名(可变部位) <--> 内容模板
                        可变部位  不变内容
```

下面通过一个例子说明。

典型的例子就是用宏封装一个计算一维数组长度的表达式：

```
int arr2[] = {1, 3, 6, 9, 8};
double a3[] = {1.2, 3.2, 5.6, 8.9};
printf("%d\n", sizeof(arr2)/ sizeof(arr2[0]));
printf("%d\n", sizeof(a3) / sizeof(a3[0]));
```

上面代码中计算数组长度的表达式是固定的模式：

sizeof(数组名)/ sizeof(数组名[0])

如果想要计算数组长度只需要把数组名的位置替换成某个数组即可。我们可以把 sizeof(数组名)/ sizeof(数组名[0]) 这个表达式当作一个模板，变化的部位就是数组名，谁使用谁提供；用#define 来实现就是：

```
#define ARR_LEN(arr_name) sizeof(arr_name)/ sizeof(arr_name[0])
```

即把变化的部位命名为一个参数名 arr_name。这样上面两个打印代码就变为：

```
printf("%d\n", ARR_LEN(arr2));
printf("%d\n", ARR_LEN(a3));
```

这样代码的简洁度、可读性就提高了。虽然用起来感觉像一个函数，但它本质是个宏。所以在使用时注意宏的格式：**宏 替换内容**，如果是有参数的宏，那么宏名称与（参数）之间不要有空格。一旦宏名与括号之间有空格，那么()就变成了替换内容而非参数的作用了。

笔试真题

编程题

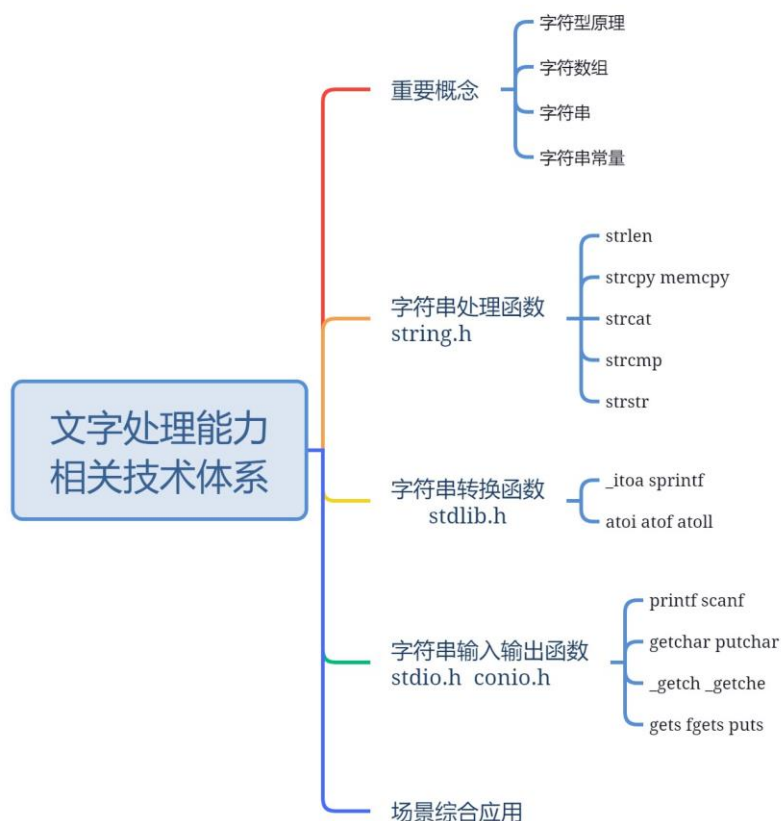
科林 C 语言

编程能力四：文字处理

能力介绍

文字处理能力就是在理解字符、字符数组、字符串、字符串常量等基本概念基础上能够使用 `string.h` 提供的字符串处理函数灵活处理诸如：文字匹配条件下的查找、替换、插入、截取等功能的实现。

相关技术构成



学习技巧建议

1. 严格区分字符数组与字符串概念的区别。理解 `\0` 的作用。
2. 重点理解字符串常量的执行原理，以及其在代码中的作用。
3. 理解主要字符串函数的用法并能适当实现其原理仿真。
4. 主动灵活运用字符串处理函数去实现文字处理的算法问题。

Part 1 字符串相关原理

1、字符数组与字符串的区别？

字符数组是存储连续字符的内存空间。字符数组不一定能构成字符串例如：

```
#include<stdio.h>
int main()
{
    char str[] = {'a','b','c'};
    printf("%s\n",str);
    return 0;
}
```

打印结果如下：

Microsoft Visual Studio 调试控制台

abc烫烫?垠縲磨

我们看到除了 abc 还有很多奇怪的字符。根源是 %s 格式是用于输出字符串的，它需从一个地址开始（str 数组名代表的是一个地址）连续输出每一个字符的影像，直到遇到一个结束标记才能停下来。这个结束标记就是 0 号的字符编码。通常这个 0 号字符编码我们用字符型的 8 进制常量的方式表示即 '\0'。很显然，我们定义的字符数组 str 里面没有这个 \0 的标记，所以它会一直把内存中的每个非 0 的字节的内容以编码对应字符影像方式输出直到遇到第一个 \0 为止。因此解决办法是：

char str[] = {'a','b','c','\0'}; 在字符数组某位置加入一个 \0。

字符数组内第一个 '\0' 之前的所有字符称为有效字符串，'\0' 后面的内容从字符串角度即为无用内容。

如果代码改为：char str[] = {'a','b','\0','c','\0'}; 则 str 数组所存储的字符串为 ab 是了。

通常对字符数组赋值采用字符串常量进行初始值设置，写法如下：

```
char str[] = "abc";
```

双引号下的 abc 称为字符串常量，其自动在后面补充 \0。用调试器可以方便的观察到：

名称	值
str	0x00d3fcd8 "abc"
[0]	97 'a'
[1]	98 'b'
[2]	99 'c'
[3]	0 '\0'

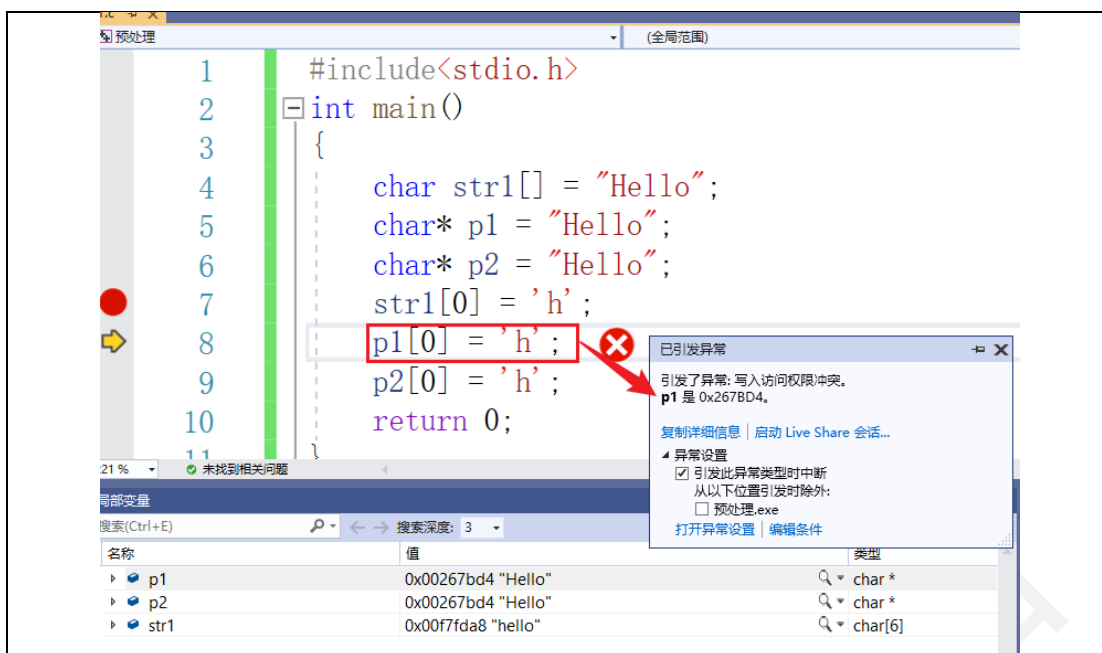
注意此写法仅用于字符数组的初始值。

2、字符串常量的原理是什么？

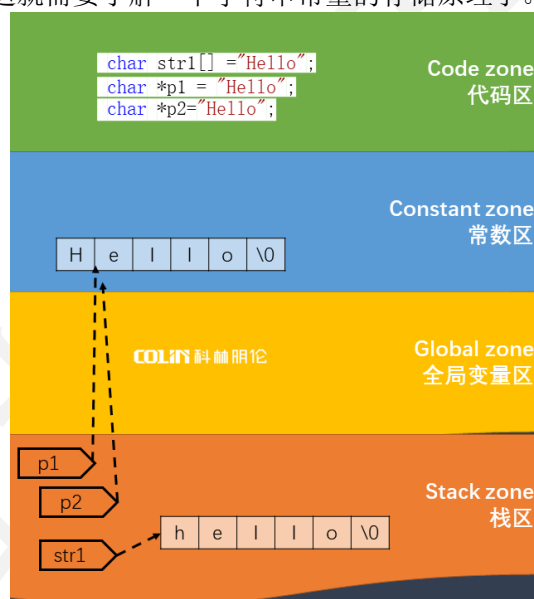
首先观察如下代码是否会有 bug？

```
#include<stdio.h>
int main()
{
    char str1[] = "Hello";
    char* p1 = "Hello";
    char* p2 = "Hello";
    str1[0] = 'h';
    p1[0] = 'h';
    p2[0] = 'h';
    return 0;
}
```

用调试器逐行运行到第 8 行时会产生如下的情况：



这个提示说的是用指针变量 p1 无法修改指向的空间，因为此时没有这个权限。是什么原因造成的呢？这就需要了解一下字符串常量的存储原理了。观察下图。



我们从上面代码的执行过程说起。当程序启动时，双引号的字符串常量Hello会首先被创建在内存的常量区。虽然源代码中有三个“Hello”但是在常量区只会被创建一份。放置在常量区的数据只能读取不能修改。

接下来main函数被调用执行。main函数内的局部变量str1字符数组等其它指针变量p1、p2被创建在栈区。接下来初始化str1字符数组：在源代码中字符串常量代表了其所在常量区的首地址。所以初始化str1的过程是把常量区的Hello\0的内容复制进str1在栈区的内存空间中。再接下来初始化p1：由于p1是指针变量，又由于字符串常量在源代码中代表了其所在常量区的地址，p1保留这个地址，所以通过p1就可以找到这个字符串常量。p2指针变量也是如此处理。通过调试器的局部变量视图可以看到p1与p2保存的是同一个地址。接下来str1[0] = 'h';这句是可以正常执行的，因为它改变的是字符数组str1所在栈区的空间内容，这是它申请的自主空间。而p1[0] = 'h';则会执行失败。因为p1指向的是常量区的地址，常量区的内容是不可以被修改的。

另外当有如下的代码时。

```
str1 = "world";
```

```
p1 = "world";
p2 = "world";
```

如上的代码中 `str1 = "world";` 是错误的，因为 `str1` 是数组名是一个地址，是一个指针常量，是不可以被修改的。而 `p1` 和 `p2` 是指针变量，它们可以指向任意的字符串（或称保留任意字符串的首地址），因此不会产生编译错误。

Part 2 字符串处理函数

3、如何找到字符串处理函数？

处理字符串时，C 语言提供了一些功能函数。它们来自 `string.h` 这个头文件。到 <https://cplusplus.com/reference/cstring/> 这个地址可以看到：

<cstring> (string.h)

C Strings

This header file defines several functions to manipulate **C strings** and arrays.

Functions

Copying:

memcpy	Copy block of memory (function)
memmove	Move block of memory (function)
strcpy	Copy string (function)
strncpy	Copy characters from string (function)

Concatenation:

strcat	Concatenate strings (function)
strncat	Append characters from string (function)

Comparison:

memcmp	Compare two blocks of memory (function)
strcmp	Compare two strings (function)
strcoll	Compare two strings using locale (function)
strncmp	Compare characters of two strings (function)
strxfrm	Transform string using locale (function)

Searching:

memchr	Locate character in block of memory (function)
strchr	Locate first occurrence of character in string (function)
strcspn	Get span until character in string (function)

我们要想更好更灵活的掌握字符串的应用，除了理解字符、字符数组、字符串、字符串常量这四者的基本概念及原理外，熟练掌握常用字符串函数的用法并且能够广泛的阅读源代码甚至适当的模仿常用函数的实现过程也是非常必要的。

4、如何得到字符数组的长度和字符串长度？

```
#include<stdio.h>
int main()
{
    char str[50] = "Hello";
    printf("字符数组长度: %d 字符串长度: %d", sizeof(str), strlen(str));
    return 0;
}
```

Microsoft Visual Studio 调试控制台

字符数组长度: 50 字符串长度: 5

字符串长度用 `strlen()` 函数得到，其参数是字符串的开始地址，它会从这个地址出发向后找到第一个 `'\0'`。这之间的字符数量就是字符串长度。注意 `'\0'` 是不会被计入到字符串长度的。例如字符串常量 `"abcd"` 的字符数组长度是 5，因为 `'\0'` 也需要占一个字节的空。但字符串长度是 4。

5、`strcpy()` 与 `memcpy()` 的区别？

`strcpy` 是字符串复制函数。其作用是把从一个地址开始到第一个 `\0`（包括 `\0`）之间的字符复制到另一个字符数组中。而 `memcpy` 函数是一个内存块的复制函数。作用是把内存中某地址开始的一片内存空间（指定字节数长度的）内容复制到另一个内存地址开始的内存内。二者相比 `memcpy` 可以适用任意类型数据，通用性更强些。

参考两者的函数声明：

```
void * memcpy ( void * destination, const void * source, size_t num );
char * strcpy ( char * destination, const char * source );
```

6、`strcat()` 的用法及仿真实现？

`strcat` 函数的功能是字符串拼接，即把一个字符串连接（复制）到另一个字符串的尾部，并把最终的字符串首地址返回。要求被连接的字符数组空间要足够大，能够容纳下新的字符串。例如：

```
char a[50]="hello ";
printf("%s\n",strcat(a,"world"));
```

其实现原理是找到第一个被连接字符串的尾部 `\0` 位置，然后把字符串 2 完整复制到 `\0` 的位置。

```
char* my_strcat(char* dest, const char* src)
{
    strcpy(dest + strlen(dest), src);
    return dest;
}
```

7、`strstr()` 字符串匹配函数的用法及仿真？

`strstr` 是字符串的匹配函数，第一个参数是一个长的字符串，第二个参数是短串（或称子串），其作用是在长串中寻找短串首次出现的指针（内存地址），并把这个指针返回，如果没有找到匹配的位置就返回 `NULL`。

```
char longStr[] = "hello world";
char* p = strstr(longStr, "world");
printf("%c %s\n", *p, p);
printf("匹配的下标: %d\n", p - longStr);
p = strstr(longStr, "you");
printf("%d %s\n", p, p);
```

Microsoft Visual Studio 调试控制台

```
w world
匹配的下标: 6
0 (null)
```

因为内存地址是连续的，是从小地址到大地址的编号方式，又由于 char 类型占 1 个字节所以 `p - longStr` 表达式利用地址差就可以得到长短串匹配的下标。由于长串中没有匹配的 you 这个字符串，所以返回了空地址。

`strstr` 的函数仿真代码如下：

```
char* my_strstr(const char* str1, const char* str2)
{
    const char* p = str1;
    const char* q = str2;
    const char* pt;
    while (*p)
    {
        q = str2;
        pt = p;
        while (*pt == *q && *q)
        {
            pt++;
            q++;
        }
        if (*q == '\0')
        {
            return p;
        }
        p++;
    }
    return NULL;
}
```

8、`strcmp()` 字符串比较函数的用法及仿真？

```
#include <stdio.h>
#include <string.h>
#include <math.h>
int my_strcmp(const char* str1, const char* str2);
int main()
{
    /*strcmp*/
    printf("%d\n", strcmp("ABCDE", "a")); //-1
    printf("%d\n", strcmp("ABCDE", "ABCDE")); //0
    printf("%d\n", strcmp("ABCDe", "ABCDE")); //1
    /*仿真字符串比较函数 strcmp */
    printf("%d\n", my_strcmp("ABCDE", "a")); //-1
    printf("%d\n", my_strcmp("ABCDE", "ABCDE")); //0
    printf("%d\n", my_strcmp("ABCDe", "ABCDE")); //1
    return 0;
}
```

`strcmp` 函数在比较字符串时是按字符的 ASCII 码的编号大小进行比较，它是区分大小写的。其比较结果有三种：

- 1、>0 说明字符串 1 大于字符串 2
- 2、==0 说明两个字符串内容完全相同
- 3、<0 说明字符串 1 小于字符串 2。

其原理仿真如下。

```

int my_strcmp(const char* str1, const char* str2)
{
    const char* p1=str1;
    const char* p2=str2;
    while ( *p1==*p2&&*p1 )
    {
        p1++;
        p2++;
    }
    // return *p1-*p2;
    return *p1 - *p2 ? (*p1 - *p2) / abs(*p1 - *p2) : 0;
}

```

9、字符串与数字类型的互相转换类型有哪些？

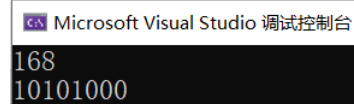
一、整数转字符串

```

#include <stdio.h>
#include<stdlib.h>
int main()
{
    int num = 168;
    char str[10];
    _itoa(num, str, 10);
    printf("%s\n", str);
    _itoa(num, str, 2);
    printf("%s\n", str);
    return 0;
}

```

_itoa() 函数是把数字变成对应的文字存储到字符数组。其中最后一个参数代表进制，可以是 2~36 之间的任意进制。



```

Microsoft Visual Studio 调试控制台
168
10101000

```

二、小数转字符串

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include<stdlib.h>
int main()
{
    double num = 3.1415926;
    char str[20] ;
    sprintf(str, "%.7f", num);
    printf("%s\n", str);
    return 0;
}

```

上面代码是个通用方式，sprintf 函数的作用是把格式化打印的结果向字符串输出。这样就可以把任意数据类型以任意的样式设置生成字符串。

三、字符串转数字

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include<stdlib.h>
int main()
{
    char str_num[50] = "168";
    char str_pi[50] = "3.1415926";
}

```

```

char str_dis[50] = "12364568";
int num = atoi(str_num); //字符串转int类型
double pi = atof(str_pi); //字符串转double类型
long long dis = atoll(str_dis); //字符串转long long类型
printf("%d %lld %.7f\n", num, dis, pi );
return 0;
}

```

10、 字符的输入输出函数有哪些？

单个字符的输入输出的方式大约有四种。

首先定义一个 char 型变量 ch

```
char ch;
```

方式一：scanf 与 printf

```
scanf("%c",&ch);
printf("%c",ch);
```

方式二：getchar 与 putchar

```
ch=getchar();
putchar(ch);
```

getchar 函数用于从键盘读取一个字符，输入完需后，需要按回车确认。此时键盘缓冲区存在两个字符一个是刚刚输入的能够看见的字符，还有一个是换行\n。但是 getchar 只能读取字符，换行的\n 还会停留在键盘缓冲区。

以上两种方式都需要#include <stdio.h>这个头文件。

如下的两种方式需要#include<conio.h>这个控制台输入输出的头文件。

方式三：

```
ch = _getch();
_putchar(ch);
```

_getch() 工作时，用户输入的字符不会被显示出来，输入完不需要按回车就会立即被读取到字符变量 ch 中。

方式四：

```
ch = _getche();
_putchar(ch);
```

和方式三的唯一区别就是输入的字符能被显示出来。

11、 字符串的输入输出函数有哪些？

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int main()
{
    char str[30];
    scanf("%s", str );
    printf("%s\n",str);

    setbuf(stdin,NULL);
    gets(str );
    puts(str);

    setbuf(stdin, NULL);
    fgets(str, 30, stdin);
}

```

```
puts(str);

return 0;
}
```

上面代码列出了主要的字符串输入输出函数。其中

`#define _CRT_SECURE_NO_WARNINGS`的目的是在VS2019中关闭安全警告的提醒。可以用三种方式实现字符串的键盘输入。

方式一：scanf函数与printf函数

它们是通用的格式化设置的输入输出函数。`%s`格式用于字符串的键盘输入。如果采用`scanf_s`这种安全函数时需要指明要保存到的内存空间的可用大小即`scanf_s("%s", str, 30)`；从键盘输入时空格或回车都可作为字符串的输入结束。但是空格或换行不会被保存到字符串内，它们还会停留在键盘缓冲区。

方式二：gets函数与puts函数

我们看到`gets`函数在调用前用了一句：`setbuf(stdin, NULL)`；是设置键盘输入`stdin`缓冲区内容的存储位置，如果位置为`NULL`起到的作用相当于清除键盘缓冲区的内容。有此句的目的是上一个`scanf`工作时，空格或换行由于不会被读取进字符数组还会停留在键盘缓冲区，此时`gets`在工作时会读取它们，`gets`以换行为结束，所以上一次的输入的换行又被作为输入的结束。因此需要把键盘缓冲区的内容清空，让用户重新输入。另外`gets`已经在C/C++新版本中不建议使用了，建议用`fgets`。

方式三：fgets函数

如果从键盘输入，`fgets`（存放地址，缓冲空间大小，`stdin`）；其实`fgets`函数也可以从文件输入。`fgets`在读取缓冲区内容时，会把换行`\n`也一并读入到字符数组。所以如果不打算保留`\n`需要写代码`str[strlen(str)-1]='\0'`；把`\n`替换成`\0`。

12、 如何实现密码输入时的掩码效果？

```
#include <stdio.h>
#include <string.h>
#include<stdlib.h>
#include <conio.h>
int main()
{
    printf("账号: ");
    char num[50];
    gets_s(num, 50);
    char pwd[50];
    printf("密码: ");
    char ch;
    int i = 0;
    do
    {
        ch = _getch();
        if (ch != '\r')
        {
            pwd[i++] = ch;
            putchar('*');
        }
        else
            break;
    } while (i < 50);
    pwd[i] = '\0';
    if (strcmp(num, "abcd") == 0 && strcmp(pwd, "123456") == 0)
```

```
{  
    printf("\n请进\n");  
}  
else  
{  
    printf("\n非法用户\n");  
}  
return 0;  
}
```

笔试真题

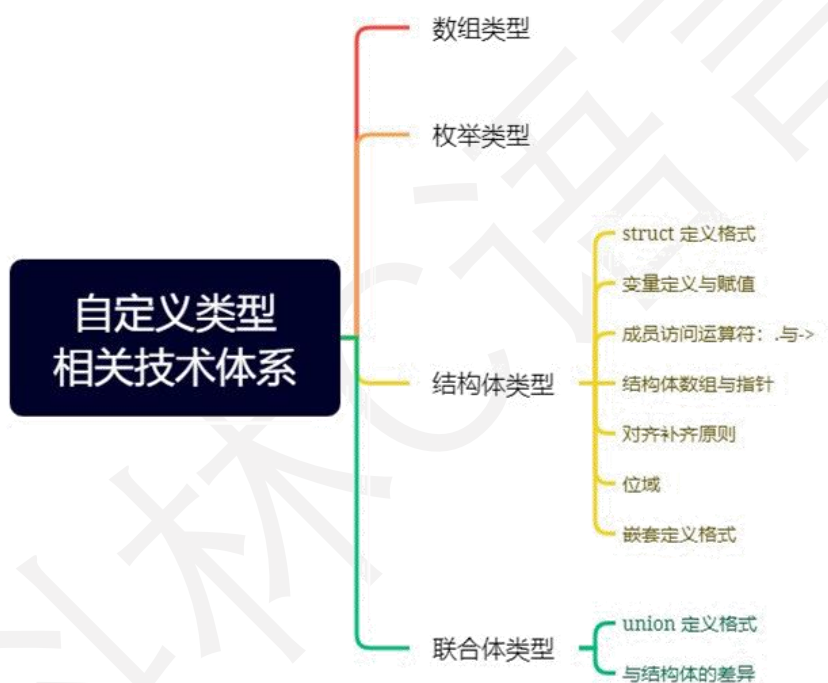
编程题

编程能力五：自定义类型

能力介绍

自定义类型就是对现实应用场景下，数据信息结构的提取能力。能够把基本数据类型、指针类型等在应用场景需求下，用 struct、union、数组等创建能够覆盖现实世界信息的数据类型。

相关技术构成



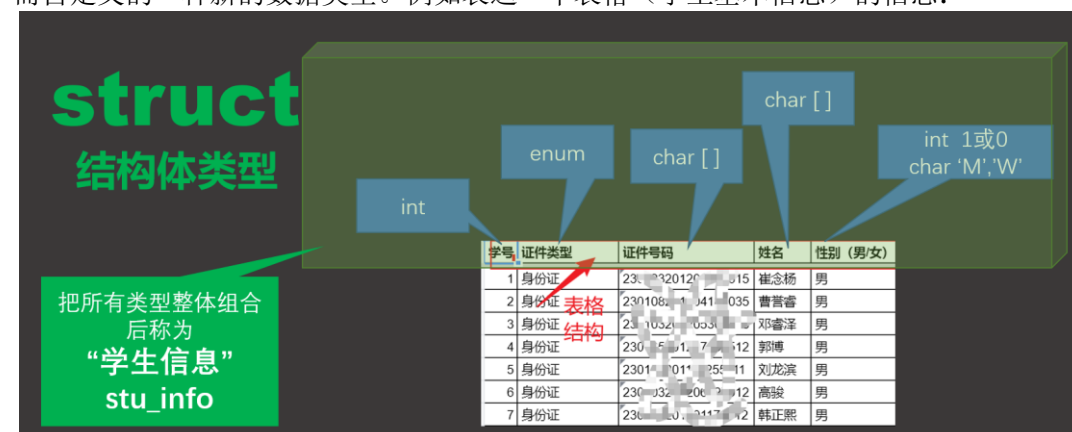
学习技巧建议

1. 用生活中的表格等复合信息练习多自定义类型的表达。
2. 理解对齐补齐原则并有意识的优化结构体成员顺序。
3. 理解结构体与联合体的内存使用区别。
4. 注意字符指针与字符数组在结构体中的使用区别。

Part 1 结构体与共用体

1、结构体的作用及定义语法？

C 语言是强类型语言。其数据类型可分为三大类：基本数据类型、指针相关类型、扩展（或称复合）数据类型。[]数组类型、enum 枚举类型、struct 结构体类型、union 共用体类型就属于扩展的数据类型。一句话概括结构体：就是根据我们数据模型的需要而自定义的一种新的数据类型。例如表达一个表格（学生基本信息）的信息：



我们可以看到表格的头部结构：学号、证件类型、证件号码、姓名、性别单独拿出来一项都可以用基本数据类型或数组来表达。但是只有把它们在一个结构中组装起来才能称为是学生的基本信息。这个结构就是——struct 结构体类型。

1、结构体类型的定义格式：

```
struct 结构体名
{
    类型 成员变量 1;
    类型 成员变量 2;
    ....
};
```

2、上面表格中证件类型可以用枚举类型来描述，因为有效证件的范围有限且固定比较适合枚举方式。

```
enum type { 身份证, 学生证, 驾驶证 };
```

3、最后学生表的结构用 struct 结构体来实现如下：

```
struct stu_info
{
    int stuNO; // 学生编号
    enum type typeID; // 证件类型
    char cardID[20]; // 证件编号
    char name[20]; // 姓名
    int sex; // 性别
}
```

2、如何创建结构体变量并为其赋值？

创建结构变量的方式有三种。

方式一：在 struct 类型描述完后，直接创建变量。例如：

```
struct stu_info
{
    int stuNO; // 学生编号
    enum type typeID; // 证件类型
    char cardID[20]; // 证件编号
    char name[20]; // 姓名
}
```

```
int sex; //性别
} stu1, stu2; //在类型后面创建变量。
```

方式二：用结构体类型名创建变量，此时 struct 关键词不可少。

```
struct stu_info stu3, stu4;
```

方式三：用 typedef 封装结构体类型并命名，可免除 struct 关键词来创建变量。

```
typedef struct stu_info STU_INFO;
```

```
STU_INFO stu5;
```

如何为结构体变量赋值？由于结构体类型是由若干个成员变量构成的。因此赋值时要为每个成员变量单独赋值。具体细节如下。

方式一：结构体变量赋初始值。

初始值时的顺序与类型值和成员定义的顺序保持一致。

```
struct stu_info stu1 = { 1, 身份证, "230102201112221234", "张睿睿", 1 };
```

```
STU_INFO stu2 = { 1, 身份证, "230102201112221234", "张睿睿", 1 };
```

或者指明某成员的值。此时用. 引出具体的成员名。例如：

```
struct stu_info s5 = { .name="李四",
                      .cardNum="23106554522155",
                      .cardType=SFZ,
                      .stuNum=5 };
```

方式二：单独为每个成员赋值。

由于结构体变量就是一个大集合，里面由具体成员构成。当需要指明具体成员是需要用到“.”运算符，来表明“其中的 xxx”这个含义。

```
stu2.stuNO = 2;
```

```
stu2.typeID = 学生证;
```

```
strcpy(stu2.cardID, "23010220112223545");
```

```
strcpy(stu2.name, "张睿睿");
```

```
stu2.sex = 1;
```

注意：如果结构成员是字符数组，记得用 strcpy 函数对其赋值。

3、如何为结构体变量的输入与输出？

结构体变量在键盘输入时需要为每个成员单独的输入对应类型的数据，此时要注意数据类型的匹配。结构体成员如果是数组，由于数组名本身就是地址，所以可省略&取地址运算符。

```
struct stu_info s3;
scanf("%d%d%s%s%d", &s3.stuNO
    , &s3.typeID
    , s3.cardID
    , s3.name
    , &s3.sex);
printf("%d %d %s %s %d",
    s3.stuNO
    , s3.typeID
    , s3.cardID
    , s3.name
    , s3.sex);
```

如果采用安全函数 scanf_s 就需要为字符相关类型再指明其内存可用空间的大小，例如：

```
scanf_s("%d%d%s%s%d",
    &s3.stuNO
    , &s3.typeID
    , s3.cardID, 20
    , s3.name, 20
    , &s3.sex);
printf("%d %d %s %s %d",
    s3.stuNO
```

```

    , s3.typeID
    , s3.cardID
    , s3.name
    , s3.sex);

```

4、如何用结构体类型创建数组并对数组成员进行排序？

如下代码定义了一个图书信息的结构体类型——BOOK。并用该类型创建了一个数组用于表达一个书架上的基本图书信息。接下来用排序算法按图书的价格进行降序排序，如果价格相同再按年代升序排序。最后打印出来查看结果。

```

#include <stdio.h>
/*定义一个图书信息的结构体类型*/
typedef struct book
{
    int year;//年代
    double price;//价格
    char bookName[50];//书名
    char author[20];//作者
} BOOK;

int main()
{
    /*创建结构体数组变量*/

    BOOK books[] = {
        {2008, 65.0, "Pointer on C C和指针", "Reek"},
        {2009, 66.0, "Expert C programming C专家编程", "Linden"},
        {2011, 65, "Effective C++", "Meyers"},
        {2005, 60, "C primer Plus", "Prata"}
    };

    for (int s=0 ;s<4-1;s++)
    {
        for (int b=s+1 ;b<4;b++)
        {
            if ( books[s].price < books[b].price
                || books[s].price == books[b].price
                && books[s].year > books[b].year)
            {
                BOOK t;
                t = books[s];
                books[s] = books[b];
                books[b] = t;
            }
        }
    }

    for (int i = 0; i < 4; i++)
    {
        printf("%-6d%6.2f %-50s%-20s\n"
            , books[i].year
            , books[i].price
            , books[i].bookName
            , books[i].author);
    }

    return 0;
}

```

```

}
选择 Microsoft Visual Studio 调试控制台
2009    66.00 Expert C programming C专家编程      Linden
2008    65.00 Pointer on C C和指针                Reek
2011    65.00 Effective C++                      Meyers
2005    60.00 C primer Plus                      Prata

```

如果按书名升序排序该如何改变排序策略呢？

首先字符串的比较需要用到 `strcmp` 函数，因此需要补加 `#include<string.h>`。然后将排序策略改为 `strcmp(books[s].bookName, books[b].bookName)>0` 但要注意的是 `strcmp` 函数是区分大小写的比较函数。如果不区分大小写，在 VS2019 中请使用 `_stricmp` 函数。

5、结构体指针变量的定义方式有哪些？

首先创建一个结构体类型

```

typedef struct book
{
    int year;//年代
    double price;//价格
    char bookName[50];//书名
    char author[20];//作者
} BOOK, * P_BOOK;

```

此处的 `P_BOOK` 类型是在前面类型基础上增加的结构体指针类型。根据上面定义的结构体类型可以用如下三种方式创建结构体指针变量。

- 1、使用结构体最原生的类型名称：`struct book* p1;`
- 2、使用 typedef 封装的结构体类型名称：`BOOK* p2;`
- 3、使用 typedef 封装的结构体指针类型名称：`P_BOOK p3;`

上面第三种写法目的是在代码中减少指针类型的*标识。让代码中*只用于间接访问运算而省略掉指针类型的标识作用。

6、结构体成员访问运算符 . 与->的用法是什么？

. 运算符用于结构体变量。使用“结构体变量.成员”的格式在整个结构体空间内使用其中的成员空间，是整体与局部的关系。而->非常形象的像个箭头，是结构体指针变量专用的运算符。用于结构体指针变量访问指向空间内的某个结构体成员空间。例如：

```

BOOK b1 = {2008, 65.0, "Pointer on C C和指针", "Reek"};
BOOK* pbook = &b1;
printf("%s\n", b1.bookName);
printf("%s\n", (*pbook).bookName);
printf("%s\n", pbook->bookName);

```

7、结构体的对齐、补齐是什么？

观察如下结构体类型，它占用的空间字节数会是多少呢？

```

struct AA
{
    int a;
    short c;
    char b;
};

```

【错误的理解方式】：结构体的每个成员的空间大小累加，即 $4 + 2 + 1$ 等于 7。如何验证呢？可用如下代码查看后

```

printf("%d\n", sizeof(struct AA));

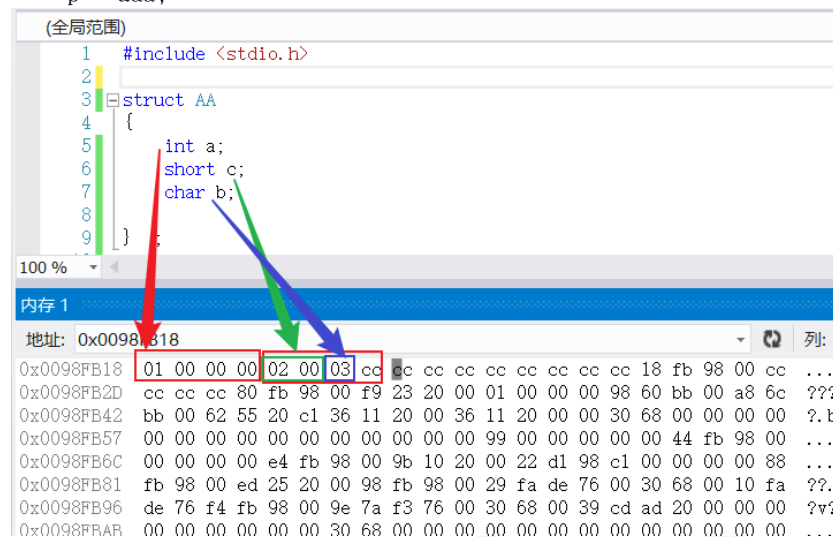
```

```
C:\WINDOWS\system32\cmd.exe
8
请按任意键继续. . .
```

为什么会是 8 呢？这就需要理解“结构体的对齐补齐原则”了。

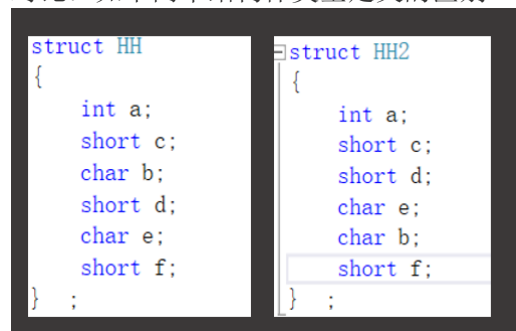
所谓结构体对齐就是以结构体成员中占数据类型字节数最多的为基准。所谓补齐就是为结构体成员分配内存空间时，先补齐一个对齐字节数，如果空间不足以存放一个结构体成员，那么再补齐一个新的对齐字节数，如此向下直到为所有结构体成员分配对应空间为止。例如：

```
struct AA* p, aa = { 1, 2, 3 }; //创建struct AA 类型的变量，用调试器观察其存储形态
p = &aa;
```



【观察上图】整个结构体成员中，空间最大类型是 int，因此它的存储策略是以 int 为基准对齐长度，然后看后面的成员能否在 4 个字节内存放，如果不足以放下，就再开辟下一组 4 个字节空间。

对比：如下两个结构体类型定义的区别



```
printf("%d\n", sizeof(struct HH));
printf("%d\n", sizeof(struct HH2));
```

```
C:\WINDOWS\system32\cmd.exe
16
12
请按任意键继续. . .
```

通过打印结果发现：同样的成员数量与类型，由于定义的顺序不同导致所占内存空间的大小不同。用如下代码调试查看内存状态：

```
struct HH* p, aa = { 1, 2, 3, 4, 5, 6 };
struct HH2* p2, bb = { 1, 2, 3, 4, 5, 6 };
p = &aa;
p2 = &bb;
printf("%d\n", sizeof(struct HH));
printf("%d\n", sizeof(struct HH2));
```

所以我们了解了结构体的对齐补齐原则后，在定义结构体成员顺序时，需要考虑结构体对齐的机制，尽量减少不必要的空间浪费。

另外要注意在测算对齐的字节数时有两种类型要额外注意，他们是数组类型与指针类型。所谓数组类型其实就是同类型的多个变量，所以数组的对齐字节数不按照成员总长度，而是按一个成员的类型大小作为对齐字节数。也就是 `int arr[5]`，从对齐字节数角度就是 4 个字节。

另一个是指针类型。指针变量的内存字节数只和 x86 和 x64 的选择有关和指向的类型无关，所以如 `char* e` 这种指针类型变量作为结构体成员，其对齐字节数在 x86 下就是 4 个字节。

8、结构体位域是什么？

结构体位域就是可以定义结构体某成员所占空间的二进制位数。比如：`int` 默认占 4 个字节 32 个位，可以用如下写法定义 `int` 成员变量，但让其只占几个二进制位。

```
typedef struct
{
    int a : 2; // a 变量占两个位的存储空间 但 int 会分配 4 个字节。
    int b : 8; // b 占 8 位，
    int c : 2; // c 占 8 位
    // 到这里：a b c 一共占 12 位 约等于 2 个字节，但是由于 int 占 4 个字节，其余空间也不会
    // 给别人的。
    int : 0; // 断开：即补齐一个新的 4 个字节空间
    int d : 2;
} TT;
```

`printf("%d\n", sizeof(TT));` // 打印的字节数为 8

如果用 `TT` 类型创建变量 `t`，其内存形态如下图。

```
int main()
{
    TT t = { 3, 1, 1 };
    printf("%d\n", t.a);
    printf("%d\n", t.b);
    printf("%d\n", t.c);
    return 0;
}
```

成员变量 `a` 占两个二进制位，但用整型方式存储。成员变量 `b` 占 8 个二进制位。当 `int:0` 被设置时，相当于舍弃剩余的 `int` 空间，重新另开辟一个 `int`。

9、结构体的嵌套定义方法？

所谓结构体嵌套就是在结构体内又定义了另一个结构体。例如：

```
#include <stdio.h>
struct student
{
    char name[30];
    int age;
    struct book // 此处book结构体类型就是一共嵌套的结构体
    {
        char bookName[28];
        double price;
    } book1,book2;
};
int main()
{
    struct student stu = { "小明", 9, {"数学", 5.9}, {"语文", 8.7} };
    printf("%s %d\n", stu.name, stu.age);
    printf("%s %.2f\n", stu.book1.bookName, stu.book1.price);
    printf("%s %.2f\n", stu.book2.bookName, stu.book2.price);
    return 0;
}
```

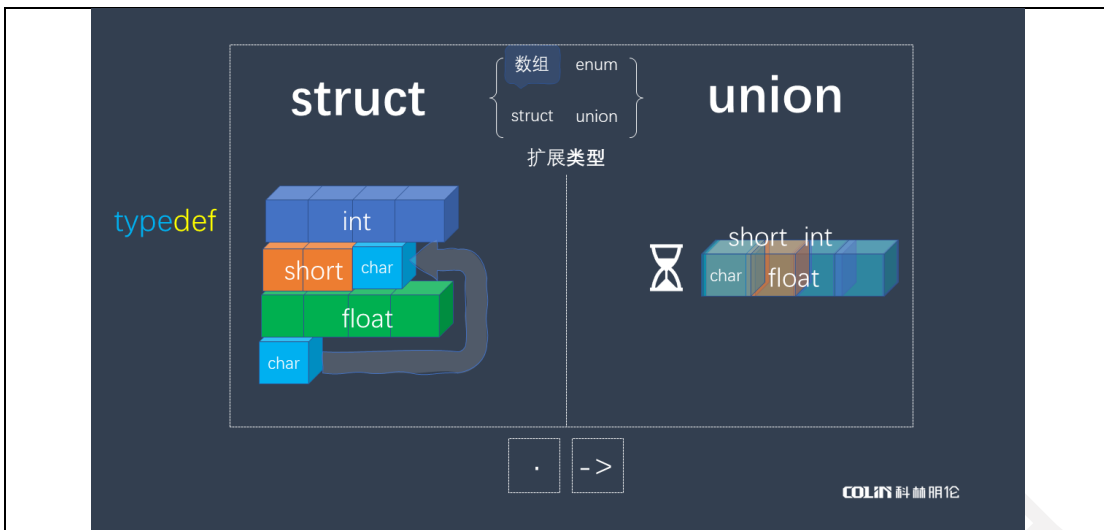
`struct book` 就是嵌套的结构体，通过调试器可以观察到成员间的结构关系：

局部变量		
搜索(Ctrl+E)		
名称	值	类型
stu	{name=0x004ff8fc "小明" age=9 book1={b...	student
name	0x004ff8fc "小明"	char[30]
age	9	int
book1	{bookName=0x004ff924 "数学" price=5.90...	book
bookName	0x004ff924 "数学"	char[28]
price	5.90000000000000004	double
book2	{bookName=0x004ff94c "语文" price=8.699...	book
bookName	0x004ff94c "语文"	char[28]
price	8.6999999999999993	double

成员变量book1、book2镶嵌在stu里面。用这种嵌套的结构可以表达出任意复杂的数据模型。当然，也可以把嵌套的结构拆解开，代码如下：

```
struct book
{
    char bookName[28];
    double price;
};
struct student
{
    char name[30];
    int age;
    struct book book1,book2;
};
```

10、 结构体与共用体的区别？



union 联合体也可称为（共用体）。由上图右侧示意图可以看到联合体所有的成员共享一段内存空间。而结构体是所有成员组合成一个大空间。所以联合体的目的是为了更省内存空间。例如：

```
#include <stdio.h>
union share
{
    short a;
    int b;
    float c;
    char d;
};
int main()
{
    printf("%d\n", sizeof(union share));
    return 0;
}
```

Microsoft Visual Studio 调试控制台

4

从打印结果可以看到虽然成员有四个，按照对齐原则看最宽成员的字节数是 4。所有成员分时共享这一段内存空间，也就是说一个时间内只有一个成员在使用。

11、 使用共用体的注意事项是什么？

观察如下代码及其执行结果：

```
#include <stdio.h>
#include <string.h>
union Data
{
    int i;
    float f;
    char str[20];
};
int main ( )
{
    union Data data;
    data.i = 10;
```

```
data.f = 220.5;
strcpy( data.str, "C Programming");
printf("data.f : %f\n", data.f);
printf( "data.i : %d\n", data.i);
printf( "data.str : %s\n", data.str);
return 0;
}
```

Microsoft Visual Studio 调试控制台

```
data.f : 4122360580327794860452759994368.000000
data.i : 1917853763
data.str : C Programming
```

我们看到只有最后一个结果是正确的，其它成员的打印结果是乱的。究其原因联合体所有成员共享一段内存空间，这就要求一个成员变量只有彻底结束使用后，才能使用另一个成员。上例这种交叉使用情况下，后面的数据把前面成员的数据进行了覆盖，而且不同数据类型的存储原理也不同（整型存储与浮点型存储），所以看到的结果是错误的。上面代码中的使用方式更适合结构体。调整一下代码顺序：

```
int main ( )
{
    union Data data;
    data.i = 10;
    printf( "data.i : %d\n", data.i);

    data.f = 220.5;
    printf( "data.f : %f\n", data.f);

    strcpy( data.str, "C Programming");
    printf( "data.str : %s\n", data.str);

    return 0;
}
```

让一个成员打印完（即用完），再使用另一个成员。打印结果如下：

Microsoft Visual Studio 调试控制台

```
data.i : 10
data.f : 220.500000
data.str : C Programming
```

笔试真题

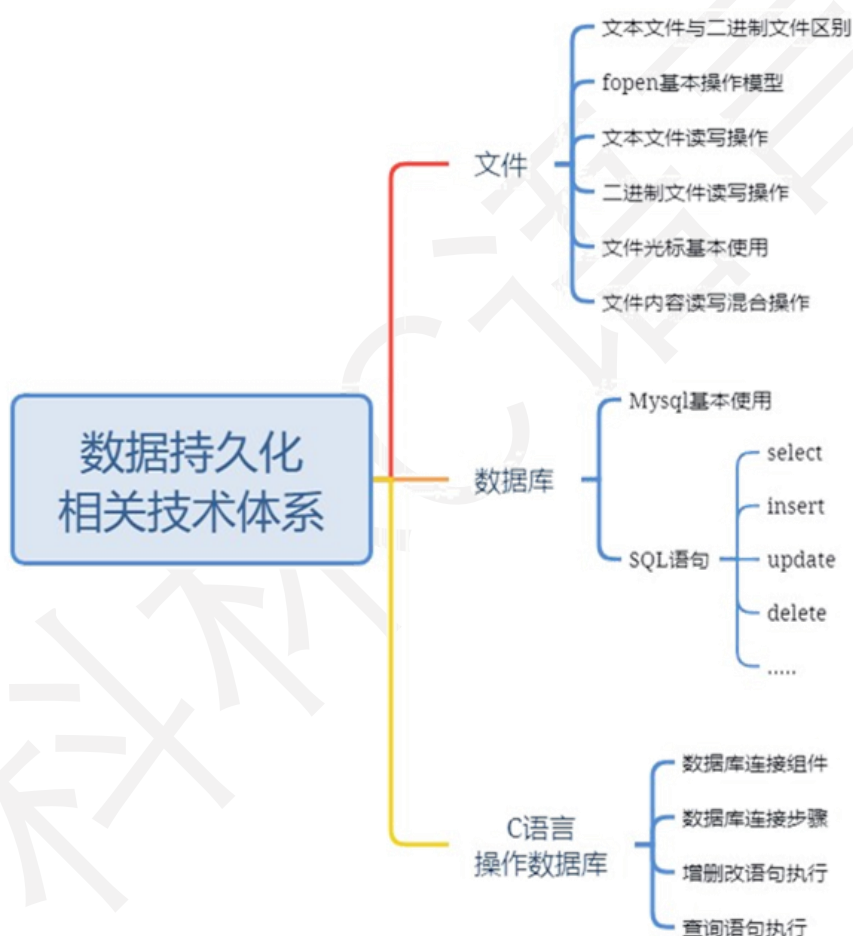
编程题

编程能力六：数据持久化

能力介绍

数据持久化就是数据被永久保存的技术总称。因为内存仅在通电状态下可以自由存取数据，但在突然掉电后数据会立即消失。所以把数据从内存中如何保存起来就是个问题。通常可以用最基本的文件的方式或用关系型数据库产品如 Mysql、Oracle、SQLServer 等对数据进行永久保存。

相关技术构成



学习技巧建议

- 1、理解文件操作模型原理与实现的代码结构（套路）。
- 2、熟练文本文件与二进制文件的读写操作。
- 3、理解文件光标与读写混合操作
- 4、理解数据结构模型在内存中与文件中的映射关系。

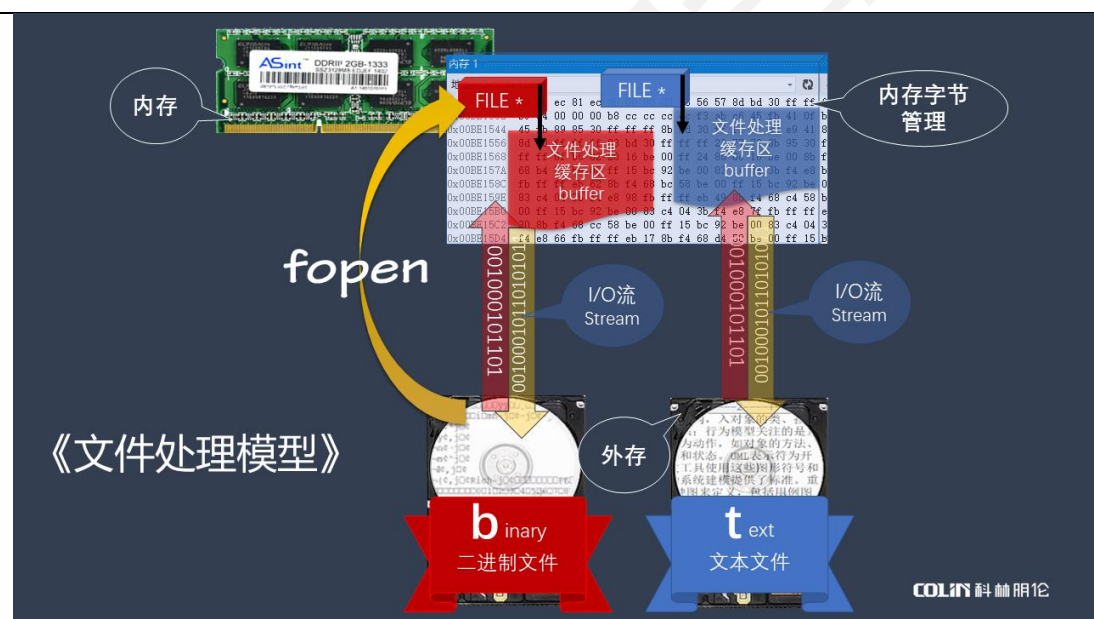
Part 1 文件处理

1、文本文件与二进制文件的区别？

所谓文本文件就是内存的数据以字符编码（比如 ASCII）的方式保存到外存（比如硬盘、优盘等）。这类文件用记事本等文本编辑器就可查看。而二进制文件是把内存中的数据及结构原封不动的平移复制到文件的过程。此时用文本编辑器打开文件通常是乱码的现象，所以得用专门的软件程序才能读取其数据进行打开。如图：同一个 pdf 文件用记事本和 pdf 阅读器打开完全不同



2、文件打开函数 fopen 的处理模型是什么？



fopen 函数来自 stdio.h。其函数声明为：

`FILE * fopen ( const char * filename, const char * mode );` 其参数 filename 是文件的存储路径及名称。参数 mode 为文件的打开方式。fopen 的工作模型如上图所示：

当我们调用 fopen 函数打开某文件时，如果文件能够找到或创建成功，会为此文件在内存中开辟一段存储空间，作为该文件的缓冲区。因为一个文件可以非常大，甚至比内存的整体空间都大，所以缓冲区相当于这个文件在内存中的“代理”。你对文件的读或写的操作只需要对其缓冲区操作即可。项目运行时系统会自动把缓冲区的内容向文件中进行读写操作。这个缓冲区的首地址用 FILE* 类型的指针来保留。如果文件没有找到或创建失败，系统是不会有这个文件创建内存缓冲区的，此时 fopen 函数返回 NULL。我们通常把数据在内存与外存间的传输（或称为数据交换）的过程形象的称为 I/O 流（stream）。基于这样的处理模型，我们可以写出文件打开的基本范本代码：

`FILE* file = fopen("路径及文件名", "打开方式");`

```

if (file!=NULL)//如果文件打开成功
{
    ... ..
    fclose(file);//关闭文件流。
    file =NULL;//释放文件指针。
}
else
{
    printf("文件打开失败\n");
}

```

当文件处理结束后务必用 `fclose` 函数关闭文件缓冲区。另外由于缓冲区已经被释放回收，所以指向文件缓冲区的指针变量应设置为 `NULL`。

3、文件的打开方式有哪些？

fopen 打开方式参数

打开方式	功能	存在
r	只读	必须存在
w	覆写	不存在：创建一个 存在：覆盖原有
a	追加	不存在：创建一个 存在：在尾部增加 新内容
r+	读写	必须存在，可读可 写、随意更新
w+	写读	w与r+的叠加
a+	追读	a与r+的叠加

b inary
二进制文件

t ext
文本文件

通常：`t` 代表文本文件、`b` 代表二进制文件。`r` 方式代表只读，其要求所处理的文件必须是已经存在的。`rt` 的组合代表的是以只读的方式打开文本文件。`r+t` 中的 `r+`则是在只读的基础上增加写入的功能。其它组合参考上表。

4、删除文件与修改文件名的函数是什么？

删除文件用 `remove` 函数，给文件改名用 `rename` 函数。其实删除文件与修改文件名包括创建文件就是调用操作系统的功能。

```
int remove ( const char * filename );
```

```
int rename ( const char * oldname, const char * newname );
```

两个函数返回值如果是 0 代表处理成功。非零就是处理失败的错误编号。

5、创建文件的方法有哪些？

可以使用 `fopen` 函数的 `w` (`write`) 方式创建文件。如果指定的文件路径及文件名存在，那么就把这个已存在的文件删除，然后再创建一个新的文件。这个过程称为覆写。其次，也可以用 `a` (`append`) 方式。其作用是文件如果不存在会创建新的文件，如果存在就不创建，并会在其尾部追加新数据。创建文件代码如下：

```
FILE* p=fopen("d:/tf/白名单.txt","wt");或
```

```
FILE* p = fopen("d:/tf/白名单.txt", "at");
```

6、向文本文件写入数据的函数有哪些？

我们用过的向屏幕输出字符或字符串的函数例如 `printf`、`puts`、`puchar` 等都可以

有对应的向文件输出的函数。例如：

```
FILE* p = fopen("d:/tf/data.txt", "wt");
if (p)
{
    //屏幕输出内容:
    printf("%d %.2f %s hello \n", 10, 3.14159, "colin");
    puts("how are you");
    putchar('A');
    //向文件输出
    fprintf(p, "%d %.2f %s hello \n", 10, 3.14159, "colin");
    fputs("how are you\n", p);
    fputc('A', p);
    fclose(p);
    p = NULL;
}
```

其中 fprintf、fputc、fputs 都是向文本文件进行写的函数。其用法就是指明写入的文件缓冲区指针即可。其它参数及用法于向屏幕输出相同。

7、如何读取一个文本文件中不确定长度的所有整数？

假设有一个文本文件 score.txt，其内容是：89, 78, 85, 69, 95, 100，如果要循环读取这些文件中的数据可以用如下代码：

```
int num;
FILE* p = fopen("d:/tf/score.txt", "rt");
if (p)
{
    while ( !feof(p) && fscanf(p, "%d,", &num)>0)
    {
        printf("%d \n", num);
    }
    fclose(p);
    p = NULL;
}
```

上面代码中 feof() 函数用于判断当前打开的文件是否读取到了文件尾部。fscanf 函数按照 “%d, ” 的格式一次读取一段数据，把满足 %d 的文本字符变为数字存储到变量里，并返回存储成功的变量个数。所以 fscanf 返回值大于 0 就代表读取成功。

8、文件光标指什么？如何得到和设置文件光标位置？

当文件以 r (read) 方式打开后，会有一个光标 (cursor) 指向文件内容的当前位置，当我们不断读取内容时，光标会不断后移，就如同我们看书时的目光。ftell 函数能够返回当前的光标位置。例如

```
FILE* p=fopen("d:/tf/letter.txt", "rt");
if ( p )
{
    //获得光标当前位置
    printf("当前光标位置: %ld\n", ftell(p));
    ///读取数据
    printf("%c\n", fgetc(p)); //读取当前光标位置的字符
    printf("当前光标位置: %ld\n", ftell(p)); //查看光标新的位置
    printf("%c\n", fgetc(p));
    printf("%c\n", fgetc(p));
    printf("当前光标位置: %ld\n", ftell(p));
    fclose(p);
    p = NULL;
}
```

```

else
{
    printf("文件打开失败\n");
}

```

 选择 Microsoft Visual Studio 调试控制台

当前光标位置: 0

1
当前光标位置: 1

2

3
当前光标位置: 3

当文件打开后, 光标位置是 0, 当我们不断读取一个字符时, 光标就会下移。当然光标也可以向回移动。可以用 `rewind` 函数让光标回到开头处。或者用 `fsetpos` 函数把光标放置在某个位置点上。

9、如何对一个文本文件的内容进行改写操作?

首先打开的文件方式需要采用 `r+` 的方式, 即在读的基础上可以改写内容。然后必须对需要改写的位置进行**定位**, 最后才能在定位点上开始利用 `fprintf`、`fputs` 等写入函数覆盖后面的内容, 不可以插入内容。

定位打开文件数据内容的位置可以用 `fseek` 函数。其函数声明为:

```

int fseek ( FILE * stream, long int offset, int origin );即
int fseek ( FILE * 文件缓冲区, long int 偏移量, int 相对位置);

```

其中相对位置的参数有三个参照点:

`SEEK_SET`: 文件开头处、`SEEK_CUR`: 当前位置、`SEEK_END`: 文件尾部

参考如下代码:

```

FILE* p=fopen("d:/tf/letter.txt", "r+t");
if ( p )
{
    fseek(p, 0, SEEK_CUR);//定位当前位置
    fputc('8', p);
    fseek(p, 2, SEEK_SET);//从文件开头处定位并偏移2个字节
    fputs("1234", p);
    fseek(p, -3, SEEK_END);//从文件尾部处定位并向回偏移2个字节
    fputc('6', p);
    fclose(p);
    p = NULL;
}
else
{
    printf("文件打开失败\n");
}

```

10、 二进制文件的读写操作函数 `fread` 与 `fwrite` 的用法?

当我们要把内存中的一片字节复制到文件或者反之把文件中的一段二进制字节复制到内存, 这样的基于二进制 01 数据的平移复制操作都可以用 `fread` 和 `fwrite` 函数来实现。其函数声明为:

```

size_t fread ( void * ptr, size_t size, size_t count, FILE * stream );

```

实际读取的大小 `fread` (void * 读取后放置的内存位置,

size_t 一个数据的字节数, size_t 数据总数量,

FILE * 从哪个文件缓冲区读取);

`fwrite` 函数的参数与返回值的逻辑类似。区别是从哪个内存地址读取, 然后写入到哪个文件缓冲区。

```

size_t fwrite ( const void * ptr, size_t size, size_t count, FILE *

```

```
stream );
```

例如把一个变量和一个数组的值以二进制方式存入到文件中：

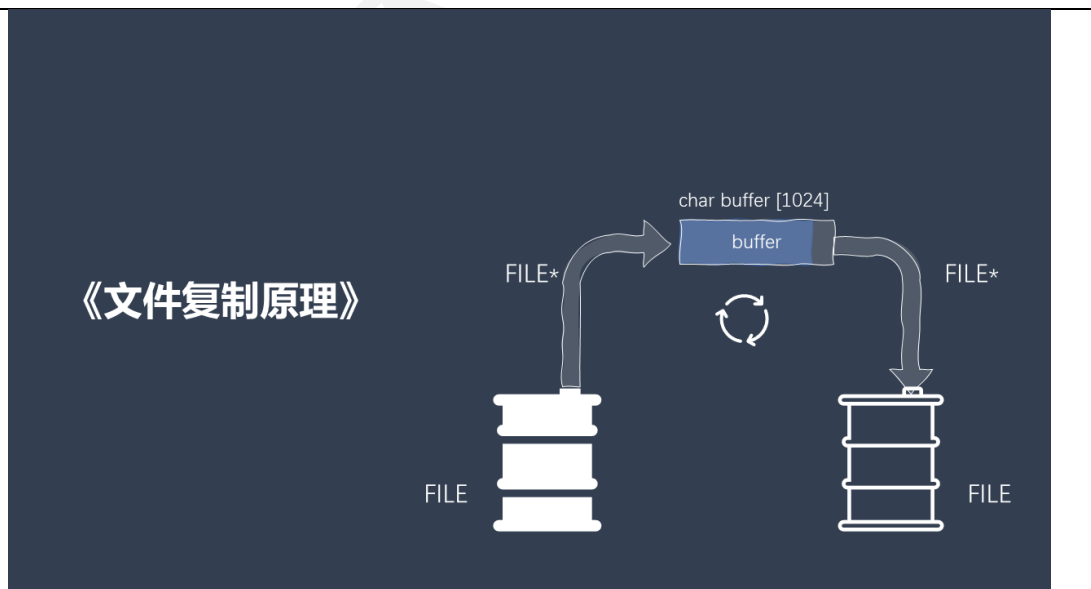
```
FILE * writeFile=fopen("d:\\tf\\data.txt", "wb");
//1 存储一个整型变量
int a = -1;
fwrite(&a, sizeof(int), 1, writeFile);
//2 存储一个数组
int arr[] = { 65, 66, 97, 98, 48, 57 };
fwrite(arr, sizeof(int), 6, writeFile);
```

反之把数据从文件中读取并存入到内存的某个位置。

```
FILE * readFile=fopen("d:\\tf\\data.txt", "rb");
//1 读取一个整型变量
int aa;
fread(&aa, sizeof(int), 1, readFile);
printf("%d\n", aa);
//2 读取一个数组
int arr[100];
int len=fread(arr, sizeof(int), 100, readFile);
for ( int i=0; i<len;i++ )
{
    printf("%d ", arr[i]);
}
```

上面代码中 fread(arr, sizeof(int), 100, readFile);虽然设置了读取 100 个 int 数据，但是其返回值是实际读取的数量。

11、 如何实现文件的复制功能？



上图表达式的是文件复制的逻辑，一个文件在硬盘上是以最小 1KB 为存储单位，所以设置一个容纳 1KB 的数组 buffer 作为缓冲区，不断循环从一个文件中读取 1KB 内容，只有读取的内容大于 0 就把读取的内容写入到另一个文件，从而实现复制的过程。

具体代码如下：

```
#include <stdio.h>
int main ()
```

```

{
    FILE * from = fopen("E:\\保你平安.mp4", "rb");
    FILE * to = fopen("d:\\tf\\bnpa.mp4", "wb");
    unsigned char buffer[1024];
    int len;
    if ( from && to)
    {
        while ( (len=fread(buffer, sizeof(char), 1024, from))>0 )
        {
            fwrite(buffer, sizeof(char), len, to);
        }
        fclose(from );
        fclose(to);
    }
    return 0;
}

```

Part 2 数据库连接

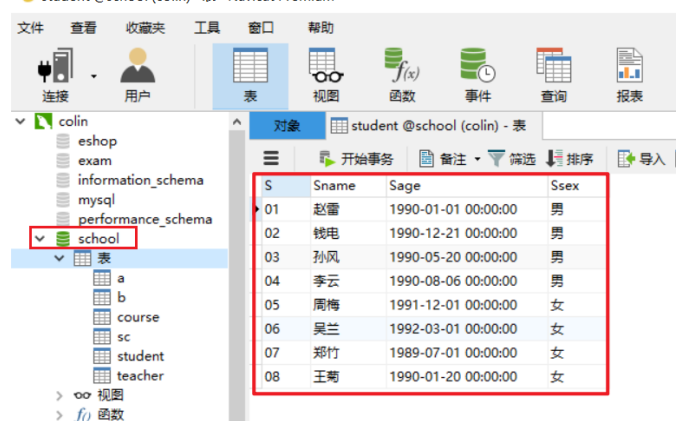
1. 数据库的准备工作有哪些？

首先把 Mysql 数据库安装好，并记住数据库的基本信息：安装位置、root 账号的密码，网络端口号默认 3306 等。这里选择的版本是 Mysql5.7。然后找到 MySQL Server 5.7\lib 文件夹，里面的 libmysql.dll 和 libmysql.lib 是 C 语言连接到 Mysql 的函数库，需要把它们复制到自己的项目的相关位置。libmysql.lib 需要复制到项目内，libmysql.dll 需要复制到生成的 exe 旁边。如果 Mysql 选择的是 x64 版本，那么运行时

请选择 

其次，当数据库环境准备好后需要创建一个数据库并在数据库中创建好表与数据。

student @school (colin) - 表 - Navicat Premium



S	Sname	Sage	Ssex
01	赵雷	1990-01-01 00:00:00	男
02	钱电	1990-12-21 00:00:00	男
03	孙风	1990-05-20 00:00:00	男
04	李云	1990-08-06 00:00:00	男
05	周梅	1991-12-01 00:00:00	女
06	吴兰	1992-03-01 00:00:00	女
07	郑竹	1989-07-01 00:00:00	女
08	王菊	1990-01-20 00:00:00	女

2. C 语言连接 MySQL 有哪些设置及流程？

```

#pragma comment(lib, "libmysql.lib") //加载数据库连接组件
#include <stdio.h>
#include <WinSock2.h> //与Windows通讯接口
#include <mysql.h>
int main()
{
    MYSQL connection; //mysql连接

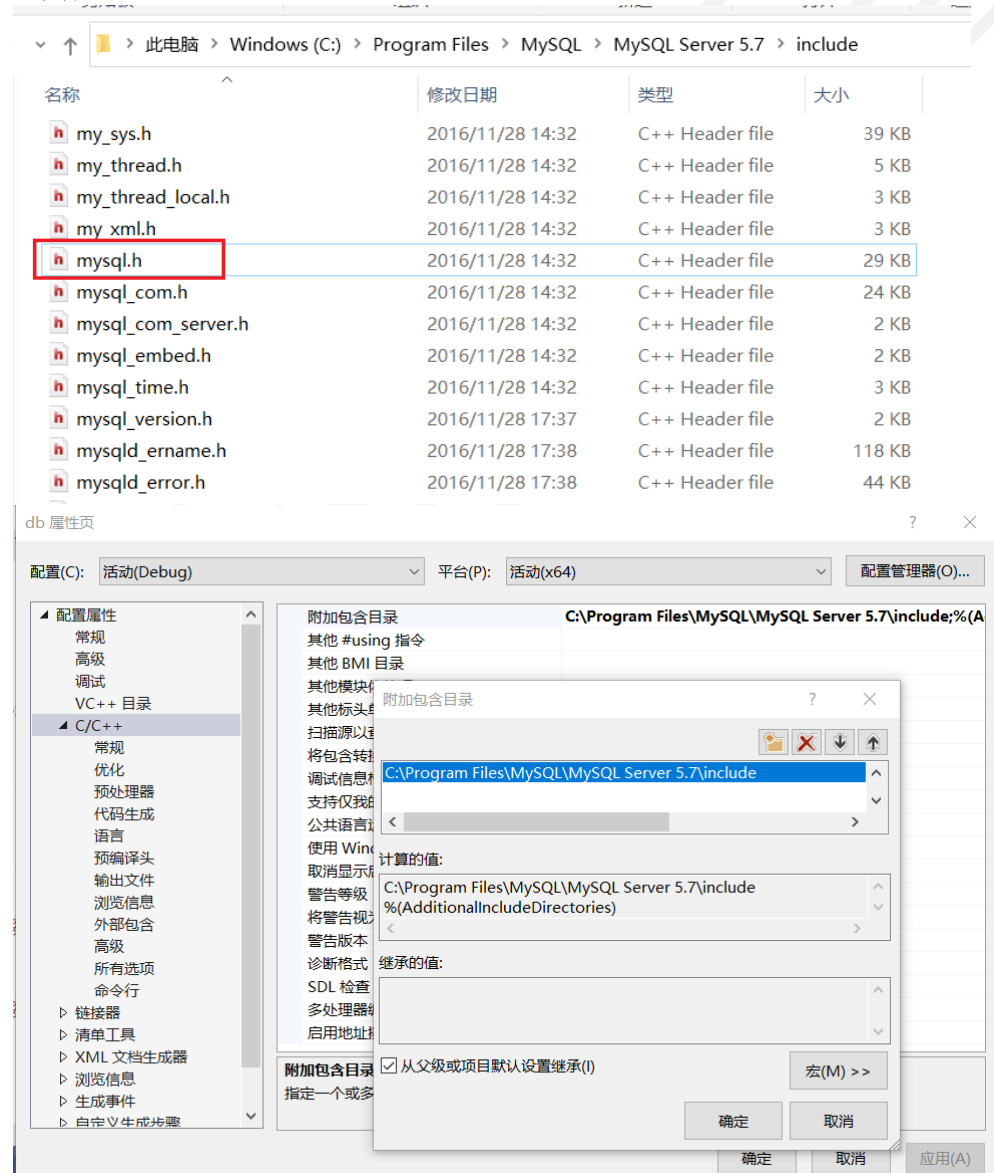
```

```

//初始化数据库
mysql_init(&connection);
//设置编码方式
mysql_options(&connection, MYSQL_SET_CHARSET_NAME, "gbk");
//连接数据库
if (mysql_real_connect(&connection, "localhost", "root", "123456", "school",
3306, NULL, 0))
{
    //主机      用户名      密码      数据库名      端口
    printf("数据库连接成功\n");
}
else {
    printf("数据库连接失败:%s \n", mysql_error(&connection));
    //输出错误信息
}
mysql_close(&connection);
return 0;
}

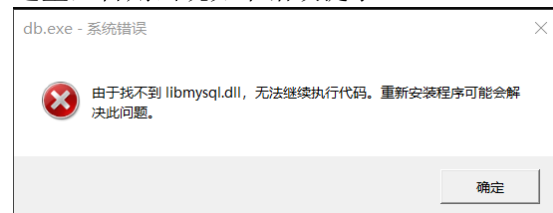
```

另外注意 mysql.h 这个头文件是在 mysql 数据库系统内的，需要在项目属性页中设置其位置。



在运行前记得设置 debug-x64 模式。然后先生成项目。在 VS2019 中找到“解决方

案所在文件夹”，进入到 x64\debug 文件夹下，把 libmysql.dll 这个动态链接库粘贴在这里，否则出现如下错误提示：



如果能够正确连接到数据库会打印出如下内容：

Microsoft Visual Studio 调试控制台

数据库连接成功

如果连接失败就要看失败的信息提示是什么，例如：

Microsoft Visual Studio 调试控制台
数据库连接失败:Access denied for user 'root'@'localhost' (using password: YES)

代表账号密码有错误。

Microsoft Visual Studio 调试控制台
数据库连接失败:Can't connect to MySQL server on 'localhost' (10061)

Mysql 的端口号问题。

Microsoft Visual Studio 调试控制台
数据库连接失败:Unknown database 'schowol'

指定数据库名不存在。

3. 在 C 语言中如何对数据库实现增加、修改、删除的数据操作？

在数据库能够连接的基础上增加如下代码：

首先准备好调试好的 sql 语句

```
const char* sql = "insert into student (s, sname, sage, ssex) VALUES ('10', '张三', '2000-01-02', '男')";
```

然后用一个 en 变量保存执行结果是否成功。0 代表成功 1 代表失败。

```
int en;
if (!(en = mysql_query(&connection, sql)))
{
    printf("增删改查成功 %d \n", en);
}
else
{
    printf("错误信息: %s\n", mysql_error(&connection));
}
```

在上面代码的基础上可以尝试修改的 sql 语句

```
sql = "update student set sname = '李东', sage = '2001-1-5' where s = '10'";
```

删除的sql语句。

```
sql = "delete from student where s = '10'";
```

4. 在 C 语言中如何实现查询结果的显示？

```
const char* sql = "select * from student";
if (!mysql_query(&connection, sql))//利用数据库连接执行sql语句
{
    MYSQL_RES* res; //定义一个查询结果集指针
    res = mysql_store_result(&connection);//从连接中得到查询结果集表格

    unsigned cols = mysql_num_fields(res);//得到结果集的总列数
```

```
my_ulonglong rows = mysql_num_rows(res); //得到结果集的总行数
printf("共查询出: %llu 行 %u 列的数据\n", rows, cols);
MYSQL_FIELD* fields = mysql_fetch_fields(res); //得到结果集的列结构信息
for (int i = 0; i < cols; i++)
{
    printf("%s ", fields[i].name); //打印列名称
}
putchar('\n');
MYSQL_ROW row; //指向一行的信息, 每个列的信息都是一个字符串
while (row = mysql_fetch_row(res)) //返回本行的信息并指向下一行
{
    printf("%s %s %s %s\n", row[0], row[1], row[2], row[3]);
}
mysql_free_result(res); //释放结果集
}
else
{
    printf("错误信息: %s\n", mysql_error(&connection));
}
```

easyX 图形化编程

easyX 技术简介

EasyX是一套用VC++开发的图形库函数，使用它可以非常容易的开发出图形化展示效果的程序。easyX提供绘制几何图形的基本函数功能包括颜色设置、图形填充方案可以方便的绘制出圆形、椭圆形、矩形、弧形、扇形、多边形等几何图形。还有简单易用的图像处理函数，可以实现图像拉伸、旋转等基本贴图功能。另外还会提供绘图窗口的事件响应处理，可以捕捉鼠标键盘的交互信息从而实现类似游戏的交互体验。基于上述easyX提供的基本功能，本教程还封装了动画框架与游戏框架，学习者可以在此框架基础上开发有创意的游戏。提升C语言开发的应用感。

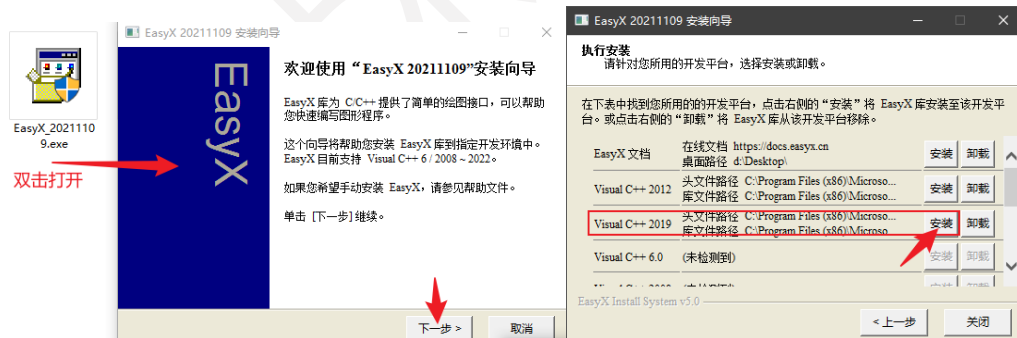
一、 环境准备

访问 <https://easyx.cn/>

1. 下载



2. 安装

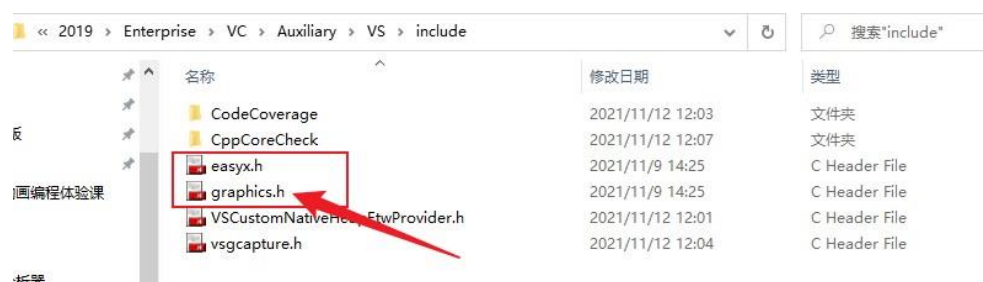


3. 查看安装位置

如果 VS2019 在安装时采用的是默认安装位置，则上述安装会把 easyx 相关组件库复制到 VS2019 文件夹下。【注意：】如果 VS2019 没有安装在默认文件夹下，则需要手动把头文件及库文件复制到你实际的安装位置。easy X 默认安装位置：

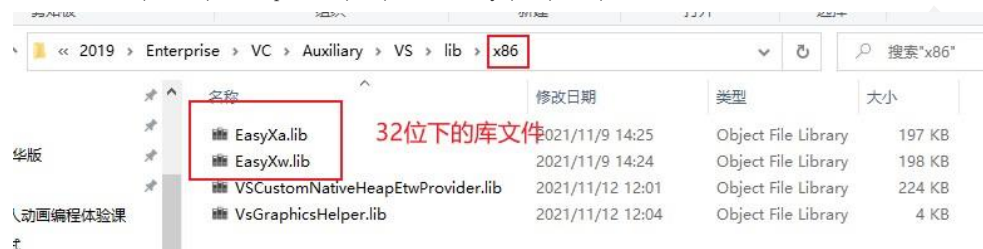
头文件的位置：

C:\Program Files (x86)\Microsoft Visual Studio\2019\Enterprise\VC\Auxiliary\VS\include

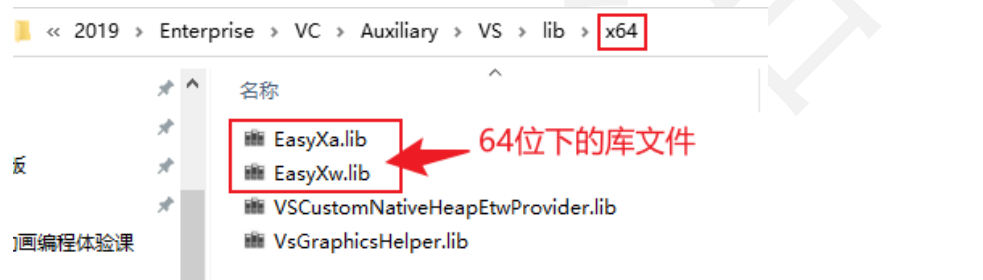


库文件的位置:

C:\Program Files (x86)\Microsoft Visual
Studio\2019\Enterprise\VC\Auxiliary\VS\lib\x86

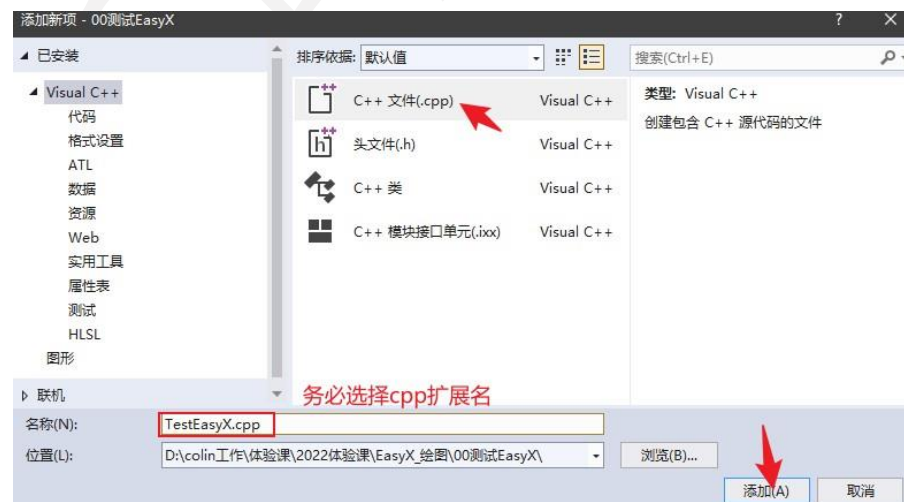


C:\Program Files (x86)\Microsoft Visual
Studio\2019\Enterprise\VC\Auxiliary\VS\lib\x64



4. 验证环境

- 1) 在 VS2019 中创建一个空项目。
- 2) 在‘源文件’ 中添加 C++源文件

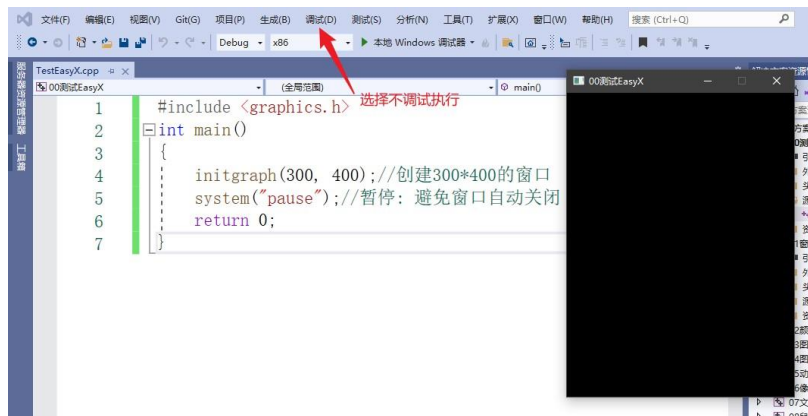


- 3) 测试是否安装成功
输入如下代码:

```
#include <graphics.h> //包含：图像库头文件

int main()
{
    initgraph(300, 400); //创建300*400的窗口
    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}
```

运行出现 300×400 的黑色窗口，代表环境配置成功。

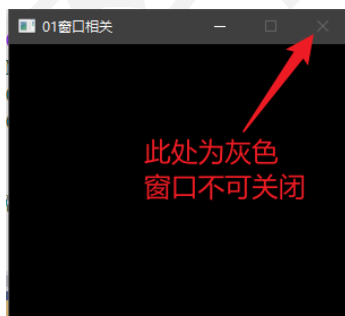


二、 窗口相关

1. 窗口设置

✓ 不显示关闭按钮

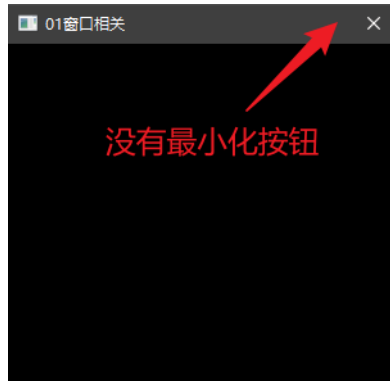
```
#include <graphics.h> //包含：图像库头文件
#include <stdio.h>
int main()
{
    initgraph(300, 400, EW_NOCLOSE); //不显示关闭按钮
    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}
```



✓ 不显示最大最小按钮

```
#include <graphics.h>
#include <stdio.h>
int main()
```

```
{
    initgraph(300, 400, EW_NOMINIMIZE); //不显示最大最小按钮
    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}
```



✓ 混合设置

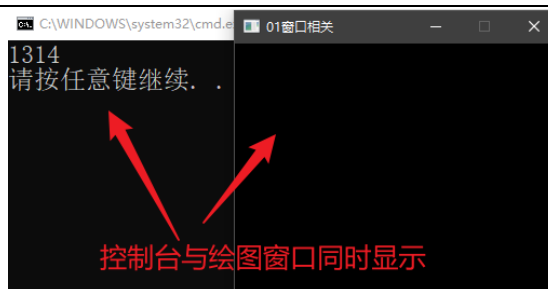
用位运算的 或 (|) 运算连接两个设置。即关闭最小化按钮并且又让关闭按钮不可用。

```
#include <graphics.h>
#include <stdio.h>
int main()
{
    initgraph(300, 400, EW_NOCLOSE|EW_NOMINIMIZE); //不显示关闭按钮
    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}
```

✓ 显示控制台

easyX 默认情况下控制台是不显示的。但是我们经常用 cout 或 printf 输出计算结果辅助调试代码。这就需要绘图窗口与控制台同时显示出来。

```
#include <graphics.h>
#include <stdio.h>
int main ()
{
    initgraph(300, 400, EW_SHOWCONSOLE); //显示控制台
    printf("%d\n", 1314); //向控制台输出一个整数：1314
    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}
```



2. 关闭窗口

可以在程序的执行过程中使用 `closegraph()` 函数关闭绘图窗口。例如： 倒数 5 个数，5 个数过后自动关闭窗口。

```
#include <graphics.h>
#include <stdio.h>
int main()
{
    initgraph(300, 400, EW_SHOWCONSOLE); //显示控制台
    //5秒后自动关闭窗口
    for (int i = 1; i <= 5; i++)
    {
        printf("%d\n", i); //向控制台打印i的值
        Sleep(1000); // “等待” 1秒后，代码继续执行。（注：1秒==1000毫秒）
    }
    closegraph(); //循环结束后 关闭当前的绘图窗口
    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}
```

3. 对话框

✓ 利用对话框窗口，可以询问一个问题，让用户输入该问题的相关答案。例如：

```
#include <graphics.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h> //语言本地化头文件
int main()
{
    initgraph(300, 400, EW_SHOWCONSOLE); //显示控制台
    setlocale(LC_ALL, "chs"); //设置语言本地化，让双字节字符串打印出“汉字”
    TCHAR info[50]; //创建一个双字节字符串数组，用以保留用户的输入
    bool answer = InputBox(info, 50, L"输入你感兴趣的岗位", L"入职调查", L"C++后端", 0, 0, true);
    printf("%S\n", info);
    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}
```



✓ 带有“确定” “取消” 按钮的对话框

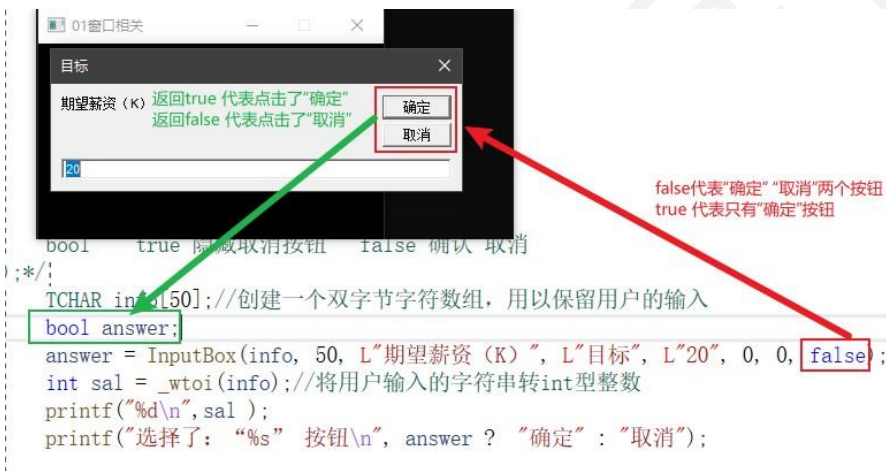
```

#include <graphics.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h> //语言本地化头文件
int main()
{
    initgraph(300, 400, EW_SHOWCONSOLE); //显示控制台
    setlocale(LC_ALL, "chs"); //设置语言本地化, 让双字节字符串打印出“汉字”

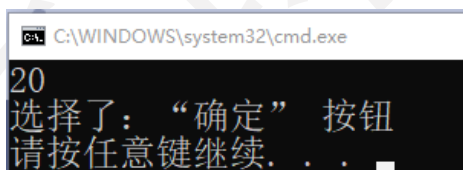
    TCHAR info[50]; //创建一个双字节字符串数组, 用以保留用户的输入
    bool answer;
    answer = InputBox(info, 50, L"期望薪资 (K)", L"目标", L"20", 0, 0, false);
    int sal = _wtoi(info); //将用户输入的字符串转int型整数
    printf("%d\n", sal);
    printf("选择了: \"%s\" 按钮\n", answer ? "确定" : "取消");

    system("pause"); //暂停: 避免窗口自动关闭
    return 0;
}

```



打印结果如下:



4. 改变“标题栏”

标题栏默认内容是项目的名称。也可以动态设置标题栏内容。

例如: 3 秒后改变标题栏内容为: hello

```

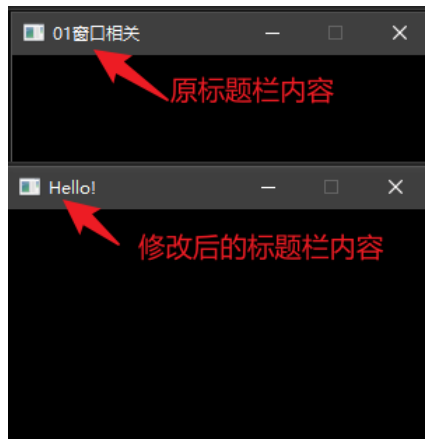
#include <graphics.h>
#include <stdio.h>
int main ()
{
    initgraph(300, 400, EW_SHOWCONSOLE); //显示控制台
    Sleep(3000); //3秒后改变标题栏
    // 获得窗口句柄
}

```

```

HWND hWnd = GetHWND();
// 使用 Windows API 修改窗口名称
SetWindowText(hWnd, L"Hello!");
system("pause");//暂停：避免窗口自动关闭
return 0;
}

```



5. 坐标原点

窗口绘图显示以坐标为参数。默认的坐标原点 (x=0, y=0) 位于窗口的左上角。

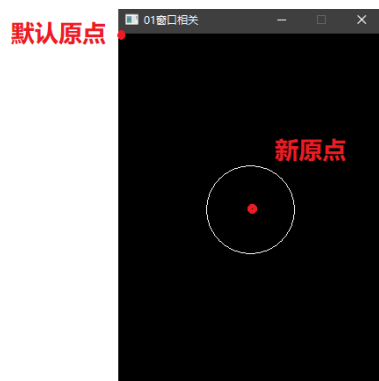


也可以设置某一个坐标位置为原点 (0, 0)。例如：

```

#include <graphics.h>
int main ()
{
    initgraph(300, 400);//创建300*400的窗口
    //设置新的坐标原点位置
    setorigin(150, 200);//让300*400的窗口中心为原点。
    circle(0, 0, 50);//在根据新“原点” 绘制圆形
    system("pause");//暂停：避免窗口自动关闭
    return 0;
}

```



6. 缩放绘图

绘图宽高默认以像素为单位 100%绘制。也可以按比例缩放绘制图形。例如

```
#include <graphics.h> //包含：图像库头文件
int main ()
{
    initgraph(300, 400); //创建300*400的窗口

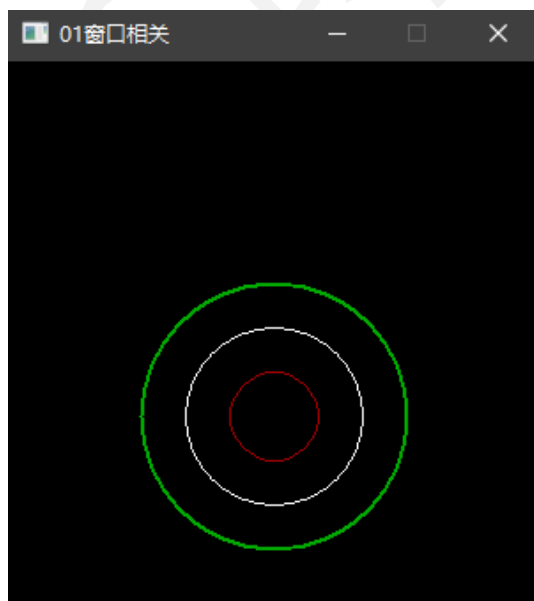
    //设置新的坐标原点位置setorigin(150, 200);
    circle(0, 0, 50); //默认100%绘制圆形

    //设置绘图缩放setlinecolor(RED);
    setaspectratio(0.5f, 0.5f); //缩放比例50%绘制圆形
    circle(0, 0, 50); //同样是50的半径 比 上面的短一半

    setlinecolor(GREEN);
    setaspectratio(1.5f, 1.5f); //缩放比例150%绘制圆形
    circle(0, 0, 50); //同样是50的半径 比 上面扩大1.5倍

    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}
```

绘制效果如下



7. 获取窗口大小

可以利用: `getwidth()`, `getheight()` 获得窗口当前的宽度与高度。

```
initgraph(300, 400, EW_SHOWCONSOLE); //创建300*400的窗口
printf("窗口绘图区的宽 %d 窗口的高 %d", getwidth(), getheight());
```



8.

三、 颜色模式

1. 颜色原理

计算机中颜色值的原理是颜色采用光的三原色即 Red 红色、Green 绿色、Blue 蓝色。缩写 RGB，使用三种原色 “调和” 成不同的颜色，然后屏幕的像素点显示颜色。想要设置一个颜色只需要分别设置构成该颜色的三个原色的值即可。可以在 word 中设置字体颜色的时候调出如下窗口。



其中红绿蓝每个颜色值为 0~255 之间，也就是一个字节。如果用十六进制表示此数就是两位十六进制数 一般情况下颜色的设置通常用十六进制方式设置。

格式为: `0xRRGGBB` 即红绿蓝色值。**注意:** 在 easyx 中默认的颜色设置顺序是蓝绿红即 `0xBBRRGG` 模式

2. 颜色设置

方式一：使用“宏”预设的颜色值



```
#include <graphics.h> //包含：图像库头文件
int main ()
{
    initgraph(300, 400); //创建300*400的窗口

    setbkcolor(BLUE); //设置背景色为蓝色。
    cleardevice(); //清除刷新界面 可以看到背景色效果

    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}
```

方式二：用十六进制设置颜色值

```
setbkcolor(0x00ff00); //蓝 + 绿 + 红
cleardevice(); //清除刷新界面
```

方式三：用十进制设置颜色值

```
setbkcolor(RGB(255, 0, 0)); //RGB(红色十进制值, 绿色十进制值, 蓝色十进制值)
cleardevice(); //清除刷新界面
```

或者

```
COLORREF color = RGB(255, 0, 0);
setbkcolor(color);
cleardevice(); //清除刷新界面
```

COLORREF 是 easyX 定义的保留颜色值的数据类型。其本质原始类型为：unsigned long

3. 获取颜色值

我们可以获得某颜色的红、绿、蓝三原色的值分别是多少。

```
#include <stdio.h>
#include <graphics.h>
int main()
{
    initgraph(300, 400, EW_SHOWCONSOLE); //创建300*400的窗口

    //分别取出对应的RGB值
```

```
printf("红色值: %d 绿色值: %d 蓝色值: %d "
      , GetRValue(BLUE), GetGValue(BLUE), GetBValue(BLUE));
system("pause");//暂停: 避免窗口自动关闭
return 0;
}
```

GetRValue(BLUE): 得到颜色参数中红色部分的值。

GetGValue(BLUE): 得到颜色参数中绿色部分的值。

GetBValue(BLUE): 得到颜色参数中蓝色部分的值。

4. 红蓝对调

一般情况下普遍习惯于用 RGB 即红绿蓝的顺序设置颜色的值。但是 easyX 默认是 BGR 的顺序。其实 easyX 也提供了调换的方法。即 BGR(颜色) 把颜色参数的颜色顺序由 BGR 替换成 RGB。如下动画例子中 背景的红蓝切换就是用的此功能。

```
#include <graphics.h>//包含: 图像库头文件
int main()
{
    initgraph(300, 400, EW_SHOWCONSOLE);//创建300*400的窗口

    setbkcolor(BLUE);
    cleardevice();

    for (int i = 0; i < 50; i++)
    {
        Sleep(300);
        setbkcolor(BGR(getbkcolor()));//把背景色进行红蓝对调
        cleardevice();
    }

    system("pause");//暂停: 避免窗口自动关闭
    return 0;
}
```

四、 绘制图形

可以利用图形函数绘制几何图形: 包括直线、圆形、椭圆形、矩形、圆角矩形、多边形、扇形、弧形等。每个图形可以绘制成: 空心描边、实心描边、实心但不描边等造型, 填充方面可以是单一颜色、预定的图案或某图片平铺等方式。

1. 直线

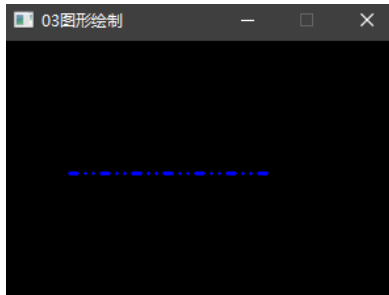
“两点确定一条直线”。

line(点 1 X 坐标, 点 1 Y 坐标, 点 2 X 坐标, 点 2 Y 坐标)

用于绘制一条直线。在绘制之前还需要对直线的外观进行相关的设置。

```
initgraph(300, 400);//创建300*400的窗口
//设置颜色
setlinecolor(0xff0000);
//设置样式(多选一) 和 粗细
//setlinestyle(PS_SOLID, 3);//实线
//setlinestyle(PS_DASH, 3);//虚线
//setlinestyle(PS_DOT, 3);//点线
//setlinestyle(PS_DASHDOT, 3);//虚 点线
```

```
setlinestyle(PS_DASHDOTDOT, 3); //虚 点点 线
//画线：两点确定一条直线
line(50, 100, 200, 100);
```

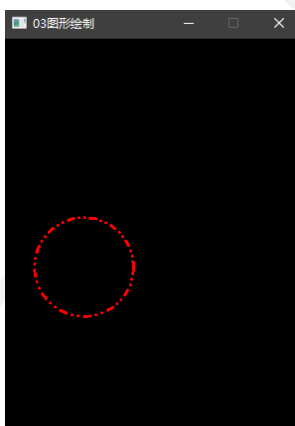


2. 圆形

绘制圆形：确定坐标位置及半径后 绘制一个正圆形。

A. 空心圆形：使用 circle(X 坐标, Y 坐标, 半径) 函数

```
initgraph(300, 400); //创建300*400的窗口
//设置颜色
setlinecolor(0x0000ff);
//设置样式(多选一) 和 粗细
//setlinestyle(PS_SOLID, 3); //实线
//setlinestyle(PS_DASH, 3); //虚线
//setlinestyle(PS_DOT, 3); //点线
//setlinestyle(PS_DASHDOT, 3); //虚 点线
setlinestyle(PS_DASHDOTDOT, 3); //虚 点点 线
//绘制空心圆形
circle(80, 230, 50);
```



B. 实心圆

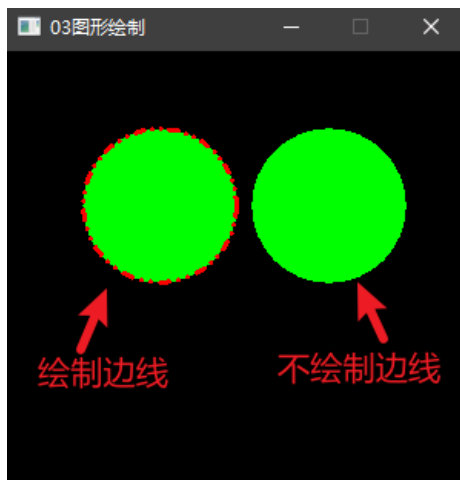
fillcircle(X 坐标, Y 坐标, 半径); 填充圆形, 但保留圆形的边线。

solidcircle(X 坐标, Y 坐标, 半径); 填充圆形, 但不保留圆形的边线。

1: 填充方式：纯颜色填充

```
setlinecolor(0x0000ff); //边线的颜色
//设置边线的样式(多选一) 和 粗细
//setlinestyle(PS_SOLID, 3); //实线
//setlinestyle(PS_DASH, 3); //虚线
//setlinestyle(PS_DOT, 3); //点线
//setlinestyle(PS_DASHDOT, 3); //虚 点线
setlinestyle(PS_DASHDOTDOT, 3); //虚 点点 线
```

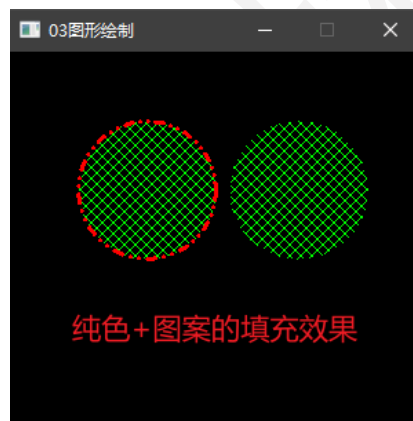
```
setfillcolor(0x00ff00); //设置填充颜色为 绿色
setfillstyle(BS_SOLID); //设置填充方式为：纯色
fillcircle(100, 100, 50);
solidcircle(210, 100, 50);
```



2. 填充方式：图案填充

可以在纯色基础上 再设置填充的预设图案。

```
//如下预设图案多选一
//setfillstyle(BS_HATCHED, HS_HORIZONTAL); //水平线填充
//setfillstyle(BS_HATCHED, HS_VERTICAL); //垂直线填充
//setfillstyle(BS_HATCHED, HS_FDIAGONAL); //右斜线填充
//setfillstyle(BS_HATCHED, HS_BDIAGONAL); //左斜线填充
//setfillstyle(BS_HATCHED, HS_CROSS); //方格线填充
//setfillstyle(BS_HATCHED, HS_DIAGCROSS); //斜网格线填充
solidcircle(210, 100, 50);
fillcircle(100, 100, 50);
```



3. 填充方式：图像（图片）填充

可以把硬盘上的图片加载到内存。方法如下。

```
IMAGE img; //定义图片变量 用于存储图片
loadimage(&img, L".\\image\\0.png", 20, 20, true); //加载图片，并缩放成 20*20 像素大小
```

注意：图片的路径分为绝对路径，例如：d:\images\0.png 和 相对路径；所谓相对路径就是根据代码文件的所在路径去寻找图片。.\image\0.png 中的“.”就代表了代码文件所在的当前位置。

```
//设置填充方式为：图像，并指明图像的来源（图像内存变量）
```

```
setfillstyle(BS_DIBPATTERN, NULL, &img);
solidcircle(210, 100, 50);
fillcircle(100, 100, 50);
```

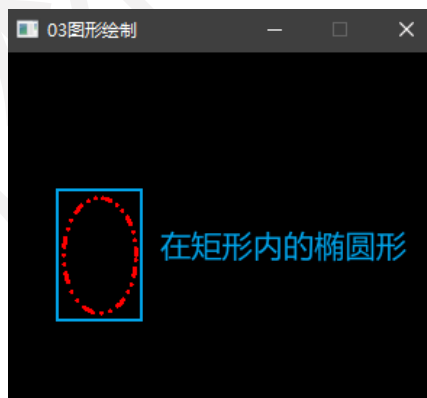


3. 椭圆形

绘制椭圆形与绘制圆形的参数不同。它是以矩形为基础的内接圆。所以它参数设置与矩形相同。

ellipse(顶点 X, 顶点 Y, 顶点 X+宽度, 顶点 Y+高度); 其它外观设置方面和圆形基本一致。

```
//设置颜色
setlinecolor(0x0000ff);
//设置样式(多选一) 和 粗细
//setlinestyle(PS_SOLID, 3); //实线
//setlinestyle(PS_DASH, 3); //虚线
//setlinestyle(PS_DOT, 3); //点线
//setlinestyle(PS_DASHDOT, 3); //虚点线
setlinestyle(PS_DASHDOTDOT, 3); //虚点点线
//绘制空心椭圆形
int w = 50, h = 80; //宽度50 高度80的范围内绘制内接椭圆形
ellipse(40, 100, 40 + w, 100 + h);
```



实心椭圆形

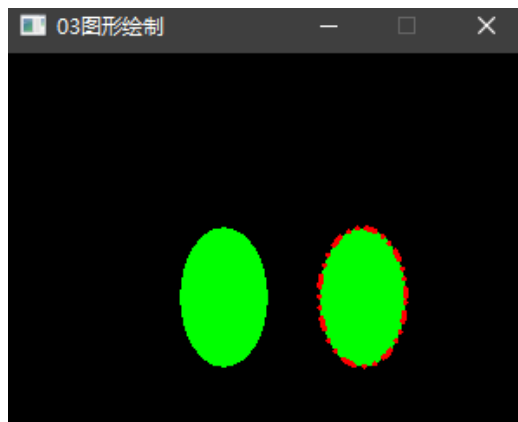
solidellipse(顶点 X, 顶点 Y, 顶点 X+宽度, 顶点 Y+高度); 填充椭圆形, 但不保留图形的边线。

fillellipse(顶点 X, 顶点 Y, 顶点 X+宽度, 顶点 Y+高度); 填充椭圆形, 但保留图形的边线

填充方式 A: 纯色

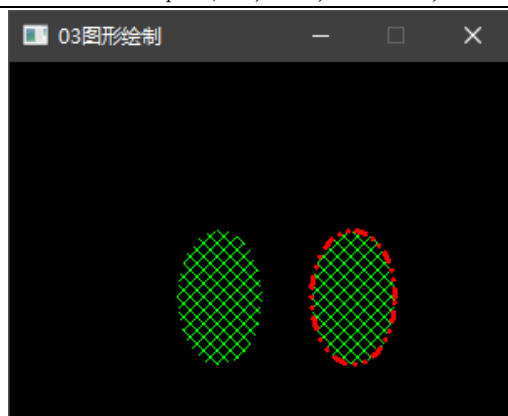
```
int w = 50, h = 80;
setfillcolor(0x00ff00); //绿色填充
```

```
setfillstyle(BS_SOLID); //设置 实心填充
solidellipse(100, 100, 100+w, 100+h);
fillellipse(180, 100, 180+w, 100+h);
```



填充方式 B: 图案

```
int w = 50, h = 80;
setfillcolor(0x00ff00); //绿色填充
setfillstyle(BS_HATCHED, HS_DIAGCROSS); //斜网格线填充
solidellipse(100, 100, 100 + w, 100 + h);
fillellipse(180, 100, 180 + w, 100 + h);
```



填充方式 C: 图像/图片

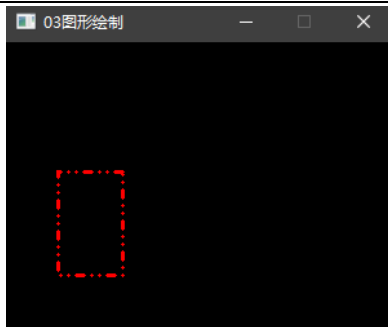
```
int w = 50, h = 80;
IMAGE img; //定义图片存储变量
loadimage(&img, L".\\image\\0.png", 20, 20, true); //加载图片, 并大小缩放
setfillstyle(BS_DIBPATTERN, NULL, &img);
solidellipse(100, 100, 100 + w, 100 + h);
fillellipse(180, 100, 180 + w, 100 + h);
```

4. 矩形

矩形绘制的两个参数默认是顶点的坐标与对角点的坐标: `rectangle(顶点 X, 顶点 Y, 对角点 X, 对角点 Y)`。也可以用宽和高的思维绘制矩形即: `rectangle(顶点 X, 顶点 Y, 顶点 X+宽度, 顶点 Y+高度)`;

```
initgraph(300, 400); //创建300*400的窗口
//设置颜色
setlinecolor(0x0000ff);
```

```
setlinestyle(PS_DASHDOTDOT, 3); //虚 点点 线
//绘制空心矩形
int w = 50, h = 80;
rectangle(40, 100, 40 + w, 100 + h); //使用宽高思维绘制矩形
```



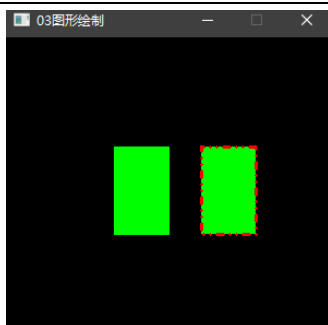
实心矩形

`solidrectangle(顶点 X, 顶点 Y, 顶点 X+宽度, 顶点 Y+高度);` 填充矩形，但不保留图形的边线。

`fillrectangle(顶点 X, 顶点 Y, 顶点 X+宽度, 顶点 Y+高度);` 填充矩形，但保留图形的边线。

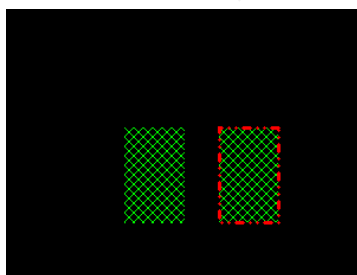
填充方式 A: 纯色

```
int w = 50, h = 80;
setfillcolor(0x00ff00); //绿色填充
setfillstyle(BS_SOLID); //设置 实心填充
solidrectangle(100, 100, 100+w, 100+h);
fillrectangle(180, 100, 180+w, 100+h);
```



填充方式 B: 图案

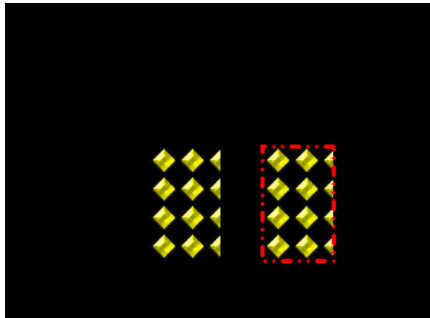
```
int w = 50, h = 80;
setfillcolor(0x00ff00); //绿色填充
setfillstyle(BS_HATCHED, HS_DIAGCROSS); //斜网格线填充
solidrectangle(100, 100, 100+w, 100+h);
fillrectangle(180, 100, 180+w, 100+h);
```



填充方式 C: 图像/图片

```
int w = 50, h = 80;
IMAGE img; //定义图片存储变量
```

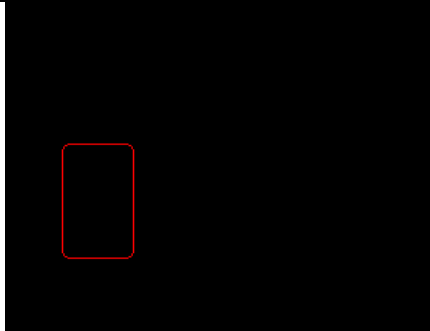
```
loadimage(&img, L".\\image\\0.png", 20, 20, true); //加载图片, 并大小缩放
setfillstyle(BS_DIBPATTERN, NULL, &img);
solidrectangle(100, 100, 100 + w, 100 + h);
fillrectangle(180, 100, 180 + w, 100 + h);
```



5. 圆角矩形

圆角矩形和矩形的用法基本相同, 只是参数增加了两个: 圆角部分的宽、高是多少像素。它会根据设置 绘制不同大小的圆角。

```
initgraph(300, 400); //创建300*400的窗口
//设置颜色
setlinecolor(0x0000ff);
setlinestyle(PS_SOLID, 1); //实线
int w = 50, h = 80;
//绘制空心圆角矩形
roundrect(40, 100, 40 + w, 100 + h, 10, 10);
```



实心圆角矩形

`solidroundrect`(顶点 X, 顶点 Y, 顶点 X+宽度, 顶点 Y+高度, 圆角宽, 圆角高); 填充圆角矩形, 但不保留图形的边线。

`fillroundrect`(顶点 X, 顶点 Y, 顶点 X+宽度, 顶点 Y+高度, 圆角宽, 圆角高); 填充圆角矩形, 但保留图形的边线。

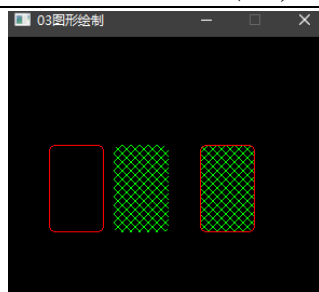
填充方式 A: 纯色

```
int w = 50, h = 80;
setfillcolor(0x00ff00); //绿色填充
roundrect(40, 100, 40 + w, 100 + h, 10, 10); //绘制空心圆角矩形
setfillstyle(BS_SOLID); //设置 实心填充
solidroundrect(100, 100, 100 + w, 100 + h, 10, 10); //绘制实心无边圆角矩形
fillroundrect(180, 100, 180+w, 100+h, 10, 10); //绘制实心有边圆角矩形
```



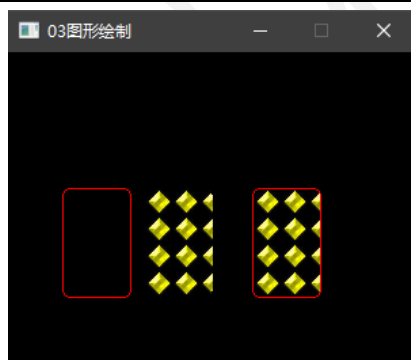
填充方式 B: 图案

```
int w = 50, h = 80;
roundrect(40, 100, 40 + w, 100 + h, 10, 10); //绘制空心圆角矩形
setfillcolor(0x00ff00); //绿色填充
setfillstyle(BS_HATCHED, HS_DIAGCROSS); //斜网格线填充
solidroundrect(100, 100, 100 + w, 100 + h, 10, 10); //绘制实心无边圆角矩形
fillroundrect(180, 100, 180+w, 100+h, 10, 10); //绘制实心有边圆角矩形
```



填充方式 C: 图像/图片

```
int w = 50, h = 80;
roundrect(40, 100, 40 + w, 100 + h, 10, 10); //绘制空心圆角矩形
IMAGE img; //定义图片存储变量
loadimage(&img, L".\\image\\0.png", 20, 20, true); //加载图片, 并大小缩放
setfillstyle(BS_DIBPATTERN, NULL, &img);
solidroundrect(100, 100, 100 + w, 100 + h, 10, 10); //绘制实心无边圆角矩形
fillroundrect(180, 100, 180+w, 100+h, 10, 10); //绘制实心有边圆角矩形
```



6. 圆弧形

所谓圆弧形就是绘制一段弧线。其基于一个矩形内接圆, 根据起始弧度绘制的一段弧线 arc(顶点 X, 顶点 Y, 顶点 X+矩形宽, 顶点 Y+ 矩形高, 开始点弧度, 结束点弧度); 注意: 其参数采用弧度值。如果你需要用角度值需要自行写出角度与弧度的转换公式。

角度转弧度: $\text{PI}/180 \times \text{角度}$

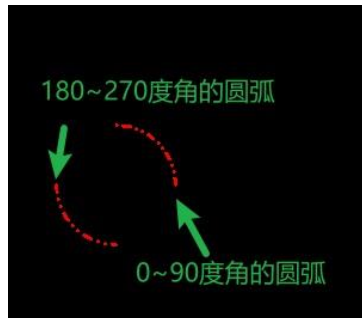
弧度转角度: $180/\text{PI} \times \text{弧度}$

```
initgraph(300, 400); //创建300*400的窗口
```

```

//设置颜色
setlinecolor(0x0000ff);
setlinestyle(PS_DASHDOTDOT, 3); //虚 点点 线
//绘制 圆弧形
double PI = 3.1415926; int w = 100, h = 100;
//绘制从0度角到90度角的弧形
arc(40, 100, 40 + w, 100 + h, PI / 180 * 0, PI / 180 * 90);
//绘制从180度角到270度角的弧形
arc(40, 100, 40 + w, 100 + h, PI / 180 * 180, PI / 180 * 270);

```



7. 扇形

扇形就是基于一个矩形内接圆，根据起始弧度绘制的部分内接扇形

pie(顶点 X, 顶点 Y, 顶点 X+矩形宽, 顶点 Y+ 矩形高, 开始点弧度, 结束点弧度);

注意：其参数采用弧度值。如果你需要用角度值需要自行写出角度与弧度的转换公式。

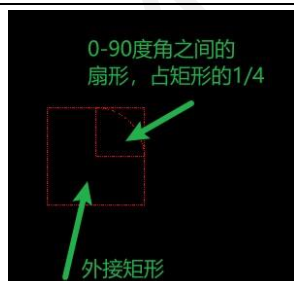
角度转弧度： $PI/180 \times \text{角度}$

弧度转角度： $180/PI \times \text{弧度}$

```

//设置颜色
setlinecolor(0x0000ff);
setlinestyle(PS_DASHDOTDOT, 1); //虚 点点 线
//绘制 圆弧形
/*
角度转弧度：PI/180*角度 弧度转角度：180/PI*弧度
*/
double PI = 3.1415926;
int w = 100, h = 100;
//绘制一个矩形，观察扇形内接在矩形中的部位
rectangle(40, 100, 40 + w, 100 + h);
//x y 宽 高 开始角 结束角
pie(40, 100, 40 + w, 100 + h, PI / 180 * 0, PI / 180 * 90);

```



实心扇形

solidrectangle(顶点 X, 顶点 Y, 顶点 X+宽度, 顶点 Y+高度); 填充扇形，但不保留图形的边线。

fillrectangle(顶点 X, 顶点 Y, 顶点 X+宽度, 顶点 Y+高度); 填充扇形，但保留图形的边

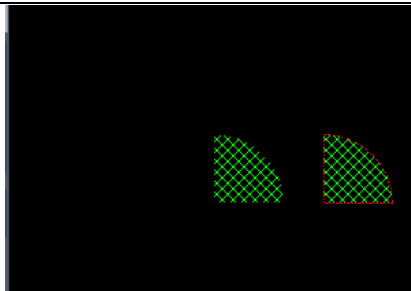
线。

填充方式 A: 纯色

```
double PI = 3.1415926;
int w = 100, h = 100;
setfillstyle(BS_SOLID); //设置实心填充
setfillcolor(0x00ff00); //绿色
solidpie(100, 100, 100+w, 100+h, PI / 180 * 0, PI / 180 * 90);
fillpie(180, 100, 180 + w, 100 + h, PI / 180 * 0, PI / 180 * 90);
```

填充方式 B: 图案

```
double PI = 3.1415926;
int w = 100, h = 100;
setfillstyle(BS_SOLID); //设置实心填充
setfillcolor(0x00ff00); //绿色
setfillstyle(BS_HATCHED, HS_DIAGCROSS); //斜网格线填充
solidpie(100, 100, 100+w, 100+h, PI / 180 * 0, PI / 180 * 90);
fillpie(180, 100, 180 + w, 100 + h, PI / 180 * 0, PI / 180 * 90);
```



填充方式 C: 图像/图片

```
double PI = 3.1415926;
int w = 100, h = 100;
IMAGE img; //定义图片存储变量
loadimage(&img, L".\\image\\0.png", 20, 20, true); //加载图片, 并大小缩放
setfillstyle(BS_DIBPATTERN, NULL, &img);
solidpie(100, 100, 100+w, 100+h, PI / 180 * 0, PI / 180 * 90);
fillpie(180, 100, 180 + w, 100 + h, PI / 180 * 0, PI / 180 * 90);
```



8. 多边形

多边形就是通过每个顶点的坐标最终连接成一个图形。所以需要用一个 POINT 类型数组保存每个点的坐标。

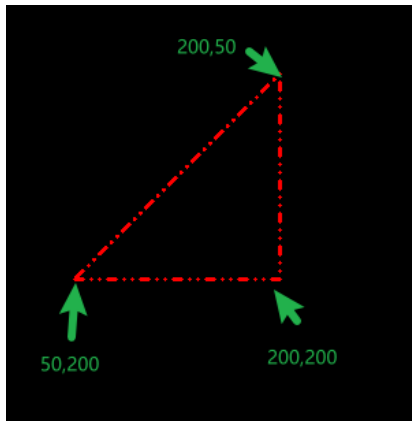
```
typedef struct tagPOINT
{
    LONG x;
    LONG y;
} POINT; //POINT坐标点类型是一个结构体
// *PPPOINT, *NPPOINT, FAR *LPPOINT;
```

例如设置三角形的三个点:

```
//设置多边形每个点的坐标位置
POINT pts[] = { {50, 200}, {200, 200}, {200, 50} };
```

然后用 polygon(坐标点数组, 点数);函数绘制多边形。

```
initgraph(300, 400); //创建300*400的窗口
setlinecolor(0x0000ff);
setlinestyle(PS_DASHDOTDOT, 3); //虚 点点 线
//绘制三角形
//设置多边形每个点的坐标位置
POINT pts[] = { {50, 200}, {200, 200}, {200, 50} };
polygon(pts, 3);
```



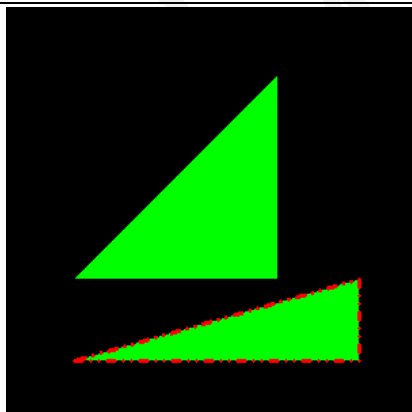
实心多边形

solidpolygon(顶点 X, 顶点 Y, 顶点 X+宽度, 顶点 Y+高度); 填充扇形, 但不保留图形的边线。

fillpolygon(顶点 X, 顶点 Y, 顶点 X+宽度, 顶点 Y+高度); 填充扇形, 但保留图形的边线。

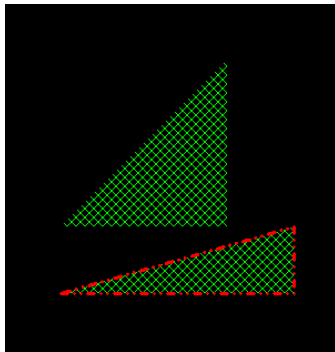
填充方式 A: 纯色

```
setfillcolor(0x00ff00); //绿色填充
setfillstyle(BS_SOLID); //设置 实心填充
POINT pts1[] = { {50, 200}, {200, 200}, {200, 50} };
solidpolygon(pts1, 3);
POINT pts2[] = { {50, 260}, {260, 260}, {260, 200} };
fillpolygon(pts2, 3);
```



填充方式 C: 图案

```
setfillcolor(0x00ff00); //绿色填充
setfillstyle(BS_SOLID); //设置 实心填充
setfillstyle(BS_HATCHED, HS_DIAGCROSS); //斜网格线填充
POINT pts1[] = { {50, 200}, {200, 200}, {200, 50} };
solidpolygon(pts1, 3);
POINT pts2[] = { {50, 260}, {260, 260}, {260, 200} };
fillpolygon(pts2, 3);
```



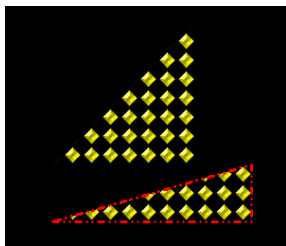
填充方式 C: 图像/图片

```

IMAGE img;//定义图片存储变量
loadimage(&img, L".\\image\\0.png", 20, 20, true);//加载图片, 并大小缩放
setfillstyle(BS_DIBPATTERN, NULL,&img);

POINT pts1[] = { {50, 200}, {200, 200}, {200, 50} };
solidpolygon(pts1, 3);
POINT pts2[] = { {50, 260}, {260, 260}, {260, 200} };
fillpolygon(pts2, 3);

```



9. 区域填充

封闭区域填充: 设置图形空间内的任意一点坐标, 再根据边线颜色。在一个封闭空间内填充指定颜色、图案、图像。封闭的判断依据就是边线的颜色值。

利用 *floodfill* (坐标 *X*, 坐标 *Y*, 边线颜色, *FLOODFILLBORDER*) 函数实现此功能。

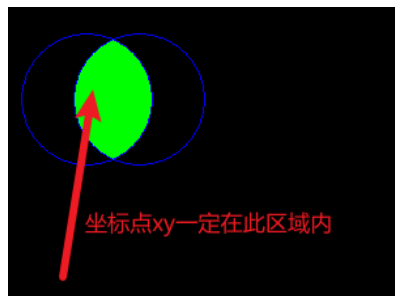
其中: *FLOODFILLBORDER* 代表: 封闭区域内填充

```

initgraph(300, 400);
setlinecolor(0xff0000);
setlinestyle(PS_SOLID, 1);//实线

circle(100, 70, 50);
circle(60, 70, 50);
//封闭区域内填充
setfillcolor(0x00ff00);//绿色
//区域内填充
// 位置点xy 区域边界颜色 封闭区域内填充
floodfill(70, 70, 0xff0000, FLOODFILLBORDER);

```



10. 镂空图形

镂空图形就是在当前绘图区“扣下”一个图形并露出背景色。 镂空图形就是在原有绘图函数前增加 clear 某图形。

```
initgraph(300, 400); //创建300*400的窗口
setbkcolor(RED); //设置背景色
cleardevice(); //刷新窗口
/*绘制一个图像*/
IMAGE imgBK ;
loadimage(&imgBK, L".\\image\\背景_03.png");
putimage(0, 0, &imgBK);
//镂空一个圆形
clearcircle(80, 80, 50);
int w = 50, h = 80;
//镂空一个椭圆形
clearellipse(40, 100, 40 + w, 100 + h);
//镂空一个矩形
clearrectangle(40, 100, 40 + w, 100 + h);
//镂空一个圆角矩形
clearroundrect(40, 100, 40 + w, 100 + h, 10, 10);

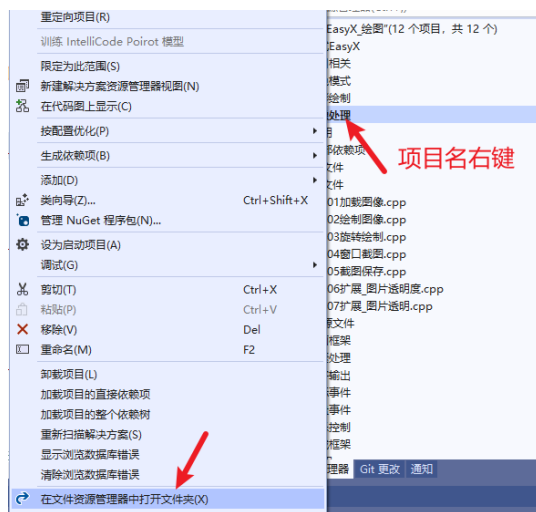
double PI = 3.1415926;
//镂空一个扇形
clearpie(40, 100, 40 + w, 100 + h, PI / 180 * 0, PI / 180 * 90);
//镂空一个三角形
POINT pts[] = { {50, 200}, {200, 200}, {200, 50} };
clearpolygon(pts, 3);
```



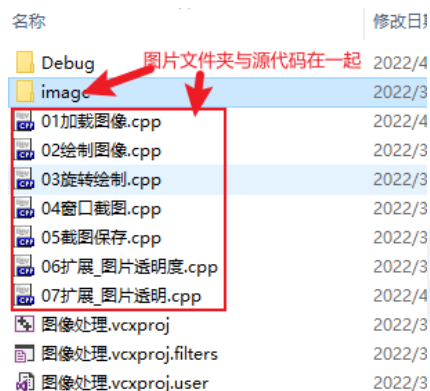
五、 图像处理

图像处理就是把硬盘的图片加载到内存一个 IMAGE 的变量中。其中图片路径分为绝对路径和相对路径。 绝对路径就是指明图片所在的盘符与路径，相对路径就是把图片或图片文件夹放到与源代码同一个文件夹内。

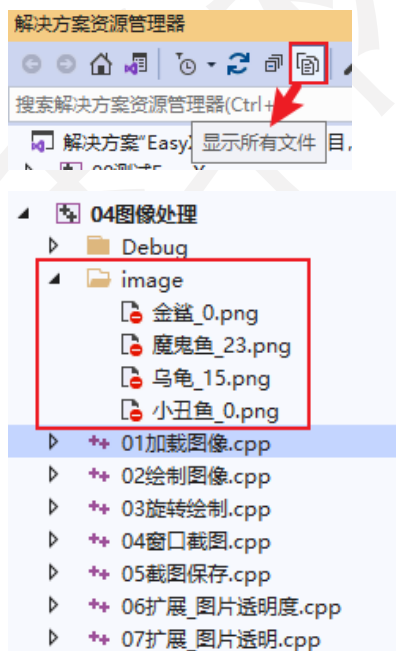
具体操作：1 在 VS2019 中打开项目所在文件夹。



2. 把图片粘贴到项目文件夹内



进入 VS “解决方案管理器” 选择项目名，选择顶端“显示所有文件”按钮，进入到文件浏览界面，可以查看到项目所在文件夹下的所有文件及文件夹



选择图片名称->右键->查看“属性”会看到如下窗口。



1. 加载图像

方式 1：按图片默认大小加载

```
IMAGE img1; //创建一个IMAGE变量，用以保存加载进内存的图片
loadimage(&img1, L".\\image\\金鲨_0.png"); //从硬盘任意位置加载图像，并默认图像大小
putimage(10, 20, &img1); //把 img1 地址的图片呈现到坐标(x,y)值是 10 20 的绘图窗口
```

运行效果如下：



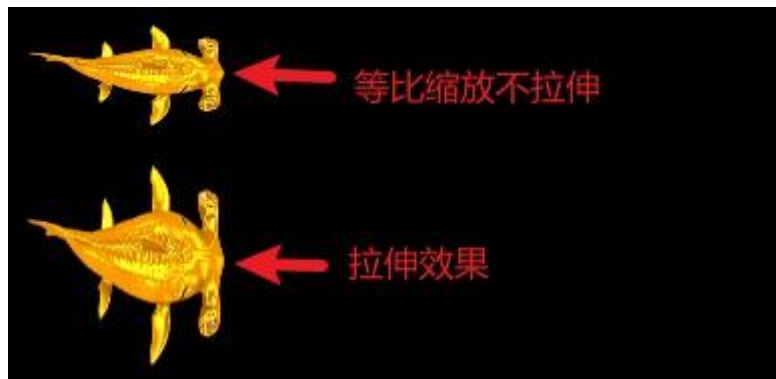
方式 2：图片等比缩放加载

```
IMAGE img2;
//从硬盘任意位置加载图像，并按100宽50高的像素空间缩放。 false代表等比缩放
loadimage(&img2, L".\\image\\金鲨_0.png", 100, 50, false);
putimage(10, 180, &img2);
```



方式 3：图片拉伸加载

```
IMAGE img3;
//从硬盘任意位置加载图像， 并按100宽50高的像素空间缩放。 true代表拉伸
loadimage(&img3, L".\\image\\金鲨_0.png", 100, 100, true);
putimage(10, 250, &img3);
```

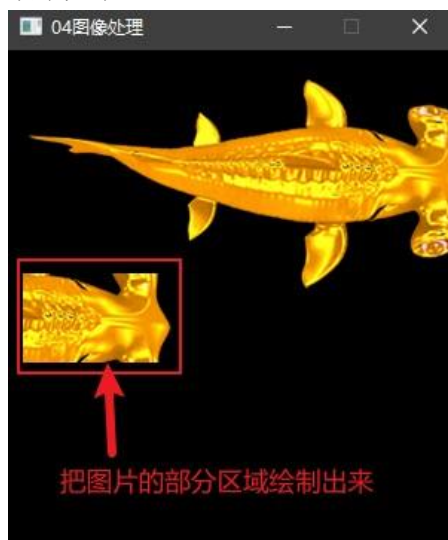


2. 绘制图像

上面例子中的 `putimage()` 函数就是绘制图像的功能。`putimage` 除了绘制完整图像外，也可以把图像的某部分绘制出来。例如：

```
IMAGE img1;
loadimage(&img1, L".\\image\\金鲨_0.png");//从硬盘任意位置加载图像，并默认图像大小
putimage(10, 20, &img1);
//显示图片的某块区域
//区域宽 区域高 区域内的x, y
putimage(10, 150, 100, 60, &img1, 200, 30);
```

效果如下：



3. 旋转绘制

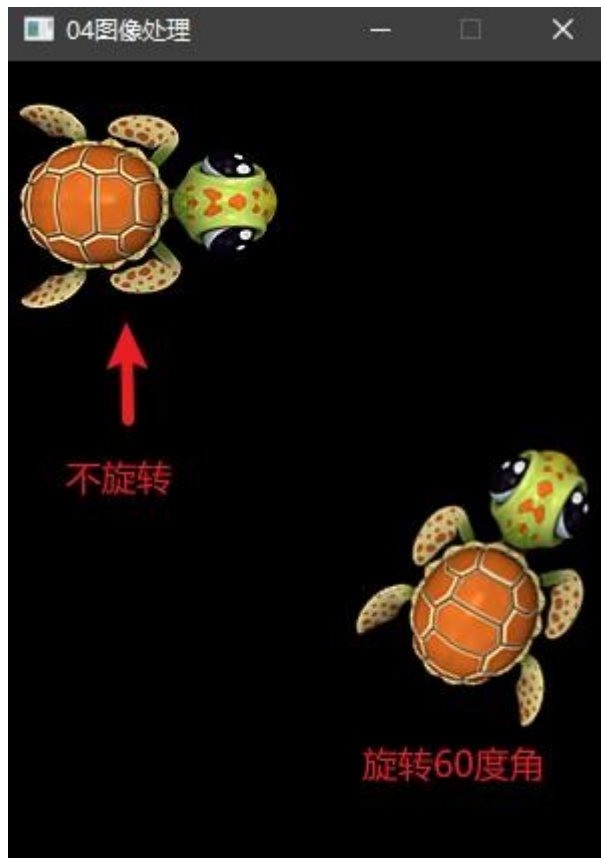
所谓图像“旋转”就是根据旋转角度把内存图片旋转赋值到另一个内存图片变量中的过程。

```
//按图片默认大小加载
IMAGE img1;
loadimage(&img1, L".\\image\\乌龟_15.png");//从硬盘任意位置加载图像，并默认图像大小
putimage( 0, 0, &img1);
IMAGE imgTemp;
/*
角度转弧度:  $\text{PI}/180 \times \text{角度}$ 
弧度转角度:  $180/\text{PI} \times \text{弧度}$ 
```

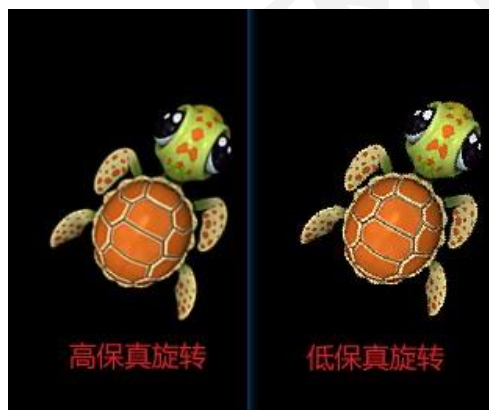
```

*/
double PI = 3.1415926;
//      旋转后      旋转前  旋转角度    空白区填色  自动大小  保真吗(高true低
false)
rotateimage(&imgTemp, &img1, PI / 180 * 60, NULL,      true,      true);
putimage(150, 160, &imgTemp);

```



最后一个参数就是旋转的图片质量清晰程度。true 为高保真，false 为低保真。



但是高保真效率低些。【注意：】在有些时候高保真旋转可能不会被显示。（也许是性能考虑）

4. 窗口截图

可以在绘图窗口内截取一个矩形区域形成新的图像。

```

//按图片默认大小加载
IMAGE img1;
loadimage(&img1, L".\\image\\乌龟_15.png");//从硬盘任意位置加载图像，并默认图像

```

大小

```
putimage(0, 0, &img1);

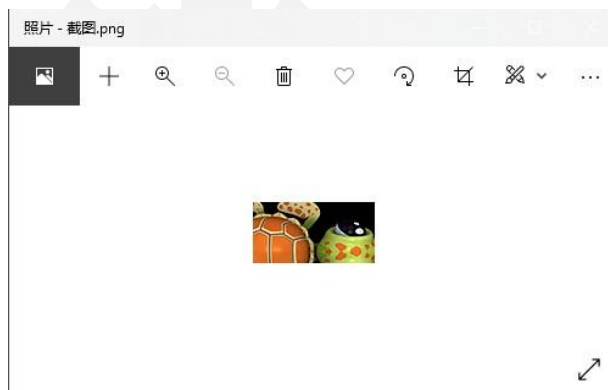
IMAGE imgTemp;
//窗口截图: 存放地 x y 宽 高
getimage(&imgTemp, 30, 30, 100, 50);
//显示查看
putimage(200, 300, &imgTemp);
```



5. 截图保存

可以把区域截图结果保存到硬盘的某个位置。

```
IMAGE img1;
loadimage(&img1, L".\\image\\乌龟_15.png"); //从硬盘任意位置加载图像, 并默认图像大小
putimage(0, 0, &img1);
IMAGE imgTemp;
//窗口截图: 存放地 x y 宽 高
getimage(&imgTemp, 30, 30, 100, 50);
//保存截图
saveimage(L"d:\\截图.png", &imgTemp);
```



六、 动画框架

1. 绘图动画原理

所谓动画绘图就是：绘制画板->等待停留->擦除画板，循环上述 3 个步骤的过程。

```
#include <graphics.h> //包含：图像库头文件
int main ()
{
    initgraph(500, 400); //创建300*400的窗口

    for (int i=10; i<400; i+=10)
    {
        circle(i, 100, 50); //画图
        Sleep(100); //等待时间间隔 100毫秒 1秒=1000毫秒
        cleardevice(); //清屏刷新
    }

    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}
```

上面代码的运行结果就是一个圆形从左边移动到右边。



2. 图像帧动画原理



上图所示：一套动画帧的图片。

我们把所有图片利用命名规则，循环加载到内存的 IMAGE 数组，使用循环绘制擦除数组的每个图片再结合 Sleep 等待时间，就可以实现动画效果。

```
#include <graphics.h>
int main()
{
    initgraph(500, 400);
    IMAGE imgs[12]; //图片数组 保留12张动画帧的图像。
    /*加载动画的每一帧到数组*/
    TCHAR path[100]; //双字节字符数组临时保留图片的硬盘存储路径
    for (int i = 0; i < 12; i++)
    {
```

```

//_sprintf 实现路径的格式化模板，并把变化的位置设置变量值
_sprintf(path, L".\\image\\蝴蝶鱼_%d.png", i); //动态拼装图片路径
loadimage(&imgs[i], path); //根据拼装路径加载图片到图片数组。
}

int imgnum = 0; //记录当前动画的第几帧
/*循环绘制动画的每一帧图像*/
while (1)
{
    putimage(20, 30, &imgs[imgnum]); //绘制动画当前帧图像
    Sleep(100); //等待100毫秒
    //切换动画帧
    if (imgnum == 11) //如果最后一张，就切回第0张
    {
        imgnum = 0;
    }
    else
    {
        imgnum++;
    }
    cleardevice(); //清屏
}
system("pause"); //暂停：避免窗口自动关闭
return 0;
}

```



3. 动画框架原理

所谓动画框架就是把界面显示部分与动画人物动作逻辑部分用函数进行分离。由于界面显示函数 `paintAnime()` 需要的数据在动画逻辑函数 `timer()` 中被改变，所以需要把两个函数都需要访问的变量设置成“全局变量”

```

#include <graphics.h>

/*初始化动画*/
static void initAnime();
/*绘制动画界面*/
static void paintAnime();
/*定时处理*/
static void timer();
/*定时器时间间隔*/
static int interval = 100;
/*设置动画屏幕宽高*/
static int screenWidth = 500;
static int screenHeight = 400;
//TODO: 全局变量

```

```

int main ()
{
    initgraph(screenWidth, screenHeight); //创建300*400的窗口
    initAnime(); //初始化加载图片
    while (1)
    {
        BeginBatchDraw(); //开启批量绘图
        cleardevice(); //清空画板
        paintAnime(); //动画绘图
        EndBatchDraw(); //结束批量绘图

        timer(); //动画逻辑

        Sleep(interval); //动画帧时间间隔
    }

    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}

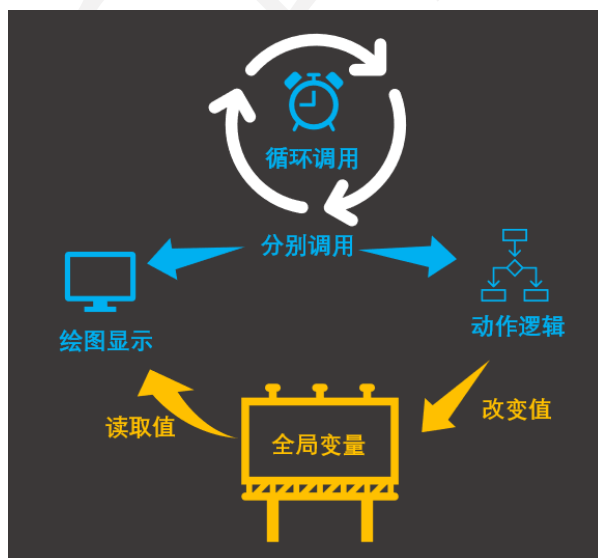
//TODO: 初始化动画
static void initAnime()
{
}

//TODO: 绘制动画界面
static void paintAnime()
{
}

//TODO: 定时处理
static void timer()
{
}

```

动画框架的逻辑概念图如下：



4. 动画简单例子

就是把《图像帧动画》的例子中的代码解耦到用动画框架的过程。所谓的解耦就是把图片一次性加载到内存，然后让图片的变化逻辑与显示功能分别写到不同的函数内，让游戏框架调用对应的函数。

```
#include <graphics.h>
/*初始化动画*/
static void initAnime();
/*绘制动画界面*/
static void paintAnime();
/*定时处理*/
static void timer();
/*定时器时间间隔*/
static int interval = 50;
/*设置动画屏幕宽高*/
static int screenWidth = 500;
static int screenHeight = 400;

//TODO: 全局变量
static IMAGE imgs[12];
static int imgnum = 0;
int main()
{
    initgraph(screenWidth, screenHeight); //创建300*400的窗口
    initAnime(); //初始化加载图片
    while (1)
    {
        BeginBatchDraw(); //开启批量绘图
        cleardevice(); //清空画板
        paintAnime(); //动画绘图
        EndBatchDraw(); //结束批量绘图

        timer(); //动画逻辑

        Sleep(interval); //动画帧时间间隔
    }

    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}

//TODO: 初始化动画
static void initAnime()
{
    TCHAR path[100];
    for (int i = 0; i < 12; i++)
    {
        _stprintf(path, L".\\image\\蝴蝶鱼_%d.png", i); //动态拼装图片路径
        loadimage(&imgs[i], path); //根据拼装路径加载图片到图片数组。
    }
}

//TODO: 绘制动画界面
static void paintAnime()
{

```

```

    putimage(20, 30, &imgs[imgnum]);
}
//TODO: 定时处理
static void timer()
{
    if (imgnum == 11)
    {
        imgnum = 0;
    }
    else
    {
        imgnum++;
    }
}
}

```

七、 像素处理

窗口中的绘图画板上的图像本质是二维表格状像素点排列的结果。每个像素点就是一个颜色值。easyX 提供对每个像素点的单独处理函数：

getpixel(x, y); 得到某坐标像素点的颜色值。

putpixel(x, y, color); 向某坐标像素点设置颜色值

```

#include <graphics.h> //包含：图像库头文件
#include <stdio.h>
int main ()
{
    initgraph(300, 400, EW_SHOWCONSOLE); //创建300*400的窗口
    //获得某坐标像素点的值
    printf("%x\n", getpixel(100, 100));
    //设置一个像素点的颜色
    putpixel(100, 100, 0xffffffff); //白色
    putpixel(100, 105, 0x00ff00); //绿色
    putpixel(105, 100, 0x0000ff); //红色
    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}

```

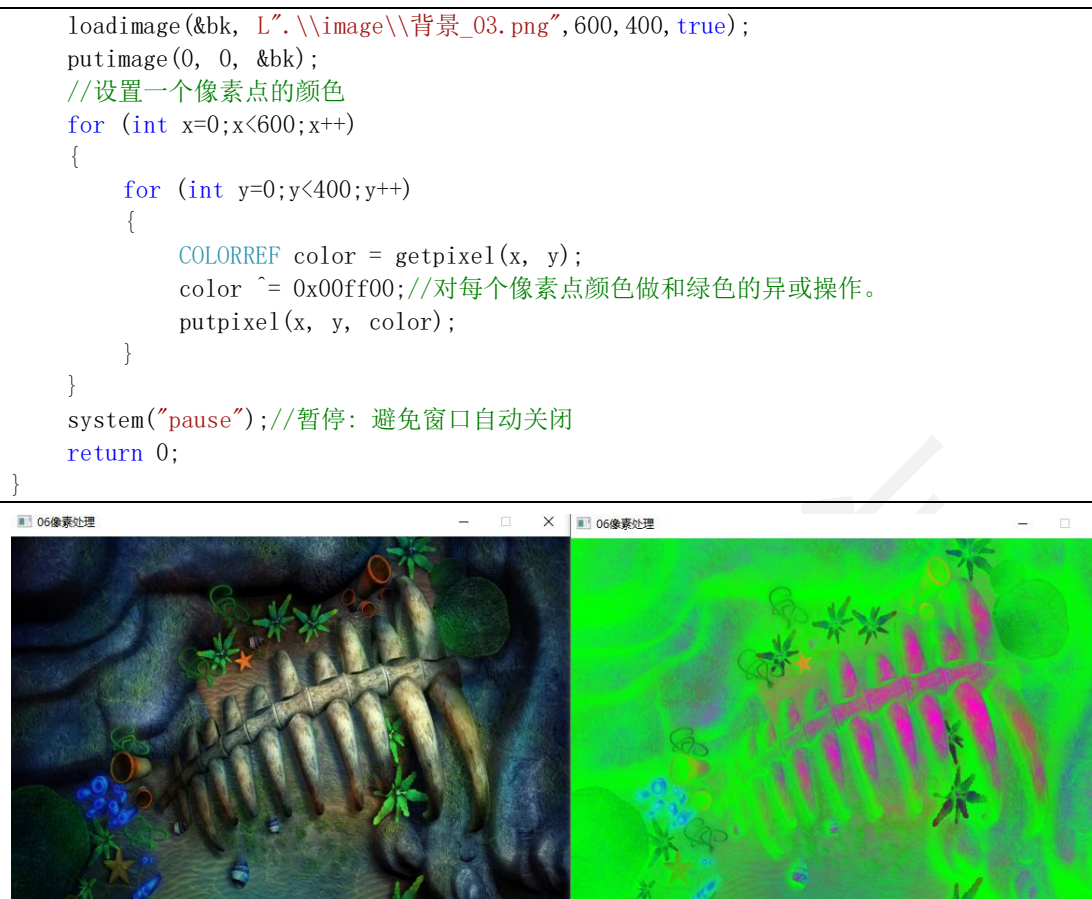


面的例子就是对整个窗口的所有像素点进行颜色改变。采用的是异或运算符。

```

#include <graphics.h>
int main()
{
    initgraph(600, 400); //创建300*400的窗口
    IMAGE bk;

```



改变像素值的前后比较。

八、 文字输出

所谓文字输出就是把文字影像绘制到窗口的方式。

1. 坐标处输出

把文字显示到某坐标 xy 的位置上。在绘制之前需要对文字外观进行设置，然后再根据设置绘制出不同的 文字效果。

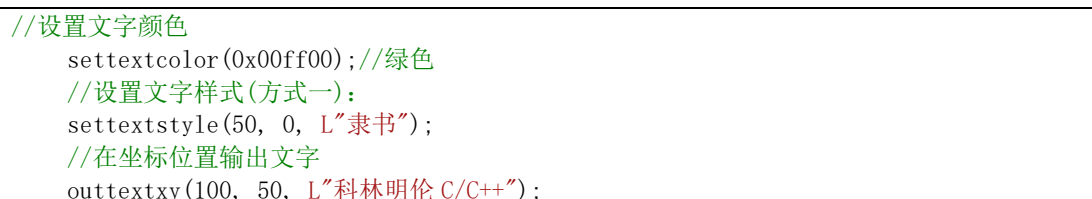
颜色设置函数：settextcolor(0x00ff00);

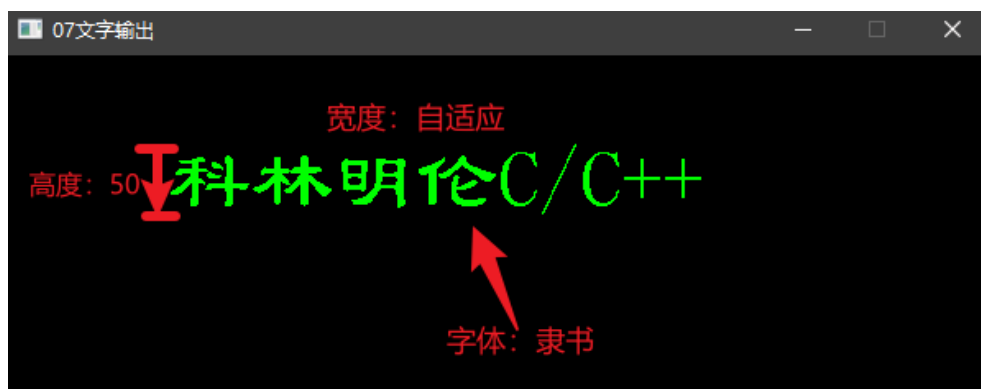
字体设置函数：

方式 1：settextstyle(文字高度, 0, L"字体名");

其中第二个参数 0 代表自适应宽度

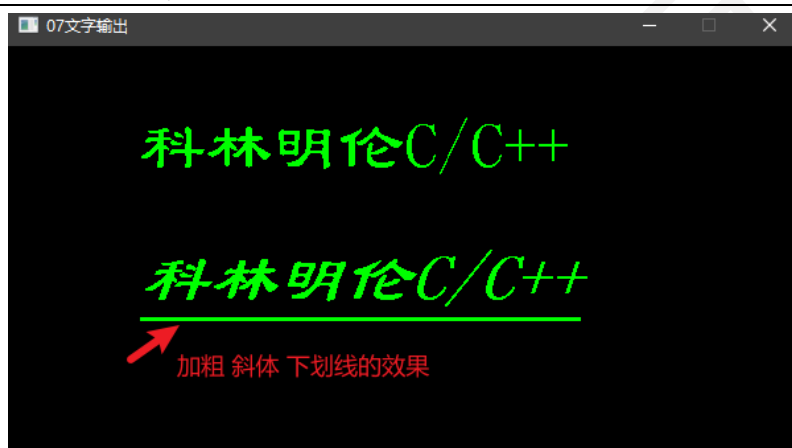
方式 2：settextstyle(高度, 0, L"隶书", 0, 0, 加粗值, 是否斜体, 是否下划线, 是否删除线); 其中加粗值是整型比如 200, 是否斜体, 是否下划线, 是否删除线三个参数值用 true 代表是 false 代表否





```
//设置文字样式 字体 大小 粗体 斜体 下划线 删除线
settextstyle(50, 0, L"隶书");
outtextxy(100, 50, L"科林明伦C/C++");

//设置字体: 加粗 斜体 下划线
settextstyle(50, 0, L"隶书", 0, 0, 800, true, true, false);
outtextxy(100, 150, L"科林明伦 C/C++");
```



2. 区域内输出

所谓“区域内”指的是先确定一个矩形的绘制区，然后在区域范围内绘制文字。由于是区域内，因此可以相对区域空间设置文字在区域内的对齐：例如水平方向：左对齐 居中 右对齐 垂直方向的顶端对齐 底端对齐等。

区域设置采用 RECT 数据类型，RECT 就是一个结构体类型，其定义如下：

```
typedef struct tagRECT
{
    LONG    left;
    LONG    top;
    LONG    right;
    LONG    bottom;
} RECT; *PRECT, NEAR *NPRECT, FAR *LPRECT;
```

drawtext() 函数实现在区域内绘制文字。

其使用参数：

drawtext(L"文字内容" , &区域类型变量, 对齐设置);

关于对齐设置：

1、垂直方向或水平方向单独设置

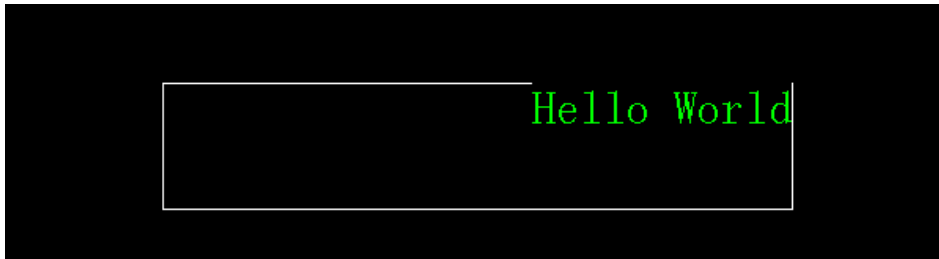
```
//设置文字颜色
```

```

settextcolor(0x00ff00);
//设置文字样式 字体 大小 粗体 斜体 下划线 删除线
settextstyle(30, 0, L"隶书");

//为观察方便，为区域设置默认白色的矩形
rectangle(100, 50, 100 + 400, 50 + 80);
//设置绘制区域坐标值
RECT r = { 100, 50, 100 + 400, 50 + 80 };
//在区域内设置文字
drawtext(L"Hello World", &r, DT_RIGHT); //右对齐

```



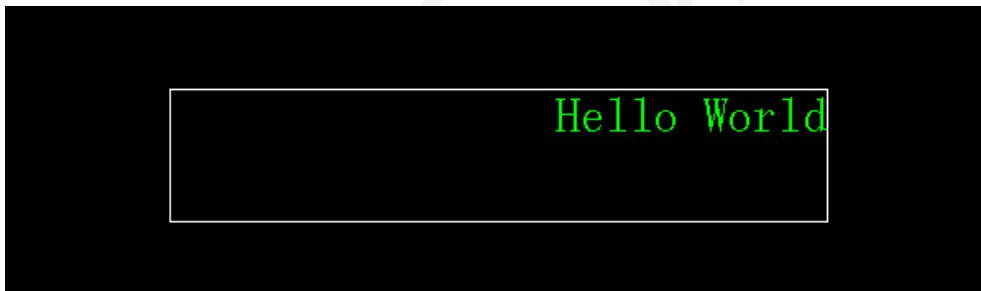
如图：我们发现文字的背景并非透明。需要在绘制前设置文字背景模式为透明。
具体代码：

```

initgraph(600, 400);
setbkmode(TRANSPARENT); //TRANSPARENT透明
//设置文字颜色
settextcolor(0x00ff00);

```

效果如下：



2、垂直方向或水平方向 混合 设置

例如：DT_CENTER | DT_VCENTER | DT_SINGLELINE 三个显示方式用 | 位或运算符连接

```

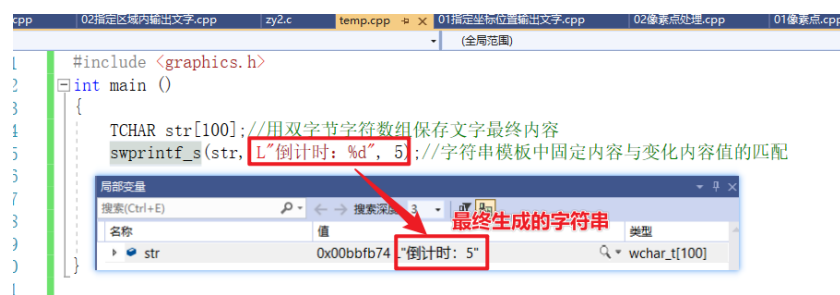
drawtext(L"Hello World ", &r
, DT_CENTER | DT_VCENTER | DT_SINGLELINE); //水平居中垂直居中 单行显示

```



3. 动态文字内容

所谓动态文字内容就是使用 `_swprintf_s()` 函数设置一个内容模板，再根据需把模板中的变化内容（如：`%d %s...`）匹配对应数据产生最终文字内容存入双字节的字符数组中。



根据上面代码拼装出字符串内容的方式完成一个倒计时的程序。

代码如下：

```

#include <graphics.h>
int main ()
{
    initgraph(600, 400);
    //设置文字颜色
    settextcolor(0x00ff00);
    //设置文字样式 字体 大小 粗体 斜体 下划线 删除线
    settextstyle(50, 0, L"隶书");
    TCHAR str[100];
    for (int s = 10; s >= 0; s--)
    {
        swprintf_s(str, L"倒计时: %d", s);
        outtextxy(100, 50, str);
        Sleep(1000);
        cleardevice();
    }
    system("pause"); //暂停：避免窗口自动关闭
    return 0;
}

```

九、 鼠标处理

鼠标处理过程就是识别出鼠标的具体动作（例如：移动 左键按下 左键抬起 双击等...）以及鼠标事件发生后获得鼠标的相关信息（例如鼠标所在的坐标）。然后再为不同的鼠标动作编写不同的处理代码。

上面所述的需求，easyX 通过“消息处理相关函数”来实现。

ExMessage 结构体类型：

当事件发生时，easyX 会把事件的相关信息存入一个 ExMessage 结构体变量中。

```
// 鼠标消息的数据
struct
{
    bool ctrl      :1;    // Ctrl 键是否按下
    bool shift     :1;    // Shift 键是否按下
    bool lbutton   :1;    // 鼠标左键是否按下
    bool mbutton   :1;    // 鼠标中键是否按下
    bool rbutton   :1;    // 鼠标右键
    short x;        // 鼠标的 x 坐标
    short y;        // 鼠标的 y 坐标
    short wheel;    // 鼠标滚轮滚动值, 为 120 的倍数
};
```

peekmessage() 消息函数

peekmessage() 函数会检测出当前是否又鼠标键盘等事件发生。并返回是否获得了信息。可以使用一个 while(true) 的死循环 不间断的调用 peekmessage() 函数, 用以监听鼠标或键盘事件是否发生, 如果发生就获得鼠标键盘的相关信息并编写相关处理代码。

peekmessage() 函数的相关参数如下:

peekmessage(&ExMessage 结构体变量, 事件类型, 是否从事件序列中移除);

事件类型分类如下:

标志	描述
EM_MOUSE	鼠标消息。
EM_KEY	按键消息。
EM_CHAR	字符消息。
EM_WINDOW	窗口消息。

典型鼠标处理模式代码:

```
#include <graphics.h>
#include <stdio.h>
int main ()
{
    //显示控制台 并且 支持双击事件
    initgraph(300, 400, EW_SHOWCONSOLE| EW_DBLCLKS);
    ExMessage mouse;//创建消息变量

    while (1)    // 游戏主循环 可借助break 结束循环
    {
        //把鼠标的消息 存入变量
        if (peekmessage(&mouse, EM_MOUSE, true))
        {
            switch (mouse.message)
            {
                case WM_MOUSEMOVE://鼠标移动
                    printf("鼠标移动 x:%d y:%d\n", mouse.x, mouse.y);
                    break;
                case WM_LBUTTONDOWN://左键按下
                    printf("左键按下 x:%d y:%d\n", mouse.x, mouse.y );
                    break;
                case WM_LBUTTONUP://左键抬起
                    printf("左键抬起 x:%d y:%d\n", mouse.x, mouse.y);
                    break;
                case WM_LBUTTONDBLCLK://左键双击
                    printf("左键双击 x:%d y:%d\n", mouse.x, mouse.y);
            }
        }
    }
}
```

```

        break;
    case WM_RBUTTONDOWN: //右键按下
        printf("右键按下 x:%d y:%d\n", mouse.x, mouse.y);
        break;
    case WM_RBUTTONUP: //右键抬起
        printf("右键抬起 x:%d y:%d\n", mouse.x, mouse.y);
        break;
    case WM_RBUTTONDOWNLCLK: //右键双击
        printf("右键双击 x:%d y:%d\n", mouse.x, mouse.y);
        break;
    }
}
return 0;
}

```

鼠标动作的预定义如下:

```

#define WM_MOUSEFIRST          0x0200
#define WM_MOUSEMOVE          0x0200
#define WM_LBUTTONDOWN        0x0201
#define WM_LBUTTONUP          0x0202
#define WM_LBUTTONDOWNLCLK    0x0203
#define WM_RBUTTONDOWN        0x0204
#define WM_RBUTTONUP          0x0205
#define WM_RBUTTONDOWNLCLK    0x0206
#define WM_MBUTTONDOWN        0x0207
#define WM_MBUTTONUP          0x0208
#define WM_MBUTTONDOWNLCLK    0x0209

```

代码执行效果: 不同的鼠标动作发生时 会被识别并打印出来。

```

C:\WINDOWS\system32\cmd.exe
左键按下 x:70 y:177
左键抬起 x:70 y:177
左键双击 x:70 y:177
左键抬起 x:70 y:177
鼠标移动 x:71 y:177
鼠标移动 x:75 y:177
鼠标移动 x:76 y:177
鼠标移动 x:80 y:178
鼠标移动 x:81 y:178
右键按下 x:81 y:178
右键抬起 x:81 y:178
右键双击 x:81 y:178
右键抬起 x:81 y:178
右键按下 x:81 y:178
右键抬起 x:81 y:178
鼠标移动 x:82 y:178
左键按下 x:82 y:178
左键抬起 x:82 y:178
左键双击 x:82 y:178

```

十、 键盘处理

和鼠标处理目的一样, 键盘处理就是监听键盘的某个按键是否被用户按下, 当按键被按下时执行对应的 代码。关于获得用户的按键又有两种方式。

方式一: 利用<conio.h>控制台输入输出头文件获得键盘按键信息。

```

#include <graphics.h> //包含: 图像库头文件
#include <stdio.h>
#include <conio.h>
int main ()
{

```

```

initgraph(300, 400, EW_SHOWCONSOLE); //创建300*400的窗口
while (1) //死循环不间断的监听键盘
{
    if (_kbhit()) //如果键盘被敲击按下了
    {
        char key = _getch(); //获得按键的ASCII码值
        switch (key)
        {
            case 'A':
            case 'a':
                printf("左\n");
                break;
            case 'D':
            case 'd':
                printf("右\n");
                break;
            case 'W':
            case 'w':
                printf("上\n");
                break;
            case 'S':
            case 's':
                printf("下\n");
                break;
            case 72: //上
                break;
            case 80: //下
                break;
            case 75: //左
                break;
            case 77: //右
                break;
        }
    }
}
system("pause"); //暂停：避免窗口自动关闭
return 0;
}

```

方式二： 用 peekmessage() 消息函数监听

```

#include <graphics.h>
#include <stdio.h>
#include <conio.h>
int main()
{
    initgraph(300, 400, EW_SHOWCONSOLE | EW_DBLCLKS); //创建300*400的窗口
    ExMessage key; //创建消息变量

    while (1)
    {
        if (peekmessage(&key, EM_KEY, true))
        {
            if (key.message == WM_KEYDOWN)

```

```

    {
        switch (key.vkcode)
        {
            case VK_LEFT:
                printf("按 左\n");
                break;
            case VK_RIGHT:
                printf("按 右\n");
                break;
            case VK_UP:
                printf("按 上\n");
                break;
            case VK_DOWN:
                printf("按 下\n");
                break;
        }

        switch (key.vkcode)
        {
            case 'A':
            case 'a':
                printf("按 A\n");
                break;
            case 'B':
            case 'b':
                printf("按 B\n");
                break;
            case 'C':
            case 'c':
                printf("按 C\n");
                break;
            case 'D':
            case 'd':
                printf("按 d\n");
                break;
        }
    }
}
}
}
}

```

方式三： 同时监听鼠标和键盘动作事件 。

```

#include <graphics.h>
#include <stdio.h>
int main()
{
    initgraph(300, 400, EW_SHOWCONSOLE | EW_DBLCLKS); //创建300*400的窗口
    ExMessage msg; //创建消息变量

    while (1) // 游戏主循环 可借助break 结束循环
    {

        if (peekmessage(&msg, -1, true))
        {
            switch (msg.message)
            {
                case WM_KEYDOWN:

```

```

        switch (msg.vkcode)
        {
        case VK_LEFT:
            printf("按 左\n");
            break;
        case VK_RIGHT:
            printf("按 右\n");
            break;
        case VK_UP:
            printf("按 上\n");
            break;
        case VK_DOWN:
            printf("按 下\n");
            break;
        }
        break;
    case WM_LBUTTONDOWN://左键按下
        printf("左键按下 x:%d y:%d\n", msg.x, msg.y);
        break;
    }
}
}
}

```

十一、 音乐播放

无论是动画还是游戏都离不开声音。C 语言通过 Winmm.lib 库函数实现系统声音的播放控制。

Winmm 是 windows multimedia 多媒体 的缩写。使用 Winmm.lib 库函数时需要 加载一下。

```
#pragma comment(lib, "Winmm.lib")
```

再使用 Winmm.lib 库中的 mciSendString() 函数向操作系统传递声音播放的相关指令。

完整代码如下：

```

#include<graphics.h>
#pragma comment(lib, "Winmm.lib")
int main()
{
    //打开音乐
    mciSendString(L"open .\\music\\背景乐_01.mp3 alias bk", 0, 0, 0);
    //播放音乐
    mciSendString(L"play bk ", 0, 0, 0);
    Sleep(20*1000); //播放20秒
    //关闭音乐
    mciSendString(L"close bk", 0, 0, 0);
    return 0;
}

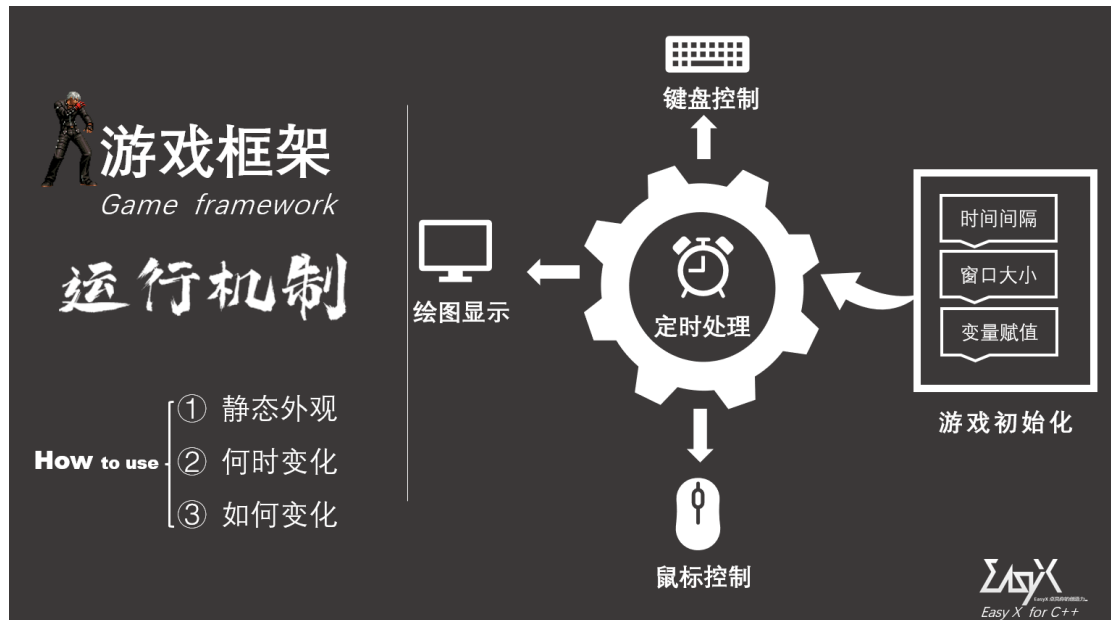
```

其中：open play close 都是播放音频的相关指令 它们通过 mciSendString 函数传递给声音设备。如果你想循环播放某个音频，在声音别名的后面加上 repeat 关键字

```
mciSendString(L"play bk repeat", 0, 0, 0);
```

十二、 游戏框架

根据前面所有的 easyX 知识点的了解，将界面绘制、全局变量、动画逻辑、鼠标键盘交互等相关元素组织起来， 形成一个方便开发小游戏的一套代码，我们只根据需要在不同位置上写入流程结构即可实现游戏编程。我们把 这套“半成品”代码称为“游戏框架”。其基本运行原理如下图：



1. 初始值基本游戏信息

```
/*设置100即每隔100毫秒刷新以下界面绘图。*/
int interval = 100; //TODO: 1 设置定时器时间间隔（单位毫秒）
// TODO: 1 设置窗口：宽度 高度
int screenWidth = 800;
int screenHeight=600;
```

2. 鼠标键盘动作的监听

```
if (peekmessage(&msg, -1, true))
{
    switch (msg.message)
    {
        case WM_MOUSEMOVE: //鼠标移动
            gameMousemove(msg.x, msg.y);
            break;
        case WM_LBUTTONDOWN: //左键按下
            gameMouseDown(msg.x, msg.y);
            break;
        case WM_LBUTTONUP: //左键抬起
            gameMouseUp(msg.x, msg.y);
            break;
        case WM_LBUTTONDBLCLK: //左键双击
            break;
        case WM_RBUTTONDOWN: //右键按下
            break;
    }
}
```

```

        case WM_RBUTTONDOWN://右键抬起
            gameMouseDown(msg.x, msg.y);
            break;
        case WM_RBUTTONDOWNDBLCLK://右键双击

            break;
        case WM_KEYDOWN:
            gameKeyPress(msg.vkcode);
            break;
    }
    paint();
    continue;
}

```

上面代码用于键盘或鼠标的动作监听。一旦发现有动作，就会调用对应的处理函数如：
`gameMouseMove(msg.x, msg.y);`、`gameMouseDown(msg.x, msg.y);`和
`gameKeyPress(msg.vkcode);`
 等。我们也可以根据鼠标的其它交互动作设置对应的处理函数。

```

//TODO: 7 处理鼠标控制位置
void gameMouseDown(int mouseX, int mouseY)
{
    printf("按下鼠标左键\n");
}

void gameMouseUp(int mouseX, int mouseY)
{
}

void gameMouseMove (int mouseX, int mouseY)
{
}

void gameMouseRightDown(int mouseX, int mouseY)
{
}

```

在写项目时，只需要在对应函数内部写入我们的代码逻辑。当鼠标或键盘有交互行为时，我们的代码逻辑就会被执行。

3. 界面绘制的处理结构

执行完我们的交互代码逻辑后，游戏框架会调用`paint();`函数进行界面的重绘刷新。

```

void paint()
{
    BeginBatchDraw();//打开批量绘制
    cleardevice();// 清屏：擦除绘图窗口
    gamePaint();// 调用游戏专属绘图程序
    EndBatchDraw();//关闭批量绘制
}

```

在进行界面绘制时，会用`BeginBatchDraw();`函数先打开批量绘制。正常的绘制是执行一个绘图函数比如`circle`，就通知显卡在屏幕上展示效果，如果绘图函数非常多，会影响性能。所以会统一把所有绘图函数执行完，再通知显卡把最终的绘制的图像在显示器上显示出来。

`cleardevice();`函数用于将绘图窗口的像素值清空(即擦除绘图窗口)。

再执行gamePaint();把本游戏的绘图代码执行完,最后用EndBatchDraw();关闭批量绘制,把我们的最终界面效果呈现出来。

其中gamePaint();在游戏框架中是一个空函数体,根据游戏界面需要由我们开发者再写明绘图显示的逻辑代码。

```
//TODO: 4 绘图处理位置
void gamePaint()
{
}
```

所以上面的游戏交互处理总逻辑是:监听鼠标键盘动作->处理动作下的逻辑->刷新界面让玩家看到。由于玩家随时按下鼠标键盘,希望按下立即有反应,所以在监听的上面写上死循环来不间断的检索监听。

```
/*游戏运行*/
void run()
{
    ExMessage msg ;
    //ExMessage key;
    while (1)    // 游戏主循环 可借助break 结束循环
    {
        if (peekmessage(&msg, -1, true))
        {
            switch (msg.message)
            {
                case WM_MOUSEMOVE://鼠标移动
                .....
            }
        }
    }
}
```

4. 机器人控制处理结构

```
/*游戏运行*/
void run()
{
    ExMessage msg ;
    //ExMessage key;
    while (1)    // 游戏主循环 可借助break 结束循环
    {
        if (peekmessage(&msg, -1, true)) { ... }

        //-----时间处理 勿动-----//
        Sleep(1);
        times++;
        if(times%(interval/10)!=0) { ... }
        times=0;
        if(stop) { ... }
        gameInterval();
        paint();// 刷新显示游戏界面
    }
}
```

在死循环 while(1)中不但有监听鼠标键盘交互动作的代码也有“机器人”不断重复处理的逻辑代码 gameInterval();我们需要计算机不间断重复处理的逻辑就写到其里面。当我们的重复处理逻辑执行完就调用 paint();函数让界面重新绘制从而实现动画效果。

```
//TODO: 5 定时处理位置
void gameInterval()
{
}
```

```
}

```

5. 退出游戏框架

由于游戏框架采用 while(1) 死循环控制游戏的主体运行，所有要结束程序的执行只需退出死循环即可。因此游戏框架设置了一个 stop 全局变量，用于标识是否结束循环。

```
int stop = 0; //是否停止游戏
```

```
void run()
{
    ExMessage msg ;
    //ExMessage key;
    while (1) // 游戏主循环 可借助break 结束循环
    {
        if (peekmessage(&msg, -1, true)) { ... }

        //-----时间处理 勿动-----//
        Sleep(1);
        times++;
        if(times%(interval/10)!=0) { ... }
        times=0;
        if(stop)
        {
            break;
        }
        gameInterval();
        paint(); // 刷新显示游戏界面
    }
}
```

当我们在游戏逻辑编写时，如果需要结束游戏只需要把 stop 这个全局变量的值设置为 1。那么 while 循环到 if (stop) 时就会执行 break 语句结束游戏的循环控制。

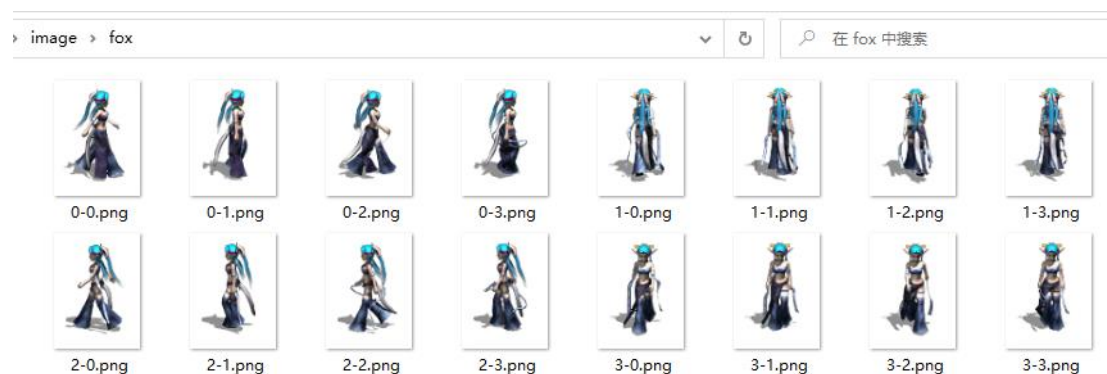
6. 游戏框架的最佳用法

根据对游戏框架各个部位的了解，我们可以总结出它们之间的执行逻辑。其界面的显示与逻辑的控制是分离的。用 while 循环不间断的处理逻辑控制，把逻辑处理的结果用全局变量来保存，逻辑处理完后再用界面绘图函数重新绘制界面产生动画的视觉感。因此使用这个基于 easyx 的游戏框架需要将一个游戏的界面绘图、交互逻辑、机器人逻辑及它们之间的共享信息分离出来，就可以完成一款小游戏的开发。具体的步骤可以分为三步走：

- 一、**静态外观**：即把界面用绘图或贴图及文字展示出来。在 gamePaint() 函数中实现
- 二、**什么变化**：在界面代码中找数据的变化，可能是坐标的变化，或图片的变化。总之把变化的部位或参数用变量替代。并把这些变量提取后放到全局变量的位置，用于函数之间的数据共享。
- 三、**何时变化**：从界面中提取的变量，什么时候变量的值发生变化？可能是鼠标键盘的交互如 void gameKeyPress(int key) 或 void gameMouseDown(int mouseX, int mouseY) 函数，或者机器人自动控制的 void gameInterval() 函数。

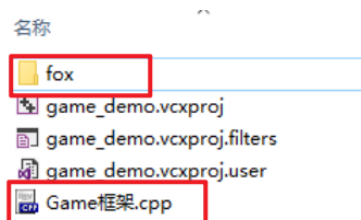
下面用一个例子说明：

首先看图片素材

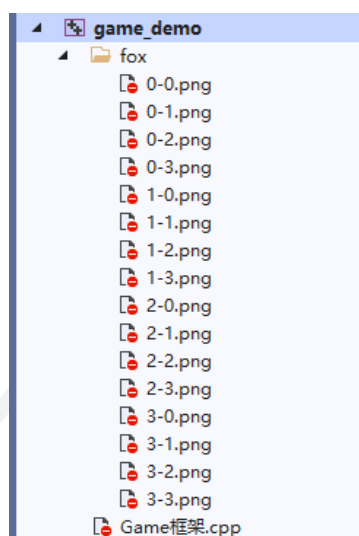


准备利用人物的四向行走图片实现键盘控制人物运动的代码。

- 1、新建一个《空项目》，并命名为 `game_demo`，并把游戏框架代码 `game 框架.cpp` 及游戏素材图片 `image` 文件夹复制粘贴到项目里。



也可以通过“显示所有文件”按钮切换到文件浏览界面查看。

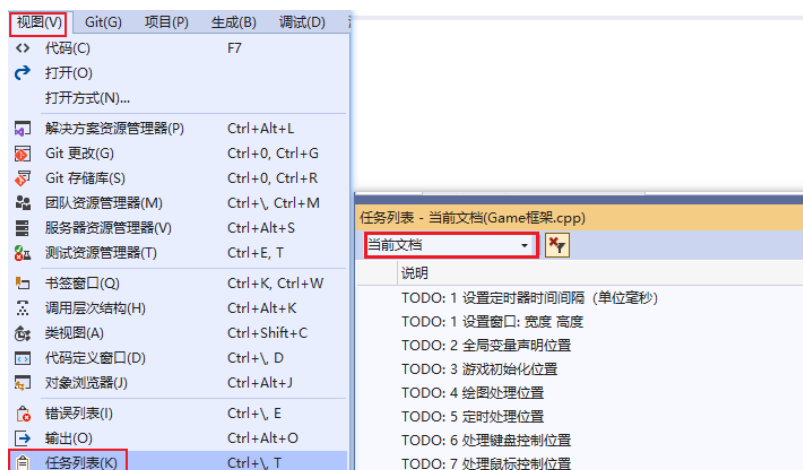


再用“显示所有文件”按钮切换回项目管理界面，把游戏框架代码 `game 框架.cpp` 加入到源代码中。



由于游戏框架中的辅助代码比较多，为了快速定位写游戏逻辑代码的部位可以用“任务列表”窗口，快速定位代码的编写位置。具体步骤如下：

首先在《视图》菜单中找到《任务列表》选项并打开对应窗口。



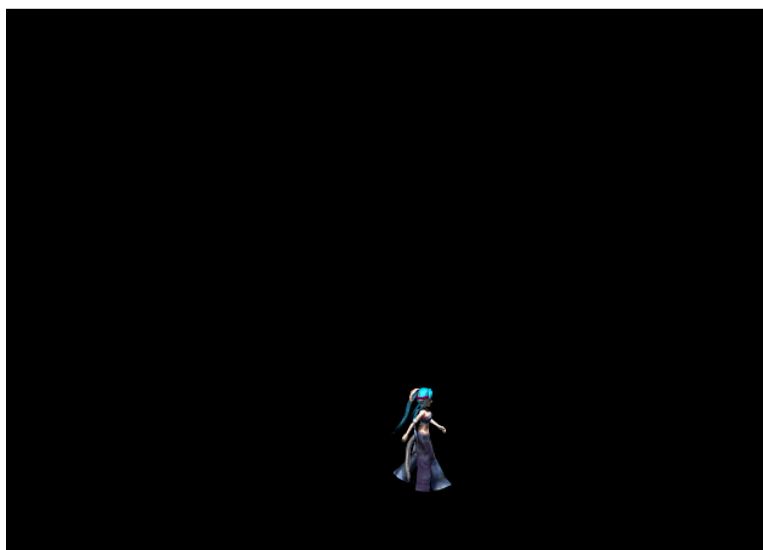
双击 todo 字样的标题，即可定位到代码的编写部位。

2、定位到：todo:4 绘图处理位置，先把一个图片展示到界面上

//TODO: 4 绘图处理位置

```
void gamePaint()
{
    IMAGE img;
    loadimage(&img, L".\\fox\\0-0.png");
    putimage( 300, 300, &img);
}
```

game_demo



接下来优化一下代码。由于 gamePaint() 函数不断的被死循环刷新界面调用。又由于 loadimage 频繁加载硬盘图片到内存会很耗时，所以图片需要加载一次到内存即可。所以上述代码做如下调整。

//TODO: 2 全局变量声明位置

IMAGE img;

//TODO: 3 游戏初始化位置

void gameInit()

```
{
    loadimage(&img, L".\\fox\\0-0.png");
}
```

```
//TODO: 4 绘图处理位置
void gamePaint()
{
    putimage( 300, 300, &img);
}
```

图片变量img放到全局位置上。loadimage()函数加载图片放到gameInit()函数内，仅被执行一次加载。putimage()留在gamePaint()内，每次刷新界面都从内存变量img调取图片进行显示。由于本图片素材人物有4个方向，每个方向有4张行走的图片，因此可以用数组保留这些图片。代码做如下修改：

```
//TODO: 2 全局变量声明位置
IMAGE img[4][4];
```

下面用字符串拼装出图片的路径，并根据路径把图片加载进对应的图片数组中。

```
//TODO: 3 游戏初始化位置
void gameInit()
{
    //loadimage(&img, L".\\fox\\0-0.png");
    wchar_t path[20];
    for ( int fx=0; fx<4; fx++ )
    {
        for (int i=0 ; i<4; i++)
        {
            swprintf_s(path, L".\\fox\\%d-%d.png", fx, i);
            loadimage(&img[fx][i], path);
        }
    }
}
```

图片显示就需要变为数组中的某行某列的图片。

```
//TODO: 4 绘图处理位置
void gamePaint()
{
    putimage( 300, 300, &img[0][0]);
}
```

3、找变化：在绘图函数 putimage(300, 300, &img[0][0]);中观察什么数据是变化的。

首先：&img[0][0]中，第一个0代表方向是变化的，所以把它提取出一个变量 fx，第二个0代表某方向下的图片编号是变化的，也把它提取出一个变量 num。

其次，图片能够被移动所以坐标也是变化的。最终的处理代码如下：

全局变量部分：

```
//TODO: 2 全局变量声明位置
IMAGE img[4][4];
int fx = 0;
int num = 0;
int x = 300;
int y = 300;
```

绘图显示部分

```
//TODO: 4 绘图处理位置
void gamePaint()
{
    putimage( x, y, &img[fx][num]);
}
```

4、确定何时变化

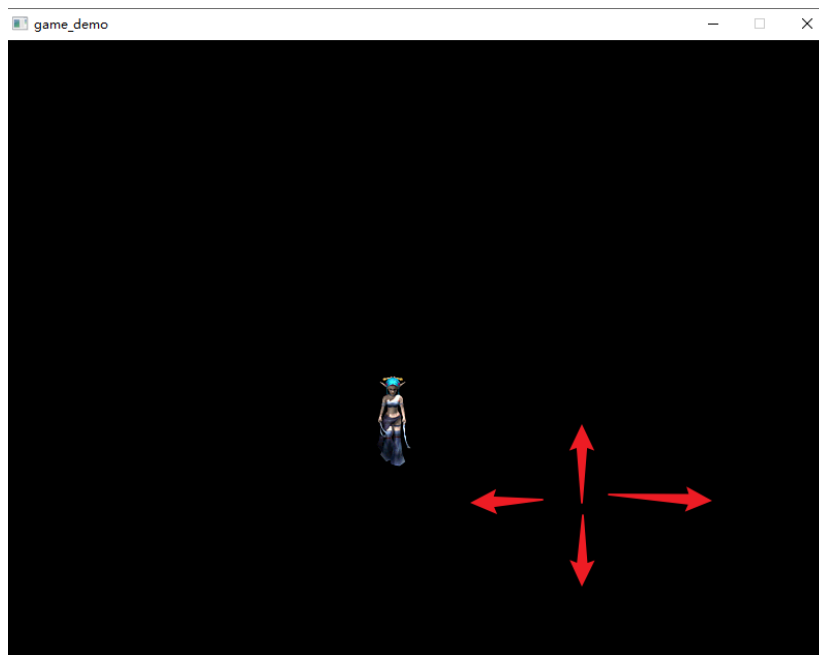
全局变量 fx 由键盘控制。因此键盘部分的代码如下：

```
//TODO: 6 处理键盘控制位置
void gameKeyPress(int key)
{
    switch (key )
    {
        case VK_LEFT:
            fx = 2;
            break;
        case VK_RIGHT:
            fx = 0;
            break;
        case VK_UP:
            fx = 1;
            break;
        case VK_DOWN:
            fx = 3;
            break;
    }
}
```

由于每个方向只有 4 张图，其编号为 0~3 。所以全局变量 num 在“机器人定时”中处理。保证变量的值在 0~3 之间不断的切换。另外全局变量 x, y 会根据方向 fx 的变化，会有不同的变化规律，也需要在定时处理中完成。最终定时处理的代码如下：

```
//TODO: 5 定时处理位置
void gameInterval ()
{
    num=(num+1) % 4; //保证num的值在0~3之间
    switch (fx)
    {
        case 0:
            x += 5;
            break;
        case 1:
            y -= 5;
            break;
        case 2:
            x -= 5;
            break;
        case 3:
            y += 5;
            break;
    }
}
```

最终执行效果如下：



按“上下左右”键可控制人物的方向，并沿着新的方向进行前进行走。

C 语言项目实战

可以说最快的掌握一门编程语言的方法就是项目实战。因为一个项目不仅仅是编程，而是一个系统化的过程。非常综合。这个完整项目的过程包括需求分析设计、技术选型、代码管理、开发规范、项目管理、版本控制还有项目部署实施、迭代升级等很多的环节。所以从学习编程的第一天就要建立项目的意识。要理解技术是为项目服务，只有在项目中才能发挥技术的价值，技术点的运用主次。可以说项目能够用到的技术点，我们必须深入研究其核心原理，反之很多技术点在实际开发中用的很少很浅，甚至用不到，那么在学习中就可以区别对待。接触的项目种类与数量越多，我们对于技术的理解与技术体系的掌握就会越透彻。

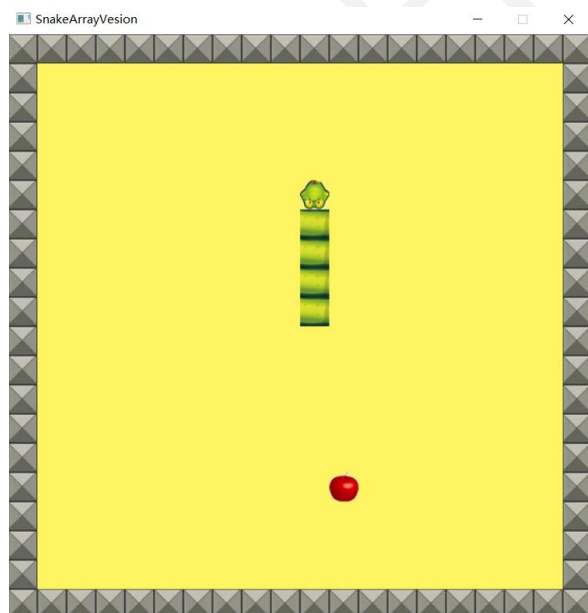
游戏项目开发

经典小游戏类是最适合新手小白的项目。首先这类项目的需求比较好理解，当我们对于项目的目标没有理解的障碍时，就可以把思维重心全部放在技术实现上。其次游戏类项目我们可以做一个局部小功能点，也可以按照自己的意愿设计出不同的玩法，非常适合初学者找到编程的兴趣与自信。

1. 经典贪吃蛇

需求分析设计：

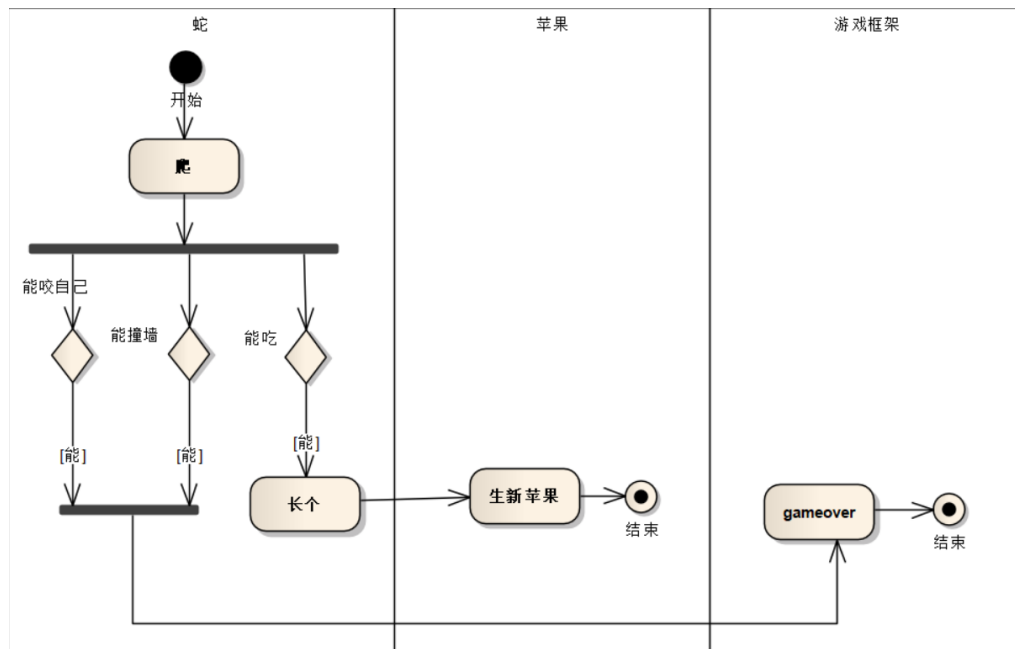
贪吃蛇游戏操作简单，可玩性比较高。玩家使用方向键操控一条长长的蛇不断吞下豆子，同时蛇身随着吞下的豆子不断变长，当蛇头撞到蛇身或障壁时游戏结束。



当我们第一次开发这个游戏项目时首先要规划这个功能边界，不要想像要做一个大而全的软件产品，先开发出其最简原型——最核心本质功能。然后可以在其上面不断优化迭代新的游戏规则与玩法。因此要先进行基本的需求分析与逻辑设计。此刻建议采用用例驱动的手法：站在使用者视角来描述用户的操作实现的功能，然后再细化每个功能的实现过程，最终分模块、分步骤的把功能开发出来。在需求设计过程中推荐采用 UML 中的用例图和活动图来描述。图例参考如下：



上图为“use case 用例图”。即从宏观视角站在不同用户的视角表达式参与者与其功能的边界。主要回答这个软件项目的使用者都有谁，能干什么的问题。



上图为“activity 活动图”，其作用是对“蛇爬行”这个用例进行细化，用以说明爬行的过程。之所以成为活动图是因为一个活动是需要多方参与，按照一个逻辑流程分工协作最终才能完成。所以这个图中用“泳道”表明了此活动的参与者有蛇、被吃的苹果（或称豆）还有支撑它们运行的游戏框架。接下来从游戏玩的过程中找到不同参与者的动作，并用一个动词准确概括，因为这里的一个动词都将成为代码中的一个函数。根据图逻辑描述可以看到从蛇开始先爬行，每爬行一步可能会发生能吃到苹果的情况，能撞墙出界的情况也可能是咬到自己的情况。如果能吃到苹果就会引发连续的后续动作：长个、生成新苹果。我们把对后续行为起到判断作用的称为前置条件。如果出现能撞墙或咬到自己的情况就引发 gameover 这个后续动作。

项目结构说明：

在开发项目的过程中，本着“高内聚、低耦合”的原则把不同的参与者单独建立一个源代码文件来管理和其有关的变量与函数功能。具体操作步骤如下：

1、先创建一个名为：globalvar.h 的头文件。

目的是把本项目中不同参与者的全局变量在这里做统一的外部变量的链接声明。因为 C 语言是面向过程的语言。其项目的代码结构相对于 C++面向对象会比较简单。在基于 easyX 的游戏框架中，其模式是把游戏所以模块都需要的变量以全局变量的方式进行。全局变量在 C 语言中的使用要求是必须只有一次定义，但可以无数次声明调用。所以

把变量定义放到各个源代码中，但是把变量的声明统一放在一处形成“目录清单”，谁需要变量，谁就#include。

- 2、为参与者“苹果”建立头文件 apple.h 用于其功能的函数声明。

```
#pragma once
#include "globalvar.h"
/* 创建 新 苹果*/
void createNewApple();
```

- 3、为参与者“蛇”建立头文件 snake.h 用于其功能的函数声明。

```
#pragma once
#include "globalvar.h"
/*蛇 爬行*/
void snakeMove();
/*
 * 能否吃到苹果
 返回: 0 不能 1 能
 */
int enabledEatApple();
/*长个*/
void grow();
/*
 * 能否咬到自己
 返回: 0 不能 1 能
 */
int enabledEatSelf();
/*
 * 能否出界
 返回: 0 不能 1 能
 */
int enabledOut();
/*关闭游戏*/
void gameover();
/*向上转*/
void turnUp();
/*向下转*/
void turnDown();
/*向左转*/
void turnLeft();
/*向右转*/
void turnRight();
```

- 4、根据用例图在游戏框架中写好不同模块的调用。

这种方式属于测试驱动开发模式，以测试的视角在功能应用的场景里调用功能，虽然功能没有实现不会运行，但是通过函头文件中的函数声明可以把调用逻辑先表达出来，依此测试评估函数的参数与返回值是否复合调用的逻辑。在含有 easyX 游戏框架的源代码文件 game.cpp 插入如下代码：

- 头文件包含增补：

```
#include "globalvar.h"
#include "apple.h"
#include "snake.h"
```

- 定时处理位置增补蛇的爬行逻辑调用：

```
void gameInterval()
```

```

{
    snakeMove();
    if (enabledEatApple() )
    {
        grow();
        createNewApple();
    }
    if ( enabledOut() )
    {
        gameOver();
    }
    if (enabledEatSelf())
    {
        gameOver();
    }
}

```

- 键盘处理位置增补蛇的转向逻辑调用:

```

void gameKeyPress(int key)
{
    switch (key)
    {
        case VK_LEFT:
            turnLeft();
            break;
        case VK_RIGHT:
            turnRight();
            break;
        case VK_UP:
            turnUp();
            break;
        case VK_DOWN:
            turnDown();
            break;
    }
}

```

5、创建 apple.cpp 源代码文件实现苹果的功能

```

#include <stdlib.h>
#include "apple.h"

int appleHang;//苹果所在行
int appleLie;//苹果所在列

void createNewApple()
{
    appleHang = rand() % 14 + 3;
    appleLie = rand() % 14 + 3;
}

```

由于创建新苹果的显示位置要求随机，所以用到了 C 语言的随机函数 rand()，又由于游戏框架中已经封装了对随机种子数的设置，所以没有 srand 函数的调用。为了让游戏不太能“恶心”到玩家，所以苹果的生成位置定位在 3~16 之间。

6、创建 sanke.cpp 源代码文件实现蛇的功能

```

#include "snake.h"
int fangXiang = 1;//蛇当前方向: 0右 1上 2左 3下
int snakeHang[100] = { 10, 11, 12, 13, 14 };

```

```

int snakeLie[100] = { 10,10,10,10,10 };
int snakeLength = 5; //蛇当前长度。
void snakeMove()
{
    for (int j=snakeLength-1 ;j>=1 ;j-- )
    {
        snakeHang[j] = snakeHang[j-1];
        snakeLie[j] = snakeLie[j-1];
    }

    switch ( fangXiang)
    {
    case 0:
        snakeLie[0]++;
        break;
    case 1:
        snakeHang[0]--;
        break;
    case 2:
        snakeLie[0]--;
        break;
    case 3:
        snakeHang[0]++;
        break;
    }
}

int enabledEatApple()
{
    return appleHang == snakeHang[0]
        && appleLie == snakeLie[0];
}

void grow()
{
    snakeLength++;
}

int enabledEatSelf()
{
    int enabled = 0; //
    for (int i = 1; i < snakeLength; i++)
    {
        if (snakeHang[0] == snakeHang[i]
            && snakeLie[0] == snakeLie[i])
        {
            return 1;
        }
    }
    return 0;
}

int enabledOut()
{
    return snakeHang[0] == 0
        || snakeHang[0] == 19

```

```

        || snakeLie[0] == 0
        || snakeLie[0] == 19;
    }

    void gameover()
    {
        stop = 1;
    }

    void turnUp()
    {
        if (fangXiang != 3)
        {
            fangXiang = 1;
        }
    }

    void turnDown()
    {
        if (fangXiang != 1)
        {
            fangXiang = 3;
        }
    }

    void turnLeft()
    {
        if (fangXiang != 0)
        {
            fangXiang = 2;
        }
    }

    void turnRight()
    {
        if (fangXiang != 2)
        {
            fangXiang = 0;
        }
    }
}

```

关于蛇的功能实现中，可以用链表实现也可以用数组实现。这里采用了数组的实现方法，但由于数组长度固定，所以用 100 节作为长度上限。让其仅显示 5 节，随着游戏的深入不断的增加显示的节数。

7、把 apple 与 snake 的全局变量在 globalvar.h 中做统一声明。

```

#pragma once
extern int appleHang;
extern int appleLie;
extern int fangXiang ;
extern int snakeHang[100] ;
extern int snakeLie[100] ;
extern int snakeLength ;
extern int stop;

```

注意：声明变量是不能赋初始值的，否则就变成了变量定义会产生重定义错误。

至此，贪吃蛇的功能逻辑部分就基本完成。

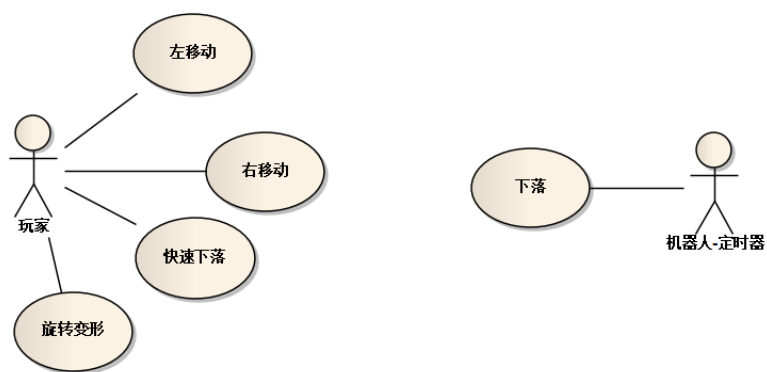
科林 C 语言

2. 俄罗斯方块

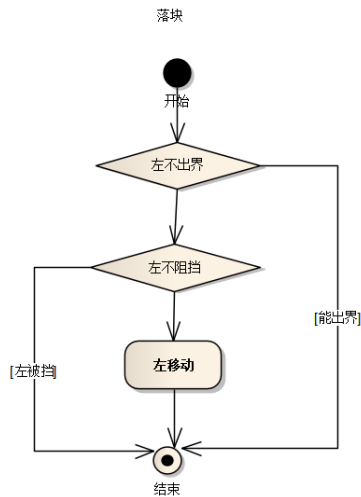
一、需求分析



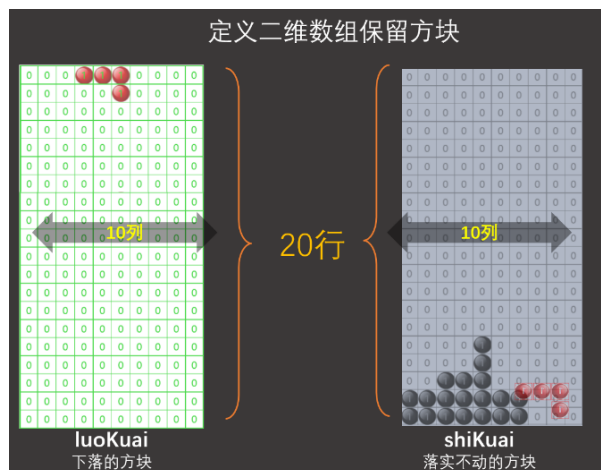
上图是本项目的运行效果。根据游戏的功能本项目的用例图如下：



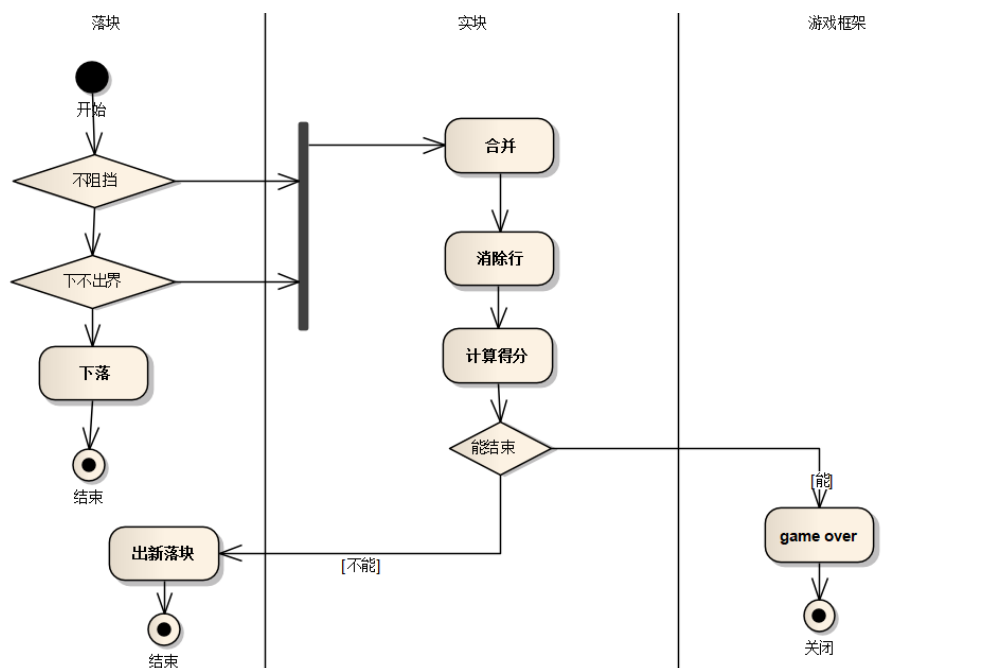
即从宏观来看，玩家控制下落方块的左右移动，当确定下落位置后，可以快速下落到指定位置。也可以旋转变形（本案例暂时不做）。机器人定时处理负责控制方块的默认下落。接下来用活动图细化每个重要的用例处理过程。



以左移逻辑为例，当玩家的落块在左移不出界或不被阻挡的前提下，可以左移方块。右移逻辑也是如此，在此省略。

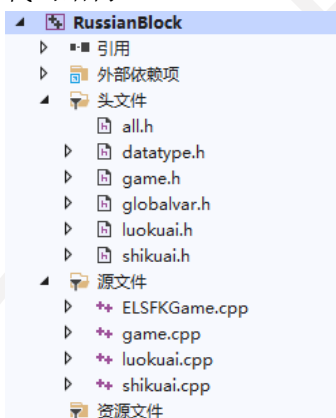


俄罗斯方块中，有两个方块体系，一个是下落中的方块我们称之为 luoKuai，另一个是不断收集不能继续落的方块，我们称之为 shiKuai。接下来把方块下落过程中的处理流程如下。



图中展示了落块、实块以及游戏框架的分工与协作关系。由于下落过程是由游戏框架中的定时处理来调用的。所以这个过程是不间断处理的。首先在向下不阻挡和不出界的情况下，方块可以正常下落。一旦出现阻挡或出界时，不能下落的方块就要和实块进行合并；合并后要对实块进行消除行的处理，消除行处理后会返回实际消除的行数；再根据消除的行数计算当前的得分。如果还有方块的下落空间，则产生新的落块让游戏继续。如果顶端没有方块的下落空间，则结束游戏。

二、代码结构



根据“高内聚、低耦合”的基本原则，结合用例图、活动图中的相关角色（游戏、落块、实块），把和游戏整体有关的功能函数放到 game 子程序中，把和下落方块有关的功能放到 luokuai 子程序中，把和收集落块等后续功能放到 shikuai 子程序中。另外从代码方便维护管理的角度，把游戏中的自定义数据类型统一管理起来放到 datatype.h，由于全局变量的定义会出现在不同的子程序源代码中，但是子程序之间会涉及到全局变量的调用，所以把全局变量的声明统一在一个 globalvar.h 文件中。最后为了方便代码调用，把本项目中所有文件的包含使用再放到一个 all.h 的文件中，其代码如下：

```
#pragma once
#include "datatype.h"
#include "globalvar.h"
#include "luokuai.h"
```

```
#include "shikuai.h"
#include "game.h"
```

这里要注意理解：头文件不仅仅是语法的作用，在项目中头文件相当于一本书的目录。通过它能够大致掌握程序的结构与功能总成。

三、模块设定

数据类型头文件 datatype.h 的代码

```
#pragma once
/*布尔型*/
#define TRUE 1 //逻辑真
#define FALSE 0 //逻辑假
typedef int _BOOL_;
/*下落方块的样式类型：Z, T, 田 L 一 等造型*/
typedef enum type { Z, T, TIAN, L, ONE } TYPE;
```

全局变量的统一声明头文件 globalvar.h

```
#pragma once
#include <graphics.h>
extern int luoKuai[20][10]; //下落的方块
extern int shiKuai[20][10]; //落实不动的方块
extern int color; //方块颜色
extern int score; //当前成绩
extern int stop; //是否退出游戏
extern IMAGE imgfk[11]; //方块的图片素材集合
```

游戏整体功能子程序的功能规划（即相关函数声明集合）game.h

```
#pragma once
/*关闭游戏*/
void gameover();
/*初始背景*/
void initBg();
/*加载方块图*/
void loadFKImage();
/*显示背景*/
void showBg();
/*显示得分*/
void showScore();
/*显示等级*/
void showLevel();
```

下落方块子程序的功能规划 luokuai.h

```
#pragma once
#include "datatype.h"
/*左不出界 */
_BOOL_ leftNotOut();
/*左不阻挡 */
_BOOL_ leftNotStop();
/*左移动*/
void leftMove();
/*右不出界 */
_BOOL_ rightNotOut();
/*右不阻挡 */
_BOOL_ rightNotStop();
/*右移动*/
void rightMove();
/*下不出界 */
```

```

_BOOL_ downNotOut();
/*下不阻挡 */
_BOOL_ downNotStop();
/*下移动*/
void downMove();
/*出新的落块*/
void createNewLuoKuai();
/*显示落块*/
void showLuoKuai();

```

收集落块的子程序功能规划 shikuai.h

```

#pragma once
#include "datatype.h"
/*合并*/
void merge();
/* 清除行
返回：被消除的行数
*/
int cleanLines();
/*
根据消除的行数计算得分
参数：lines 行数
*/
void setScore(int lines);
/*能否结束*/
_BOOL_ enabledOver();
/*显示实块*/
void showShiKuai();

```

四、开发流程

上面把函数功能规划完之后，具体定义开发那个函数呢？建议用测试驱动的方式先把函数的调用位置及它们之间的调用逻辑表达清除，再根据游戏玩的进程逐步实现对应的函数定义。根据用例图的左移动、右移动、快速下落的交互功能，确定在游戏框架中的键盘处理部位实现功能的逻辑调用，再根据对应的活动图中的逻辑其具体代码如下：

```

//TODO: 6 处理键盘控制位置
void gameKeyPress(int key)
{
    switch (key)
    {
        case VK_LEFT:
            if ( leftNotOut() && leftNotStop() )
            {
                leftMove();
            }
            break;
        case VK_RIGHT:
            if (rightNotOut() && rightNotStop())
            {
                rightMove();
            }
            break;
        case VK_UP:
            break;
        case VK_DOWN:

```

```

        while ( downNotOut() && downNotStop() )
        {
            downMove();
        }
        break;
    }
}

```

机器人定时进行处理的“下落”逻辑代码如下：

```

//TODO: 5 定时处理位置
void gameInterval()
{
    if ( downNotOut() && downNotStop() )
    {
        downMove();
    }
    else
    {
        merge();
        setScore(cleanLines());
        if (enabledOver())
        {
            gameover();
        }
        else
        {
            createNewLuoKuai();
        }
    }
}

```

另外游戏背景、落块、实块的初始化的调用如下：

```

//TODO: 3 游戏初始化位置
void gameInit()
{
    initBg();
    loadFKImage();
    createNewLuoKuai();
}

```

游戏角色显示的调用如下：

```

//TODO: 4 绘图处理位置
void gamePaint()
{
    showBg();
    showLuoKuai();
    showShiKuai();
    showScore();
    showLevel();
}

```

当调用逻辑确定好后就按游戏的推进分块实现对应的功能

- 一、 为 game.h、luokuai.h、shikuai.h 等三个头文件创建同名但扩展名是.cpp 的源代码文件，用于定义实现对应的函数声明。



二、把全局变量定义出来。把初始化与界面功能的函数开发出来，并运行后能看到对应角色。

具体代码如下：

game.cpp 中的源代码：

```
#include "all.h"
#include <graphics.h>
int score = 0; //总得分
IMAGE imgbg;
IMAGE imgfk[11];
TCHAR imgPath[100];

void initBg()
{
    loadimage(&imgbg, L".\\images\\2\\bg-1.png");
}

void loadFKImage()
{
    for (int c = 0; c < 11; c++)
    {
        _stprintf(imgPath, L".\\images\\2\\%d.png", c);
        loadimage(&imgfk[c], imgPath);
    }
}

void showBg()
{
    putimage(0, 0, &imgbg);
}

void showScore()
{
    settextstyle(18, 0, _T("黑体"));
    settextcolor(0xffffffff);
    TCHAR str[100];
    _stprintf(str, L"%d分", score);
    outtextxy(390, 228, str);
}

void showLevel()
{
    settextstyle(18, 0, _T("黑体"));
    TCHAR str[100];
    _stprintf(str, L"Level %d", 1);
    outtextxy(390, 294, str);
}

void gameover()
{
    stop = 1;
}
```

```
}
```

luokuai.cpp 的代码如下:

```
#include "all.h"
#include<stdlib.h>
int luoKuai[20][10] = { 0 };//下落的方块
int color;

void showLuoKuai()
{
    extern void drawImage(int picture_x, int picture_y, IMAGE * picture);
    for (int h = 0; h < 20; h++)
    {
        for (int l = 0; l < 10; l++)
        {
            if (luoKuai[h][l] != 0)
            {
                drawImage(30 + l * 30, 5 + h * 30, &imgfk[luoKuai[h][l]]);
            }
        }
    }
}

/*左不出界 */
_BOOL_ leftNotOut()
{
    return TRUE;
}

/*左不阻挡 */
_BOOL_ leftNotStop()
{
    return TRUE;
}

/*左移动*/
void leftMove()
{
}

/*右不出界 */
_BOOL_ rightNotOut()
{
    return TRUE;
}

/*右不阻挡 */
_BOOL_ rightNotStop()
{
    return TRUE;
}

/*右移动*/
void rightMove()
{
}
```

```

/*下不出界 */
_BOOL_ downNotOut ()
{
    return TRUE;
}

/*下不阻挡 */
_BOOL_ downNotStop ()
{
    return TRUE;
}

/*下移动*/
void downMove ()
{
}

/*出新的落块*/
void createNewLuoKuai ()
{
    //1方块种类: 随机
    TYPE type = (TYPE) (rand() % 5);
    //2颜色: 随机
    color = rand() % 10 + 1; //随机颜色
    switch (type)
    {
        case Z:
            luoKuai[0][3] = luoKuai[0][4] = color;
            luoKuai[1][4] = luoKuai[1][5] = color;
            break;
        case T:
            luoKuai[0][3] = luoKuai[0][4] = luoKuai[0][5] = color;
            luoKuai[1][4] = color;
            break;
        case TIAN:
            luoKuai[0][3] = luoKuai[0][4] = color;
            luoKuai[1][3] = luoKuai[1][4] = color;
            break;
        case L:
            luoKuai[0][3] = color;
            luoKuai[1][3] = color;
            luoKuai[2][3] = luoKuai[2][4] = color;
            break;
        case ONE:
            luoKuai[0][3] = color;
            luoKuai[1][3] = color;
            luoKuai[2][3] = color;
            luoKuai[3][3] = color;
            break;
    }
}

```

注意上面的代码仅仅把和显示的功能开发出来，其它函数仅保留定义的结构，这样编译器就不会有语法错误而能够执行项目让我们看到界面效果。这里需要注意一点，如果函数的返回值不是 void 那么必须用 return 语句明确返回一个复合返回类型的值。

shikuai.cpp 的代码如下:

```
#include "all.h"
int shiKuai[20][10] = { 0 }; //落实 不动的方块
void showShiKuai()
{
    extern void drawImage(int picture_x, int picture_y, IMAGE * picture);
    for (int h = 0; h < 20; h++)
    {
        for (int l = 0; l < 10; l++)
        {
            if (shiKuai[h][l] != 0)
            {
                drawImage(30 + l * 30, 5 + h * 30, &imgfk[shiKuai[h][l]]);
            }
        }
    }
}

/*合并*/
void merge()
{
}

/* 清除行
返回: 行数
*/
int cleanLines()
{
    return 0;
}

/*
根据消除的行数 计算得分
*/
void setScore(int lines)
{
}

/*能否结束*/
_BOOL_ enabledOver()
{
    return FALSE;
}
```

至此游戏界面的相关代码已经开发完成, 如果能够运行并看到游戏基本界面就进入下面的开发步骤。

三、 根据游戏玩的推进步骤建议按如下顺序开发函数的功能:

第一组功能: 下落、左移动、右移动

```
/*下移动*/
void downMove()
{
    for (int h=18 ;h>=0 ;h--)
    {
        for ( int k=0; k<10;k++)
        {
            luoKuai[h + 1][k] = luoKuai[h][k];
            luoKuai[h][k] = 0;
        }
    }
}
```

```

    }
}
/*左移动*/
void leftMove()
{
    for (int k=1 ;k<10 ;k++ )
    {
        for (int h=0 ; h<20; h++)
        {
            luoKuai[h][k - 1] = luoKuai[h][k];
            luoKuai[h][k] = 0;
        }
    }
}
/*右移动*/
void rightMove()
{
    for (int k = 8; k >=0; k--)
    {
        for (int h = 0; h < 20; h++)
        {
            luoKuai[h][k + 1] = luoKuai[h][k];
            luoKuai[h][k] = 0;
        }
    }
}
}

```

第二组功能：下不出界、左不出界、右不出界

```

/*下不出界 */
_BOOL_ downNotOut ()
{
    for (int k=0 ;k<10 ;k++)
    {
        if ( luoKuai[19][k]!=0)
        {
            return FALSE;
        }
    }
    return TRUE;
}
/*左不出界 */
_BOOL_ leftNotOut ()
{
    for (int h=0 ;h<20 ;h++ )
    {
        if (luoKuai[h][0])
        {
            return FALSE;
        }
    }
    return TRUE;
}
/*右不出界 */
_BOOL_ rightNotOut ()

```

```

{
    for (int h = 0; h < 20; h++)
    {
        if (luoKuai[h][9])
        {
            return FALSE;
        }
    }
    return TRUE;
}

```

第三组功能：合共 shuikuai.cpp

```

/*合并*/
void merge()
{
    for ( int h=0; h<20;h++ )
    {
        for (int k=0 ;k<10 ;k++ )
        {
            if ( luoKuai[h][k] )
            {
                shiKuai[h][k] = luoKuai[h][k];
                luoKuai[h][k] = 0;
            }
        }
    }
}

```

第四组功能：下不被阻挡、左不被阻挡、右不被阻挡

```

/*下不阻挡 */
_BOOL_ downNotStop()
{
    for (int h=18 ;h>=0 ;h-- )
    {
        for ( int k=0; k<10; k++)
        {
            if ( luoKuai[h][k] && shiKuai[h+1][k] )
            {
                return FALSE;
            }
        }
    }
    return TRUE;
}

/*左不阻挡 */
_BOOL_ leftNotStop()
{
    for (int k = 1; k < 10; k++)
    {
        for (int h = 0; h < 20; h++)
        {
            if (luoKuai[h][k]&&shiKuai[h][k-1])
            {
                return FALSE;
            }
        }
    }
}

```

```

        return TRUE;
    }
    /*右不阻挡 */
    _BOOL_ rightNotStop()
    {
        for (int k = 8; k >= 0; k--)
        {
            for (int h = 0; h < 20; h++)
            {
                if (luoKuai[h][k]&&shiKuai[h][k+1])
                {
                    return FALSE;
                }
            }
        }
        return TRUE;
    }
}

```

第五组：消除行、计算得分、是否结束

```

/* 清除行
返回：行数
*/
int cleanLines()
{
    int count = 0; //记录消除的行数
    //1 遍历所有行
    for ( int h=1; h<20; h++)
    {
        //2 看哪行能消除
        _BOOL_ neng = TRUE; //假设能消除
        for (int k=0 ;k<10 ;k++ )
        {
            if ( shiKuai[h][k]==0)
            {
                neng = FALSE; //推翻假设
                break;
            }
        }
        //
        if ( neng )
        {
            //3 消除
            for ( int xh=h-1; xh>=0;xh--)
            {
                for (int xk=0 ;xk<10 ;xk++ )
                {
                    shiKuai[xh + 1][xk] = shiKuai[xh][xk];
                    shiKuai[xh][xk] = 0;
                }
            }
            //4 记录行数
            count++;
        }
    }
    return count;
}
/*

```

根据消除的行数 计算得分

```

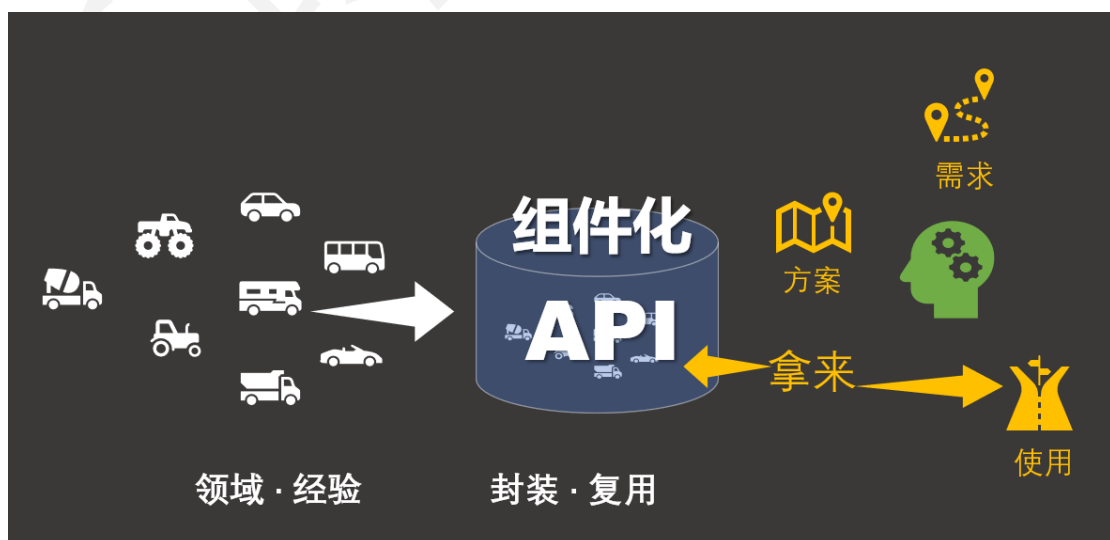
*/
void setScore(int lines)
{
    switch (lines)
    {
        case 1:
            score += 20;
            break;
        case 2:
            score += 50;
            break;
        case 3:
            score += 100;
            break;
        case 4:
            score += 200;
            break;
    }
}

/*能否结束*/
_BOOL_ enabledOver()
{
    for (int k=3 ;k<7 ;k++)
    {
        if (shiKuai[0][k])
        {
            return TRUE;
        }
    }
    return FALSE;
}

```

至此俄罗斯方块这个基于二维数组实现的游戏就开发完成了。

组件化项目开发



组件化是指我们在某些领域的开发中，积累了一定的应用经验与可复用的代码。我们可以把这些代码按照应用的主题、场景把它们组织起来并进行封装打包成组件包。这样一

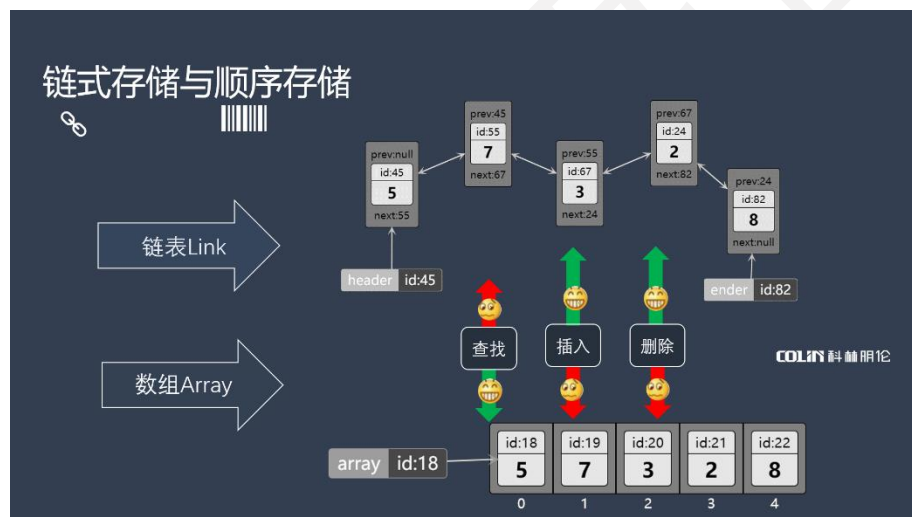
来，在其它项目中如果需要某些代码的功能，只需要按步骤加载组件就实现了功能的使用。所以组件化项目就是让我们知道在项目代码中把共性的模块抽取、沉淀、打包、发布的过程。组件又可称为 API（Application programming interface 应用程序接口）。简而言之就是现有的功能程序，根据我们自身项目的需要找到它、使用它。是一种代码复用提高开发效率的手段。下面我们就以数据结构的链表为例，把它开发成通用性更强的组件化项目，并以静态库、动态链接库的形式进行打包发布。

链表组件化项目

一、链表的基本概念

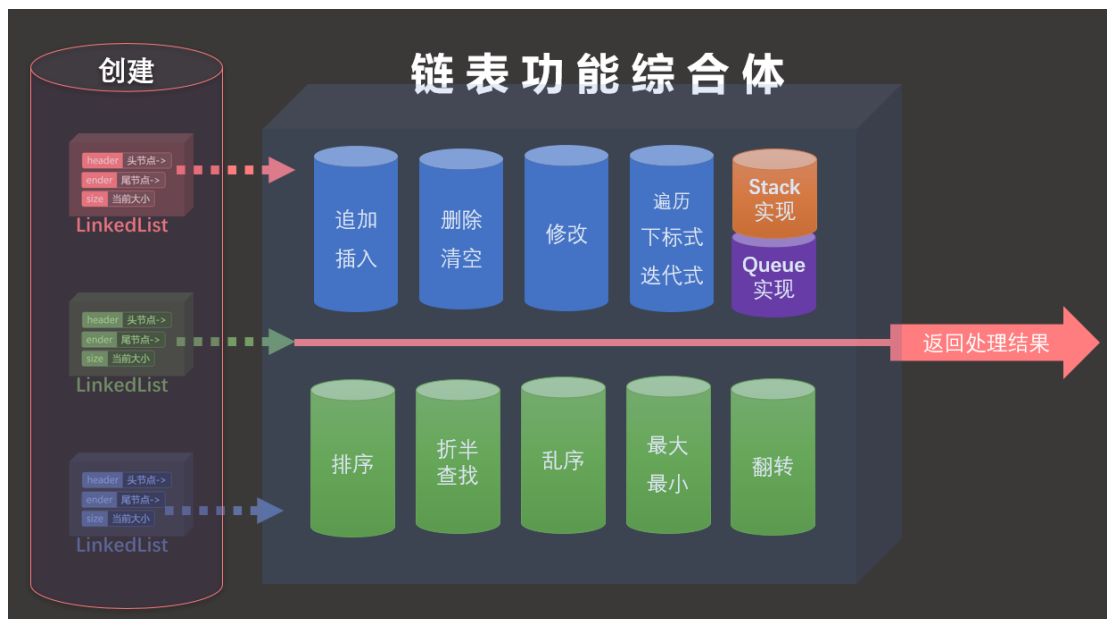
链表是数据结构中线性表（list）的一种实现手段。

什么是数据结构？数据结构是描述现实世界的数据集合的关系与管理逻辑的一项技术。现实世界的数据高度抽象后可以简化为三类集合关系，分别是 1 对 1 关系、1 对多关系、多对多关系。这其中的线性表逻辑就是 1 对 1 关系。每个数据知道自己的一个前驱数据和一个后续数据。生活中的超市购物小票上的购买清单、春晚的节目单、朋友圈中的各种“晒”都是线性表的逻辑。线性表的技术实现分为顺序存储方式（以数组技术实现）和链式存储方式（链表实现）。它们的区别如下图：



- 1、数组在定义时，计算机为其分配连续的内存空间。数组名代表连续空间的首地址。当我们要访问数组某成员时用 数组名[下标]就可以访问。这是因为用：数组名的地址 + 下标位置 × 数据类型字节数，是可以计算出数组成员空间的位置。所以很容易实现线性表的 1 对 1 数据逻辑。但数组也有缺点，比如长度是固定的，当我们需要向数组的中间插入数据或删除数据时，会让数组中的数据向后或向前移动，而且还会牺牲掉一个数据，当频繁插入删除数据时其运行效率会比较低，当然如果仅仅是遍历查找，数组的效率非常高；因为每个成员的地址是可被计算得出的。基于数组的缺点，另一种链式存储的手段被提出来。
- 2、链式存储时，数据是被分散在堆区非连续性存储的。但是按照线性表的逻辑要求，如何实现一个数据知道其前一个和后一个呢，方法就是“记住”前一个和后一个数据在内存中的地址（指针），这样做会增加存储的开销，不但要存储数据本身还需要存储前一个和后一个数据的指针。当然其优点是长度可以延长不会固定长度，另外删除和插入数据变得容易许多，只需要改变前后数据的地址即可。当然因为不是连续的存储空间，在寻找中间某个节点时需要从头进行遍历。

二、链表组件设计目标



上图列出了链表组件的功能结构。共分为三大块功能区。

- 1、可以根据需要能够创建出多条链表。而且要求链表中节点的管理与数据是分离的。即链表可以管理任意数据如字符串或自定义结构体等。
- 2、数据维护方面包括：增加、插入、删除、修改、查找、遍历等另外能够实现栈与队列的数据管理。
- 3、能够实现类似一维数组功能的：排序、折半查找、打乱顺序、翻转、最大值与最小值等。

三、链表组件开发过程

- 1、分别创建三个头文件来规划基本功能

datatype.h 头文件用于规划自定义类型。代码如下：

```
#pragma once
/*链表内部的节点类型*/
typedef struct node
{
    struct node* prev; //前一个节点指针
    void* data; //数据域指针
    struct node* next; //下一个节点指针
} Node, * PNode;

/*链表管理类型*/
typedef struct
{
    PNode header; //头节点
    PNode ender; //尾部节点
    int size; //当前节点总数
    PNode nextNode; //迭代器的下一个节点
} LinkedList, * PLinkedList;

/*定义一个可指向比较函数的函数指针
此函数用于用户自定义结构数据的比较
比较结果分为
大于0 等于0 和小于0来表达两个参数的大小关系
```

```
*/
typedef int (*Compare) (void*, void*);
```

说明：

- 1)、由于采用的是双向链表模式，即每个节点需要知道其前一个及后一个的地址。
- 2)、由于节点与数据采用分离模式，即任何的数据都可以存入到链表。因为数据只要在内存中就会有地址，如果链表仅仅用于保存地址，使用者必须明确是什么样的类型数据存入到链表，这样他只需要用强转的方式操作数据，我们的链表只负责存储其地址即可，这样就保证了链表管理数据的通用性。因此其数据域采用的类型是 void*。这种指针类型变量仅用于保存地址，不能利用存储地址间接访问存储空间，因为 void 代表无类型或不确定类型。为了简化开发为节点类型创建了类型别名是 Node，为了减少代码中的*号数量，给节点指针类型重名为 PNode。
- 3)、每条链表的管理都需要头节点、尾部节点和当前大小。所以把这些能够记录链表特征信息的数据也形成了一个类型称为 LinkedList 即链式实现的线性表。其指针类型称为 PLinkedList。
- 4) 关于 nextNode 与函数指针类型 Compare 在后面使用时再进行详细说明

linkedList.h 头文件用于规划增删改查等数据维护功能。代码如下：

```
#pragma once
#include <stdio.h>
#include <stdlib.h>
#include "datatype.h"
/*创建链表*/
PLinkedList createLinkedList();
/*创建节点*/
PNode createNode( void * data);

/*增加新的节点*/
void add(PLinkedList list , void * data);

/*在某节点的位置上插入新节点*/
void insert(PLinkedList list ,int index, void * data);

/*删除 指定 下标的节点*/
void* removeIndex(PLinkedList list ,int index);

/*删除等于参数的首个节点*/
void removeData(PLinkedList list , void* data, Compare fun);

/*某下标对应的 节点*/
void* get(PLinkedList list ,int index);
/*找到节点*/
PNode findNode(PLinkedList list ,int index);

/*某下标对应的 节点数据进行更新 */
void set(PLinkedList list ,int index, void* newdata);

/*某数据对应的下标*/
int indexOf(PLinkedList list , void* data, Compare fun );

/*节点数量*/
int size(PLinkedList list );
```

```

/*清空链表*/
void clear(PLinkedList list);

/*迭代器*/
//产生新的迭代器
void iterator(PLinkedList list);
//是否有下一个节点
int hasNext(PLinkedList list);
//取得下一个节点数据
void* next(PLinkedList list);

/*实现栈stack的功能: LIFO: last in first out */
/* 入栈*/
void push(PLinkedList list, void* data);

/*出栈*/
void* pop(PLinkedList list);

/*实现队列queue的功能: FIFO: first in first out */
void addFirst(PLinkedList list, void* data);
void addLast(PLinkedList list, void* data);
void* removeFirst(PLinkedList list);
void* removeLast(PLinkedList list);

```

collections.h 头文件用于规划排序查找等数据管理功能。代码如下:

```

#pragma once
#include "linkedlist.h"
/*将链表2 追加到链表1的后面*/
void addAll__(PLinkedList list1, PLinkedList list2);
/*返回最大数据域*/
void* max__(PLinkedList list, Compare fun);
/*返回最小数据域*/
void* min__(PLinkedList list, Compare fun);
/*交换 两个下标对应的数据域*/
void swap__(PLinkedList list, int i, int j);
/*对链表 按 Compare比较规则 进行排序 */
void sort__(PLinkedList list, Compare fun);
/*使用折半查找思路 查询某key数据的下标*/
int binarySearch__(PLinkedList list, void * key, Compare fun);
/*打乱链表的顺序*/
void shuffle__(PLinkedList list);
/*对链表成员进行首尾倒置*/
void reverse__(PLinkedList list);
/*把符合fun比较规则的数据域 替换成新的数据域*/
void replaceAll__(PLinkedList list, void *oldVal, void *newVal, Compare fun);

```

2、为 linkedlist.h 和 collections.h 头文件创建两个对应的源代码文件 LinkedList.cpp 和 Collections.cpp 来定义实现声明的函数功能。继续再创建两个源代码文件 useLinkedList.cpp 和 useCollections.cpp 用于测试驱动开发组件的功能。

3、准备以测试驱动的过程开发链表的增删改查等数据管理功能。

1) 创建链表，并向链表添加数据。

进入 useLinkedList.cpp 文件准备写好测试用例

```
#include "linkedlist.h"
#include "string.h"
int main()
{

    PLinkedList list = createLinkedList();
    add(list, "apple");//0
    add(list, "orange");//1
    add(list, "grape");//2
    add(list, "banana");//3
    add(list, "peach");//4
    return 0;
}
```

进入 LinkedList.cpp 源代码文件来实现测试中用到的 createLinkedList();函数和 add() 函数，还有用于把数据打包到堆内存的 createNode() 函数。具体代码如下：

```
#include "linkedlist.h"

PLinkedList createLinkedList()
{
    PLinkedList newList = (PLinkedList)malloc(sizeof(LinkedList));
    if (newList!=NULL)
    {
        newList->header = NULL;
        newList->ender = NULL;
        newList->size = 0;
        newList->nextNode = NULL;
    }
    return newList;
}

PNode createNode(void* data)
{
    PNode newnode = (PNode)malloc(sizeof(Node));
    newnode->prev = NULL;
    newnode->next = NULL;
    newnode->data = data;
    return newnode;
}

void add(PLinkedList list, void* data)
{
    PNode newNode=createNode(data);
    if (list->size==0)//空链表
    {
        list->header = list->ender = newNode;
    }
    else
    {
        list->ender->next = newNode;
        newNode->prev = list->ender;
        list->ender = newNode;
    }
}
```

```

    }
    list->size++;
}

```

如果要看到代码目前的功能效果，只能通过调试器观察链表的存储情况：

局部变量		
名称	值	类型
list	0x015e5170 {header=0x015e05a0 {prev=0x00000000 <NULL> ...	LinkedList *
header	0x015e05a0 {prev=0x00000000 <NULL> data=0x00e89b30 (组...	node *
prev	0x00000000 <NULL>	node *
data	0x00e89b30 (组件化_LinkedList.exe!string) "apple"	void *
next	0x015e51b0 {prev=0x015e05a0 {prev=0x00000000 <NULL> dat...	node *
prev	0x015e05a0 {prev=0x00000000 <NULL> data=0x00e89b30 (组...	node *
data	0x00e89b38 (组件化_LinkedList.exe!string) "orange"	void *
next	0x015e42d8 {prev=0x015e51b0 {prev=0x015e05a0 {prev=0x00...	node *
prev	0x015e51b0 {prev=0x015e05a0 {prev=0x00000000 <NULL> dat...	node *
data	0x00e89b80 (组件化_LinkedList.exe!string) "grape"	void *
next	0x015e4310 {prev=0x015e42d8 {prev=0x015e51b0 {prev=0x01...	node *
prev	0x015e42d8 {prev=0x015e51b0 {prev=0x015e05a0 {prev=0x00...	node *
data	0x00e89b48 (组件化_LinkedList.exe!string) "banana"	void *
next	0x015edd20 {prev=0x015e4310 {prev=0x015e42d8 {prev=0x01...	node *
prev	0x015e4310 {prev=0x015e42d8 {prev=0x015e51b0 {prev=0x01...	node *
data	0x00e89b40 (组件化_LinkedList.exe!string) "peach"	void *
next	0x00000000 <NULL>	node *
size	5	int
nextNode	0x00000000 <NULL>	node *
ender	0x015edd20 {prev=0x015e4310 {prev=0x015e42d8 {prev=0x01...	node *
prev	0x015e4310 {prev=0x015e42d8 {prev=0x015e51b0 {prev=0x01...	node *
data	0x00e89b40 (组件化_LinkedList.exe!string) "peach"	void *
next	0x00000000 <NULL>	node *

2) 向链表某下标位置插入数据。

准备好测试用例，在 useLinkedList.cpp 的 main 函数内增加如下代码：

```

insert(list, 2, "red");//中间位置插入
insert(list, 0, "blue");//头部位置插入
insert(list, 8, "green");//尾部位置插入

```

再到 LinkedList.cpp 文件实现 insert 函数的功能

```

void insert(PLinkedList list, int index, void* data)
{
    //空链表          //插入尾部
    if (list->size==0 || index>=list->size)
    {
        add(list, data);
        return;
    }
    PNode newNode = createNode(data);
    //插入到头部

    if (index<=0)
    {
        newNode->next = list->header;
        list->header->prev = newNode;
        list->header = newNode;
    }
    else
    {
        //理想的 中间插入
        PNode p = findNode(list, index);
        PNode q = p->prev;//插入点的前面的节点
        newNode->next = p;
        p->prev = newNode;
        q->next = newNode;
    }
}

```

```

        newnode->prev = q;
    }
    list->size++;
}

```

由于根据下标查找节点的功能会被很多地方使用，所以把它封装成一个独立的函数

```

PNode findNode(PLinkedList list, int index)
{
    PNode p = list->header;
    for (int i = 0; i < index; i++)
    {
        p = p->next;
    }
    return p;
}

```

3) 按下标号删除节点

准备好测试用例，在 useLinkedList.cpp 的 main 函数内增加如下代码：

```

const char*bssj =(const char*)removeIndex(list,2);
printf("被删的是: %s\n", bssj);
bssj = (const char*)removeIndex(list, 0);
printf("被删的是: %s\n", bssj);
bssj = (const char*)removeIndex(list, 4);
printf("被删的是: %s\n", bssj);

```

一般来讲，删除某节点的数据是需要把被删除数据返回的。再到 LinkedList.cpp 文件实现 removeIndex 函数的功能。

```

void* removeIndex(PLinkedList list, int index)
{
    if (list->size==0)
    {
        return NULL;
    }
    PNode p =NULL;//p指向被删除的节点
    if (list->size==1)//唯一节点时的处理
    {
        p = list->header;
        list->header = list->ender = NULL;
    }
    else if (index<=0)//删除头节点
    {
        p = list->header;
        list->header = p->next;
        list->header->prev = NULL;
    }
    else if (index>=list->size-1)//删除尾部节点
    {
        p = list->ender;
        list->ender = p->prev;
        list->ender->next = NULL;
    }
    else
    {
        //理想的 中间删除
        p = findNode(list, index);
        PNode q = p->prev;
    }
}

```

```

        PNode m = p->next;
        q->next = m;
        m->prev = q;
    }
    //保护被删节点内的数据
    if (p!=NULL)
    {
        void* temp = p->data;
        free(p);
        list->size--;
        return temp;
    }
    return NULL;
}

```

4) 修改节点存储的值

准备好测试用例，在 useLinkedList.cpp 的 main 函数内增加如下代码：

```
set(list, 2, "GRAPE");
```

再到 LinkedList.cpp 文件实现 set 函数的功能。

```

void set(PLinkedList list, int index, void* newdata)
{
    PNode p=findNode(list, index);
    if ( p!=NULL)
    {
        p->data = newdata;
    }
}

```

5) 下标式遍历节点

准备好测试用例，在 useLinkedList.cpp 的 main 函数内增加如下代码：

```

for (int i = 0; i < size(list); i++)
{
    const char* data = (const char*)get(list, i);
    printf("%s\n", data);
}

```

再到 LinkedList.cpp 文件实现函数的功能。

```

int size(PLinkedList list)
{
    return list->size;
}

void* get(PLinkedList list, int index)
{
    PNode p = findNode(list, index);
    return p!=NULL? p->data : NULL;
}

```

6) 迭代式遍历节点

准备好测试用例，在 useLinkedList.cpp 的 main 函数内增加如下代码：

```

iterator(list);
while (hasNext(list))
{
    const char* data = (const char*)next(list);
}

```

```
    printf("%s\n", data);
}
```

再到LinkedList.cpp 文件实现函数的功能。

```
void iterator(PLinkedList list)
{
    list->nextNode = list->header;
}

int hasNext(PLinkedList list)
{
    return list->nextNode != NULL;
}

void* next(PLinkedList list)
{
    if (list->nextNode != NULL)
    {
        void* data = list->nextNode->data;
        //为下一次做准备
        list->nextNode = list->nextNode->next;
        return data;
    }
    return NULL;
}
```

7) 寻找数据所在下标

观察indexOf函数的基本结构，`int indexOf(PLinkedList list, void* data, Compare fun)` 它需要一个函数指针作为比较的参数。因为链表为了通用性，仅仅保留的内存中数据的地址，但由于不知道指针类型，就无法通过此地址获得空间读取对应的数据，所以对于数据指针指向空间的数据比较就需要链表使用者提供一种能够比较两个地址空间数据的比较函数。由于Compare类型的函数指针的定义是`typedef int (*Compare) (void*, void*)`；所以我们需要符合这个类型特点创建一个自定义的比较函数。在useLinkedList.cpp中增加如下函数：

```
int my_strcmp(void* s1, void* s2)
{
    const char* ps1 = (const char*)s1;
    const char* ps2 = (const char*)s2;
    return strcmp(ps1, ps2);
}
```

在main函数内容增加测试用例：

```
char str[] = "peach";
printf("peach的下标: %d\n", indexOf(list, str, my_strcmp));
```

在LinkedList.cpp源代码中实现indexOf函数的功能：

```
int indexOf(PLinkedList list, void* data, Compare fun)
{
    iterator(list);
    for (int i=0 ;hasNext(list) ;i++ )
    {
        if ( fun(next(list), data)==0 )
        {
            return i;
        }
    }
}
```

```

    }
}
return -1;
}

```

8) 删除第一个满足的数据所在下标

可以在上面 indexOf 功能的基础上完成此功能，首先在 useLinkedList.cpp 中增加测试用例：

```
removeData(list, "peach", my_strcmp);
```

在 LinkedList.cpp 源代码中实现 removeData 函数的功能：

```

void removeData(PLinkedList list, void* data, Compare fun)
{
    int i=indexOf(list, data, fun);
    if (i>=0)
    {
        removeIndex(list, i);
    }
}

```

9) 清空链表、释放链表

清空链表就是把链表的所有在堆内存的节点统一释放掉，但是不能释放掉节点指向的数据，因为节点指向的数据不确定一定在堆内存空间，可能在全局静态区，也可以在常量区甚至在栈区。所以这些用户的数据释放交由使用者自行处理。useLinkedList.cpp 的测试用例如下：

```
clear(list);
```

LinkedList.cpp 中的源代码如下：

```

void clear(PLinkedList list)
{
    PNode p = list->header;
    PNode q = NULL;
    while (p!=NULL)
    {
        q = p;
        p = p->next;
        free(q);
    }
    list->header = list->ender = NULL;
    list->size = 0;
}

```

最后如果想要彻底释放链表的内存空间需要在 useLinkedList.cpp 中补充测试用例：

```

//释放链表的堆空间
free(list);
list = NULL;

```

10) 实现栈的逻辑 LIFO

栈从存储角度讲是个线性表，但是其存取逻辑是 Last in first out 即后进先出。所以我们在链式存储的线性表基础上增加两个函数 push 压入栈，pop 弹出栈。其测试用例如下：

```

PLinkedList stack = createLinkedList();
push(stack, "车");
push(stack, "马");
push(stack, "炮");
push(stack, "象");

const char* qz=(const char*)pop(stack);
printf("%s\n",qz);
qz = (const char*)pop(stack);
printf("%s\n", qz);
qz = (const char*)pop(stack);
printf("%s\n", qz);
qz = (const char*)pop(stack);
printf("%s\n", qz);
qz = (const char*)pop(stack);
printf("%s\n", qz);

```

LinkedList.cpp 中实现的代码如下：

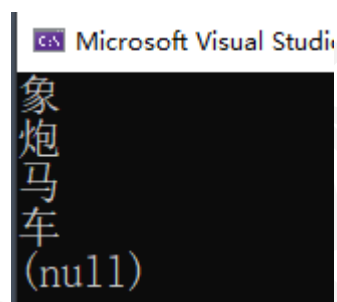
```

void* pop(PLinkedList list)
{
    return removeIndex(list,size(list)-1);
}

void addFirst(PLinkedList list, void* data)
{
    insert(list, 0, data);
}

```

运行结果如下：



11) 实现队列的逻辑 FIFO

队列的存储也是线性表，其存取逻辑是 First in first out 即先进先出。我们把链表的头部节点称为 first 尾部节点称为 last。测试用例如下：

```

PLinkedList queue = createLinkedList();
addFirst(queue, "aa");
addFirst(queue, "bb");
addFirst(queue, "cc");
addFirst(queue, "dd");
const char* str = (const char*)removeLast(queue);
printf("%s\n",str);
str = (const char*)removeLast(queue);
printf("%s\n", str);
str = (const char*)removeLast(queue);
printf("%s\n", str);
str = (const char*)removeLast(queue);
printf("%s\n", str);
str = (const char*)removeLast(queue);

```

```
printf("%s\n", str);
```

LinkedList.cpp 中实现的代码如下：

```
void addFirst(PLinkedList list, void* data)
{
    insert(list, 0, data);
}

void addLast(PLinkedList list, void* data)
{
    add(list, data);
}

void* removeFirst(PLinkedList list)
{
    return removeIndex(list, 0);
}

void* removeLast(PLinkedList list)
{
    return removeIndex(list, size(list)-1);
}
```

运行结果如下：

```
Microsoft Visual Studio 调试控制台
aa
bb
cc
dd
(null)
```

12) 连接两个链表

在 useCollections.cpp 源代码中增加测试用例：

```
PLinkedList list1 = createLinkedList();
PLinkedList list2 = createLinkedList();
add(list1, "apple");
add(list1, "orange");
add(list1, "peach");
add(list1, "banana");

add(list2, "one");
add(list2, "two");
add(list2, "three");
addAll__(list1, list2);

//遍历list1
iterator(list1);
while ( hasNext(list1))
{
    const char* str =(const char*)next(list1);
    printf("%s\n", str);
}
```

在 Collections 中增加 addAll 函数的定义实现：

```
void addAll__(PLinkedList list1, PLinkedList list2)
{
```

```

    iterator(list2);
    while (hasNext(list2))
    {
        add(list1, next(list2));
    }
}

```

13) 交互两个节点的数据

根据两个下标找到对应节点，互换它们指向的数据：

```

void swap__(PLinkedList list, int i, int j)
{
    PNode pi = findNode(list, i);
    PNode pj = findNode(list, j);
    void* t;
    t = pi->data;
    pi->data = pj->data;
    pj->data = t;
}

```

14) 寻找链表中的最大值与最小值

在 useCollections.cpp 源代码中增加测试用例：

```

const char* max_str = (const char*)max__(list1, my_strcmp);
printf("最大字符串: %s\n", max_str);
const char* min_str = (const char*)min__(list1, my_strcmp);
printf("最小值: %s\n", min_str);

```

在 Collections 中增加 addAll 函数的定义实现：

```

void* max__(PLinkedList list, Compare fun)
{
    if (list->size == 0) return NULL;
    void* max_value = list->header->data;
    iterator(list);
    while (hasNext(list))
    {
        void* data = next(list);
        if ( fun(max_value, data) < 0 )
        {
            max_value = data;
        }
    }
    return max_value;
}

void* min__(PLinkedList list, Compare fun)
{
    if (list->size == 0) return NULL;
    void* min_value = get(list, 0);
    for (int i=1 ; i<size(list) ; i++)
    {
        void* data = get(list, i);
        if ( fun(min_value, data) > 0 )
        {
            min_value = data;
        }
    }
}

```

```

    return min_value;
}

```

15) 对链表节点的数据进行升序排序

```

void sort__(PLinkedList list, Compare fun)
{
    PNode p_suo = list->header;
    PNode p_bi = NULL;
    for (int suo = 0; suo < size(list) - 1; suo++)
    {
        p_bi = p_suo->next;
        for (int bi = suo + 1; bi < size(list); bi++)
        {
            if (fun(p_suo->data, p_bi->data) > 0)
            {
                void* t;
                t = p_suo->data;
                p_suo->data = p_bi->data;
                p_bi->data = t;
            }
            p_bi = p_bi->next;
        }
        p_suo = p_suo->next;
    }
}

```

16) 折半查找链表中的数据

```

int binarySearch__(PLinkedList list, void* key, Compare fun)
{
    int low = 0;
    int high = size(list) - 1;
    int mid;
    do
    {
        mid = (low + high) / 2;
        void* data = get(list, mid);
        int r = fun(data, key);
        if (r == 0)
        {
            return mid;
        }
        else if (r < 0) {
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    } while (low <= high);
    return -1;
}

```

17) 打乱链表的数据顺序

增加随机数需要的头文件

```
#include <stdlib.h>
#include<time.h>
```

```
void shuffle__(PLinkedList list)
{
    srand((unsigned) time(NULL));
    for (int i=0 ;i<size(list) ;i++ )
    {
        int a = rand() % size(list);
        int b = rand() % size(list);
        swap__(list, a, b);
    }
}
```

18) 对链表数据进行翻转

```
void reverse__(PLinkedList list)
{
    PNode p = list->header;
    PNode q = list->ender;
    void* t;
    for (int i = 0, j = size(list) - 1; i < j; i++, j--)
    {
        t = p->data;
        p->data = q->data;
        q->data = t;
        p = p->next;
        q = q->prev;
    }
}
```

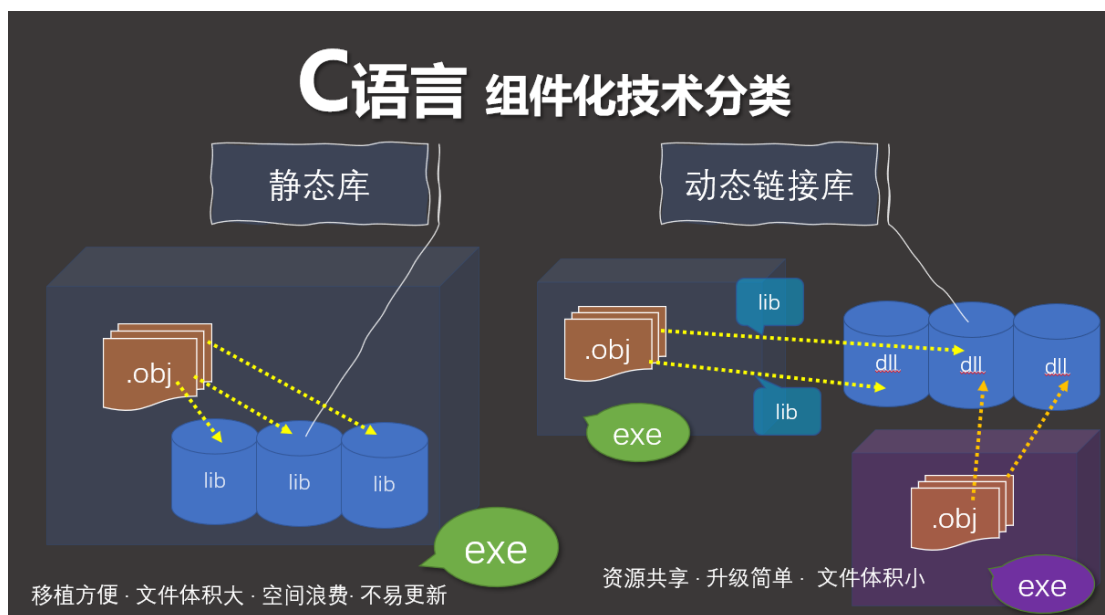
19) 替换链表中符合的数据为新数据。

```
void replaceAll__(PLinkedList list, void* oldVal, void* newVal, Compare fun)
{
    PNode p = list->header;
    while (p!=NULL)
    {
        if (fun(p->data, oldVal)==0)
        {
            p->data = newVal;
        }
        p = p->next;
    }
}
```

四、链表组件化打包

把开发并测试好的组件代码包括（头文件和源代码文件）准备好后，需要创建新的库项目，把写好的组件代码放入其中，并进行必要的设置即可实现组件库的打包。打包的方式可以

分为静态库及动态链接库 DLL。其区别如下：



我们开发好的组件代码会被打包进一个 lib 扩展名的文件。如果一个项目需要组件的功能，只需要把 lib 放入项目中即可实现对其中功能的使用。之所以称为静态库是因为项目自身的目标文件 obj 会和 lib 组件库一同被编译进 exe。这样会造成 exe 文件体积会大。优点是便于移植。缺点是如果这个组件库被很多项目使用，而且每个项目都被启动执行，那么内存中就会有多个相同的库就造成了空间浪费的情况。所以有动态链接库，它会把组件独立出来成为独立的 dll 文件，这样内存中仅用一份，形成共享模式。又能减少内存资源浪费还能减少 exe 文件的体积，而且升级相对简单。

1、静态库的实现过程

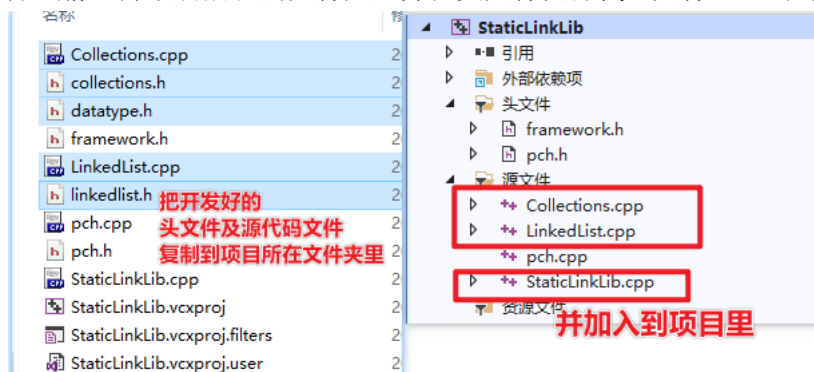
在解决方案上新建项目。此时选择静态库



项目名为：StaticLinkLib，当点击创建按钮后会出现如下的项目结构



此时不需要修改移出这里的任何文件。仅需要把我们开发好的头文件及源代码文件复制粘贴到静态库项目所在的文件夹，并把头文件及源代码文件加入到对应的位置上。



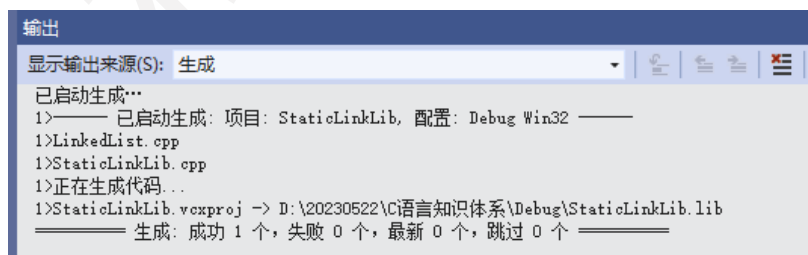
接下来在要参加打包的源代码里增加代码：`#include "pch.h"` 意为此源代码需要被打包进 lib 文件，但务必将其放在第一行处，否则在生成时会产生错误提示。

这里 Collections.cpp 和 LinkedList.cpp 都需要在第一行增加。

到这里就可以点击生成菜单来把项目变为 lib 组件库了。



如果有如下提示，说明组件库生成成功。



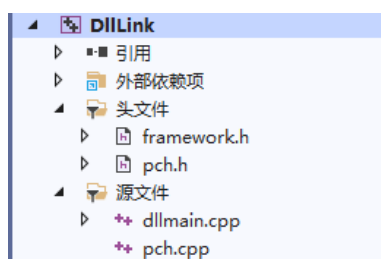
按照其生成提示位置可以找到对应的 lib 文件。在这里，要根据组件的应用场景还要区分 32 位和 64 位系统，需要分别进行生成。

2、动态链接库 DLL

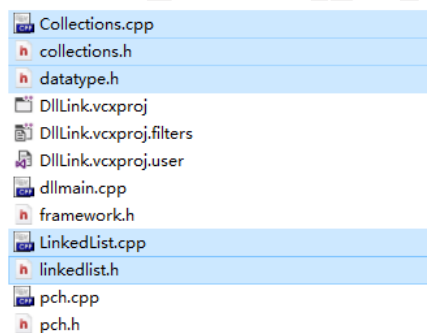
在解决方案上新建项目。此时选择动态链接库（DLL）如下图：



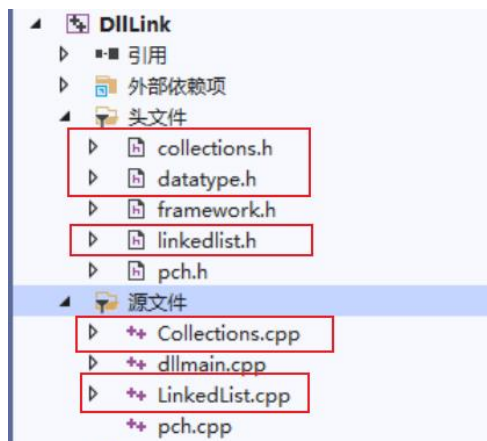
输入项目名：DllLink 后点击创建按钮后，查看其文件结构如下：



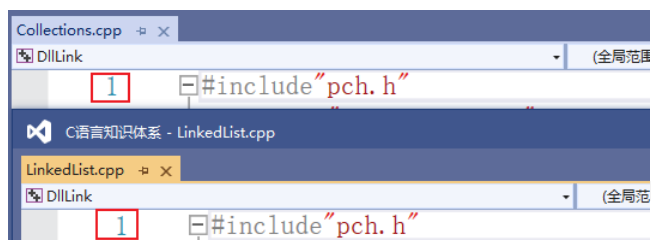
和静态库的模式类似，此时需要把开发好的组件代码（头文件及源代码文件）复制到项目所在文件夹。



在到项目管理中把头文件和源代码文件加入到项目管理中来。



和静态库一样的地方就是每个被打包的源代码文件的第一句要加入“#include” pch.h”。



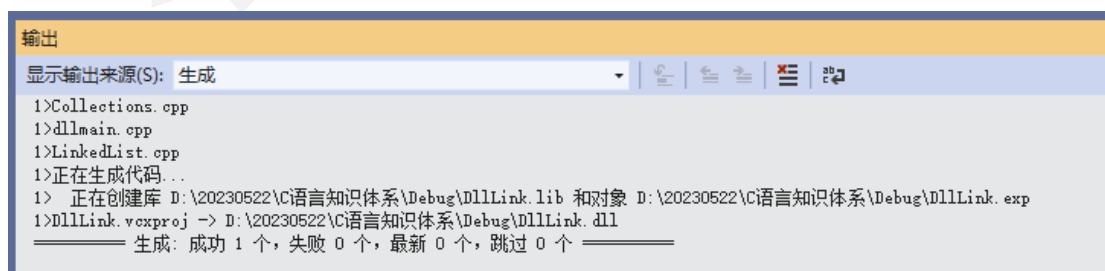
和静态库不一样的地方是需要在头文件中，把需要被外部使用函数声明处增加 `_declspec(dllexport)` 这样的关键字，使得此函数成为动态库对外的接口函数。每个函数都需要添加此关键字。collections.h 中的处理如下图。

```

/*将链表2 追加到链表1的后面*/
_declspec(dllexport) void addAll__(PLinked
/*返回最大数据域*/
_declspec(dllexport) void* max__(PLinkedLi
/*返回最小数据域*/
_declspec(dllexport) void* min__(PLinkedLi
/*交换 两个下标对应的数据域*/
_declspec(dllexport) void swap__(PLinkedLis
/*对链表 按 Compare比较规则 进行排序 */
_declspec(dllexport) void sort__(PLinkedLis
/*使用折半查找思路 查询某key数据的下标*/
_declspec(dllexport) int binarySearch__(PLi
/*打乱链表的顺序*/
_declspec(dllexport) void shuffle__(PLinked
/*对链表成员进行首尾倒置*/
_declspec(dllexport) void reverse__(PLinke
/*把符合fun比较规则的数据域 替换成新的数据域
_declspec(dllexport) void replaceAll__(PLin

```

linklist.h 头文件的处理亦如此。最后点击生成菜单的生成 DllLink（项目名）如果有如下提示说明生成成功。



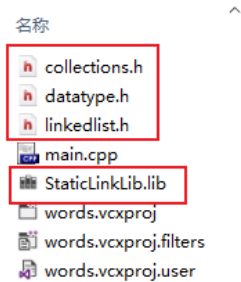
其产生的两个文件是我们需要的，一个是 dll 文件另一个是 lib 文件。其中 dll 是运行时用到的，函数实现的流程代码就打包在 dll 中，程序运行时就可调用。这里的 lib 文件是编译时用到的，其包含了函数所在 dll 文件的位置信息。

五、组件的场景应用

无论是静态库还是动态链接库，它们属于中间件，不是独立运行的 exe 可执行文件，它们需要被 exe 项目调用才能执行。下面用一个例子说明其用法。

无论哪种库都需要创建一个 exe 可执行的项目。我们以一串单词的排序为例说明。

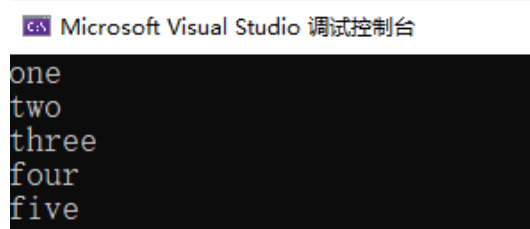
- 1、创建一个空项目，名字为 words。
- 2、添加一个名为 main.cpp 源代码文件
- 3、以静态库为例，把生成的 StaticLinkLib.lib 和有关的 3 个头文件复制到项目所在文件夹。



- 4、在 main.cpp 源代码中增加如下代码：

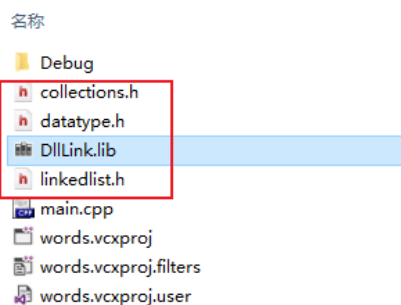
```
#include<stdio.h>
#include"collections.h"
#pragma comment(lib,"StaticLinkLib.lib")//加载组件库
int main()
{
    PLinkedList list = createLinkedList();
    add(list, (void*) "one");
    add(list, (void*) "two");
    add(list, (void*) "three");
    add(list, (void*) "four");
    add(list, (void*) "five");
    iterator(list);
    while (hasNext(list))
    {
        printf("%s\n", (const char*)next(list));
    }
    return 0;
}
```

如果出现如下运行结果则代码组件加载使用成功。



如果是动态链接库，其做法如下：

- 1、和静态库一样把头文件放到项目里，把 DllLink.lib 引导调用 dll 代码的文件也放到项目里。



- 2、DllLink.dll 这个文件不需要放到项目里，默认操作只需要把它放在生成的 exe 的旁边就可以调用了。
- 3、main.cpp 源代码稍做如下改动

```
#include<stdio.h>
#include"collections.h"
#pragma comment(lib,"DllLink.lib")//加载组件库
```

- 4、执行项目，就可看到和静态库执行一样的效果。
- 如果出现如下提示：



说明动态链接库不存在或没找到，此时把它复制到 exe 所在文件夹即可。

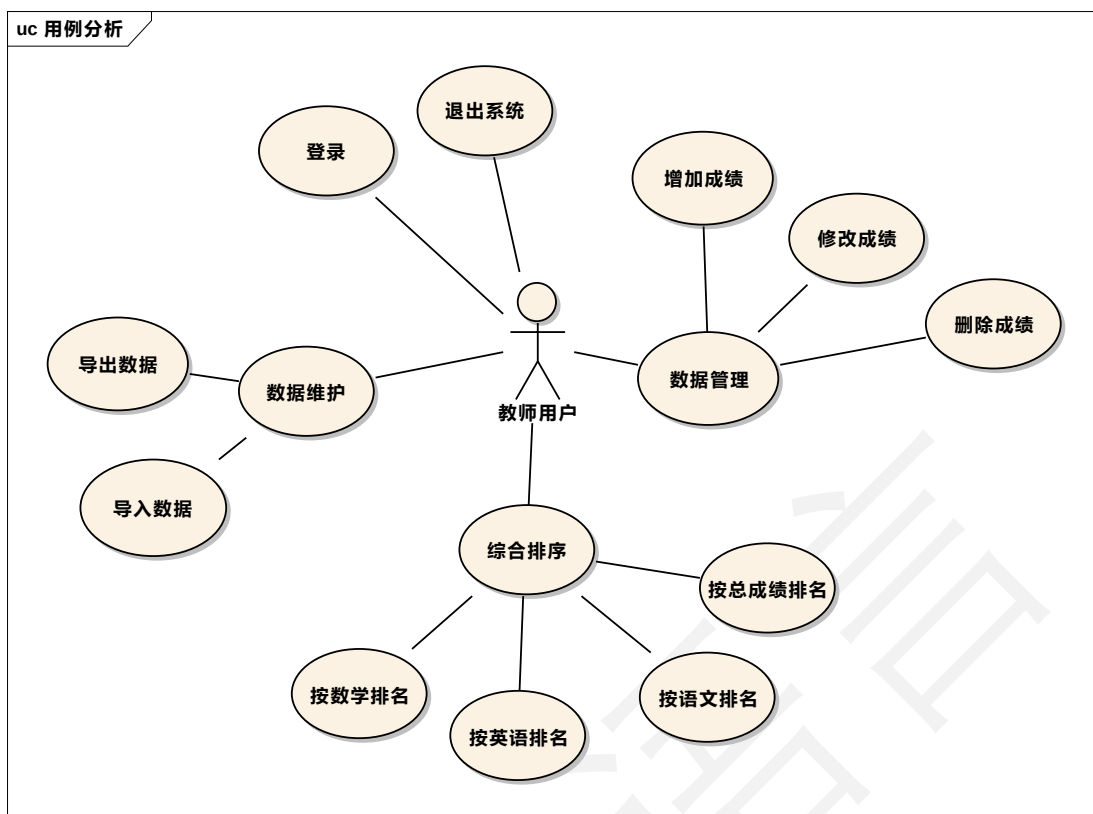
MIS 管理信息系统开发

管理信息系统（Management Information System，简称 MIS）。其实我们开发的软件中信息管理类的软件占比是很多高的。比如学校的学生管理、图书馆的图书管理。信息管理软件开发的基本功能概括起来四个字：增删改查。所以新手小白第一个开发的 MIS 类系统就是如何实现信息的：查询展示、增加新数据、修改既有数据、删除数据并在这个基础上再增添良好的用户交互体验，数据的完整与安全等。我们用 C 语言开发的第一个 MIS 只需要做成单机版即可，对于数据的持久化方面可以选择数据库技术，也可以使用自定义数据格式的文件存储。

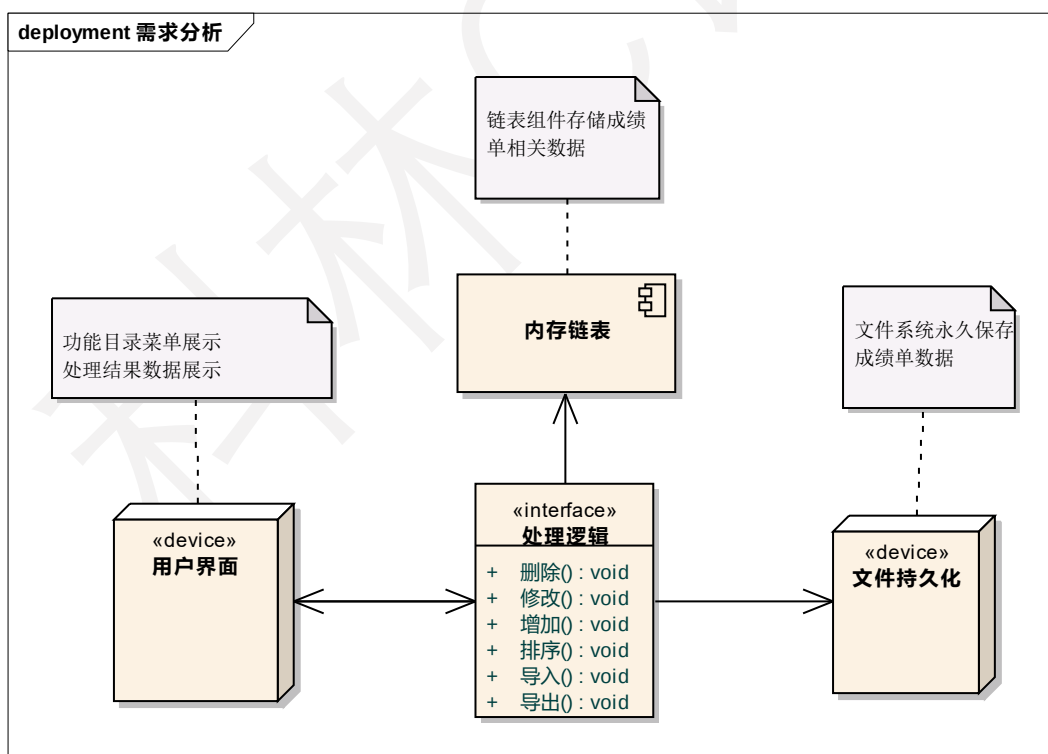
成绩单管理系统

需求分析：

本系统为中小学班主任老师开发的简易成绩排名系统。功能如下：

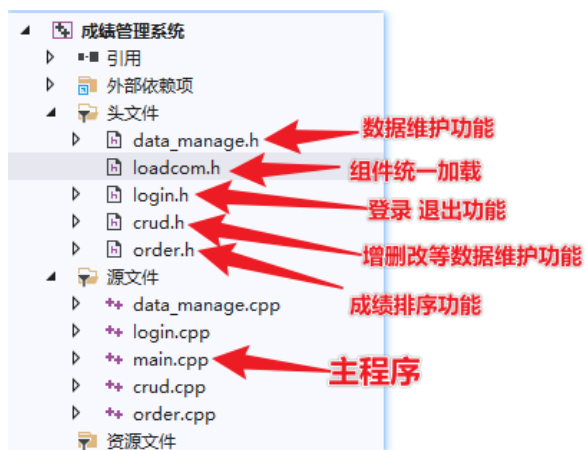


本项目的运行部署模型如下：



用链表组件的链表功能管理学生的成绩单数据，并把数据在增加、修改、删除等更新时能够持久化到文件中进行永久保存。

代码结构：



根据用例图规划的功能菜单，按照高内聚低耦合的原则对子程序进行分离。

子程序功能规划：

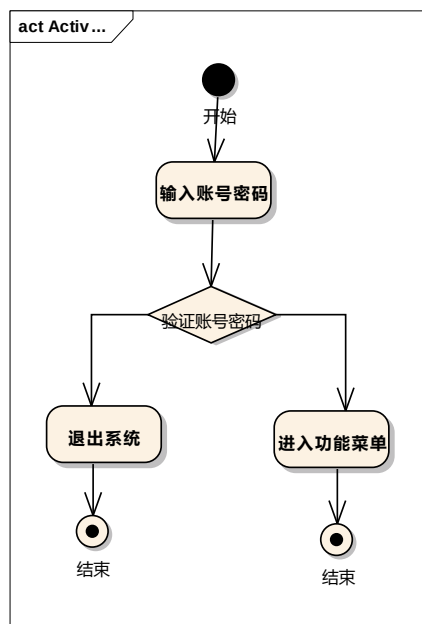
登录模块的用户体验：

```

D:\20230522\C语言知识体系\Debug\成绩管理系统.exe
*****欢迎使用成绩单管理系统*****
输入账号: colin
输入密码: *****
-----数据管理-----
*   r. 显示所有学生   *
*   a. 增加学生成绩   *
*   u. 修改学生成绩   *
*   d. 删除学生成绩   *
*
-----综合排序-----
*   m. 数学排名       *
*   e. 英语排名       *
*   c. 语文排名       *
*   s. 总成绩排名     *
*
-----数据维护-----
*   i. 导入数据       *
*   p. 导出数据       *
*
*   q. 退出系统       *
*
指令/>

```

登录的处理逻辑：



登录子程序 login.h 的函数声明代码如下：

```

#pragma once
#define LOGIN_ERR 0
#define LOGIN_SUCCESS 1

/*
  登录输入
  参数: user账号的存放处
        pwd密码密码的存放处
*/
void loginInput(char user[], char pwd[]);

/*登录验证*/
int login(char user[], char pwd[]);

/*显示教师菜单*/
void showTeacherMenu();

/*处理 教师的菜单功能*/
void optTeacherMenu();

/*退出系统*/
void quit();
  
```

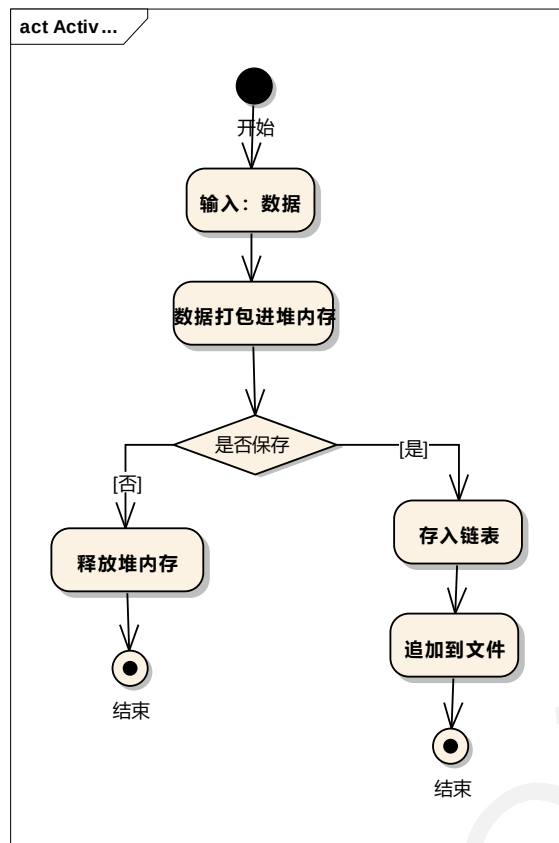
数据维护子程序的用户体验：

增加数据：

```

指令 />a*****增加新数据*****
请输入姓名: LiLy
请输入学号: 230801
请输入数学成绩: 89
请输入英语成绩: 97
请输入语文成绩: 79
是否保存新数据<Y/N>?y
保存成功
指令 />
  
```

增加功能的逻辑活动图：



修改数据：

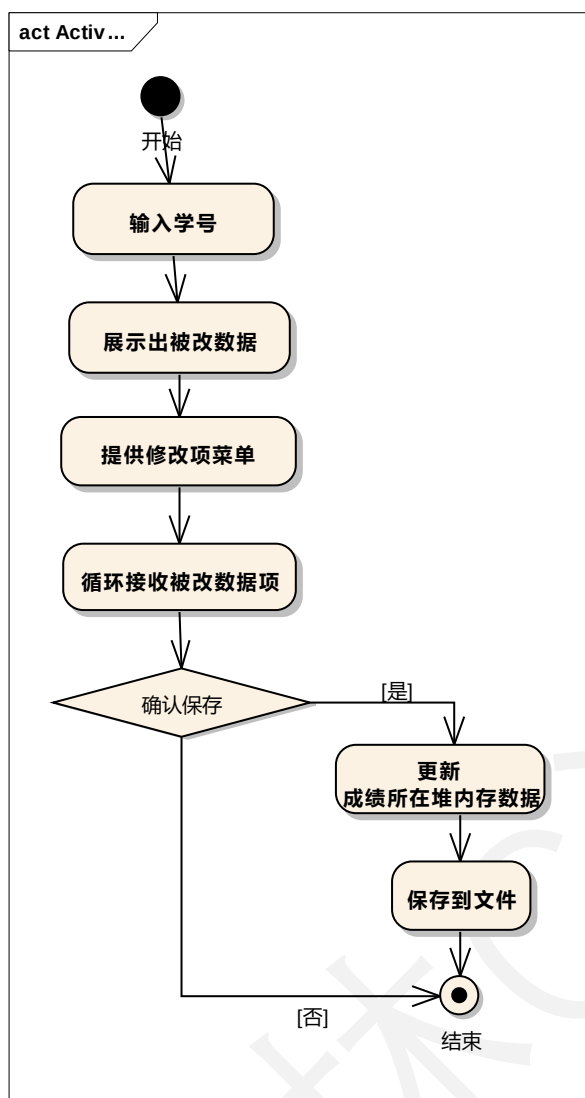
```

指令/>u*****修改数据*****
请输入学号：230801

姓名    学号    数学    英语    语文
LiLy    230801    89     97     79

*****准备更新学号为：230801的数据*****
** 0:更新学号:
** 1:更新姓名:
** 2:更新数学:
** 3:更新英语:
** 4:更新语文:
** y:保存更新:
** n:取消更新:
更新操作选项/>3新的英语成绩：100
更新操作选项/>4新的语文成绩：99
更新操作选项/>y数据更新保存成功
指令/>
  
```

修改数据的活动图：

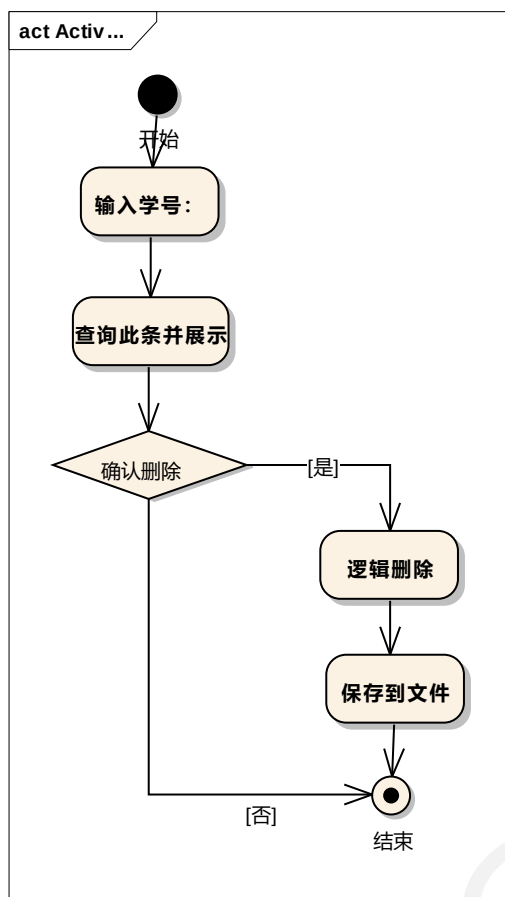


删除数据：

```
指令 />d*****删除数据*****
请输入学号：230801

姓名    学号    数学    英语    语文
LiLy    230801    89     100    99
确认删除吗？ <Y/N>y
指令 />
```

删除数据的活动图：



查询显示所有按学号排序的数据:

```

指令/>r*****按学号顺序显示*****
姓名    学号      数学   英语   语文
孙悟空  220101      88    99    78
孙悟空  220101      88    99    78
唐生    220102      88    99    78
唐生    220102      88    99    78
猪悟能  220103      77    99   100
猪悟能  220103      77    99   100
Jack    230102156    99    98    77
wang    2301025     69    88    79
王武    23010256    98    68   100
Jim     23010256    99   100    69
李      23010523    59    98    64
lucy    2301205     98    68    79
指令/>
  
```

根据成绩单数据模型及功能规划，其子程序 crud.h 的源代码如下：

```

#pragma once
#include "loadcom.h"
/*成绩管理子程序*/
/*成绩的数据模型*/
typedef struct score
{
    char name[28]; //姓名
    char stuNum[20]; //学号
    int math;
  
```

```

    int english;
    int chinese;
    int del;//逻辑删除标记 0 不删除 1 删除
} SCORE, * P_Score;
/*成绩单 列表*/
extern PLinkedList listScore;
/*初始化数据*/
void initData();
/*查询所有的成绩，按学号排序*/
void showAllScore();

/*输入成绩*/
P_Score inputNewScore();

/*在堆区申请空间 存储成绩*/
P_Score createScore(const char* name, const char* stuNum,
    int math, int english, int chinese);

/*追加到文件：ab方式*/
void appendToFile(P_Score ps);
/*增加新成绩*/
void addNewScore();
/*更新一条*/
void updateScore();
/*删除成绩*/
void deleteScore();
/*
查询一条
index:形参返回 符号的下标号
返回：复合学号的 成绩
*/
P_Score queryOneScore( int*const index);
/*更新数据到文件*/
void updateToFile( P_Score p, int index);

```

另外统一加载组件的头文件 loadcom.h 源代码如下：

```

#pragma once
#include "lib/string_pro.h"
#include "lib/collections.h"
#pragma comment(lib, ".\\lib\\StaticLinkLib.lib")
#pragma comment(lib, ".\\lib\\StaticStringLib.lib")

```

排序子程序的用户体验：

指令/>>g*****数学排名*****					指令/>>e*****英语排名*****					指令/>>c*****语文排名*****					指令/>>s*****总成绩排名*****				
姓名	学号	数学	英语	语文	姓名	学号	数学	英语	语文	姓名	学号	数学	英语	语文	姓名	学号	数学	英语	语文
Jack	230102156	99	98	77	Jim	23010256	99	100	69	猪悟能	220103	77	99	100	猪悟能	220103	77	99	100
Jim	23010256	99	100	69	孙悟空	220101	88	99	78	猪悟能	220103	77	99	100	猪悟能	220103	77	99	100
王武	23010256	98	68	100	孙悟空	220101	88	99	78	王武	23010256	98	68	100	Jack	230102156	99	98	77
Lucy	2301205	98	68	79	唐生	220102	88	99	78	wang	2301025	69	88	79	Jim	23010256	99	100	69
孙悟空	220101	88	99	78	唐生	220102	88	99	78	lucy	2301205	98	68	79	王武	23010256	98	68	100
孙悟空	220101	88	99	78	猪悟能	220103	77	99	100	孙悟空	220101	88	99	78	孙悟空	220101	88	99	78
唐生	220102	88	99	78	Jack	230102156	99	98	77	唐生	220102	88	99	78	唐生	220102	88	99	78
猪悟能	220103	77	99	100	李	23010523	59	98	64	唐生	220102	88	99	78	lucy	2301205	98	68	79
猪悟能	220103	77	99	100	wang	2301025	69	88	79	Jack	230102156	99	98	77	wang	2301025	69	88	79
wang	2301025	69	88	79	王武	23010256	98	68	100	Jim	23010256	99	100	69	李	23010523	59	98	64
李	23010523	59	98	64	Lucy	2301205	98	68	79	李	23010523	59	98	64	指令/>>				

排序子程序 order.h 规划的函数源代码如下：

```

#pragma once
/*排序规则函数*/
/*学号比较规则*/
int ruleStuNumOrder(void* s1, void* s2);

```

```

/*按学号的升序排序*/
void stuNumOrder();

/*数学比较规则*/
int ruleMathOrder(void* s1 ,void*s2 );
/*按数学的 成绩 降序排序*/
void mathOrder();

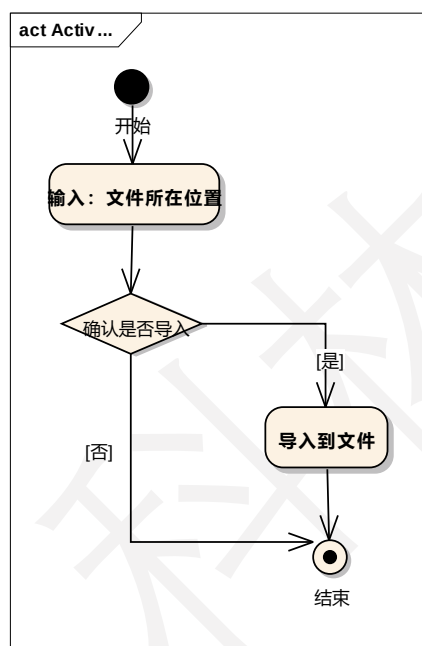
/*英语比较规则*/
int ruleEnglishOrder(void* s1, void* s2);
/*按英语的 成绩 降序排序*/
void englishOrder();

/*语文比较规则*/
int ruleChineseOrder(void* s1, void* s2);
/*按语文的 成绩 降序排序*/
void chineseOrder();

/*总成绩比较规则*/
int ruleAllScoreOrder(void* s1, void* s2);
/*按总成绩的降序排序*/
void allScoreOrder();

```

数据维护子程序的活动图：



对应的 data_manage.h 源代码如下：

```

#pragma once
#define IMPORT_SUCCESS 1
#define IMPORT_ERR 0
#define EXPORT_SUCCESS 1
#define EXPORT_ERR 0

/*显示导入的用户界面*/
void showImportUI();

/*导入数据
参数：数据的存储来源

```

```

返回：是否成功 0 失败,1成功
*/
int importData(const char* dataPath);

/*导出界面*/
void showExportUI();
/*导出数据
参数：数据的存放位置
返回：是否成功 0 失败,1成功
*/
int exportData(const char* dataPath);

```

最终源代码：

主模块 main.cpp

```

#include <stdio.h>
#include "login.h"
#include "crud.h"
int main()
{
    printf("*****欢迎使用成绩单管理系统*****\n");
    char user[20];
    char pwd[20];
    loginInput(user, pwd);
    switch ( login(user, pwd) )
    {
        case LOGIN_SUCCESS:
            showTeacherMenu();
            initData();
            optTeacherMenu();
            break;
        case LOGIN_ERR:
            printf("\n非法用户 请离开\n");
            return 0;
        default:
            break;
    }
    return 0;
}

```

登录子程序 login.cpp 源代码：

```

#include "login.h"
#include <stdio.h>
#include <conio.h>
#include "loadcom.h"
#include "crud.h"
#include "order.h"
#include "data_manage.h"
void loginInput(char user[], char password[])
{
    printf("输入账号：");
    gets_s(user, 20);
    printf("输入密码：");
    char pwd;
    int len = 0;

```

```

do
{
    pwd = _getch();
    if (pwd != '\r')
    {
        putchar('*');
        password[len++] = pwd;
    }
    else
        break;
} while (len < 19);
password[len] = '\0';
//去掉首尾部空格
rightTrim(leftTrim(user));
}

int login(char user[], char password[])
{
    if (strcmp(user, "colin") == 0
        && strcmp(password, "123456") == 0)
    {
        return LOGIN_SUCCESS;
    }
    return LOGIN_ERR;
}

void showTeacherMenu()
{
    printf("\n-----数据管理-----\n");
    printf("*   r. 显示所有学生   *\n");
    printf("*   a. 增加学生成绩   *\n");
    printf("*   u. 修改学生成绩   *\n");
    printf("*   d. 删除学生成绩   *\n");
    printf("\n-----综合排序-----\n");
    printf("*   m. 数学排名   *\n");
    printf("*   e. 英语排名   *\n");
    printf("*   c. 语文排名   *\n");
    printf("*   s. 总成绩排名 *\n");
    printf("\n-----数据维护-----\n");
    printf("*   i. 导入数据   *\n");
    printf("*   p. 导出数据   *\n");
    printf("-----\n");
    printf("*   q. 退出系统   *\n");
    printf("-----\n");
}

void optTeacherMenu()
{
    char menuID; //menu菜单 ID编号
    do
    {
        printf("指令/>");
        menuID = _getche();
        switch (menuID)
        {
            case 'r':
            case 'R':

```

```

        printf("*****按学号顺序显示*****\n");
        stuNumOrder();
        break;
    case 'a':
    case 'A':
        printf("*****增加新数据*****\n");
        addNewScore();
        break;
    case 'u':
    case 'U':
        printf("*****修改数据*****\n");
        updateScore();
        break;
    case 'd':
    case 'D':
        printf("*****删除数据*****\n");
        deleteScore();
        break;
    case 'm':
    case 'M':
        printf("*****数学排名*****\n");
        mathOrder();
        break;
    case 'e':
    case 'E':
        printf("*****英语排名*****\n");
        englishOrder();
        break;
    case 'c':
    case 'C':
        printf("*****语文排名*****\n");
        chineseOrder();
        break;
    case 's':
    case 'S':
        printf("*****总成绩排名*****\n");
        allScoreOrder();
        break;
    case 'i':
    case 'I':
        printf("*****导入数据*****\n");
        showImportUI();
        break;
    case 'p':
    case 'P':
        printf("*****导出数据ing*****\n");
        showExportUI();
        break;
    case 'q':
    case 'Q':
        quit();
        break;
    default:
        printf("无此指令\n");
        showTeacherMenu();
        break;

```

```

    }
    } while (!(menuID == 'Q' || menuID == 'q'));
}
void quit()
{
    printf("\n正在退出系统\n");
    iterator(listScore);
    while (hasNext(listScore))
    {
        free(next(listScore));
    }
    clear(listScore);
    free(listScore);
}

```

增删改查等功能的数据维护子程序 crud.cpp 的源代码：

```

#define _CRT_SECURE_NO_WARNINGS
#include "crud.h"
#include <conio.h>
#define SHOW_TABLE_FIELD printf("\n%-8s%-10s%6s%6s%6s\n", "姓名", "学号", "数学", "英语", "语文");
#define SHOW_TABLE_ONE_LINE(POINTER) printf("%-8s%-10s%6d%6d%6d\n", POINTER->name, POINTER->stuNum, POINTER->math, POINTER->english, POINTER->chinese);
/*成绩单 列表*/
PLinkedList listScore;

void initData()
{
    listScore = createLinkedList();
    //加载文件的数据 到 链表
    FILE* p = fopen("./data.db", "rb");
    if (p!=NULL )
    {
        P_Score ps =(P_Score) malloc(sizeof(SCORE));
        while ( ps&& fread(ps, sizeof(SCORE), 1, p) >0 )
        {
            add(listScore, ps);
            ps = (P_Score)malloc(sizeof(SCORE));
        }
        free(ps);
        fclose(p);
        p = NULL;
    }
}

P_Score createScore(const char* name, const char* stuNum,
    int math, int english, int chinese)
{
    P_Score p = (P_Score)malloc(sizeof(SCORE));
    strcpy(p->name, name);
    strcpy(p->stuNum, stuNum);
    p->math = math;
    p->english = english;
    p->chinese = chinese;
    p->del = 0;
}

```

```

    return p;
}

P_Score inputNewScore()
{
    //键盘输入一组数据
    char name[28]; //姓名
    char stuNum[20]; //学号
    int math;
    int english;
    int chinese;
    printf("\n请输入姓名: ");
    gets_s(name, 28);
    trimAll(name);
    printf("请输入学号: ");
    gets_s(stuNum, 20);
    trimAll(stuNum);

    printf("请输入数学成绩: ");
    scanf_s("%d", &math);
    printf("请输入英语成绩: ");
    scanf_s("%d", &english);
    printf("请输入语文成绩: ");
    scanf_s("%d", &chinese);
    //清除键盘缓冲区
    setbuf(stdin, NULL);
    return createScore(name, stuNum, math, english, chinese);
}

void appendToFile(P_Score ps)
{
    FILE* p = fopen("./data.db", "ab");
    if (p != NULL)
    {
        //二进制文件的写入
        fwrite(ps, sizeof(SCORE), 1, p);
        fclose(p);
        p = NULL;
    }
}

void addNewScore()
{
    P_Score newScore = inputNewScore();
    printf("是否保存新数据<Y/N>?");
    char yes = getchar();
    //清除键盘缓冲区
    setbuf(stdin, NULL);
    if (yes == 'Y' || yes == 'y')
    {
        add(listScore, newScore);
        appendToFile(newScore);
        printf("保存成功\n");
    }
    else
    {
        free(newScore);
    }
}

```

```

        newScore = NULL;
    }
}

void updateScore()
{
    int index;
    P_Score ps = queryOneScore( &index);
    if (ps == NULL)
    {
        printf("没有此学生的数据\n");
        return;
    }
    //展示
    SHOW_TABLE_FIELD
    SHOW_TABLE_ONE_LINE(ps)
    setbuf(stdin, NULL);
    printf("\n*****准备更新学号为: %s的数据*****\n", ps->stuNum);
    printf("** 0:更新学号: \n");
    printf("** 1:更新姓名: \n");
    printf("** 2:更新数学: \n");
    printf("** 3:更新英语: \n");
    printf("** 4:更新语文: \n");
    printf("** y:保存更新: \n");
    printf("** n:取消更新: \n");
    char updateMenuID;
    SCORE temp;
    memcpy(&temp, ps, sizeof(SCORE));

    do
    {
        printf("更新操作选项/>");
        updateMenuID = _getche();
        switch (updateMenuID)
        {
            case '0':
                printf("新的学号: ");
                gets_s( temp.stuNum, 20);
                break;
            case '1':
                printf("新的姓名: ");
                gets_s( temp.name, 28);
                break;
            case '2':
                printf("新的数学成绩: ");
                scanf_s("%d", &temp.math);
                break;
            case '3':
                printf("新的英语成绩: ");
                scanf_s("%d", &temp.english);
                break;
            case '4':
                printf("新的语文成绩: ");
                scanf_s("%d", &temp.chinese);
                break;

```

```

        case 'y':
        case 'Y':
            memcpy(ps, &temp, sizeof(SCORE));
            //保存到文件
            updateToFile(ps, index);
            printf("数据更新保存成功\n");
            return;
        }
        setbuf(stdin, NULL);
    } while (!(updateMenuID=='n' || updateMenuID=='N'));
}

void deleteScore()
{
    int index;
    P_Score ps=queryOneScore( &index);
    if (ps==NULL)
    {
        printf("没有此学生的数据\n");
        return;
    }
    //展示
    SHOW_TABLE_FIELD
    SHOW_TABLE_ONE_LINE(ps)
    setbuf(stdin, NULL);
    printf("确认删除吗? <Y/N>");
    char yes = getchar();
    setbuf(stdin, NULL);
    if ( yes=='Y' || yes=='y')
    {
        ps->del = 1; //数据做了逻辑删除
        //保存到文件
        updateToFile(ps, index);
    }
}

P_Score queryOneScore( int* const index)
{
    printf("请输入学号: ");
    char stuNum[20];
    gets_s(stuNum, 20);

    iterator(listScore);
    *index = -1;
    int i = 0;
    while ( hasNext(listScore))
    {
        P_Score p = (P_Score)next(listScore);
        if ( strcmp(p->stuNum, stuNum )==0)
        {
            *index = i;
            return p;
        }
        i++;
    }
    return NULL;
}

```

```

}

void updateToFile(P_Score p , int index)
{
    FILE* file = fopen("./data.db", "r+b");
    if (file)
    {
        fseek(file, index*sizeof(SCORE) , SEEK_SET );
        fwrite(p, sizeof(SCORE), 1, file);
        fclose(file);
    }
}

void showAllScore ()
{
    //printf("\n姓名 学号 数学 英语 语文\n");
    SHOW_TABLE_FIELD
    iterator(listScore);
    while (hasNext(listScore))
    {
        P_Score p = (P_Score)next(listScore);
        if (p->del == 0)
        {
            SHOW_TABLE_ONE_LINE(p)
        }
    }
}

```

各种成绩排序子程序 order.cpp 的源代码:

```

#include "order.h"
#include "loadcom.h"
#include "crud.h"
int ruleStuNumOrder(void* s1, void* s2)
{
    P_Score ps1 = (P_Score)s1;
    P_Score ps2 = (P_Score)s2;
    return strcmp(ps1->stuNum, ps2->stuNum);
}
void stuNumOrder()
{
    sort__(listScore, ruleStuNumOrder);
    showAllScore();
}
int ruleMathOrder(void* s1, void* s2)
{
    P_Score ps1 = (P_Score)s1;
    P_Score ps2 = (P_Score)s2;
    if (ps2->math == ps1->math)
    {
        return strcmp(ps1->stuNum, ps2->stuNum);
    }
    return ps2->math - ps1->math;
}

void mathOrder()
{
    sort__(listScore, ruleMathOrder);
}

```

```
    showAllScore();
}

int ruleEnglishOrder(void* s1, void* s2)
{
    P_Score ps1 = (P_Score)s1;
    P_Score ps2 = (P_Score)s2;
    if (ps2->english == ps1->english)
    {
        return strcmp(ps1->stuNum, ps2->stuNum);
    }
    return ps2->english - ps1->english;
}

void englishOrder()
{
    sort__(listScore, ruleEnglishOrder);
    showAllScore();
}

int ruleChineseOrder(void* s1, void* s2)
{
    P_Score ps1 = (P_Score)s1;
    P_Score ps2 = (P_Score)s2;
    if (ps2->chinese == ps1->chinese)
    {
        return strcmp(ps1->stuNum, ps2->stuNum);
    }
    return ps2->chinese - ps1->chinese;
}

void chineseOrder()
{
    sort__(listScore, ruleChineseOrder);
    showAllScore();
}

int ruleAllScoreOrder(void* s1, void* s2)
{
    P_Score ps1 = (P_Score)s1;
    P_Score ps2 = (P_Score)s2;
    int s1Sum = ps1->math + ps1->english + ps1->chinese;
    int s2Sum = ps2->math + ps2->english + ps2->chinese;

    if (s1Sum == s2Sum)
    {
        return strcmp(ps1->stuNum, ps2->stuNum);
    }
    return s2Sum - s1Sum;
}

void allScoreOrder()
{
    sort__(listScore, ruleAllScoreOrder);
    showAllScore();
}
```

最后数据维护导入导出功能 data_manage.cpp 的源代码:

```
#define _CRT_SECURE_NO_WARNINGS
#include "data_manage.h"
#include<stdio.h>
#include"crud.h"
void showImportUI ()
{
    printf("请输入导入数据的来源: ");
    char path[50];
    scanf_s("%s", path, 50);
    if ( importData(path)==IMPORT_SUCCESS)
    {
        printf("\n数据导入成功\n");
    }
    else
    {
        printf("\n数据导入失败\n");
    }
}

int importData(const char* dataPath)
{
    FILE* pImportData = fopen(dataPath, "rt");
    if (pImportData ==NULL) {
        return IMPORT_ERR;
    }
    FILE* pData = fopen(".\\data.db", "ab");
    if (pData!=NULL)
    {
        SCORE temp;
        while ( fscanf_s(pImportData, "%s%s%d%d%d"
                        , temp.name, 28
                        , temp.stuNum, 20
                        , &temp.math
                        , &temp.english
                        , &temp.chinese)>=5 )
        {
            temp.del = 0;
            //存入文件
            fwrite(&temp, sizeof(SCORE), 1, pData);
            //放入链表
            P_Score ps =(P_Score) malloc(sizeof(SCORE));
            memcpy(ps, &temp, sizeof(SCORE));
            add(listScore, ps);
        }
        fclose(pData);
    }
    fclose(pImportData);
    return IMPORT_SUCCESS;
}

void showExportUI ()
{
    printf("导出的位置: ");
```

```
char path[50];
scanf_s("%s", path, 50);
if ( exportData(path)==EXPORT_SUCCESS )
{
    printf("\n数据导出成功\n");
}
else
{
    printf("\n数据导出失败\n");
}
}

int exportData(const char* dataPath)
{
    FILE* pExport = fopen(dataPath, "wt");
    if (pExport!=NULL)
    {
        fprintf(pExport, "\n%-8s%-10s%6s%6s%6s\n", "姓名", "学号", "数学", "英语", "语文");
        //把链表的数据 向文件输出
        iterator(listScore);
        while (hasNext(listScore) )
        {
            P_Score p =(P_Score)next(listScore);
            if (p->del==0)
            {
                fprintf(pExport, "%-8s%-10s%6d%6d%6d\n", p->name, p->stuNum,
                    p->math, p->english, p->chinese);
            }
        }

        fclose(pExport);
        return EXPORT_SUCCESS;
    }
    return EXPORT_ERR;
}
```